

Lynxer: A VOLE-in-the-Head Signature Scheme based on Only Symmetric-key Primitives

Algorithm Specifications and Supporting Documentation

(Version 1.1)

Kang Yang¹, Yu Yu^{2,7}, Lin Jiao¹, Hongrui Cui², Xiaojie Guo³, Qunxiong Zheng⁴, Jiang Zhang¹, Hongsen Yang⁴, Yituo He², Yonglin Hao¹, Chun Guo^{5,8} and Jiangxia Ge⁶

¹ State Key Laboratory of Cryptology, Beijing, China

² Shanghai Jiao Tong University, Shanghai, China

³ Shanghai Qi Zhi Institute, Shanghai, China

⁴ Information Engineering University, Zhengzhou, China

⁵ Shandong University, Qingdao, China

⁶ China Telecom Quantum Group Ltd., Hefei, China

⁷ East China Normal University, Shanghai, China

⁸ State Key Laboratory of Cryptography and Digital Economy Security, Qingdao, China

June 30, 2026

Table of Contents

1	Introduction	5
1.1	Features and Advantages	5
1.2	Hardness Assumption	8
1.3	Organization	9
2	Design Rationale	9
2.1	Symmetric-key Primitive for VOLE-in-the-Head	9
2.1.1	Choice of Field	9
2.1.2	Choice of Structure	9
2.1.3	Choice of Nonlinear Components	10
2.1.4	Configuration of Linear Layers	11
2.1.5	Setting of Constants	11
2.2	Design Approach of Lynxer from VOLEitH-friendly Primitive Lynx	11
2.2.1	Zero-Knowledge Proofs	12
2.2.2	Vector Oblivious Linear Evaluation	12
2.2.3	Proving Lynx OWF with QuickSilver	12
2.2.4	VOLE-in-the-Head	13
2.3	TCCR Hash Function	15
3	Preliminaries	16
3.1	Notation	16
3.1.1	Sampling and Assignment	16
3.1.2	Integer Ranges	16
3.1.3	Vectors and Matrices	16
3.1.4	Tuples and Bit Strings	16
3.2	Hash Function	17
3.3	Pseudorandom Generator	18
3.4	Zero-Knowledge Proof	19
3.5	Information-Theoretic Message Authentication Code	20
3.6	Digital Signature Scheme	21
3.7	Signature Size of VOLEitH	22
4	Lynx: VOLEitH-Friendly Primitive	22
4.1	Nonlinear Components	23
4.2	Linear Components	23
4.3	Concrete Instances	23
5	Specification of Symmetric-based Signature Scheme Lynxer	24
5.1	Parameter Structure	25
5.2	Top-Level APIs	25
5.2.1	Key Generation	25
5.2.2	Signing	26
5.2.3	Verification	28
5.3	Vector Oblivious Linear Evaluation	29
5.3.1	VOLE Challenge Decoding	29
5.3.2	Decoding All VOLE Challenges	30

5.3.3	VOLE Commitment	30
5.3.4	VOLE Reconstruction	31
5.3.5	VOLE Correction	32
5.3.6	Seed Conversion to VOLE	32
5.4	Batch All-but-one Vector Commitment	33
5.4.1	BAVC Commitment	34
5.4.2	BAVC Opening	35
5.4.3	BAVC Verification	36
5.4.4	Leaf Interleaving	37
5.4.5	Leaf Deinterleaving	37
5.4.6	Minimum-Span Opening	37
5.4.7	Leaf Hashing	38
5.5	Lynx One-Way Function	38
5.5.1	Lynx Public-Parameter Sampling	38
5.5.2	Lynx Witness Extension	39
5.5.3	Lynx Output Evaluation	40
5.6	QuickSilver Proof System	40
5.6.1	QuickSilver Proving	41
5.6.2	QuickSilver Verification	43
5.7	Universal Hash Functions	46
5.7.1	VOLE Linear Universal Hash	46
5.7.2	ZK Linear Universal Hash	47
5.8	Utility Functions	47
5.8.1	Bits to Integer	48
5.8.2	Integer to Bits	48
5.8.3	Bits to Field	48
5.8.4	Field to Bits	48
5.9	Symmetric Primitives	49
5.9.1	Message Hash	49
5.9.2	Signing-Coin Hash	49
5.9.3	BAVC Commitment Hash	50
5.9.4	Matrix-and-Vector Expansion Hash	50
5.9.5	First Fiat-Shamir Hash	50
5.9.6	Second Fiat-Shamir Hash	51
5.9.7	Third Fiat-Shamir Hash	51
5.9.8	TCCR Hash Function	51
5.9.9	Pseudorandom Generator	52
5.9.10	Extendable-Output Function	52
5.9.11	Block-Cipher Interface	53
6	Parameter Selection of VOLE-in-the-Head	53
6.1	Classical and Quantum Security Levels	53
6.2	Selection Approach of VOLEitH Parameters	54
6.3	Failure Probability Analysis	54
6.4	VOLE-in-the-Head Parameters	55
7	Performance Evaluation	56
7.1	Instantiations of Standard Symmetric-key Primitives	56
7.2	Implementation Platform	57
7.3	Software Performance	58
7.4	Performance Comparison	60
7.5	Discussion	60

8	Concrete Security Analysis	61
8.1	Concrete Security of the Lynx One-Way Function	61
8.1.1	Algebraic Attacks	61
8.1.2	Statistical Attacks	66
8.1.3	Quantum Attacks	67
8.2	Known Attacks	68
8.2.1	Brute-Force the Public Key	68
8.2.2	Brute-Force the TCCR Tree Generation	68
8.2.3	Brute-Force the PRG	69
8.2.4	Attack Soundness of the ZK Protocol	69
8.2.5	Multi-User Attacks	69
9	Security Proof	69
9.1	Correctness Analysis	70
9.2	Security Proof in the QROM and QICM	73
9.3	Security Proof in the ROM and ICM	97
9.4	Proofs of Subroutine Lemmas and Theorems	99
9.4.1	Local Emulation of Ideal Cipher	99
9.4.2	Proof of Lemma 3.1	100
9.4.3	(Quantum) Security of the CTR-based PRG	104
9.4.4	TCCR Security	111
A	Proving Knowledge of Secret Keys in NEV	130
A.1	Definition of NEV	130
A.2	Expressing Public/Seret Key Relation as Polynomials	131
A.3	Estimating the Proof Size	132
B	Illustrations of the Nonlinear Component	133
B.1	Algebraic Metrics	133
B.2	Statistic Metrics	133

1 Introduction

This document describes and specifies the post-quantum digital signature scheme **Lynx**, which is constructed in the VOLE-in-the-Head (VOLEitH) framework [BBD⁺23b]¹, and builds upon a VOLEitH-friendly symmetric-key cryptographic primitive (referred to as **Lynx**). The primitive **Lynx** is specially designed for VOLEitH-based post-quantum signatures by the design team, and is publicly available in [JYC⁺26]. The design goal of **Lynx** is to guarantee security against adversaries with quantum capabilities. The design of **Lynx** only depends on symmetric-key primitives, i.e., **Lynx** and the standard symmetric-key primitives (i.e., cryptographic hash functions and block ciphers). We rigorously prove the existential unforgeability under chosen message attacks (EUF-CMA) of the **Lynx** scheme in the Quantum Random Oracle Model (QROM) and Quantum Ideal Cipher Model (QICM). To provide a formal proof in the QROM and QICM, we further require that **Lynx** satisfies the pseudorandomness given only a single pair of message and output. In the classical ROM and ICM, the “one-time” pseudorandomness is unnecessary, and it is sufficient to assume that **Lynx** is a secure one-way function. The “one-time” pseudorandomness of **Lynx** is required by the proof technique in the QROM and QICM, and may be eliminated by developing new proof techniques in the future. This document provides the design rationale, algorithm specification, parameter selection, provable security, concrete security evaluation and performance evaluation of the **Lynx** scheme.

1.1 Features and Advantages

Techniques. Our symmetric-based post-quantum signature scheme **Lynx** builds upon the recent VOLEitH framework [BBD⁺23b], and uses **QuickSilver** [YSWW21] to instantiate the underlying VOLE-based zero-knowledge proof. Compared to **FAEST** [BBB⁺26], a symmetric-based post-quantum signature scheme in the VOLEitH framework, which has proceed to the third round of NIST’s Post-Quantum Cryptography (PQC) competition for additional post-quantum signatures, the **Lynx** scheme has the following technical novelties:

- We design a VOLEitH-friendly one-way function **Lynx**, targeting four security levels required by the Next Generation Commercial Cryptography (NGCC) competition [Ins25]. The **Lynx** primitive features a multi-branch structure centered on a finite-field-multiplication-based truncation function, as well as customized nonlinear components and interleaved linear layers. It minimizes the number of VOLE correlations, which determines the efficiency of VOLEitH-based signature schemes. The symmetric-key primitive **Lynx** offers abundant security against the full spectrum of state-of-the-art cryptanalysis, for both classical and quantum adversaries.
- We combine the recent construction of large GGM tree [BBM⁺24] with the **Half-Tree** construction [GYW⁺23] to generate batch all-but-one vector commitments (BAVCs). Moreover, we propose an efficient evaluation approach on the opening size of BAVCs, which can be used to estimate the distribution of BAVC opening size and signing time.
- To instantiate the underlying hash function in **Half-Tree** with classical security levels from 160 bits to 512 bits, we propose a new construction of Tweakable Circular Correlation Robust (TCCR) hash functions, which achieves beyond n -bit classical security from any n -bit block cipher, and is provably secure in both ICM and QICM. Previously, the state-of-the-art TCCR construction with high security was proposed by the design team in [CGW⁺24]. Compared with [CGW⁺24], our new construction enjoys two improvements. On the one hand, the two blockcipher calls involved in [CGW⁺24] use distinct keys for separation, whereas the two calls

¹Vector Oblivious Linear Evaluation (VOLE) [ADI⁺17] is an arithmetic generalization of Oblivious Transfer (OT). The VOLEitH framework adopts the approach in the SoftSpokenOT protocol [Roy22] to generate VOLE correlations from $(N - 1)$ -out-of- N random OT.

involved in our new construction adopt the same key. Thus, the new construction saves one call to the key schedule function, which allows us to improve the computational efficiency. On the other hand, the construction in [CGW⁺24] makes two (or multiple) calls to a linear orthomorphism σ , which was instantiated as $\sigma(x_L \| x_R) = x_R \oplus x_L \| x_L$ where x_L, x_R are the left and right halves of the input x , whereas our construction gets rid of this. This further saves one or two CPU cycles per hash call.

Performance advantage of VOLEitH-friendly symmetric-key primitive Lynx. The design of symmetric-key primitive Lynx minimizes the witness length ℓ and the degree d of polynomials to be proven, which in turn minimizes the “public key + signature” size of symmetric-based signature schemes in the VOLEitH framework. Specifically, for the λ -bit classical security level, Lynx supports $\ell = 3\lambda$ and $d = 2$, which are optimal in the VOLEitH framework. Furthermore, Lynx minimizes the number of finite-field multiplications for the polynomials to be proven. As a result, Lynxer achieves smaller communication cost and competitive or better computation cost, compared to FAEST [BBB⁺26], AIMER [KCH⁺26] (a symmetric-based post-quantum signature scheme in the finalist of Korea’s PQC competition [Kpq26]), and Rainier [DKR⁺22]. See below for a concrete performance comparison.

Security advantage of Lynx. For symmetric-based signature schemes in the VOLEitH framework, each secret key sk is uniquely bound to a random initial vector iv_{owf} , and then they jointly produce a single public key pk . Moreover, the constants and most of linear layers in Lynx are sampled randomly for each pair (sk, pk) . Therefore, an attacker only has access to a single plaintext-ciphertext pair under each key; and the statistical attacks (e.g., differential attacks, linear attacks, and integral attacks), requiring multiple plaintext-ciphertext pairs, are inapplicable. In the presence of both classical and quantum adversaries, Lynx is rigorously validated by covering all possible attacks. In particular, it is secure against all known cryptanalytic attacks targeting state-of-the-art tailored symmetric primitives—including RAIN and AIM—as rigorously evaluated in recent foundational works, which covers the full spectrum of advanced algebraic attacks [BFU26, YZY⁺24, YZY25, ZWY⁺23, SS21, CKPS00, Din21, LMØM23, LMM24, SCL⁺25] targeting key recovery. We also assess the resilience of Lynx against quantum attacks, accounting for the quadratic speedup of Grover’s algorithm.

The necessity of evaluating security with XL attacks. When evaluating the security of Lynx, we analyze its security against the XL algorithm [CKPS00]. Below, we argue that the XL algorithm is necessary to evaluate the algebraic security of symmetric primitives towards post-quantum signatures. Specifically, based on the observation from prior work (CCS’23) by the AIM design team [KHS⁺23], the XL algorithm [CKPS00] should be analyzed separately from the Gröbner basis method [SS21]. The work [KHS⁺23] demonstrates that, for primitives built upon power-mapping S-boxes over binary extension fields, the complexity of XL attacks can be significantly lower than that of Gröbner basis attacks, and this gap directly affects parameter selection. This discrepancy arises because the XL algorithm operates via linearization over $\mathbb{F}_2$, where the exact number of Boolean quadratic equations derived from the S-box critically influences the size of the Macaulay matrix and the working degree D . In contrast, estimating the complexity of a Gröbner basis computation using the generic degree of regularity formula assumes that the algorithm must run up to the maximal degree D_{reg} to terminate, which often fails to capture early-termination phenomena in overdetermined or structured systems and relect the true behaviour of F4/F5 [BFP09, ACG⁺19]. Consequently, for the post-quantum signature schemes whose security relies on the one-wayness of a symmetric primitive over $\mathbb{F}_2$, a comprehensive security assessment must treat XL as an independent and potentially dominant threat, following the worst-case principle by evaluating both algorithms under their respective optimal conditions.

Provable security. We rigorously prove the EUF-CMA security of the Lynxer scheme in the QROM and QICM, assuming that Lynx satisfies the “one-time” pseudorandomness (i.e., a sin-

gle ciphertext is computationally indistinguishable from a random string under each key). Our proof follows the approach used to prove the security of the second version of FAEST in the QROM [BBB⁺25, BBB⁺26]. For completeness, we also fill in proof details that are implicit or omitted in the FAEST specification. We use statistical distinguishers and standardized randomness tests to validate the “one-time” pseudorandomness of Lynx. When proving the EUF-CMA security of the Lynxer scheme in the classical ROM and ICM, we only need to assume that Lynx is a secure one-way function. If new proof techniques are developed in the QROM and QICM, the “one-time” pseudorandomness may be unnecessary.

Table 1: Summary of public-key and signature sizes, running time, and concrete security of VOLEitH-friendly primitive Lynx for the Lynxer signature scheme under different parameter sets. The values in the final section for concrete security of Lynx are reported as the logarithm (base 2) of cost estimates of attacks. The security estimate against the Grover quantum algorithm does not consider the NIST metric of a “MAXDEPTH” constraint, i.e., assuming that the depth of a quantum circuit could be unlimited.

Scheme	Comm. (bytes)		Runtime (ms)			Security of Lynx(bits)					
	pk	σ	KeyGen	Sign	Verify	XL	GB	Dinur	MP	Resu.	Grover
Lynxer-160s	40	4,607	0.080	2.414	2.044	184	235	409	346	699	102
Lynxer-160f	40	5,801	0.081	0.378	0.342	184	235	409	346	699	102
Lynxer-256s	64	12,191	0.111	4.737	4.127	296	468	644	471	1119	151
Lynxer-256f	64	15,097	0.111	0.588	0.553	296	468	644	471	1119	151
Lynxer-384s	96	27,495	0.287	18.474	17.608	429	821	959	681	1678	216
Lynxer-384f	96	34,109	0.287	2.551	2.447	429	821	959	681	1678	216
Lynxer-512s	128	48,879	0.618	47.901	45.204	612	6133	1272	993	2237	281
Lynxer-512f	128	61,091	0.619	5.874	5.737	612	6133	1272	993	2237	281

“Comm.” represents communication cost, “pk” represents Public Key, and “sig” represents signature.

“GB” represents Gröbner Basis, “MP” represents Magic Pot, and “Resu.” represents resultant attack.

Competitive performance. In Table 1, we summarize the communication cost, computational cost, and the security of Lynx for the Lynxer signature scheme under different parameter sets. The running time is taken from the optimized AVX implementation. The performance of Lynx is competitive among symmetric-based post-quantum signature schemes. For the security estimate of Lynx, we only show the attacks with lower costs, and give the costs of other attacks in Section 8.1. We note that the recent Magic Pot attack [BFU26] also fails to attack Lynx. When evaluating the security of Lynx against the Grover quantum algorithm [Gro96a], we do *not* limit the maximum depth of the quantum circuit, which allows us to obtain a conservative security estimate. If the “MAXDEPTH” (ranging from 2^{40} to 2^{96}) by NIST is taken into account, the security margin will be higher, e.g., the quantum security of both Lynxer-160s and Lynxer-160f is 157 bits when the MAXDEPTH is set as 2^{40} .

At the 128-bit quantum security level (or the 256-bit classical security level), Lynxer reduces the “public key + signature” size by 40.9% (resp., 43.0%) for the small (resp., fast) variant, compared to the state-of-the-art symmetric-based signature scheme FAEST [BBB⁺26]. In terms of computational efficiency, compared to FAEST, our Lynxer scheme improves the signing (resp., verification) time by a factor of $1.93\times$ (resp., $2.18\times$) for the small variant; it speeds up the signing (resp., verification) time by a factor of $2.34\times$ (resp., $2.33\times$) for the fast variant. Compared to FAEST-EM (which is the Even-Mansour variant of FAEST [BBB⁺26]), at the 128/256-bit quantum/classical security level, Lynxer reduces the “public key + signature” size by 32.1% (resp., 35.6%) for the small (resp., fast) variant. In terms of computational efficiency, compared to FAEST-EM, the Lynxer scheme improves the signing (resp., verification) time by a factor of

1.51 $\times$ (resp., 1.67 $\times$) for the small variant; it speeds up the signing (resp., verification) time by a factor of 1.91 $\times$ (resp., 1.87 $\times$) for the fast variant.

Flexibility. The VOLEitH framework offers a number of parameters affecting the size/runtime characteristics of the signature scheme built on top of it. Therefore, for each parameter set, we choose a small variant targeting smaller signature size and a fast variant targeting fast running time. Further parameter tuning is also possible and our BAVC opening size estimation could provide guidance.

Extensibility. The Non-Interactive Zero-Knowledge (NIZK) proof framework in Lynxer can be used to prove a broader range of statements beyond proving knowledge of secret keys for the Lynx primitive. Specifically, the underlying VOLEitH proof system excels at proving systems of low-degree constraints, and is particularly efficient when the underlying statement has a relatively short witness. Such statements arise in applications, for example, when proving knowledge of secret keys for lattice-based Key Encapsulation Mechanisms (KEMs). NIZK proofs for such statements can be applied in post-quantum Public Key Infrastructure (PKI) systems for Transport Layer Security (TLS) and Authenticated Key Exchange (AKE) protocols built from lattice-based KEMs. For instance, we can use the VOLEitH-based NIZK proof system in Lynxer to prove knowledge of the secret keys on public keys for lattice-based KEMs, e.g., Kyber [SAB⁺20], NEV [ZFY23] and Aegis-enc [ZYF⁺20]. As an example, we show how to prove knowledge of the secret key on a public key for NEV-KEM in Appendix A.

Simplicity. The Lynxer signature scheme is built entirely from symmetric-key primitives: block ciphers for pseudorandom generation and TCCR hashing, cryptographic hash functions for Fiat-Shamir hashing and public randomness sampling, and the Lynx one-way function. No number-theoretic operations (e.g., number-theoretic transforms or Gaussian sampling) or floating-point arithmetic are required. This makes the Lynxer design conceptually simple, easy to audit, and straightforward to implement in constant time. It also simplifies the protection against side-channel attacks such as timing attacks and power-analysis attacks.

Compatibility. Apart from Lynx, the cryptographic primitives utilized in Lynxer are block ciphers and cryptographic hash functions. Currently, we instantiate them with standard algorithms (i.e., AES, SHA2, and SHA3) that have been widely studied. Nevertheless, Lynxer only has black-box access to these primitives, and so it is straightforward to replace them with symmetric primitives standardized in the future.

1.2 Hardness Assumption

The security of the Lynxer signature scheme relies on the one-wayness of the VOLEitH-friendly symmetric primitive Lynx. In the VOLEitH-based signature scheme, each random secret key sk is uniquely bound to a randomly sampled initial vector iv_{owf} , and together they produce exactly one output pk . Consequently, the attacker cannot accumulate statistically observations tied to sk . Moreover, the linear transformation within Lynx is sampled independently and uniformly for each individual user. Given the per-user randomness, Lynx naturally resists all known statistical cryptanalysis techniques, such as differential, linear, and integral attacks, even in the multi-target setting. Therefore, we focus on analyzing the security of Lynx against algebraic attacks. The structure of Lynx produces cross-terms among pairwise witness variables, strengthening the resilience against large-field algebraic attacks (e.g., Magic Pot attack [BFU26], resultant attack [YZY⁺24] and linearized polynomial decomposition attacks [YZY25, SCL⁺25]). Furthermore, linear layers of Lynx are strategically interleaved to isolate nonlinear components. This ensures high-degree terms for every witness variable, and prevents variable elimination from spawning additional Boolean equations [LMØM23, LMM24]. Specific functions, such as the Kasami mapping and the extended truncation function, are chosen to compress multiple

multiplications into a single nonlinear layer. This simultaneously reduces the overdefined degree of the Boolean equation system, which enhances the resistance against $\mathbb{F}_2$ -based algebraic attacks (e.g., XL attack [CKPS00], Gröbner Basis attack [SS21]). Besides, we analyze the security of Lynx in the presence of the Grover’s attack.

1.3 Organization

The rest of this document is organized as follows. [Section 2](#) explains the design rationale of Lynx, including the overall construction approach, the parameter-selection methodology, the failure-probability analysis, and other design considerations. [Section 3](#) introduces the notation and recalls the formal definition of digital signature schemes. [Section 4](#) specifies the Lynx One-Way Function (OWF). [Section 5](#) presents the full algorithm description, including the parameter structure, the top-level algorithms, the underlying VOLE, BAVC, Lynx OWF, QuickSilver, universal-hash, utility, and symmetric-cryptographic components, as well as the selected parameter sets. [Section 7](#) reports on the implementation setting and benchmark results for the current software implementations. [Section 8](#) discusses known attacks and evaluates the concrete security of the Lynx one-way function and the symmetric primitives used in the scheme. [Section 9](#) gives the correctness analysis and shows the formal security proof in the QROM and QICM. [Section A](#) shows an example of how to prove knowledge of secret keys in the NEV-KEM scheme.

2 Design Rationale

2.1 Symmetric-key Primitive for VOLE-in-the-Head

We first describe the design rationale of the symmetric-key primitive Lynx that is friendly for VOLE-in-the-Head (VOLEitH). When using a VOLEitH-based zero-knowledge proof to prove knowledge of the preimage of the Lynx One-Way Function (OWF), we transform such a proof into a proof on a set of multi-variable polynomials, where the witness is extended to include the intermediate variables.

2.1.1 Choice of Field

Lynx is a symmetric-key primitive defined over $\mathbb{F}_{2^\lambda}$. Its security level matches the field extension degree λ . Tailored for VOLEitH, it is built on nonlinear operations over $\mathbb{F}_{2^\lambda}$, exactly the underlying field adopted in VOLE, which incurs no field conversion overhead, and can fully leverage modern CPU instruction sets for acceleration. Linear layers are realized as matrix multiplications over $\mathbb{F}_2$, thereby preserving the proof size while achieving a high algebraic degree when viewed as linearized polynomials over $\mathbb{F}_{2^\lambda}$.

2.1.2 Choice of Structure

The signature size for a VOLEitH-based signature scheme is linear in

$$\hat{\ell} = \ell + d \cdot \lambda + B, \tag{1}$$

where ℓ is the length of the secret witness, comprising the secret key input and the intermediate variables that need to be committed; d is the maximum degree of the multi-variable polynomials to be proven (that are called constraint polynomials hereafter); and B is a constant associated with the VOLE consistency check [YSWW21, BBD⁺23b]. For a fixed $\hat{\ell}$, reducing the number of field multiplications (as well as Frobenius automorphisms) in the constraint polynomials improves both signing and verification efficiency. Our goal is to minimize the “public key + signature” size, when accelerating the running time.

To minimize the signature size, we jointly reduce both d and ℓ . Nonlinear components are essential for cryptographic security; hence $d \geq 2$. The Lynx primitive adopts $d = 2$ to maintain a tight bound. Then, we turn to ℓ . Suppose the primitive contains ϑ nonlinear components; then generally, $\vartheta - 1$ intermediate variables must be committed—only the nonlinear component connected to the primitive’s output can be directly checked and requires no intermediate commitment. Thus, $\ell = (1 + \vartheta - 1)\lambda = \vartheta\lambda$, and the objective reduces to minimizing ϑ .

- Can ϑ be 2? No —the guess-and-determine S-box linearization technique from [ZWY+23] enables key recovery by linearizing one nonlinear component and solving the lower-dimensional quadratic system induced by the remaining nonlinear component, thereby posing a serious threat to such primitives as two-round RAIN.
- Lynx adopts $\vartheta = 3$. These three nonlinear components require careful design and placement to resist all potential attacks. Specifically, $\mathbb{F}_{2^\lambda}$ -based algebraic attacks (e.g., Magic Pot attack [BFU26], resultant attack [YZY+24] and linearized polynomial decomposition attacks [YZY25, SCL+25]) exploit structural properties of the primitive including S-box composition and linear layer design. In contrast, $\mathbb{F}_2$ -based algebraic attacks (such as XL, Gröbner Basis Attacks [SS21, CKPS00]) rely critically on the overdefined degree of the quadratic Boolean equation system derived from constraint polynomials. To achieve one-wayness, we depart from conventional truncation designs (especially those relying on linear operations) and introduce a multi-input, single-output nonlinear component. Finite-field multiplication is the natural choice. To realize “multi-input”, our design adopts a multi-branch framework: each branch applies an S-box to an affine transformation of the secret input in parallel; the resulting outputs are then multiplied collectively over $\mathbb{F}_{2^\lambda}$. Combined with a feed-forward of all witnesses, this produces the final output. Moreover, the truncation function based on finite-field multiplication inherently introduces cross-terms among intermediate variables, thereby strengthening resilience against these large-field algebraic attacks.

Therefore, $\hat{\ell} = 5\lambda + B$ remains optimal across all security levels within the VOLEitH paradigm.

2.1.3 Choice of Nonlinear Components

To support four security levels ($\lambda = 160, 256, 384, 512$) while maintaining structural uniformity, we tailor the nonlinear components to each security level. Besides the finite-field multiplication, our candidate functions include power permutations over $\mathbb{F}_{2^\lambda}$: the inverse function, Mersenne function, Kasami function, and their respective inverses. Lynx employs a power mapping in one branch and an inverse power mapping in another branch, thereby ensuring a high algebraic degree in both forward and backward evaluations. The candidate functions all admit quadratic constraint polynomials; however, they differ in (i) the number of finite-field multiplications and Frobenius automorphisms in their constraint polynomials, and (ii) the number of independent quadratic Boolean equations they induce. The former affects the signing and verification runtime under the same communication cost, while the latter determines resilience against algebraic attacks over $\mathbb{F}_2$.

For example, both the Kasami function and the three-input truncation function (see Figure 1) require two multiplications in their constraint polynomials, yet each yields only a single set of quadratic Boolean equations. It reduces the overdefined degree of the resulting equation system, thereby strengthening resilience against Gröbner basis attack and XL attacks. This is equivalent to compressing multiple multiplications within a single nonlinear component, enhancing the security of the primitive without adding a witness. In particular, when the final truncation function is $f(x, y, z) = x \cdot y \cdot z^{-1}$, this operation is realized via a dedicated parallel branch, whose output is multiplied with those of other parallel branches. This branch is configured without a linear layer and uses the field inverse as its S-box. Since this S-box contributes directly to Lynx’s output alongside the final truncation, the witness count ϑ remains 3.

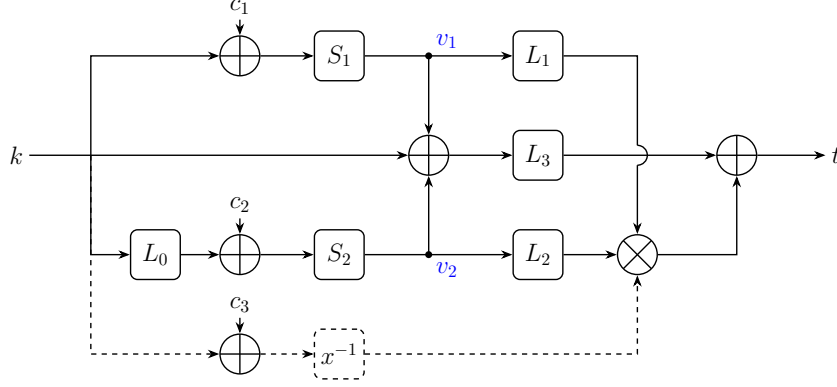


Figure 1: The structure of Lynx (Input: key k and iv_{owf} ; Output: t ; Intermediate variable: v_1 and v_2 ; Nonlinear and linear components: S_i and L_i ; Constant: c_i). The dashed branch appears only in Lynx-256, -384, -512. L_0, L_1, L_2 and c_1, c_2, c_3 are derived from $\text{XOF}(\text{iv}_{\text{owf}})$.

2.1.4 Configuration of Linear Layers

When setting the linear layers, we ensure that high-degree, dense linearized polynomials separate all nonlinear components (including those between input and output, across branches, and between branches and the final truncation function). This prevents variable elimination from generating additional quadratic Boolean equations and guarantees that the overdefined degree of the Boolean equation system derived from constraint polynomials is sufficiently low [LMM24]. Moreover, the secret key and intermediate variables are all routed through these linear layers, ensuring their appearance with high-degree terms in Lynx’s constraint polynomial system over $\mathbb{F}_{2^\lambda}$, thereby providing resistance against large-field algebraic attacks. Consequently, Lynx employs exactly four linear layers.

Moreover, Lynx incorporates randomly generated linear layers derived from the user’s IV via an extendable output function (XOF). Equipping each branch with a randomly sampled linear layer for every user suffices to endow Lynx with instance-specific security against attacks requiring multiple input-output pairs. To achieve efficiency, we fix the linear layer in the feed-forward path. It also satisfies strict cryptographic criteria, thereby eliminating the risk of random degradation.

2.1.5 Setting of Constants

Constants in each branch must be set to prevent attackers from using identical guesses to conduct attacks. They should also be iv_{owf} -dependent to thwart strong key-reuse attacks, wherein the same secret key is used with different iv_{owf} ². Furthermore, to support the security proof in the QROM, Lynx requires not only one-wayness but also “one-time” pseudorandomness. It also provides resilience against both statistical distinguishers and standardized randomness tests.

2.2 Design Approach of Lynxer from VOLEitH-friendly Primitive Lynx

Lynxer is a digital signature scheme based on the VOLEitH-friendly symmetric primitive Lynx. A key pair in Lynxer describes an input-output pair of Lynx and a signature of Lynxer is a Non-Interactive Zero-Knowledge (NIZK) proof on proving knowledge of the Lynx input. The NIZK is constructed in the VOLE-in-the-Head (VOLEitH) framework. Below, we introduce the main components of the Lynxer signature scheme.

²Although such a scenario is excluded under standard security models and rarely occurs in practice, we include this countermeasure for defense in depth.

2.2.1 Zero-Knowledge Proofs

Zero-knowledge proofs [GMR85, GMW91] allow a prover to convince a verifier that a statement is valid, without revealing anything beyond the validity of the statement. A standard paradigm of designing signature schemes is to use an NIZK scheme to prove knowledge of the preimage of a One-Way Function (OWF), while binding the message to be signed with the proving process. Under this paradigm, a signature scheme's security builds on the security of the one-way function and the proof system. Therefore, by combining NIZK with an OWF that resists quantum attacks, one can naturally obtain a post-quantum digital signature scheme. As a signature consists of an NIZK transcript, the proof size affects the resulting signature size. Hence, choosing a suitable NIZK scheme is crucial for the signature scheme's efficiency. The Lynxer scheme utilizes the VOLEitH framework [BBD⁺23b] to construct the underlying NIZK. The VOLEitH framework combines the VOLE-based QuickSilver proof system [YSWW21] with a publicly-verifiable scheme of producing Vector Oblivious Linear Evaluation (VOLE) correlations [Roy22, BBD⁺23b]. We present a bottom-up overview of the VOLEitH framework in the following.

2.2.2 Vector Oblivious Linear Evaluation

Given a binary field $\mathbb{F}_2$, its extension field $\mathbb{F}_{2^\lambda}$, and a vector length n , VOLE allows a sender to obtain a vector of messages $\mathbf{u} \in \mathbb{F}_2^n$ and MAC tags $\mathbf{v} \in \mathbb{F}_{2^\lambda}^n$ and a receiver to get the local keys $\mathbf{q} \in \mathbb{F}_{2^\lambda}^n$ and a global key $\Delta \in \mathbb{F}_{2^\lambda}$, subject to

$$\mathbf{q} = \mathbf{u} \cdot \Delta + \mathbf{v}. \quad (2)$$

VOLE can be viewed as a two-party commitment scheme. For any $i \in [\ell]$, the message $\mathbf{u}[i]$ is hidden from the receiver, as $\mathbf{v}[i]$ offers a one-time padding to mask $\mathbf{u}[i]$, which satisfies the hiding property. Moreover, the receiver can ask the sender to open $\mathbf{u}[i]$ by providing $\mathbf{v}[i]$ alongside $\mathbf{u}[i]$, while it checks $\mathbf{q}[i] = \mathbf{u}[i] \cdot \Delta + \mathbf{v}[i]$. Any attempt to provide a different pair $(\tilde{\mathbf{u}}[i], \tilde{\mathbf{v}}[i])$ while still passing the check reduces to guessing Δ , since $\Delta = (\tilde{\mathbf{u}}[i] - \mathbf{u}[i])^{-1} \cdot (\mathbf{v}[i] - \tilde{\mathbf{v}}[i])$. This happens except with negligible probability for Δ with sufficient min-entropy, so the scheme is also binding.

An alternative view of VOLE commitment is placing the committed message $\mathbf{u}[i]$ in the leading coefficient of a degree-1 polynomial $f(X) = \mathbf{u}[i] \cdot X + \mathbf{v}[i]$. The receiver's global key Δ is a random point, and its local key $\mathbf{q}[i]$ is the polynomial's evaluation on this point $f(\Delta)$. Naturally, we can generalize this pattern to higher-degree polynomials. Moreover, by relating commitments to polynomials, homomorphic operations on committed messages are also straightforward, as we are essentially operating on polynomials. This implies that VOLE commitments are also homomorphic.

VOLE can also be viewed as a vector of Information-Theoretic Message Authentication Codes (IT-MACs). We recall the IT-MAC definition in Section 3.5. In the following, we show that the homomorphism of VOLE/IT-MAC gives rise to efficient zero-knowledge protocols. Hereafter, we use $\llbracket x \rrbracket^d$ to denote a degree- d VOLE/IT-MAC commitment of $x \in \mathbb{F}_{2^\lambda}$ for $d \in \mathbb{N}^+$, and denote by $\llbracket \mathbf{x} \rrbracket^d$ a vector of degree- d VOLE/IT-MAC commitments for $\mathbf{x} \in \mathbb{F}_{2^\lambda}^\ell$.

2.2.3 Proving Lynx OWF with QuickSilver

QuickSilver [YSWW21] is an efficient zero-knowledge protocol for proving NP relations with IT-MACs. We first recall the basic approach of QuickSilver, and then show how to use it to prove the Lynx OWF. For our purpose, we only use QuickSilver to prove degree-2 relations.

QuickSilver. Given $\mathbf{k} \in \mathbb{F}_2^\ell$, suppose that a prover Prv and a verifier Ver already hold $\llbracket \mathbf{k} \rrbracket^1$. Then, for a degree-2 ℓ -variate polynomial f , Prv can prove $f(\mathbf{k}) = 0$ by letting both parties compute $\llbracket e \rrbracket^2 := f(\llbracket \mathbf{k} \rrbracket^1)$, and then reducing the check of $f(\mathbf{k}) = 0$ to that of $e = 0$, which is

accomplished via the IT-MAC opening. Based on the linear homomorphism of IT-MACs, we can verify multiple degree-2 relations in a batch. Specifically, given the authenticated witness $\llbracket \mathbf{k} \rrbracket^1$, suppose that $f_i(\mathbf{k}) = 0$ for each $i \in [m]$ needs to be proven, where each f_i is a degree-2 ℓ -variate polynomial. The QuickSilver protocol executes as follows:

- Prv and Ver compute $\llbracket e_i \rrbracket^2 := f_i(\llbracket \mathbf{k} \rrbracket^1)$ for each $i \in [m]$.
- Ver samples $\chi_1, \dots, \chi_m \leftarrow \mathbb{F}_{2^\lambda}$ uniformly at random, and then sends them to Prv.
- Prv and Ver compute $\llbracket \bar{e} \rrbracket^2 := \sum_{i=1}^m \chi_i \cdot \llbracket e_i \rrbracket^2$, and Ver checks $\bar{e} = 0$ via the IT-MAC opening.

Let $p_{\bar{e}}(X)$ be the degree-1 polynomial on $\llbracket \bar{e} \rrbracket^2$ as $\bar{e} = 0$. Since the coefficients of $p_{\bar{e}}(X)$ depends on the secret witness $\mathbf{k}$, to ensure the zero-knowledge property, Prv masks $\llbracket \bar{e} \rrbracket^2$ with a random IT-MAC of zero, i.e., $\llbracket 0 \rrbracket^2$ where the underlying polynomial $p_0(X)$ is a random degree-1 polynomial. The $\llbracket 0 \rrbracket^2$ can be generated by aggregating λ random degree-1 IT-MACs of elements from $\mathbb{F}_2$ into a single IT-MAC on a random element in $\mathbb{F}_{2^\lambda}$.

Proving Lynx OWF in zero-knowledge. The computation of Lynx is defined by two S-boxes S_1, S_2 and the $\mathbb{F}_2$ -linear maps L_0, L_1, L_2, L_3 . Crucially, S_1 and S_2 computes exponentiations over $\mathbb{F}_{2^\lambda}$, and the exponents are specifically chosen such that for $i \in \{1, 2\}$, verifying $y = S_i(x)$ can be done by a quadratic constraint over the Frobenius maps of x, y . When viewing $\mathbb{F}_{2^\lambda}$ elements as vectors over $\mathbb{F}_2^\lambda$, the Frobenius map is linear and can be described by a public matrix. Therefore, given the bit decomposition of the S-box input and output, verifying their validity can be described by a quadratic polynomial over these bits. Finally, given the S-box outputs and Lynx input, verifying that the public Lynx output value is correctly computed also reduces to checking a quadratic equation. These constraints can be efficiently proven via the QuickSilver zero-knowledge proof system.

Therefore, in Lynxer, the signer acts as the QuickSilver prover to prove knowledge of a Lynx input k with respect to the Lynx output t in the public key as follows. The signer first extends the witness k from its signing key into the extended witness, which contains k, v_1, v_2 , i.e., the Lynx input as well as the outputs of the S-boxes S_1, S_2 . Using the VOLEitH framework, the signer generates IT-MAC commitments on the bit decomposition of the extended witness. Then, it invokes QuickSilver to prove that the quadratic constraints corresponding to the correct evaluation of Lynx hold.

As we follow the QROM proof idea of FAEST [BBB⁺26], the one-way function co-domain is required to be larger than its domain. Therefore, as in FAEST, we artificially require that the first two bits of the Lynx input k are not both 1. This can also be enforced efficiently in QuickSilver by checking $\mathbf{k}[0] \cdot \mathbf{k}[1] = 0$, where $\mathbf{k} \in \mathbb{F}_2^\lambda$ is the vector form of $k \in \mathbb{F}_{2^\lambda}$. We note that the constraint of $\mathbf{k}[0] \cdot \mathbf{k}[1] = 0$ is just used for obtaining a proof in the QROM. The constraint may be eliminated if new proof techniques are proposed in the future. Furthermore, when proving Lynxer in the classical ROM, the constraint is unnecessary and removed. When Lynxer with the constraint of $\mathbf{k}[0] \cdot \mathbf{k}[1] = 0$ is provably secure in the QROM, it is naturally secure without the constraint, as the key $\mathbf{k}$ has a higher min-entropy. Therefore, the specification of Lynxer shown in Section 5 does not involve such a constraint.

Since the QuickSilver zero-knowledge protocol is private-coin and thus designated-verifier (i.e., only the verifier, who knows Δ , can verify the proof), it is not directly suitable for constructing a signature scheme. In Section 2.2.4, we describe the VOLE-in-the-Head framework, which generates VOLE correlations in a public-coin fashion, thereby transforming QuickSilver into a publicly verifiable zero-knowledge proof and subsequently into a digital signature scheme.

2.2.4 VOLE-in-the-Head

Based on the approach in the SoftSpokenOT protocol [Roy22], Baum et al. [BBD⁺23b] proposed a VOLE generation scheme, which converts QuickSilver from *private coin* to *public coin*. The

verifier does not need to maintain a secret internal state, and so the scheme becomes publicly verifiable. Specifically, the scheme consists of two phases.

- In the first phase, the prover commits to its VOLE output $(\mathbf{u}, \mathbf{v})$. After this phase, the prover cannot change its output.
- In the second phase, the verifier samples Δ at random and reveals it to open $\mathbf{q} = \mathbf{u} \cdot \Delta + \mathbf{v}$.

Between the two phases, the prover uses its VOLE output to prove a given NP relation via QuickSilver. After the second phase, the verifier uses $\mathbf{q}$ to check the proof. Since Δ is revealed to the prover only after the QuickSilver protocol has terminated, this information cannot compromise the security of QuickSilver. This methodology is called VOLE-in-the-Head (VOLEitH). Using the VOLEitH framework, we can construct a public-coin zero-knowledge proof for the input/output relation of the Lynx function. The prover commits to the (bit decomposition) of the elements over $\mathbb{F}_{2^\lambda}$ in the extended witness, and then use QuickSilver to prove their validity with respect to the quadratic constraints defined by Lynx. It ensures that the prover indeed knows the Lynx input corresponding to the Lynx output in the public key.

At a high level, the VOLEitH-based zero-knowledge proofs work as follows. Let $\text{PRG} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\ell$ be a pseudorandom generator, and suppose the construction uses τ small VOLE instances over $\mathbb{F}_{2^k}$. The prover first commits to a collection of seeds, and derives a random vector of degree-1 IT-MACs from these seeds. Then, the prover transforms random IT-MACs into chosen IT-MACs depending on the witness. Next, he runs QuickSilver on top of these IT-MACs; afterwards, the verifier obtains enough information to reconstruct its verification material. More concretely, the main steps are shown as follows:

1. Using a batch all-but-one vector commitment (BAVC) scheme, the prover commits to τ sets of 2^k seeds (using τ k -depth GGM trees), denoted by $\{\text{sd}_j^i\}_{i \in [0, \tau), j \in [0, 2^k)}$.
2. For each $i \in [0, \tau)$, the prover expands these seeds and derives a small-field VOLE correlation by computing

$$\mathbf{u}^i := \sum_{j \in \mathbb{F}_{2^k}} \text{PRG}(\text{sd}_j^i), \quad \mathbf{v}^i := \sum_{j \in \mathbb{F}_{2^k}} (-j) \cdot \text{PRG}(\text{sd}_j^i),$$

where we implicitly identify $\mathbb{F}_{2^k}$ with the index set of size 2^k .

3. To merge these τ instances into one VOLE correlation over a large field, the prover computes $\mathbf{c}^i := \mathbf{u}^0 - \mathbf{u}^i$ for each $i \in [1, \tau)$, and then sends them to the verifier. The prover defines $\mathbf{u} := \mathbf{u}^0$ and $\mathbf{v} := (\mathbf{v}^0, \dots, \mathbf{v}^{\tau-1})$. Together, $(\mathbf{u}, \mathbf{v})$ define a vector of degree-1 IT-MACs.
4. To ensure that $\{\mathbf{c}^i\}_{i \in [1, \tau)}$ are consistent with the committed seeds, the prover and verifier execute the SoftSpokenOT consistency check [Roy22].
5. Using the random IT-MACs, the prover and verifier run the QuickSilver protocol. This is possible because the verifier messages used by QuickSilver are only random challenges, and thus can be generated before the verifier has reconstructed its verification material.
6. After the QuickSilver messages are fixed, the verifier samples challenge indices $\Delta^i \in [0, 2^k)$ for all $i \in [0, \tau)$. Using the BAVC opening procedure, the prover opens all committed seeds except those at the challenged positions. From the opened seeds, the verifier reconstructs

$$\mathbf{q}^i := \sum_{j \in \mathbb{F}_{2^k}} (\Delta^i - j) \cdot \text{PRG}(\text{sd}_j^i),$$

and additionally adds the correction term $\mathbf{c}^i \cdot \Delta^i$ for each $i \in [1, \tau)$. The verifier defines $\mathbf{q} := (\mathbf{q}^0, \dots, \mathbf{q}^{\tau-1})$, which is exactly the verification material. It then uses $\mathbf{q}$ to verify the QuickSilver consistency check.

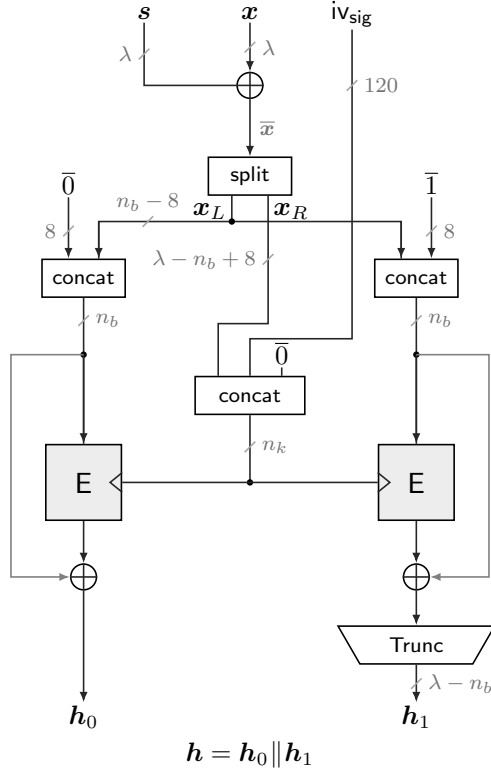


Figure 2: The new TCCR hash construction in the two-block case $n_b < \lambda \leq 2n_b$.

Fiat-Shamir transform. We use the Fiat-Shamir heuristic [FS87] to transform a public-coin zero-knowledge protocol into a publicly verifiable NIZK proof. In particular, since the verifier’s messages are public coins, the prover can simulate the verifier’s behavior using a cryptographic hash function modeled as a random oracle: (1) the prover generates the verifier’s messages by hashing the message to be signed together with the protocol transcript up to this point; (2) it uses the transcript as a signature on the message. For a One-Way Function (OWF), we can naturally define an NP relation based on the input and output of the OWF, with the corresponding preimage as the witness. Therefore, the final piece of Lynxer is to apply the Fiat-Shamir transform to the public-coin zero-knowledge protocol for the Lynx relation in the VOLE-in-the-Head framework.

2.3 TCCR Hash Function

We propose a new construction of TCCR hash functions, which is provably secure in both ICM and QICM. In particular, the TCCR hashing is constructed from a blockcipher with n_b -bit block size and n_k -bit key length, and the hashing should offer $\lambda > n_b$ bit security. This means that the hashing needs to have the input and output of $\lambda > n_b$ bits. Previous constructions like [GKWY20, GKW⁺20, GWYY25] focus on the $\lambda = n_b$ case. Intuitively, setting the security parameter higher than the block length requires making more than one blockcipher call per hash input.

Fortunately, we have $n_b + n_k \gg \lambda$, meaning that the λ -bit input can be fully absorbed into each blockcipher call. Specifically, for the parameters relevant to Lynxer, we have $\lambda \leq 2n_b$. In this parameter regime, the design team previously proposed a TCCR hash construction which makes two blockcipher calls [CGW⁺24]. We sketch the basic idea below, as the new construction in this specification follows the same generic idea. To avoid various attacks, the two blockcipher calls need to employ strongly correlated yet distinct inputs. By this, the idea consists of: (i) appending the λ -bit input with the 8-bit encoding of an integer $i \in \{0, 1, \dots, 2^8 - 1\}$; (ii) padding

the result with $n_b + n_k - \lambda - 8$ zeros; (iii) breaking the result into an n_b -bit part and a n_k -bit part and setting them as the input to the $(i + 1)^{\text{th}}$ blockcipher call.

Compared to [CGW⁺24], our new construction incorporates further optimizations by arranging the integer i in the plaintext input part of the blockcipher calls, so that the two blockcipher calls can use the same key. This design allows us to avoid one costly key schedule function evaluation. Moreover, we remove the usage of linear orthomorphism in the new design, which brings further performance optimization. The detailed construction of the new TCCR hash function is described in Section 5.9.8. We show the architecture of the new TCCR design in Figure 2. In this figure, “split” denotes splitting operation of breaking the λ -bit $\bar{x} := x \oplus s$ into the $(n_b - 8)$ -bit left part x_L and the $(\lambda - n_b + 8)$ -bit right part x_R ; “concat” denotes the concatenation operation; the shaded “E” block denotes the ideal cipher with n_b -bit block size (the vertical arrows correspond to plaintext input and ciphertext output) and n_k -bit key size (the horizontal arrow corresponds to the key input); “Trunc” denotes the truncation operation that takes the least significant $\lambda - n_b$ bits from the n_b -bit input. Note that the two ideal cipher calls share the same key, which is x_R concatenated with iv_{sig} and zero-padded to n_k bits. This saves one key schedule for implementation.

3 Preliminaries

3.1 Notation

3.1.1 Sampling and Assignment

We write $x \leftarrow S$ to denote the sampling of x from S : if S is a set, then x is sampled uniformly at random from S ; if S is a distribution, then x is sampled according to S . We write $x \leftarrow y$ to denote deterministic assignment.

3.1.2 Integer Ranges

For a positive integer n , we write $[n] := \{0, 1, \dots, n - 1\}$; more generally, for integers $a \leq b$, we write $[a, b] := \{a, a + 1, \dots, b - 1\}$.

3.1.3 Vectors and Matrices

Vectors are denoted by bold lowercase letters and matrices by bold uppercase letters, with entries accessed via bracket notation. For $\mathbf{v} \in \mathbb{F}^n$, we write $\mathbf{v}[i]$ for the i -th entry ($i \in [0, n)$) and $\mathbf{v}[i, j]$ for the sub-vector from index i to $j - 1$. For $\mathbf{V} \in \mathbb{F}^{m \times n}$, we write $\mathbf{V}[i, :]$ for the i -th row ($i \in [0, m)$) and $\mathbf{V}[:, j]$ for the j -th column ($j \in [0, n)$). Concatenation uses brackets: for $\mathbf{x} \in \mathbb{F}^m$ and $\mathbf{y} \in \mathbb{F}^n$, we write $[\mathbf{x}, \mathbf{y}] \in \mathbb{F}^{m+n}$; for matrices $\mathbf{A} \in \mathbb{F}^{m \times n}$ and $\mathbf{B} \in \mathbb{F}^{m \times p}$, we write $[\mathbf{A}, \mathbf{B}] \in \mathbb{F}^{m \times (n+p)}$ for horizontal concatenation, and similarly for vertical concatenation. For $\mathbf{x}, \mathbf{y} \in \mathbb{F}^n$, their *inner product* is $\langle \mathbf{x}, \mathbf{y} \rangle := \sum_{i \in [0, n)} \mathbf{x}[i] \cdot \mathbf{y}[i] \in \mathbb{F}$.

3.1.4 Tuples and Bit Strings

We write $\{0, 1\}^n$ for a bit string of length n and $\{0, 1\}^*$ for an arbitrary-length bit string. For $\mathbf{a} \in \{0, 1\}^m$ and $\mathbf{b} \in \{0, 1\}^n$, we define $\mathbf{c} := \mathbf{a} \parallel \mathbf{b} \in \{0, 1\}^{m+n}$. We write 0^k and 1^k for all-zero and all-one strings of length k , respectively. We denote bit-wise XOR by $\oplus$ and, for $\mathbf{a} \in \{0, 1\}^n$, we write $\text{HW}(\mathbf{a})$ for the *Hamming weight* of $\mathbf{a}$, i.e., the number of nonzero entries. We use (a, b, c) to denote a tuple of elements a, b, c (not necessarily from the same set). For an indexed family of elements, we write $(x_i)_{i \in I}$ when each element x_i is a single object; when the elements are themselves tuples, we use outer braces to disambiguate, i.e., we write $\{(x_i, y_i)\}_{i \in I}$ rather than $(x_i, y_i)_{i \in I}$.

3.2 Hash Function

In our signature scheme, we require the following hash functions.

Definition 3.1 (Universal Linear Hash Function). *Let $\mathcal{H} = \{H : \mathcal{X} \rightarrow \mathcal{Y}\}$ be a family of hash functions for two Abelian groups $(\mathcal{X}, +)$ and $(\mathcal{Y}, \oplus)$. $\mathcal{H}$ is ε -universal linear if*

- **Universality.** *For any non-zero $x \in \mathcal{X}$, $\Pr_{H \leftarrow \mathcal{H}} [H(x) = 0] \leq \varepsilon$.*
- **Linearity.** *For any $x, x' \in \mathcal{X}$, $H(x + x') = H(x) \oplus H(x')$.*

Definition 3.2 (Hiding of Hash Function). *Let $\mathcal{H} = \{H : \mathbb{F}_q^{n+h} \rightarrow \mathcal{Y}\}$ be a family of hash functions. $H \in \mathcal{H}$ is $\mathbb{F}_q^n$ -hiding if the distribution of $H(x)$ is independent of $x[0, n]$ conditioned on $x[n, n+h] \leftarrow \mathbb{F}_q^h$ being uniformly random. $\mathcal{H}$ is $\mathbb{F}_q^n$ -hiding if every $H \in \mathcal{H}$ is $\mathbb{F}_q^n$ -hiding.*

Definition 3.3 (Circular Correlation Robust (CCR) Hash Function [GKWW20]). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $H^{\mathcal{O}} : \{0, 1\}^{\ell} \rightarrow \{0, 1\}^{\ell}$ be a hash function using these oracles. For $x, \Delta \in \{0, 1\}^{\ell}$ and $b \in \{0, 1\}$, let $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{CCR}}(x, b) = H^{\mathcal{O}}(x \oplus \Delta) \oplus b \cdot \Delta$ be the CCR oracle. $H^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, Q, \chi, \varepsilon)$ -CCR if for any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q queries to its second oracle (the CCR oracle $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{CCR}}$ or a random function f),*

$$\text{AdvCCR}_{\mathcal{A}}^{\mathcal{O}, H}(Q_{\mathcal{O}}, Q, \chi) = \left| \Pr_{\Delta \leftarrow \chi} [\mathcal{A}^{\mathcal{O}, \mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{CCR}}}() = 1] - \Pr_{f \leftarrow \mathcal{F}_{\ell+1, \ell}} [\mathcal{A}^{\mathcal{O}, f}() = 1] \right| \leq \varepsilon,$$

where $\mathcal{A}$ cannot query $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{CCR}}$ with both $(x, 0)$ and $(x, 1)$ for any $x \in \{0, 1\}^{\ell}$, χ is a distribution over $\{0, 1\}^{\ell}$, and $\mathcal{F}_{\ell+1, \ell} = \{f : \{0, 1\}^{\ell} \times \{0, 1\} \rightarrow \{0, 1\}^{\ell}\}$ is a function family.

Definition 3.4 (Tweakable Circular Correlation Robust Hash Function (TCCR)). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $H^{\mathcal{O}} : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^{\lambda}$ be a hash function using these oracles, $x, T, \Delta \in \{0, 1\}^{\lambda}$ and $b \in \{0, 1\}$, define the TCCR oracle $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{TCCR}}(x, T, b) \leftarrow H^{\mathcal{O}}(x \oplus \Delta, T) \oplus b \cdot \Delta$. $H^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, Q, \chi, \varepsilon)$ -TCCR if for any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q queries to its second oracle (the TCCR oracle $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{TCCR}}$ or a random function f), and every distribution χ over $\{0, 1\}^{\lambda}$ with min-entropy at least ρ , it holds that*

$$\begin{aligned} & \text{AdvTCCR}_{\mathcal{A}, H}(Q, \rho) \\ &= \left| \Pr_{\Delta \leftarrow \chi} [\mathcal{A}^{\mathcal{O}, \mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{TCCR}}}(1^{\lambda}) = 1] - \Pr_{f \leftarrow \mathcal{F}_{\lambda+1, \lambda}} [\mathcal{A}^f(1^{\lambda}) = 1] \right| \leq \text{negl}(\lambda), \end{aligned}$$

where for any $x, T \in \{0, 1\}^{\lambda}$, the adversary $\mathcal{A}$ is not allowed to query both $(x, T, 0)$ and $(x, T, 1)$.

The quantum analogs of CCR and TCCR are obtained by using a quantum adversary $\mathcal{A}$ in Definition 3.3 and granting $\mathcal{A}$ access to $|\mathcal{O}\rangle$, i.e., oracles $\mathcal{O}$ implemented by quantum circuits (but the construction oracles $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{CCR}}$, $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{TCCR}}$ and f still only appear in the non-quantum form).

Definition 3.5 (Natural and Non-trivial Query (NNQ) [GYW⁺23]). *Let $\mathcal{H} : \{0, 1\}^{\ell} \rightarrow \{0, 1\}^{\ell}$ be a hash function, $\mathcal{Q} : \{0, 1\}^{\ell} \times \{0, 1\} \rightarrow \{0, 1\}^{\ell}$ be an oracle, and $\mathcal{T} \subseteq \{0, 1\}^{\ell} \setminus \{0^{\ell}\}$ be a set of linearly independent strings. We say that a query q_i is a natural and non-trivial query if the output $x_i \in \{0, 1\}^{\ell}$ defined by that query satisfies the following:*

- **Naturality.** x_i is defined by one of the following types of computation:
 - Type 1: $x_i \leftarrow v$, where $v \in \{0^{\ell}\} \cup \mathcal{T}$.
 - Type 2: $x_i \leftarrow x_j \oplus x_k$, where x_j, x_k were defined by two previous distinct NNQs q_j, q_k .
 - Type 3: $x_i \leftarrow \mathcal{H}(x_j)$, where x_j was defined by a previous NNQ q_j .

– Type 4: $x_i \leftarrow \mathcal{Q}(x_j, b_j)$, where x_j was defined by a previous NNQ q_j and $b_j \in \{0, 1\}$.

- **Non-triviality.** x_i syntactically differs from any x_j defined by another NNQ q_j .

Let $\text{Real}_{\mathcal{H}_S, \Delta}^{\mathcal{O}}$ be the real-world oracle that, given an NNQ as input, implements $\mathcal{H} = \mathcal{H}_S^{\mathcal{O}} : x \mapsto \mathcal{H}^{\mathcal{O}}(x \oplus S)$ and $\mathcal{Q} = \mathcal{O}_{\mathcal{H}_S, \Delta}^{\mathcal{O}, \text{CCR}}$, executes the query according to the definition, and returns the result. Meanwhile, the ideal-world oracle $\text{Ideal}_{\mathcal{H}_S}^{\mathcal{O}}$ is identical to $\text{Real}_{\mathcal{H}_S, \Delta}^{\mathcal{O}}$ except that it replaces $\mathcal{Q}$ with an oracle that returns a fresh uniformly random string on each invocation.

The following lemma is an improved version of the original one of [GYW⁺23] by counting the ideal-model queries to capture the ideal-model-based constructions of CCR hash function. Its formal proof is provided in Section 9.4.2.

Lemma 3.1 (CCR for Natural and Non-trivial Queries). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $\mathcal{H}^{\mathcal{O}} : \{0, 1\}^{\ell} \rightarrow \{0, 1\}^{\ell}$ be a $(Q'_{\mathcal{O}}, Q, \chi, \varepsilon)$ -CCR hash function using these oracles, where $Q'_{\mathcal{O}} = Q_{\mathcal{O}} + (Q, \dots, Q)$ corresponds to Q more queries for each oracle in $\mathcal{O}$. For any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q NNQs to its second oracle (the real-world oracle $\text{Real}_{\mathcal{H}_S, \Delta}^{\mathcal{O}}$ or the ideal-world oracle $\text{Ideal}_{\mathcal{H}_S}^{\mathcal{O}}$), where each NNQ only depends on the results of the previous NNQs (with the initial NNQ only depending on an empty result),*

$$\left| \Pr_{\Delta, S \leftarrow \mathcal{S}_{\chi}} \left[\mathcal{A}^{\mathcal{O}, \text{Real}_{\mathcal{H}_S, \Delta}^{\mathcal{O}}}() = 1 \right] - \Pr_{S \leftarrow \mathcal{S}_{\chi, c} \leftarrow \mathcal{U}} \left[\mathcal{A}^{\mathcal{O}, \text{Ideal}_{\mathcal{H}_S}^{\mathcal{O}}}() = 1 \right] \right| \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, \mathcal{H}}(Q'_{\mathcal{O}}, Q, \chi) + \frac{Q^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}},$$

where $\mathcal{U}$ is the uniform distribution of some proper unspecified length, and c is the random coin used for Type 4 NNQs in $\text{Ideal}_{\mathcal{H}_S}^{\mathcal{O}}$.

3.3 Pseudorandom Generator

In our signature scheme, we use an IV-based tweakable PRG, which ensures pseudorandomness given a secret seed. For the soundness of an interactive proof system underlying the signature scheme, we also require the PRG to be almost injective.

Definition 3.6 (IV-based Tweakable PRG). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $G^{\mathcal{O}} : \mathcal{X} \times \mathcal{I} \times \mathcal{T} \rightarrow \mathcal{Y}$ be a PRG using these oracles. $G^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, \varepsilon)$ -secure if for any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and any $(\text{iv}, t) \in \mathcal{I} \times \mathcal{T}$,*

$$\text{AdvPRG}_{\mathcal{A}}^{\mathcal{O}, G}(Q_{\mathcal{O}}) = \left| \Pr_{x \leftarrow \mathcal{S}_{\chi}} \left[\mathcal{A}^{\mathcal{O}}(G^{\mathcal{O}}(x, \text{iv}, t)) = 1 \right] - \Pr_{y \leftarrow \mathcal{S}_{\mathcal{Y}}} \left[\mathcal{A}^{\mathcal{O}}(y) = 1 \right] \right| \leq \varepsilon.$$

Definition 3.7 (Almost Injective PRG [BBB⁺25]). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $G^{\mathcal{O}} : \mathcal{X} \times \mathcal{I} \times \mathcal{T} \rightarrow \mathcal{Y}$ be a PRG using these oracles. $G^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, L, \varepsilon)$ -almost-injective if for any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$,*

$$\text{AdvInj}_{\mathcal{A}}^{\mathcal{O}, G}(Q_{\mathcal{O}}, L) = \Pr \left[\text{GameInj}_{\mathcal{A}}^{\mathcal{O}, G}(Q_{\mathcal{O}}, L) = 1 \right] \leq \varepsilon,$$

where the almost injectivity game $\text{GameInj}_{\mathcal{A}}^{\mathcal{O}, G}$ is defined in Fig. 3.

Game $\text{GameInj}_{\mathcal{A}}^{\mathcal{O},G}(Q_{\mathcal{O}}, L)$	
1 :	$(\text{iv}, \{(t_i, y_i)\}_{i \in [L]}) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}}()$
2 :	if $\exists i \in [L], x \neq x' \in \mathcal{X} : G^{\mathcal{O}}(x, \text{iv}, t_i) = G^{\mathcal{O}}(x', \text{iv}, t_i) = y_i :$
3 :	return 1
4 :	else
5 :	return 0

Figure 3: The almost injectivity game.

3.4 Zero-Knowledge Proof

A zero-knowledge proof (ZK) is a two-party protocol. In this document, we only consider zero-knowledge proofs for languages in the complexity class NP. In this setting, the goal of ZK is for the prover to convince the verifier that a public string x belongs to an NP language $\mathcal{L}$. Since $\mathcal{L}$ is an NP language, there is an efficient algorithm TM such that the following holds: if $x \in \mathcal{L}$, then there exists a *witness* vector w with $\text{TM}(x, w) = 1$; conversely, if $x \notin \mathcal{L}$, then $\text{TM}(x, w) = 0$ for all w . For the Lynxer algorithm, the NP language of interest consists of instances (H, y) where $y = H \cdot e$ for a regular noise vector e . Because we use the systematic form $H = [I \mid H_B]$, the actual committed witness is the truncated blockwise representation of e_B , from which the prover and verifier can reconstruct the full regular witness e .

We require the interactive protocol to satisfy the following properties³:

- **Completeness:** If $x \in \mathcal{L}$, then the verifier should accept the proof.
- **Soundness:** If $x \notin \mathcal{L}$, then the verifier should reject with high probability.
- **Knowledge soundness:** If the verifier accepts the proof, then we can extract w by interacting with the prover. (This property is a strengthening of soundness.)
- **Zero-knowledge:** If $x \in \mathcal{L}$, then the proof should not leak the witness w .

In 1987, Goldreich, Micali, and Wigderson proved that every NP language admits a ZK protocol [GMW87]. Through ZK, we can construct *identification protocols*, in which the prover demonstrates to the verifier that it possesses the witness w corresponding to public information x . In this process, the knowledge soundness of ZK guarantees that if the verifier accepts the proof, then the prover must hold the corresponding witness; meanwhile, the zero-knowledge property ensures that the protocol execution does not leak any information about w .

Proof system. We formalize the properties of an interactive proof (IP) system to be used in our security proofs.

Definition 3.8 (Soundness). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $\Pi^{\mathcal{O}} = (\mathcal{P}^{\mathcal{O}}, \mathcal{V}^{\mathcal{O}})$ be a proof system for an NP relation $\mathcal{R}$ and its language $\mathcal{L}_{\mathcal{R}}$ using these oracles. $\Pi^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, \varepsilon)$ -sound if for any (possibly unbounded) adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$,*

$$\text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}, \Pi}(Q_{\mathcal{O}}) = \Pr \left[\mathbb{x} \leftarrow_{\$} \mathcal{A}^{\mathcal{O}}(), b \leftarrow_{\$} \langle \mathcal{A}^{\mathcal{O}}, \mathcal{V}^{\mathcal{O}} \rangle(\mathbb{x}) : \mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge b = 1 \right] \leq \varepsilon,$$

where $\langle \mathcal{A}^{\mathcal{O}}, \mathcal{V}^{\mathcal{O}} \rangle(\mathbb{x}) \in \{0, 1\}$ returns 1 if and only if the execution for the instance $\mathbb{x}$ between the prover $\mathcal{A}$ and the verifier $\mathcal{V}$ makes the verifier output accept.

³As a core concept of modern cryptography, the rigorous definition of zero knowledge can be found in Goldreich's textbook [Gol01, Chapter 4]. Since our focus is on the security of the signature scheme, we only provide a brief introduction here.

Definition 3.9 (Interactive Proof System). For an NP relation $\mathcal{R}$ with its language $\mathcal{L}_{\mathcal{R}}$ and $k \in \mathbb{N}^+$, a k -round IP system $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$ is defined as follows:

- The prover $\mathcal{P}_{\text{IP}}$ takes an instance $\mathbf{x}$ and a witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ as its initial input, while the verifier $\mathcal{V}_{\text{IP}}$ only takes $\mathbf{x}$ as its initial input.
- In the i -th round, where $i \in [k]$, the prover $\mathcal{P}_{\text{IP}}$ sends a message pmsg_i to the receiver $\mathcal{V}_{\text{IP}}$, who sends back a challenge chall_i to the prover $\mathcal{P}_{\text{IP}}$.
- Upon receiving the last message pmsg_k from the prover $\mathcal{P}_{\text{IP}}$, the verifier $\mathcal{V}_{\text{IP}}$ outputs

$$\{\text{reject}, \text{accept}\} \leftarrow \mathcal{V}_{\text{IP}}(\mathbf{x}, \{(\text{pmsg}_i, \text{chall}_i)\}_{i \in [k]}, \text{pmsg}_k).$$

A transcript tr of IP is an ordered list of messages and has one of the following cases:

- $i \in [k] \cup \{k\}$ and the prover moves next: $\text{tr}_{2i} = (\text{pmsg}_0, \text{chall}_0, \dots, \text{pmsg}_{i-1}, \text{chall}_{i-1})$. As a special case, $\text{tr}_0 = \emptyset$ for $i = 0$ is an empty transcript.
- $i \in [k] \cup \{k\}$ and the verifier moves next: $\text{tr}_{2i+1} = (\text{pmsg}_0, \text{chall}_0, \dots, \text{pmsg}_i)$. As a special case, $\text{tr}_{2k+1} = (\text{pmsg}_0, \text{chall}_0, \dots, \text{pmsg}_{k-1}, \text{chall}_{k-1}, \text{pmsg}_k)$ is a full transcript.

Definition 3.10 (Round-by-Round Soundness [CCH⁺19]). Let $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$ be a k -round IP system for some NP relation $\mathcal{R}$ with a language $\mathcal{L}_{\mathcal{R}}$. IP is ε -RBR-sound if there exists a deterministic (not necessarily polynomial-time) algorithm state, which takes an instance $\mathbf{x}$ and a transcript as input and outputs reject or accept, such that

- If $\mathbf{x} \notin \mathcal{L}_{\mathcal{R}}$, then $\text{state}(\mathbf{x}, \emptyset) = \text{reject}$ for the empty transcript.
- In the i -th round, where $i \in [k]$, for any transcript tr_{2i} and prover's message pmsg_i , we have

$$\Pr_{\text{chall}_i} [\text{state}(\mathbf{x}, \text{tr}_{2i} \parallel \text{pmsg}_i \parallel \text{chall}_i) = \text{accept} \mid \text{state}(\mathbf{x}, \text{tr}_{2i}) = \text{reject}] \leq \varepsilon_i \leq \varepsilon,$$

where the probability is taken over the uniform chall_i .

- For any full transcript tr_{2k+1} , if $\text{state}(\mathbf{x}, \text{tr}_{2k+1}) = \text{reject}$, then $\mathcal{V}_{\text{IP}}(\mathbf{x}, \text{tr}_{2k+1}) = \text{reject}$.

3.5 Information-Theoretic Message Authentication Code

We adopt the generalized Information-Theoretic Message Authentication Code (IT-MAC) definition of Agarwal et al. [ABBS25]. The original presentation in [ABBS25] supports general modulus, whereas Lynxer only needs modulus 2. In Definition 3.11, we simplify the notation accordingly.

Definition 3.11. For $d \in \mathbb{Z}$, a degree- d IT-MAC for $a \in \mathbb{F}_{2^\lambda}$ is defined as $\llbracket a \rrbracket^d := (p_a(x), k_a, \Delta)$, where a prover Prv holds a polynomial $p_a(x) \in \mathbb{F}_{2^\lambda}[x]$ with $\deg(p_a) = d$ and leading coefficient a ; while a verifier Ver holds the evaluation $k_a = p_a(\Delta)$, for a global key $\Delta \in \mathbb{F}_{2^\lambda}$ unknown to Prv . The degree superscript d is omitted when clear from context.

Let $\llbracket a \rrbracket^{d_a}$ and $\llbracket b \rrbracket^{d_b}$ be two IT-MACs, with corresponding polynomials $p_a(x), p_b(x)$ and evaluations k_a, k_b , and assume w.l.o.g. that $d_a \geq d_b$. IT-MAC supports the following local homomorphic operations:

- **Addition.** To compute $\llbracket a + b \rrbracket^{d_a} = \llbracket a \rrbracket^{d_a} + \llbracket b \rrbracket^{d_b}$, Prv computes $p_{a+b}(x) = p_a(x) + p_b(x) \cdot x^{d_a-d_b}$, while Ver computes $k_{a+b} = k_a + k_b \cdot \Delta^{d_a-d_b}$.
- **Multiplication.** To compute $\llbracket ab \rrbracket^{d_a \cdot d_b} = \llbracket a \rrbracket^{d_a} \cdot \llbracket b \rrbracket^{d_b}$, Prv computes $p_{ab}(x) = p_a(x) \cdot p_b(x)$, while Ver computes $k_{ab} = k_a \cdot k_b$.

- **Constants.** For any $c \in \mathbb{F}_{2^\lambda}$ and $d \in \mathbb{Z}$, define $\llbracket c \rrbracket^d$ by $p_c(x) = cx^d$ and $k_c = c\Delta^d$. Combining this with the above operations, constant addition and scalar multiplication preserve degree. Subtraction follows as $\llbracket a \rrbracket - \llbracket b \rrbracket := \llbracket a \rrbracket + \llbracket -1 \rrbracket^1 \cdot \llbracket b \rrbracket$. Moreover, to de-randomize a random IT-MAC $\llbracket r \rrbracket^1$ (where $r \leftarrow \mathbb{F}_2$) into a target value $x \in \mathbb{F}_2$, Prv publishes $\delta := x - r$, and both parties compute $\llbracket x \rrbracket^1 := \llbracket r \rrbracket^1 + \delta$.

IT-MAC satisfies the following hiding and binding properties:

- **Hiding.** Random degree-1 IT-MACs are typically generated from VOLE protocols, which inherently ensure that the committed value a is independent of Ver's authentication material. Thus, hiding is guaranteed.
- **Binding.** To open a committed value a , Prv sends its (purported) associated polynomial $\tilde{p}_{a'}(x)$ to Ver, who checks whether $\tilde{p}_{a'}(\Delta) = k_a$. Any attempt by Prv to open a different value $a' \neq a$ while passing this check implies $\tilde{p}_{a'}(\Delta) = p_a(\Delta)$. Assuming $\Delta \leftarrow \mathbb{F}_{2^\lambda}$, the probability of such a collision is at most $\frac{\deg(p_a)}{2^\lambda}$.

3.6 Digital Signature Scheme

Definition 3.12 (Digital Signature Scheme). A digital signature scheme $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ consists of three algorithms:

- $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$: A probabilistic key generation algorithm that takes as input the security parameter 1^λ and outputs a secret key sk and a public key pk .
- $\sigma \leftarrow \text{Sign}(\text{sk}, \text{msg})$: A (possibly probabilistic) signing algorithm that takes as input the secret key sk and a message $\text{msg} \in \{0, 1\}^*$, and outputs a signature σ .
- $b \leftarrow \text{Verify}(\text{pk}, \text{msg}, \sigma)$: A deterministic verification algorithm that takes as input the public key pk , a message msg , and a signature σ , and outputs $b \in \{\text{accept}, \text{reject}\}$.

We say Σ is correct if for all $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$ and all $\text{msg} \in \{0, 1\}^*$:

$$\Pr[\text{Verify}(\text{pk}, \text{msg}, \text{Sign}(\text{sk}, \text{msg})) = \text{accept}] = 1. \quad (3)$$

We consider two standard security notions of a signature scheme: existential unforgeability against key-only attacks (EUF-KO) and existential unforgeability under chosen message attacks (EUF-CMA).

Definition 3.13 (EUF-KO and EUF-CMA Security). Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $\Sigma^{\mathcal{O}} = (\text{KeyGen}^{\mathcal{O}}, \text{Sign}^{\mathcal{O}}, \text{Verify}^{\mathcal{O}})$ be a signature scheme using these oracles. $\Sigma^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, \varepsilon)$ -EUF-KO-secure if for any stateful adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$,

$$\text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}) = \Pr \left[\text{GameEUFKO}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}) = 1 \right] \leq \varepsilon,$$

where the EUF-KO game $\text{GameEUFKO}_{\mathcal{A}}^{\mathcal{O}, \Sigma}$ is defined in Fig. 4. Meanwhile, $\Sigma^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, Q_{\text{sig}}, \varepsilon)$ -EUF-CMA-secure if for any stateful adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q_{sig} queries to the signature oracle OSig_{sk} ,

$$\text{AdvEUF CMA}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}, Q_{\text{sig}}) = \Pr \left[\text{GameEUF CMA}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] \leq \varepsilon,$$

where the EUF-CMA game $\text{GameEUF CMA}_{\mathcal{A}}^{\mathcal{O}, \Sigma}$ is defined in Fig. 4.

Game $\text{GameEUFKO}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}})$	
1 :	$\mathcal{A}^{\mathcal{O}}()$
2 :	$(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}^{\mathcal{O}}()$
3 :	$(\text{msg}, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pk})$
4 :	return $\text{Verify}^{\mathcal{O}}(\text{pk}, \text{msg}, \sigma) = \text{accept}$
Game $\text{GameEUF CMA}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}, Q_{\text{sig}})$	$\text{OSig}_{\text{sk}}(\text{msg})$
1 :	$\mathcal{A}^{\mathcal{O}}()$
2 :	$(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}^{\mathcal{O}}()$
3 :	$M \leftarrow \emptyset$
4 :	$(\text{msg}, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}, \text{OSig}_{\text{sk}}}(\text{pk})$
5 :	return $\text{Verify}^{\mathcal{O}}(\text{pk}, \text{msg}, \sigma) = \text{accept} \wedge \text{msg} \notin M$
	1 : $M \leftarrow M \cup \{\text{msg}\}$
	2 : return $\text{Sign}^{\mathcal{O}}(\text{sk}, \text{msg})$

Figure 4: The EUF-KO and EUF-CMA games, where the blue parts are different.

3.7 Signature Size of VOLEitH

We recall the signature size formula in Eq.(4), as it critically informs the parameter selection process. Some messages of the VOLEitH framework can be reconstructed by the verifier locally, so after the Fiat-Shamir transform we can omit these from the protocol transcript, saving communication. The meaning of the parameters in Eq.(4) is specified in Table 4. We fix the BAVC leaf-node commitment size to 2λ bits [BBB⁺26]. Also, as the design of Lynx ensures that verifying the validity of an extended witness (which is three $\mathbb{F}_{2^\lambda}$ elements) only requires quadratic constraints, we have $d = 2$ in Lynxer.

$$\begin{aligned}
\text{Size} = & \underbrace{\ell_{\text{wit}}}_{\text{Fixing witness}} + \underbrace{(\tau - 1)(\ell_{\text{wit}} + d\lambda + \text{B})}_{c_1, \dots, c_{\tau-1}} + \underbrace{(T_{\text{open}} + 2\tau)\lambda}_{\text{BAVC Opening}} \\
& + \underbrace{(d-1)\lambda}_{\text{QuickSilver}} + \underbrace{\lambda + \text{B}}_{\text{SoftSpoken}} + \underbrace{\lambda}_{\text{chall}_3} + \underbrace{120}_{\text{iv}_{\text{sig}}} + \underbrace{32}_{\text{Counter}}
\end{aligned} \tag{4}$$

4 Lynx: VOLEitH-Friendly Primitive

Lynx is an iv-based symmetric primitive over $\mathbb{F}_{2^\lambda}$. It takes a secret key k and an initial vector iv_{owf} as inputs, and outputs $t \in \mathbb{F}_{2^\lambda}$. Lynx provides $\lambda = 160$ -, 256-, 384-, 512-bit security against preimage recovery on k and pseudorandomness distinguishing on t . The construction combines (i) nonlinear components of power-function-defined S-boxes S_i (including inverse function) for $i = 1, 2, 3$ and field multiplications as truncation function, with (ii) linear components of random constant additions c_i for $i = 1, 2, 3$ and binary matrix multiplications L_i for $i = 0, 1, 2, 3$. The schematic of Lynx is shown in Figure 1, where the dashed branch is included exclusively in Lynx-256, Lynx-384, Lynx-512 versions. Let v_i be the outputs of S_i , for $i = 1, 2$. Lynx is defined as

$$v_1 = S_1(k + c_1), \tag{5}$$

$$v_2 = S_2(L_0(k) + c_2), \tag{6}$$

$$t = \delta_\lambda(L_1(v_1) \cdot L_2(v_2), (k + c_3)^{-1}) + L_3(k + v_1 + v_2), \tag{7}$$

where $\delta_\lambda : \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda} \rightarrow \mathbb{F}_{2^\lambda}$ is

$$\delta_\lambda(x, y) = \begin{cases} x, & \text{if } \lambda \in \{160\}, \\ x \cdot y, & \text{if } \lambda \in \{256, 384, 512\}. \end{cases}$$

Table 2: Nonlinear Components and Cryptographic Properties

Nonlinear component over $\mathbb{F}_{2^\lambda}$, λ is even.		Verification			Security metrics			
		Deg.	Mult.	F.T.	$\deg_2$	#LIQE	Diff.Unif.	cor
inverse/power	$d = -1$	2	1	0	$\lambda - 1$	5λ	$2^{2-\lambda}$	$2^{1-\lambda/2}$
perm. $f(x) = x^d$, d and $2^\lambda - 1$ are coprime.	$d = 2^i - 1$	2	1	1	i	3λ	$\leq \frac{d-1}{2^\lambda}$	$\leq \frac{d-1}{2^{\lambda/2}}$
	$d = (2^i - 1)^{-1} \bmod 2^\lambda - 1$	2	1	1	$\text{wt}_2(d)$	3λ		
	$d = 2^{2i} - 2^i + 1$ $d = (2^{2i} - 2^i + 1)^{-1} \bmod 2^\lambda - 1$	2	2	2	$\frac{i+1}{2}$ $\frac{\lambda}{2} + 1$	λ		
Mult. over $\mathbb{F}_{2^\lambda}$	$f(x, y) = x \cdot y$	2	1	0	2	3λ	$2^{-\lambda}$	$2^{-\lambda}$
	$f(x, y, z) = x \cdot y \cdot z^{-1}$	2	2	0	$\geq \lambda - 1$	λ	$2^{1-\lambda} - 2^{-2\lambda}$	$2^{-\lambda} - 2^{-2\lambda}$

† For the constraints, Deg. denotes the degree of corresponding constraint polynomial, while Mult. and F.A. denote the number of finite-field multiplications and Frobenius automorphisms (realized by binary matrix multiplications) in the constraint polynomial, respectively.

† For the security metrics, $\deg_2$, Diff. Unif. and cor denote the algebraic degree over $\mathbb{F}_2$, differential uniformity and correlation of the nonlinear component, respectively; #LIQE denotes the number of linearly independent quadratic Boolean equations derived from its constraint polynomial.

4.1 Nonlinear Components

Throughout this paper, the inverse function over $\mathbb{F}_{2^\lambda}$ is denoted by $f(x) = x^{-1}$, with the convention that

$$f(x) = x^{2^\lambda-2} = \begin{cases} x^{-1}, & \text{if } x \neq 0, \\ 0, & \text{if } x = 0. \end{cases}$$

The nonlinear permutations are defined as $S_i(x) = x^{d_i}$ for $i = 1, 2$, and $S_3(x) = x^{-1}$. A requirement for Lynx is that the key be randomly selected to ensure no S-box receives a zero input. To support configurable security levels while maintaining a unified structure and compact signature size, Lynx systematically diversifies the nonlinear components. Their constraint polynomials and cryptanalysis resistance are summarized in Table 2, with full details provided in Section B. Recommended parameters for 160-, 256-, 384-, and 512-bit security levels are given in Table 3.

4.2 Linear Components

Every $\lambda \times \lambda$ binary matrix multiplication L can be represented as a linearized polynomial over $\mathbb{F}_{2^\lambda}$:

$$L(x) = \sum_{j=0}^{\lambda-1} a_j x^{2^j}, \quad a_j \in \mathbb{F}_{2^\lambda}.$$

The L_i for $i = 0, 1, 2$ and c_i for $i = 1, 2, 3$ are generated by an XOOF seeded with iv_{owf} . L_3 is a fixed invertible matrix⁴ whose characteristic polynomial $\det(xI - L_3) \in \mathbb{F}_2[x]$ is irreducible.

4.3 Concrete Instances

The irreducible polynomials for constructing extension fields are as follows, which are widely adopted in cryptographic applications—specifically recommended by NIST and used in OpenSSL:

$$- \mathbb{F}_{2^{160}} : X^{160} + X^5 + X^3 + X^2 + 1$$

⁴The specific L_3 is given in the implementation code. For a randomly sampled L , the invertibility is guaranteed by standard generation methods—e.g., rejection sampling or LU decomposition—and the density and maximum degree hold with overwhelming probability. $c_i \in \mathbb{F}_{2^\lambda}$ are also pairwise distinct with overwhelming probability.

Table 3: The d_i for S_i , $i = 1, 2$ in Lynx instantiation

scheme	d_1	d_2
Lynx-160	-1	$2^{67} - 1$
Lynx-256	$(2^5 - 1)^{-1} \bmod 2^{256} - 1$	$2^{53} - 1$
Lynx-384	$(2^{13} - 1)^{-1} \bmod 2^{384} - 1$	$2^{60} - 2^{30} + 1$
Lynx-512	$(2^{12} - 2^6 + 1)^{-1} \bmod 2^{512} - 1$	$2^{180} - 2^{90} + 1$

- $\mathbb{F}_{2^{256}} : X^{256} + X^{10} + X^5 + X^2 + 1$
- $\mathbb{F}_{2^{384}} : X^{384} + X^{16} + X^{15} + X^6 + 1$
- $\mathbb{F}_{2^{512}} : X^{512} + X^8 + X^5 + X^2 + 1$

5 Specification of Symmetric-based Signature Scheme Lynxer

In this section, we specify the Lynxer post-quantum digital signature scheme. We present the algorithms in a top-down manner by first defining the top-level APIs and then specifying their building blocks. The principal component of Lynxer is an efficient zero-knowledge proof of the Lynx one-way function in the VOLEitH framework. Lynxer consists of the following components.

- [Section 5.2](#) describes the top-level APIs, including the [Lynxer.KeyGen](#), [Lynxer.Sign](#), and [Lynxer.Verify](#) algorithms. They offer the overall signature functionality.
- [Section 5.3](#) describes algorithms related to vector oblivious linear evaluation relations, including [VOLE.Commit](#), [VOLE.Reconstruct](#), [VOLE.ConvertToVOLE](#), [VOLE.Fix](#), [VOLE.ChalDec](#), and [VOLE.ChalDecAll](#). They serve to generate IT-MACs (i.e., VOLE correlations) as well as providing a de-randomization process of IT-MACs.
- [Section 5.4](#) describes Batch All-but-one Vector Commitment, including the [BAVC.Commit](#), [BAVC.Open](#), [BAVC.Verify](#), [BAVC.Interleave](#), [BAVC.Deinterleave](#), [BAVC.MinSpan](#), and [LeafHash](#) algorithms. They provide the vector commitment functionality for use of the VOLE component.
- [Section 5.5](#) describes procedures related to the Lynx one-way function, including [Lynx.SampleMatVec](#), [Lynx.ExtendWitness](#), and [Lynx.EvalOutput](#). They provide public-parameter sampling from iv_{owf} , plaintext evaluation of the Lynx one-way function, and the witness extension process used by the QuickSilver proof system.
- [Section 5.6](#) describes the QuickSilver proof system, including the [QS.LynxProve](#) and [QS.LynxVerify](#) algorithms. They utilize the QuickSilver proof system and the linear sketching technique to prove that the IT-MAC-committed Lynx witness satisfies the required OWF constraints.
- [Section 5.7](#) describes linear universal hash functions, including the [VOLEHash](#) and [ZKHash](#) algorithms. They are two linear universal hash functions that correspond to the SoftSpoken VOLE check and QuickSilver batch quadratic relation check, respectively.
- [Section 5.8](#) describes utility functions, including the [BitsToInt](#), [IntToBits](#), [FieldToBits](#), and [BitsToField](#) algorithms. They offer data-representation conversion utilities.
- [Section 5.9](#) describes symmetric primitives, including the fundamental symmetric-key algorithms [Enc](#) and [XOF](#), as well as the pseudorandom generator [PRG](#), the tweakable circular correlation robust hash function [TCCR](#), and the domain-separated hash functions [MsgHash](#), [CoinHash](#), [BAVCHash](#), [MatVecHash](#), [FSHash₁](#), [FSHash₂](#), and [FSHash₃](#).

We begin with a detailed specification of the parameter structure in Lynxer, followed by the pseudocode of the eight components, and conclude with selected parameter sets of Lynxer. Unless stated otherwise, vectors and matrices that participate in linear-algebra operations are written over $\mathbb{F}_2$. Bitstrings that are sampled uniformly or returned by [XOF](#) or [PRG](#) are written over $\{0, 1\}$. When an object over $\mathbb{F}_2$ is used as input to a hash function or serialized into a public key or signature, we identify $\mathbb{F}_2 = \{0, 1\}$ and serialize its coordinates in the natural order.

5.1 Parameter Structure

We define the parameters of Lynxer in [Table 4](#). For $\text{param}_{\text{BAVC}}$, we have $k_1 = k_0 + 1$, and for $\alpha \in [0, \tau)$, if $\alpha < t_1$, then $n_\alpha = k_1$ and $N_\alpha = 2^{k_1}$; otherwise, $n_\alpha = k_0$ and $N_\alpha = 2^{k_0}$. The total number of leaves in the BAVC tree is defined to be $L = \sum_{\alpha \in [0, \tau)} N_\alpha$. For $\text{param}_{\text{VOLE}}$, we fix $B = 16$ following the choice of [FAEST](#). The design of Lynx ensures that $\ell_{\text{wit}} = 3\lambda$ for all Lynxer instances.

Table 4: Parameter meanings

Group	Parameter	Description
param	λ	Security parameter
	$\text{param}_{\text{OWF}}$	OWF parameter structure
	$\text{param}_{\text{BAVC}}$	BAVC parameter structure
	$\text{param}_{\text{VOLE}}$	VOLE parameter structure
$\text{param}_{\text{BAVC}}$	τ	Number of VOLE instances
	k_0	Dimension of the first small-field type
	t_0	Number of instances of the first type
	k_1	Dimension of the second small-field type
	t_1	Number of instances of the second type
	$\{N_\alpha\}_{\alpha \in [0, \tau)}$	Number of leaves in each VOLE instance's binary tree
	$\{n_\alpha\}_{\alpha \in [0, \tau)}$	Dimension of each VOLE instance
	L	Total number of leaves in the BAVC tree
$\text{param}_{\text{OWF}}$	T_{open}	Threshold for the number of opened BAVC nodes
	$\mathbf{L}_3$	The fixed linear transform matrix in Lynx
	$\mathbf{F}_n$	The fixed matrix that computes the $x \mapsto x^{2^n}$ Frobenius map for $x \in \mathbb{F}_{2^\lambda}$
$\text{param}_{\text{VOLE}}$	ℓ_{wit}	Encoded length of the one-way function input
	ℓ_{vole}	VOLE commitment length
	B	Security redundancy constant for the universal hash function, fixed to be 16
	w_{grind}	Proof-of-work level

5.2 Top-Level APIs

This subsection presents the public interface of Lynxer. It contains the following algorithms.

- [Lynxer.KeyGen](#): generates a public Lynx instance and the corresponding signing secret.
- [Lynxer.Sign](#): produces a Fiat-Shamir signature proving knowledge of a valid Lynx witness.
- [Lynxer.Verify](#): reconstructs the public transcript and checks the signature.

5.2.1 Key Generation

The key-generation algorithm takes a parameter set param as input, which fixes the security parameter λ , the VOLEitH parameters, and the Lynx OWF parameters. It repeatedly samples

a public seed iv_{owf} and an OWF input k until the sampled instance avoids the zero inputs to the inversion boxes used by Lynx. The sampling of k is otherwise uniform over $\mathbb{F}_{2^\lambda}$. The public seed determines the sampled linear layers and constants through [Lynx.SampleMatVec](#). The sampled secret k is extended to (k, v_1, v_2) by [Lynx.ExtendWitness](#), and the public OWF output t is computed by [Lynx.EvalOutput](#). The public key is $\text{pk} = (\text{iv}_{\text{owf}}, t)$, while the secret key is $\text{sk} = (\text{iv}_{\text{owf}}, k)$.

[Lynxer.KeyGen](#)(param)

Input: (param, param_{OWF})

Output: $\left(\begin{array}{l} \text{pk} := (\text{iv}_{\text{owf}} \in \{0, 1\}^\lambda, t \in \mathbb{F}_{2^\lambda}) \\ \text{sk} := (\text{iv}_{\text{owf}} \in \{0, 1\}^\lambda, k \in \mathbb{F}_{2^\lambda}) \end{array} \right)$

```

1: Extract  $\lambda$  from param
2: while  $\top$  do
3:    $\text{iv}_{\text{owf}} \leftarrow \{0, 1\}^\lambda$ 
4:    $k \leftarrow \mathbb{F}_{2^\lambda}$ 
5:    $(L_0, L_1, L_2, c_1, c_2[, c_3]) := \text{Lynx.SampleMatVec}(\text{iv}_{\text{owf}}; \text{param})$   $\triangleright c_3$  is needed only for
      $\lambda \in \{256, 384, 512\}$ 
6:    $(v_1, v_2) := \text{Lynx.ExtendWitness}(L_0, L_1, L_2, c_1, c_2[, c_3], k; \text{param})$ 
7:   if  $\lambda = 160 \wedge v_1 = 0$  then
8:     goto Step 3
9:   end if
10:  if  $\lambda \in \{256, 384, 512\} \wedge k = c_3$  then
11:    goto Step 3
12:  end if
13:   $t := \text{Lynx.EvalOutput}(L_0, L_1, L_2, L_3, c_1, c_2[, c_3], k, v_1, v_2; \text{param})$ 
14:   $\text{pk} := (\text{iv}_{\text{owf}}, t)$ 
15:   $\text{sk} := (\text{iv}_{\text{owf}}, k)$ 
16:  break
17: end while
18: return (pk, sk)

```

5.2.2 Signing

The signing algorithm takes a secret key $\text{sk} = (\text{iv}_{\text{owf}}, k)$ and a message msg . It reconstructs the public Lynx parameters from iv_{owf} , extends the secret OWF input to (k, v_1, v_2) , and recomputes the public value t . This reconstructs the public key $\text{pk} = (\text{iv}_{\text{owf}}, t)$, which is hashed together with the message by [MsgHash](#) to obtain the digest μ and the salt s used by [VOLE.Commit](#).

The signing procedure then runs in four phases, following the VOLEitH and QuickSilver transcript structure. In the first phase, the signer samples signature randomness ρ (or sets $\rho = 0^\lambda$ in deterministic signing) and derives the BAVC root seed r and IV iv_{sig} using [CoinHash](#). It calls [VOLE.Commit](#) to create the BAVC commitment com , the decommitment state decom , correction vectors $(c_i)_{i \in [1, \tau]}$, and prover-side VOLE values (u, V) . The first Fiat-Shamir challenge chall_1 is then derived from the commitment and correction vectors.

In the second phase, the signer performs the SoftSpokenOT-style VOLE consistency check and commits to the Lynx witness. In particular, the signer use a linear universal hash function [VOLEHash](#) to compress the generated IT-MACs $[\![u]\!]^1$ into $[\![\tilde{u}]\!]^1$ (algorithmically, the signer hashes u and its matrix-form tag V with [VOLEHash](#)), encodes the extended witness as $w := \text{FieldToBits}(k; \lambda) \parallel \text{FieldToBits}(v_1; \lambda) \parallel \text{FieldToBits}(v_2; \lambda)$, and masks the extended witness as $d := w \oplus u[0, \ell_{\text{wit}}]$. The second challenge chall_2 is obtained by hashing chall_1 , the opening of $[\![\tilde{u}]\!]^1$ (namely, $\tilde{u}$ and $\tilde{V}$), and the masked witness.

In the third phase, the signer restricts $\mathbf{u}$ and $\mathbf{V}$ to the rows needed by the QuickSilver check and invokes [QS.LynxProve](#). This produces $(\tilde{a}_1, \tilde{a}_0)$, which are the batched QuickSilver values proving that the committed witness satisfies the public Lynx OWF constraints.

In the final phase, the signer derives the last challenge chall_3 by hashing the QuickSilver response and a counter ctr . The counter is increased until two conditions hold: the last w_{grind} bits of chall_3 are zero, and the BAVC opening for the decoded challenge indices fits within the threshold T_{open} . The signature consists of the correction vectors, the compressed VOLE value $\tilde{\mathbf{u}}$, the masked witness $\mathbf{d}$, the QuickSilver value $\tilde{a}_1$, the partial BAVC opening decom_I , the final challenge, the IV, and the counter.

[Lynxer.Sign](#)(sk, msg; param)

Input: $\left(\begin{array}{l} \text{sk} := (\text{iv}_{\text{owf}} \in \{0, 1\}^\lambda, k \in \mathbb{F}_{2^\lambda}), \\ \text{msg} \in \{0, 1\}^*; \text{param} \end{array} \right)$

Output: $\sigma := \left(\begin{array}{l} (\mathbf{c}_i)_{i \in [1, \tau]} \in (\mathbb{F}_2^{\ell_{\text{vole}}})^{\tau-1}, \tilde{\mathbf{u}} \in \{0, 1\}^{\lambda+B}, \mathbf{d} \in \mathbb{F}_2^{\ell_{\text{wit}}}, \\ \tilde{a}_1 \in \{0, 1\}^\lambda, \text{decom}_I \in \{0, 1\}^{(T_{\text{open}}+2\tau) \cdot \lambda}, \\ \text{chall}_3 \in \{0, 1\}^\lambda, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}, \text{ctr} \in \{0, 1\}^{32} \end{array} \right)$

- 1: Extract $\lambda, \ell_{\text{wit}}, \tau, B, T_{\text{open}}$ from **param**
- 2: $(\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, c_1, c_2[, c_3]) := \text{Lynx.SampleMatVec}(\text{iv}_{\text{owf}}; \text{param})$
- 3: $(v_1, v_2) := \text{Lynx.ExtendWitness}(\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, c_1, c_2[, c_3], k; \text{param})$
- 4: $t := \text{Lynx.EvalOutput}(\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3, c_1, c_2[, c_3], k, v_1, v_2; \text{param})$
- 5: $\text{pk} := (\text{iv}_{\text{owf}}, t)$
- 6: $(\mu, \mathbf{s}) := \text{MsgHash}(\text{pk}, \text{msg})$
- 7: $\rho \leftarrow \{0, 1\}^\lambda$ ▷ For deterministic signing, set $\rho = 0^\lambda$
- 8: $(\mathbf{r}, \text{iv}_{\text{sig}}) := \text{CoinHash}(\text{sk}, \mu, \rho)$
- 9: $(\text{com}, \text{decom}, \{\mathbf{c}_i\}_{i \in [1, \tau]}, \mathbf{u}, \mathbf{V}) := \text{VOLE.Commit}(\mathbf{r}, \text{iv}_{\text{sig}}, \mu, \mathbf{s}; \text{param})$
- 10: $\mathbf{V} := [\mathbf{V}, 0^{\ell_{\text{vole}}}, \dots, 0^{\ell_{\text{vole}}}]$ ▷ Extend $\mathbf{V}$ from $\lambda - w_{\text{grind}}$ to λ columns.
- 11: $\text{chall}_1 := \text{FSHash}_1(\text{com}, \{\mathbf{c}_i\}_{i \in [1, \tau]})$
- 12: $\tilde{\mathbf{u}} := \text{VOLEHash}(\text{chall}_1, \mathbf{u})$
- 13: $\tilde{\mathbf{V}} := \text{VOLEHash}(\text{chall}_1, \mathbf{V})$
- 14: $\mathbf{w} := \text{FieldToBits}(k; \lambda) \parallel \text{FieldToBits}(v_1; \lambda) \parallel \text{FieldToBits}(v_2; \lambda)$
- 15: $\mathbf{d} := \mathbf{w} \oplus \mathbf{u}[0, \ell_{\text{wit}})$
- 16: $\text{chall}_2 := \text{FSHash}_2(\text{chall}_1, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d})$
- 17: $\mathbf{u} := \mathbf{u}[0, \ell_{\text{wit}} + \lambda)$
- 18: $\mathbf{V} := \mathbf{V}[0, \ell_{\text{wit}} + \lambda)$
- 19: $(\tilde{a}_1, \tilde{a}_0) := \text{QS.LynxProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, \mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3, c_1, c_2[, c_3], t, \text{chall}_2; \text{param})$
- 20: $\text{ctr} := 0$
- 21: $\text{decom}_I := \perp$
- 22: **while** $\text{decom}_I = \perp$ **do**
- 23: $\text{chall}_3 := \text{FSHash}_3(\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$
- 24: **if** $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda) \neq 0^{w_{\text{grind}}}$ **then**
- 25: $\text{ctr} := \text{ctr} + 1$
- 26: **continue**
- 27: **end if**
- 28: $I := \text{VOLE.ChalDecAll}(\text{chall}_3; \text{param})$
- 29: $\text{decom}_I := \text{BAVC.Open}(\text{decom}, I, \text{iv}_{\text{sig}}; \text{param})$
- 30: **end while**
- 31: **return** $\sigma := ((\mathbf{c}_i)_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \text{decom}_I, \text{chall}_3, \text{iv}_{\text{sig}}, \text{ctr})$

Deterministic vs. randomized signing. We explicitly list the master randomness ρ in the signing algorithm [Lynx.Sign](#). So deterministic signing can be achieved by setting it to all zeros.

5.2.3 Verification

The verification algorithm takes a public key $\mathbf{pk} = (\text{iv}_{\text{owf}}, t)$, a message msg , and a signature σ . It parses the signature into the correction vectors, compressed VOLE value, masked witness, QuickSilver response value, BAVC opening, final challenge, IV, and grinding counter. The verifier recomputes the message digest μ and salt $\mathbf{s}$ from $\mathbf{pk}$ and msg , so the same public key is bound to the transcript checked by the verifier.

The verifier first reconstructs its IT-MAC/VOLE local keys. Using chall_3 , the partial opening decom_I , the correction vectors, and the IV, it calls [VOLE.Reconstruct](#) to recover the commitment com and the matrix $\mathbf{Q}$, where each row corresponds to a local key. It then recomputes chall_1 from com and the correction vectors. As in signing, it hashes the VOLE local keys with [VOLEHash](#). To verify the opening of $[\tilde{u}]^1$, the verifier applies [VOLE.Fix](#) with $\tilde{u}$ and the final challenge as the global IT-MAC key. For a correct $\tilde{u}$ value, this reconstructs the verifier's local key to a degree-1 authentication of zero, making the fixed $\tilde{\mathbf{Q}}$ identical to the signer's $\tilde{\mathbf{V}}$.

Next, the verifier recomputes chall_2 from chall_1 , $\tilde{u}$, $\tilde{\mathbf{Q}}$, and the masked witness $\mathbf{d}$. It truncates $\mathbf{Q}$ to the rows needed by the Lynx QuickSilver check and calls [QS.LynxVerify](#). This routine reconstructs the missing value $\tilde{a}_0$ from the verifier-side IT-MAC keys, the masked witness, the public Lynx instance, and chall_3 .

Finally, the verifier recomputes $\text{chall}'_3 := \text{FSHash}_3(\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$. It accepts if and only if chall'_3 equals the supplied chall_3 and chall_3 satisfies the proof-of-work suffix condition.

[Lynx.Verify](#)($\mathbf{pk}, \text{msg}, \sigma; \text{param}$)

Input:
$$\left(\begin{array}{l} \mathbf{pk} := (\text{iv}_{\text{owf}} \in \{0, 1\}^\lambda, t \in \mathbb{F}_{2^\lambda}), \\ \text{msg} \in \{0, 1\}^*, \\ \sigma := \left(\begin{array}{l} (\mathbf{c}_i)_{i \in [1, \tau]} \in (\mathbb{F}_2^{\ell_{\text{vole}}})^{\tau-1}, \tilde{\mathbf{u}} \in \{0, 1\}^{\lambda+B}, \mathbf{d} \in \mathbb{F}_2^{\ell_{\text{wit}}}, \\ \tilde{a}_1 \in \{0, 1\}^\lambda, \text{decom}_I \in \{0, 1\}^{(T_{\text{open}}+2\tau) \cdot \lambda}, \\ \text{chall}_3 \in \{0, 1\}^\lambda, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}, \text{ctr} \in \{0, 1\}^{32} \end{array} \right) \end{array} \right)$$

Output: $b \in \{0, 1\}$ $\triangleright 1 = \text{accept}, 0 = \text{reject}.$

- 1: Extract $\lambda, \tau, \ell_{\text{wit}}, B$ from param
- 2: $(\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, c_1, c_2, c_3) := \text{Lynx.SampleMatVec}(\text{iv}_{\text{owf}}; \text{param})$
- 3: $(\mu, \mathbf{s}) := \text{MsgHash}(\mathbf{pk}, \text{msg})$
- 4: $(\text{com}, \mathbf{Q}) := \text{VOLE.Reconstruct}(\text{chall}_3, \text{decom}_I, \{\mathbf{c}_i\}_{i \in [1, \tau]}, \text{iv}_{\text{sig}}, \mu, \mathbf{s}; \text{param})$
- 5: $\mathbf{Q} := [\mathbf{Q}, 0^{\ell_{\text{vole}}}, \dots, 0^{\ell_{\text{vole}}}]$ $\triangleright$ Extend $\mathbf{Q}$ from $\lambda - w_{\text{grind}}$ to λ columns.
- 6: $\text{chall}_1 := \text{FSHash}_1(\text{com}, \{\mathbf{c}_i\}_{i \in [1, \tau]})$
- 7: $\mathbf{Q}' := \text{VOLEHash}(\text{chall}_1, \mathbf{Q})$
- 8: $\tilde{\mathbf{Q}} := \text{VOLE.Fix}(\mathbf{Q}', \tilde{\mathbf{u}}, \text{chall}_3; \lambda, \lambda + B)$
- 9: $\text{chall}_2 := \text{FSHash}_2(\text{chall}_1, \tilde{\mathbf{u}}, \tilde{\mathbf{Q}}, \mathbf{d})$
- 10: $\mathbf{Q} := \begin{bmatrix} \mathbf{Q}[0, :] \\ \vdots \\ \mathbf{Q}[\ell_{\text{wit}} + \lambda - 1, :] \end{bmatrix}$ $\triangleright$ Take the first $\ell_{\text{wit}} + \lambda$ rows of $\mathbf{Q}$.
- 11: $\tilde{a}_0 := \text{QS.LynxVerify}(\mathbf{d}, \mathbf{Q}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3, c_1, c_2, c_3, t; \text{param})$
- 12: $\text{chall}'_3 := \text{FSHash}_3(\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$
- 13: **if** $\text{chall}_3 = \text{chall}'_3 \wedge \text{chall}_3[\lambda - w_{\text{grind}}, \lambda) = 0^{w_{\text{grind}}}$ **then**
- 14: **return** 1
- 15: **else**
- 16: **return** 0

17: **end if**

5.3 Vector Oblivious Linear Evaluation

This subsection specifies the generation of VOLE correlations over $\mathbb{F}_{2^\lambda}$ in VOLEitH. The external interfaces include **VOLE.Commit** and **VOLE.Reconstruct**, which generate IT-MAC authentication materials of the prover and verifier respectively, and **VOLE.Fix**, which computes the verifier's IT-MAC local key during the IT-MAC de-randomization process. Conceptually, the algorithms first commit to τ small VOLE instances through BAVC, then convert the committed seeds into small-field VOLE correlations, and finally use correction vectors to align all small instances so that the committed messages become identical, making a VOLE correlation over $\mathbb{F}_{2^\lambda}$. The remaining procedures in this subsection are internal. It contains the following algorithms.

- **VOLE.ChalDec** and **VOLE.ChalDecAll**: decode the final Fiat-Shamir challenge into one hidden leaf index per VOLE instance.
- **VOLE.Commit**: commits to seed trees, converts their leaves into prover-side shares of the VOLE correlations (i.e., committed random messages and tags), and outputs correction vectors.
- **VOLE.Reconstruct**: verifies a partial BAVC opening and reconstructs the verifier-side VOLE local keys.
- **VOLE.Fix**: applies the correction vector to the IT-MAC local key.
- **VOLE.ConvertToVOLE**: converts a complete binary tree (resp. a binary tree with all-but-one leaf node) into prover-side (resp. verifier-side) VOLE correlations.

5.3.1 VOLE Challenge Decoding

The challenge decoding algorithm extracts the part of the final Fiat-Shamir challenge chall_3 assigned to one VOLE instance. The decoded value determines the leaf that remains hidden in the corresponding all-but-one vector commitment opening. Due to the proof-of-work condition, the last w_{grind} bits of chall_3 are fixed to zero and the remaining challenge bits are divided among the τ VOLE instances.

Because the BAVC tree depths can differ by one, not every instance receives the same number of challenge bits. For the first t_1 instances, **VOLE.ChalDec** reads a k_1 -bit substring and interprets it as an integer in $[0, 2^{k_1})$. For the remaining instances, it reads a k_0 -bit substring and obtains an integer in $[0, 2^{k_0})$. The all-instance decoder simply applies this rule to every instance and returns $I = (\Delta_0, \dots, \Delta_{\tau-1})$.

VOLE.ChalDec($\text{chall}_3, i; \text{param}$)

Input: ($\text{chall}_3 \in \{0, 1\}^\lambda, i \in [0, \tau)$)

Output: $\delta \in \{0, 1\}^{k_0} \cup \{0, 1\}^{k_1}$

1: Extract k_0, t_0, k_1, t_1 from **param**

2: **if** $i < t_1$ **then**

3: $\text{low} := i \cdot k_1$

4: $\text{high} := (i + 1) \cdot k_1$

5: $\Delta := \text{BitsToInt}(\text{chall}_3[\text{low}, \text{high}]; k_1)$

6: **else**

7: $\text{low} := t_1 \cdot k_1 + (i - t_1) \cdot k_0$

▷ Due to PoW, the last w_{grind} bits of chall_3 are 0.

▷ Output is k_1 bits if $i < t_1$, otherwise k_0 bits.

```

8:   high :=  $t_1 \cdot k_1 + (i + 1 - t_1) \cdot k_0$ 
9:    $\Delta := \text{BitsToInt}(\text{chall}_3[\text{low}, \text{high}]; k_0)$ 
10: end if
11: return  $\Delta$ 

```

5.3.2 Decoding All VOLE Challenges

The all-instance decoding algorithm applies [VOLE.ChalDec](#) to each of the τ VOLE instances. The resulting vector I specifies the leaves that remain hidden in the BAVC opening. Equivalently, each Δ_α corresponds to the verifier's VOLE global key for the α -th small VOLE instance.

[VOLE.ChalDecAll](#)($\text{chall}_3; \text{param}$)

Input: ($\text{chall}_3 \in \{0, 1\}^\lambda$)
Output: $I := (\Delta_0, \dots, \Delta_{\tau-1}) \in [0, N_0) \times \dots \times [0, N_{\tau-1})$

```

1: Extract  $\tau$  from param
2: for  $\alpha \in [0, \tau)$  do
3:    $\Delta_\alpha := \text{VOLE.ChalDec}(\text{chall}_3, \alpha; \text{param})$ 
4: end for
5:  $I := (\Delta_0, \dots, \Delta_{\tau-1})$ 
6: return  $I$ 

```

5.3.3 VOLE Commitment

The VOLE commitment algorithm is the prover-side interface of VOLE correlation generation in VOLEitH. It takes the root seed $\mathbf{r}$, the IV iv_{sig} , and the hashed message μ and salt $\mathbf{s}$. It first invokes [BAVC.Commit](#), which expands $\mathbf{r}$ into the large pcGGM tree and returns a global commitment com , a decommitment state decom , and the per-leaf seeds for all τ logical VOLE instances.

For each instance α , the algorithm calls [VOLE.ConvertToVOLE](#) on the seeds of that instance. This produces a secret committed vector $\mathbf{u}_\alpha$ and its authentication tag in matrix form $\mathbf{V}_\alpha$. In this form, the columns $\mathbf{V}_\alpha$ correspond to the bits of the small-field challenge for that instance. The matrices $\mathbf{V}_0, \dots, \mathbf{V}_{\tau-1}$ are concatenated to form the prover-side tag matrix $\mathbf{V}$ used by the later VOLE consistency check and QuickSilver proof.

Initially, the different small VOLE instances have independent committed vectors. To aggregate the small VOLE instances into VOLE over $\mathbb{F}_{2^\lambda}$, the algorithm designates $\mathbf{u}_0$ as the shared vector $\mathbf{u}$. For every $\alpha \geq 1$, it publishes the correction vector $\mathbf{c}_\alpha := \mathbf{u} \oplus \mathbf{u}_\alpha$. These correction vectors allow the verifier, after seeing the challenge-selected hidden leaves, to shift the receiver-side local keys so that all reconstructed columns are consistent with the same vector $\mathbf{u}$.

[VOLE.Commit](#)($\mathbf{r}, \text{iv}_{\text{sig}}, \mu, \mathbf{s}; \text{param}$)

Input: ($\mathbf{r} \in \{0, 1\}^\lambda, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}, \mu \in \{0, 1\}^{2\lambda}, \mathbf{s} \in \{0, 1\}^\lambda$)
Output: $\left(\text{com} \in \{0, 1\}^{2\lambda}, \text{decom} \in \{0, 1\}^\lambda, \right. \\ \left. (\mathbf{c}_\alpha)_{\alpha \in [1, \tau)} \in (\mathbb{F}_2^{\ell_{\text{vole}}})^{\tau-1}, \mathbf{u} \in \mathbb{F}_2^{\ell_{\text{vole}}}, \mathbf{V} \in \mathbb{F}_2^{\ell_{\text{vole}} \times \lambda} \right)$

```

1: Extract  $\lambda, \ell_{\text{vole}}, \tau, \{N_\alpha\}_{\alpha \in [0, \tau)}$  from param
2: ( $\text{com}, \text{decom}, (\text{seed}_i^\alpha)_{\alpha \in [0, \tau), i \in N_\alpha} := \text{BAVC.Commit}(\mathbf{r}, \text{iv}_{\text{sig}}, \mu, \mathbf{s}; \text{param})$ )
3: for  $\alpha \in [0, \tau)$  do

```

```

4:    $(\mathbf{u}_\alpha, \mathbf{V}_\alpha) := \text{VOLE.ConvertToVOLE}((\text{seed}_i^\alpha)_{i \in N_\alpha}, \text{iv}_{\text{sig}}; \text{param})$ 
5: end for
6:  $\mathbf{V} := [\mathbf{V}_0, \dots, \mathbf{V}_{\tau-1}]$ 
7:  $\mathbf{u} := \mathbf{u}_0$ 
8: for  $\alpha \in [1, \tau)$  do
9:    $\mathbf{c}_\alpha := \mathbf{u} \oplus \mathbf{u}_\alpha$ 
10: end for
11: return  $(\text{com}, \text{decom}, (\mathbf{c}_\alpha)_{\alpha \in [1, \tau)}, \mathbf{u}, \mathbf{V})$ 

```

5.3.4 VOLE Reconstruction

The VOLE reconstruction algorithm is the verifier-side counterpart of [VOLE.Commit](#). It takes the final challenge chall_3 , the partial BAVC opening decom_I , the correction vectors, the IV, and the hashed message μ and salt $\mathbf{s}$. It first decodes chall_3 into $I = (\Delta_0, \dots, \Delta_{\tau-1})$. The value Δ_α identifies the unopened leaf in the α -th logical VOLE instance.

The algorithm then calls [BAVC.Verify](#) on the partial opening. If the opening is invalid, too large, or inconsistent with the leaf commitments, reconstruction returns $\perp$. Otherwise, [BAVC.Verify](#) returns the commitment com and all leaf seeds except the hidden seed at index Δ_α for each instance. This recovered commitment is later hashed again in the top-level verifier, which allows the verifier to produce the same transcript as the signer.

For each α , the verifier re-indexes the opened seeds by XOR-ing their indices with Δ_α . This moves the hidden seed to position 0. The verifier represents that seed by $\perp$ and converts the partially hidden list with [VOLE.ConvertToVOLE](#). The result is an uncorrected local-key matrix $\mathbf{Q}'_\alpha$. The bit decomposition of Δ_α acts as the small instance VOLE global key Δ_α . Applying [VOLE.Fix](#) with Δ_α and the published correction vector shifts the columns of $\mathbf{Q}'_\alpha$ to match the unified committed message $\mathbf{u}_0$ chosen by the prover. Finally, the matrices for all τ instances are concatenated into the verifier-side matrix $\mathbf{Q}$.

[VOLE.Reconstruct](#)($\text{chall}_3, \text{decom}_I, (\mathbf{c}_\alpha)_{\alpha \in [1, \tau)}, \text{iv}_{\text{sig}}, \mu, \mathbf{s}; \text{param}$)

Input: $\left(\text{chall}_3 \in \{0, 1\}^\lambda, \text{decom}_I \in \{0, 1\}^{(T_{\text{open}} + 2\tau)\lambda}, \right.$
 $\left. (\mathbf{c}_\alpha)_{\alpha \in [1, \tau)} \in (\mathbb{F}_2^{\ell_{\text{vole}}})^{\tau-1}, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}, \mu \in \{0, 1\}^{2\lambda}, \mathbf{s} \in \{0, 1\}^\lambda \right)$

Output: $(\text{com}, \mathbf{Q}) \in \{0, 1\}^{2\lambda} \times \mathbb{F}_2^{\ell_{\text{vole}} \times \lambda}$

```

1: Extract  $\lambda, \ell_{\text{vole}}, \tau, \{n_\alpha\}_{\alpha \in [0, \tau)}, \{N_\alpha\}_{\alpha \in [0, \tau)}, L, T_{\text{open}}$  from  $\text{param}$ 
2:  $I := \text{VOLE.ChalDecAll}(\text{chall}_3; \text{param})$ 
3: Parse  $I = (\Delta_0, \dots, \Delta_{\tau-1})$ 
4:  $\text{out} := \text{BAVC.Verify}(I, \text{iv}_{\text{sig}}, \mu, \mathbf{s}, \text{decom}_I; \text{param})$ 
5: if  $\text{out} = \perp$  then
6:   return  $\perp$ 
7: else
8:   Parse  $\text{out} := (\text{com}, \text{seed}_i^\alpha)_{\alpha \in [0, \tau), i \neq \Delta_\alpha}$ 
9: end if
10: for  $\alpha \in [0, \tau)$  do
11:    $\text{seed}_0^\alpha := \perp$ 
12:   for  $i \in [1, N_\alpha)$  do
13:      $i' := \text{BitsToInt}(\text{IntToBits}(i; n_\alpha) \oplus \text{IntToBits}(\Delta_\alpha; n_\alpha); n_\alpha)$ 
14:      $\text{seed}_i^\alpha := \text{seed}_{i'}^\alpha$ 
15:   end for

```

```

16:    $(\mathbf{u}'_\alpha, \mathbf{Q}'_\alpha) := \text{VOLE.ConvertToVOLE}((\overline{\text{seed}}_i^\alpha)_{i \in N_\alpha}, \text{iv}_{\text{sig}}, \ell_{\text{vole}})$ 
17:    $\mathbf{Q}_\alpha := \text{VOLE.Fix}(\mathbf{Q}'_\alpha, \mathbf{c}_\alpha, \Delta_\alpha; n_\alpha, \ell_{\text{vole}})$ 
18: end for
19:  $\mathbf{Q} := [\mathbf{Q}_0, \dots, \mathbf{Q}_{\tau-1}]$ 
20: return  $(\text{com}, \mathbf{Q})$ 

```

5.3.5 VOLE Correction

The correction algorithm turns an uncorrected verifier-side matrix $\mathbf{Q}'$ into the corrected local-key matrix $\mathbf{Q}$. It is a linear post-processing step. For every challenge bit $\Delta[i]$, the algorithm either leaves the corresponding column unchanged or XORs the correction vector $\mathbf{c}$ into it. Thus, if the α -th small VOLE instance was generated with committed vector $\mathbf{u}_\alpha$, the correction vector $\mathbf{c}_\alpha = \mathbf{u}_0 \oplus \mathbf{u}_\alpha$ changes the verifier's local keys from authenticating $\mathbf{u}_\alpha$ into authenticating $\mathbf{u}_0$. The same operation is also used later by the top-level verifier when deriving the prover's IT-MAC tag for the compressed VOLE consistency checking value. This is done by correcting the verifier's local key with the compressed VOLE consistency checking value itself, essentially performing an IT-MAC opening check.

VOLE.Fix($\mathbf{Q}', \mathbf{c}, \Delta; k, \ell$)

Input: $(\mathbf{Q}' \in \mathbb{F}_2^{\ell \times k}, \mathbf{c} \in \mathbb{F}_2^\ell, \Delta \in \{0, 1\}^k)$

Output: $\mathbf{Q} \in \mathbb{F}_2^{\ell \times k}$

```

1: for  $i \in [0, k)$  do
2:    $\mathbf{q}_i := \mathbf{Q}'[:, i] \oplus \Delta[i] \cdot \mathbf{c}$   $\triangleright \Delta[i] \in \{0, 1\}$ .
3: end for
4:  $\mathbf{Q} := [\mathbf{q}_0, \dots, \mathbf{q}_{k-1}]$ 
5: return  $\mathbf{Q}$ 

```

5.3.6 Seed Conversion to VOLE

This conversion algorithm expands the leaves of a binary seed tree into VOLE correlations. It is used in two modes. During **VOLE.Commit**, all leaf seeds are known to the signer and the algorithm outputs the prover-side vector $\mathbf{u}$ and its authentication tag in matrix form $\mathbf{V}$. During **VOLE.Reconstruct**, one leaf is hidden and represented as $\perp$; the algorithm treats this missing seed as the zero vector and outputs the verifier-side IT-MAC local key in matrix form before correction.

To make the connection with the high-level expression explicit, define

$$\mathbf{r}_{0,i} := \text{PRG}(\text{seed}_i, \text{iv}_{\text{sig}}, \lceil 2\lambda/128 \rceil; \ell_{\text{vole}}), \quad \mathbf{b}_i := \text{IntToBits}(i; k).$$

Here, $\mathbf{b}_i$ is the little-endian binary representation of the leaf index i . For each row position $h \in [0, \ell_{\text{vole}})$, the small-field VOLE tag can be viewed as

$$v[h] := \sum_{i \in [0, N)} \text{BitsToField}(\mathbf{b}_i; k) \cdot \mathbf{r}_{0,i}[h] \in \mathbb{F}_{2^k}.$$

Equivalently, $\mathbf{V}$ stores these tags in bit-sliced form rather than as explicit $\mathbb{F}_{2^k}$ elements. Its j -th column is

$$\mathbf{V}[:, j] = \mathbf{v}_j := \bigoplus_{i \in [0, N): \mathbf{b}_i[j]=1} \mathbf{r}_{0,i},$$

so for every row h , the row vector $\mathbf{V}[h, :]$ is exactly the k -bit binary representation of the field element $v[h]$. Applying $\text{BitsToField}(\mathbf{V}[h, :]; k)$ therefore recovers the authentication tag corresponding to the h -th committed bit. Since the field has characteristic two, the sign in the overview formula $\sum_i (-i) \cdot \text{PRG}(\text{seed}_i)$ does not change this representation.

The algorithm first expands every available seed with PRG into a vector in $\mathbb{F}_2^{\ell_{\text{vole}}}$. It then folds the complete binary tree upward by XORing sibling vectors. At level j , the XOR of the odd children is accumulated into the column vector $\mathbf{v}_j$. This is exactly the same as XORing all expanded leaves whose j -th index bit is 1. After all levels have been processed, the remaining root value is the committed message vector $\mathbf{u} = \bigoplus_{i \in [0, N)} \mathbf{r}_{0,i}$, while the collected odd-child sums form the columns of $\mathbf{V}$. This folding procedure is the bridge between a committed seed tree and the VOLE/IT-MAC correlations consumed by the rest of the proof system.

VOLE.ConvertToVOLE $((\text{seed}_i)_{i \in [0, N)}, \text{iv}_{\text{sig}}; \text{param})$

Input: $((\text{seed}_i)_{i \in [0, N)} \in \{0, 1\}^{N \cdot \lambda}, \text{iv}_{\text{sig}} \in \{0, 1\}^{120})$ $\triangleright N = 2^k$.

Output: $(\mathbf{u} \in \mathbb{F}_2^{\ell_{\text{vole}}}, \mathbf{V} \in \mathbb{F}_2^{\ell_{\text{vole}} \times k})$

```

1: Extract  $\ell_{\text{vole}}$  from param
2: if  $\text{seed}_0 = \perp$  then
3:    $\mathbf{r}_{0,0} := 0^{\ell_{\text{vole}}}$ 
4: else
5:    $\mathbf{r}_{0,0} := \text{PRG}(\text{seed}_0, \text{iv}_{\text{sig}}, \lceil \frac{2\lambda}{128} \rceil; \ell_{\text{vole}})$   $\triangleright$  Increased index to avoid collision with
   LeafHash.
6: end if
7: for  $i \in [1, N)$  do
8:    $\mathbf{r}_{0,i} := \text{PRG}(\text{seed}_i, \text{iv}_{\text{sig}}, \lceil \frac{2\lambda}{128} \rceil; \ell_{\text{vole}})$   $\triangleright$  Increased index to avoid collision with
   LeafHash.
9: end for
10:  $k = \log_2(N)$ 
11: for  $i \in [0, k)$  do
12:    $\mathbf{v}_i := 0^{\ell_{\text{vole}}}$ 
13: end for
14: for  $j \in [0, k)$  do
15:   for  $i \in [0, \frac{N}{2^{j+1}})$  do
16:      $\mathbf{v}_j := \mathbf{v}_j \oplus \mathbf{r}_{j,2i+1}$ 
17:      $\mathbf{r}_{j+1,i} := \mathbf{r}_{j,2i} \oplus \mathbf{r}_{j,2i+1}$ 
18:   end for
19: end for
20:  $\mathbf{u} := \mathbf{r}_{k,0}$ 
21:  $\mathbf{V} := [\mathbf{v}_0, \dots, \mathbf{v}_{k-1}]$ 
22: return  $(\mathbf{u}, \mathbf{V})$ 
```

5.4 Batch All-but-one Vector Commitment

This subsection describes the batch all-but-one vector commitment used to realize the many small VOLE instances compactly. The algorithms generate a large pcGGM tree, which is conceptually τ small trees, with leaf nodes mapped via the interleaving rule. For each logical small VOLE instance α , the challenge index Δ_α derived from chall_3 identifies the one leaf that remains hidden. The opening then reveals enough internal nodes to reconstruct every other leaf, while also revealing the commitments to the hidden leaves so that the verifier can recompute the global BAVC commitment. This subsection contains the following algorithms.

- **BAVC.Commit**: expands a root seed into the large pcGGM tree and commits to all leaf commitments.
- **BAVC.Open**: opens all leaves except the challenge-selected leaves, subject to the threshold T_{open} .
- **BAVC.Verify**: checks a partial opening, reconstructs the opened leaf seeds, and recomputes the commitment.
- **BAVC.Interleave** and **BAVC.Deinterleave**: translate between per-instance leaf indices and leaf positions in the large pcGGM tree.
- **BAVC.MinSpan**: compresses a set of opened leaves to a minimal set of internal tree nodes.
- **LeafHash**: derives the seed used by VOLE conversion and the corresponding leaf commitment.

In the algorithms **BAVC.Commit** and **BAVC.Verify**, we use two constants $\text{const}_0, \text{const}_1 \in \{0, 1\}^\lambda$ satisfying $\text{const}_0 \neq \text{const}_1$. In our implementation, we choose $\text{const}_0 = \text{IntToBits}(0; \lambda)$ and $\text{const}_1 = \text{IntToBits}(1; \lambda)$.

5.4.1 BAVC Commitment

The BAVC commitment algorithm takes a root seed $\mathbf{r}$, the IV iv_{sig} , and the hashed message μ and salt $\mathbf{s}$. It expands $\mathbf{r}$ into the large pcGGM tree with L leaves. For internal nodes in the upper part of the tree, the left child is obtained by applying **TCCR** to the parent seed, and the right child is derived by XOR-ing the left child with the parent seed. For the bottom internal nodes, the two children are derived by hashing the parent seed XOR-ed with the two fixed constants const_0 and const_1 . This gives a deterministic tree expansion from the root seed, the salt, and the IV.

Each leaf index $\text{id} \in [L, 2L)$ in the large pcGGM tree is mapped back to a small VOLE instance α and a leaf index i_α in its corresponding small tree by **BAVC.Deinterleave**. The leaf value is then processed by **LeafHash**, which outputs the seed $\text{seed}_{i_\alpha}^\alpha$ used later by **VOLE.ConvertToVOLE** and the corresponding leaf commitment $\text{com}_{i_\alpha}^\alpha$. After all leaves are processed, **BAVCHash** compresses all leaf commitments together with $\mu, \mathbf{s}$, and iv_{sig} into the global commitment com . The decommitment state is the root seed $\text{decom} := \mathbf{r}_1$, which allows the signer to reproduce the whole tree when opening.

BAVC.Commit($\mathbf{r}, \text{iv}_{\text{sig}}, \mu, \mathbf{s}; \text{param}$)

Input: ($\mathbf{r} \in \{0, 1\}^\lambda, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}, \mu \in \{0, 1\}^{2\lambda}, \mathbf{s} \in \{0, 1\}^\lambda$)

Output: $\left(\text{com} \in \{0, 1\}^{2\lambda}, \text{decom} \in \{0, 1\}^\lambda, \left(\text{seed}_i^\alpha \right)_{\alpha \in [0, \tau), i \in N_\alpha} \in (\{0, 1\}^\lambda)^{N_0} \times \dots \times (\{0, 1\}^\lambda)^{N_{\tau-1}} \right)$

- 1: Extract $\lambda, \tau, \{N_\alpha\}_{\alpha \in [0, \tau)}, L$ from **param**
- 2: $\mathbf{r}_1 := \mathbf{r}$
- 3: **for** $\text{id} \in [1, L/2)$ **do**
- 4: $\mathbf{r}_{2:\text{id}} := \text{TCCR}(\mathbf{r}_{\text{id}}, \mathbf{s}, \text{iv}_{\text{sig}})$
- 5: $\mathbf{r}_{2:\text{id}+1} := \mathbf{r}_{2:\text{id}} \oplus \mathbf{r}_{\text{id}}$
- 6: **end for**
- 7: **for** $\text{id} \in [L/2, L)$ **do**
- 8: $\mathbf{r}_{2:\text{id}} := \text{TCCR}(\mathbf{r}_{\text{id}} \oplus \text{const}_0, \mathbf{s}, \text{iv}_{\text{sig}})$
- 9: $\mathbf{r}_{2:\text{id}+1} := \text{TCCR}(\mathbf{r}_{\text{id}} \oplus \text{const}_1, \mathbf{s}, \text{iv}_{\text{sig}})$
- 10: **end for**
- 11: **for** $\text{id} \in [L, 2L)$ **do**

```

12:   $(\alpha, i_\alpha) := \text{BAVC.Deinterleave}(\text{id}; \text{param})$ 
13:   $(\text{seed}_{i_\alpha}^\alpha, \text{com}_{i_\alpha}^\alpha) := \text{LeafHash}(\mathbf{r}_{\text{id}}, \text{iv}_{\text{sig}})$ 
14: end for
15:  $\text{com} := \text{BAVCHash}(\mu, \mathbf{s}, (\text{com}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]}, \text{iv}_{\text{sig}})$ 
16:  $\text{decom} := \mathbf{r}_1$ 
17: return  $(\text{com}, \text{decom}, (\text{seed}_i^\alpha)_{\alpha \in [0, \tau], i \in N_\alpha})$ 

```

5.4.2 BAVC Opening

The BAVC opening algorithm takes the root decommitment and a challenge vector I . It re-computes the pcGGM tree from the root seed and parses $I = (\Delta_0, \dots, \Delta_{\tau-1})$. For small VOLE instance α , the leaf Δ_α is the one that must remain hidden in its corresponding small tree. The algorithm maps each pair (α, Δ_α) into its position id_α in the large pcGGM tree with [BAVC.Interleave](#) and stores the commitment to that hidden leaf in the opening.

All other leaves in the large pcGGM tree are considered opened. Rather than revealing every opened leaf seed individually, the algorithm calls [BAVC.MinSpan](#) on the set of opened leaves. This replaces sibling pairs by their parent whenever possible and produces a compact set of internal node seeds from which the verifier can re-expand all opened leaves. If this compact node set is larger than T_{open} , the algorithm returns $\perp$. This abort is handled by the signing algorithm via grinding (deriving a different chall_3 using an incremented counter). Otherwise, the partial opening decom_I consists of the τ hidden leaf commitments and the selected internal node seeds, padded to a fixed length.

[BAVC.Open](#)($\text{decom}, I, \text{iv}_{\text{sig}}; \text{param}$)

Input: $(\text{decom} \in \{0, 1\}^\lambda, I \in [0, N_0] \times \dots \times [0, N_{\tau-1}], \text{iv}_{\text{sig}} \in \{0, 1\}^{120})$

Output: $\text{decom}_I \in \{0, 1\}^{(T_{\text{open}} + 2\tau)\lambda} \cup \{\perp\}$

```

1: Extract  $\lambda, \tau, L, T_{\text{open}}, \{N_\alpha\}_{\alpha \in [0, \tau]}$  from param
2:  $\mathbf{r}_1 := \text{decom}$ 
3: for  $\text{id} \in [1, L/2)$  do
4:    $\mathbf{r}_{2:\text{id}} := \text{TCCR}(\mathbf{r}_{\text{id}}, \mathbf{s}, \text{iv}_{\text{sig}})$ 
5:    $\mathbf{r}_{2:\text{id}+1} := \mathbf{r}_{2:\text{id}} \oplus \mathbf{r}_{\text{id}}$ 
6: end for
7: for  $\text{id} \in [L/2, L)$  do
8:    $\mathbf{r}_{2:\text{id}} := \text{TCCR}(\mathbf{r}_{\text{id}} \oplus \text{const}_0, \mathbf{s}, \text{iv}_{\text{sig}})$ 
9:    $\mathbf{r}_{2:\text{id}+1} := \text{TCCR}(\mathbf{r}_{\text{id}} \oplus \text{const}_1, \mathbf{s}, \text{iv}_{\text{sig}})$ 
10: end for
11: Parse  $I = (\Delta_0, \dots, \Delta_{\tau-1})$   $\triangleright$  For  $\alpha \in [0, \tau)$ ,  $\Delta_\alpha \in N_\alpha$ .
12: for  $\alpha \in [0, \tau)$  do
13:    $\text{id}_\alpha := \text{BAVC.Interleave}(\alpha, \Delta_\alpha; \text{param})$ 
14:    $(\text{seed}_{\Delta_\alpha}^\alpha, \text{com}_{\Delta_\alpha}^\alpha) := \text{LeafHash}(\mathbf{r}_{\text{id}_\alpha}, \text{iv}_{\text{sig}})$ 
15: end for
16:  $\text{id-list} := \text{BAVC.MinSpan}([L, 2L) \setminus (\text{id}_0, \dots, \text{id}_{\tau-1}); \text{param})$ 
17: if  $|\text{id-list}| \leq T_{\text{open}}$  then
18:    $\text{decom}_I := ((\text{com}_{\Delta_\alpha}^\alpha)_{\alpha \in [0, \tau)}, (\mathbf{r}_{\text{id}})_{\text{id} \in \text{id-list}}) \| 0^{(T_{\text{open}} - |\text{id-list}|) \cdot \lambda}$ 
19: else
20:    $\text{decom}_I := \perp$ 
21: end if
22: return  $\text{decom}_I$ 

```

5.4.3 BAVC Verification

The BAVC verification algorithm takes the challenge vector I , the IV, the hashed message μ and salt $\mathbf{s}$, and the partial opening decom_I . It first recomputes the hidden leaf positions in the large pcGGM tree from I and computes the same minimum-span node set that the signer should have opened. If this set exceeds T_{open} , the verifier returns $\perp$.

The verifier then parses decom_I into the commitments of the hidden leaves and the internal node seeds for the opened part of the tree. Starting from those internal node seeds, it expands the tree downward using the same tree expansion rules as in [BAVC.Commit](#). For every reconstructed opened leaf, it runs [LeafHash](#) to recover both the seed returned to [VOLE.Reconstruct](#) and the leaf commitment used for the final hash. The verifier combines the reconstructed and hidden leaf commitments, then recomputes the global BAVC hash. The output is the recomputed global commitment com together with all leaf seeds except the hidden seeds $\text{seed}_{\Delta_\alpha}^\alpha$. Thus the verifier obtains exactly the seeds needed for all-but-one reconstruction, without learning the hidden leaf seeds.

BAVC.Verify($I, \text{iv}_{\text{sig}}, \mu, \mathbf{s}, \text{decom}_I; \text{param}$)

Input: $\left(\begin{array}{l} I := (\Delta_0, \dots, \Delta_{\tau-1}) \in [0, N_0] \times \dots \times [0, N_{\tau-1}), \\ \text{iv}_{\text{sig}} \in \{0, 1\}^{120}, \mu \in \{0, 1\}^{2\lambda}, \mathbf{s} \in \{0, 1\}^\lambda, \\ \text{decom}_I := ((\text{com}_{i_\alpha}^\alpha)_{\alpha \in [0, \tau)} \in \{0, 1\}^{2\tau\lambda}, (r_i)_{i \in \text{id-list}} \in \{0, 1\}^{T_{\text{open}} \cdot \lambda}) \end{array} \right)$

Output: $(\text{com}, (\text{seed}_i^\alpha)_{\alpha \in [0, \tau), i \neq \Delta_\alpha}) \cup \{\perp\}$

```

1: Extract  $\lambda, \tau, L, T_{\text{open}}, \{N_\alpha\}_{\alpha \in [0, \tau)}$  from param
2: for  $\alpha \in [0, \tau)$  do
3:    $\text{id}_\alpha := \text{BAVC.Interleave}(\alpha, \Delta_\alpha; \text{param})$ 
4: end for
5:  $\text{id-list} := \text{BAVC.MinSpan}([L, 2L) \setminus (\text{id}_0, \dots, \text{id}_{\tau-1}); \text{param})$ 
6: if  $|\text{id-list}| > T_{\text{open}}$  then
7:   return  $\perp$ 
8: end if
9: Parse  $\text{decom}_I = ((\text{com}_{\Delta_\alpha}^\alpha)_{\alpha \in [0, \tau)}, (r_{\text{id}})_{\text{id} \in \text{id-list}})$ 
10: for  $\text{id} \in [1, L)$  do
11:   if  $\text{id} \in \text{id-list}$  then
12:     Remove  $\text{id}$  from  $\text{id-list}$ 
13:     Add  $2 \cdot \text{id}$  and  $2 \cdot \text{id} + 1$  to  $\text{id-list}$ 
14:     if  $\text{id} < L/2$  then
15:        $r_{2 \cdot \text{id}} := \text{TCCR}(r_{\text{id}}, \mathbf{s}, \text{iv}_{\text{sig}})$ 
16:        $r_{2 \cdot \text{id} + 1} := r_{2 \cdot \text{id}} \oplus r_{\text{id}}$ 
17:     else
18:        $r_{2 \cdot \text{id}} := \text{TCCR}(r_{\text{id}} \oplus \text{const}_0, \mathbf{s}, \text{iv}_{\text{sig}})$ 
19:        $r_{2 \cdot \text{id} + 1} := \text{TCCR}(r_{\text{id}} \oplus \text{const}_1, \mathbf{s}, \text{iv}_{\text{sig}})$ 
20:     end if
21:   end if
22: end for
23: for  $\text{id} \in \text{id-list}$  do ▷  $\text{id-list}$  has  $L - \tau$  indices.
24:    $(\alpha, i_\alpha) := \text{BAVC.Deinterleave}(\text{id}; \text{param})$ 
25:    $(\text{seed}_{i_\alpha}^\alpha, \text{com}_{i_\alpha}^\alpha) := \text{LeafHash}(r_{\text{id}}, \text{iv}_{\text{sig}})$ 
26: end for
27:  $\text{com} = \text{BAVCHash}(\mu, \mathbf{s}, (\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha)}, \text{iv}_{\text{sig}})$ 
28: return  $(\text{com}, (\text{seed}_i^\alpha)_{\alpha \in [0, \tau), i \neq \Delta_\alpha})$ 

```

5.4.4 Leaf Interleaving

The interleaving algorithm maps a leaf index inside one of the τ small trees to the corresponding leaf position in the large pcGGM tree. The first 2^{k_0} leaf positions are valid for all τ small instances. The mapping therefore interleaves them with stride τ . The remaining leaves exist only for the t_1 instances with depth k_1 , and these extra leaves are interleaved with stride t_1 . This layout lets the implementation commit to all VOLE instances with a single large pcGGM tree while still supporting per-instance challenge indices of length k_0 or k_1 .

BAVC.Interleave($\alpha, i_\alpha; \text{param}$)

Input: ($\alpha \in [0, \tau), i_\alpha \in [0, N_\alpha)$)

Output: $\text{id} \in [L, 2L)$

```

1: Extract  $\tau, k_0, t_0, k_1, t_1, L$  from param
2: if  $i_\alpha < 2^{k_0}$  then
3:    $\text{id} = L + i_\alpha \cdot \tau + \alpha$ 
4: else ▷ Tree  $\alpha$  of depth  $k_1$ ;  $\alpha \in [0, t_1)$ .
5:    $\text{id} = L + 2^{k_0} \cdot \tau + (i_\alpha - 2^{k_0}) \cdot t_1 + \alpha$ 
6: end if
7: return  $\text{id}$ 
```

5.4.5 Leaf Deinterleaving

The deinterleaving algorithm is the inverse of **BAVC.Interleave**. Given a leaf position in the large pcGGM tree, it recovers the index α of the small VOLE instance and the leaf index i_α inside that instance. It first checks whether the position lies in the common 2^{k_0} -leaf region. If so, division by τ gives i_α , while the remainder gives α . Otherwise, the position lies in the extra region for the t_1 larger instances, so the same computation is performed with stride t_1 and offset 2^{k_0} .

BAVC.Deinterleave($\text{id}; \text{param}$)

Input: $\text{id} \in [L, 2L)$

Output: ($\alpha \in [0, \tau), i_\alpha \in [0, N_\alpha)$)

```

1: Extract  $\tau, k_0, t_0, k_1, t_1, L$  from param
2: if  $\text{id} < L + \tau \cdot 2^{k_0}$  then
3:    $\alpha := (\text{id} - L) \bmod \tau$ 
4:    $i_\alpha := \lfloor \frac{\text{id} - L}{\tau} \rfloor$ 
5: else
6:    $\alpha := (\text{id} - L - \tau \cdot 2^{k_0}) \bmod t_1$ 
7:    $i_\alpha := \lfloor \frac{\text{id} - L - 2^{k_0} \cdot \tau}{t_1} \rfloor + 2^{k_0}$ 
8: end if
9: return ( $\alpha, i_\alpha$ )
```

5.4.6 Minimum-Span Opening

The minimum-span algorithm compresses a set of leaves or nodes in the binary tree into the smallest ancestor set that spans exactly the same leaves. It scans the tree bottom-up. Whenever both children of a node are in the opening set, the two children are replaced by their parent. Repeatedly applying this rule produces the compact internal-node set included in the BAVC

opening. This is the mechanism that makes the all-but-one opening short and motivates the threshold T_{open} used during signing.

BAVC.MinSpan(id-list; param)

Input: id-list $\subseteq [1, 2L)$

Output: id-list $\subseteq [1, 2L)$

```

1: for id from  $L - 1$  down to 1 do
2:   if  $2 \cdot \text{id} \in \text{id-list} \wedge 2 \cdot \text{id} + 1 \in \text{id-list}$  then
3:     Remove  $2 \cdot \text{id}$  and  $2 \cdot \text{id} + 1$  from id-list
4:     Add id to id-list
5:   end if
6: end for
7: return id-list

```

5.4.7 Leaf Hashing

The leaf hashing algorithm splits a leaf value into two roles. The leaf itself is used as the seed for **VOLE.ConvertToVOLE**, while a 2λ -bit commitment to that seed is derived with **PRG** and included in the BAVC commitment hash. The IV is included in the PRG call so that leaf commitments are domain-separated across signatures.

LeafHash($x, \text{iv}_{\text{sig}}$)

Input: ($x \in \{0, 1\}^\lambda, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}$)

Output: (seed $\in \{0, 1\}^\lambda, \text{com} \in \{0, 1\}^{2\lambda}$)

```

1: seed :=  $x$ 
2: com := PRG( $x, \text{iv}_{\text{sig}}, 0^8; 2\lambda$ )
3: return (seed, com)

```

5.5 Lynx One-Way Function

This subsection gives the algorithms related to the underlying Lynx one-way function. Besides sampling the public parameters and forward evaluation, it also defines the witness extension interface that converts the secret OWF input into the extended witness verified by QuickSilver. For Lynx, the witness consists of the secret input k and two intermediate field elements v_1, v_2 . Given the public parameters and the output t , the relation among (k, v_1, v_2) can be checked by a small set of degree-2 constraints. This subsection contains the following algorithms.

- **Lynx.SampleMatVec**: expands iv_{owf} into the public linear maps L_0, L_1, L_2 and field constants c_1, c_2, c_3 .
- **Lynx.ExtendWitness**: evaluates the nonlinear witness-extension layer and computes v_1, v_2 from k .
- **Lynx.EvalOutput**: evaluates the final Lynx output t from the extended witness and public parameters.

5.5.1 Lynx Public-Parameter Sampling

The public-parameter sampling algorithm expands the public seed iv_{owf} using **MatVecHash**. The output buffer is parsed into three invertible linear maps and two or three field constants. For

each $i \in \{0, 1, 2\}$, the algorithm samples an upper-triangular and a lower-triangular matrix from the buffer and multiplies them to obtain L_i . The constants c_1 and c_2 are always sampled; c_3 is additionally sampled for $\lambda \in \{256, 384, 512\}$, where the final Lynx layer contains an additional inversion box. The fourth linear map L_3 is fixed by the parameter set and is therefore not sampled from iv_{owf} .

Lynx.SampleMatVec($\text{iv}_{\text{owf}}; \text{param}$)

Input: $\text{iv}_{\text{owf}} \in \{0, 1\}^\lambda$

Output: $L_0, L_1, L_2 \in \mathbb{F}_2^{\lambda \times \lambda}, c_1, c_2, c_3 \in \mathbb{F}_{2^\lambda} \quad \triangleright c_3 \text{ is only returned if } \lambda \in \{256, 384, 512\}$

```

1: Extract  $\lambda$  from param
2: if  $\lambda = 160$  then
3:    $\text{buf} := \text{MatVecHash}(\text{iv}_{\text{owf}}; 3(\lambda - 1)\lambda + 2\lambda)$ 
4: else  $\triangleright \lambda \in \{256, 384, 512\}$ 
5:    $\text{buf} := \text{MatVecHash}(\text{iv}_{\text{owf}}; 3(\lambda - 1)\lambda + 3\lambda)$ 
6: end if
7: for  $i \in [0, 2]$  do
8:   Pack  $\text{buf}[2i \cdot (\lambda - 1)\lambda, (2i + 1) \cdot (\lambda - 1)\lambda]$  into a upper-triangular matrix  $U'_i$ , in
   row-major order.
9:   Pack  $\text{buf}[(2i + 1) \cdot (\lambda - 1)\lambda, (2i + 2) \cdot (\lambda - 1)\lambda]$  into a lower-triangular matrix  $L'_i$ , in
   row-major order.
10:   $L_i := U'_i \cdot L'_i$ 
11: end for
12: for  $i \in [1, 2]$  do
13:   $c_i := \text{BitsToField}(\text{buf}[3(\lambda - 1)\lambda + i \cdot \lambda, 3(\lambda - 1)\lambda + (i + 1) \cdot \lambda]; \lambda)$ 
14: end for
15: if  $\lambda = 160$  then
16:  return  $(L_0, L_1, L_2, c_1, c_2)$ 
17: else  $\triangleright \lambda \in \{256, 384, 512\}$ 
18:   $c_3 := \text{BitsToField}(\text{buf}[3(\lambda - 1)\lambda + 3 \cdot \lambda, 3(\lambda - 1)\lambda + 4 \cdot \lambda]; \lambda)$ 
19:  return  $(L_0, L_1, L_2, c_1, c_2, c_3)$ 
20: end if
```

5.5.2 Lynx Witness Extension

The witness-extension algorithm computes the two intermediate field elements v_1 and v_2 from the secret OWF input k and the public values sampled from iv_{owf} . The exact exponentiation rules depend on the security parameter. When a linear layer is applied to a field element, the field element is first serialized by **FieldToBits**, multiplied by the corresponding $\mathbb{F}_2$ -matrix, and then parsed back with **BitsToField**. This convention makes the plaintext evaluation match the bit-level constraints used by QuickSilver.

For $\lambda = 160$, the first nonlinear layer uses inversion, written explicitly as exponentiation by $2^\lambda - 2$. For $\lambda \in \{256, 384, 512\}$, the first intermediate value uses the inverse exponent corresponding to the selected power map. The second intermediate value is computed from $L_0(k) + c_2$ using the exponent prescribed by the Lynx design for the given security parameter.

Lynx.ExtendWitness($L_0, L_1, L_2, c_1, c_2, c_3, k; \text{param}$)

Input: $L_0, L_1, L_2 \in \mathbb{F}_2^{\lambda \times \lambda}, c_1, c_2, c_3, k \in \mathbb{F}_{2^\lambda} \quad \triangleright c_3 \text{ is only required if } \lambda \in \{256, 384, 512\}$

Output: $v_1, v_2 \in \mathbb{F}_{2^\lambda}$

```

1: Extract  $\lambda$  from param
2: if  $\lambda = 160$  then
3:    $v_1 := (k + c_1)^{2^\lambda - 2}$ 
4:    $v_2 := (\text{BitsToField}(\mathbf{L}_0 \cdot \text{FieldToBits}(k; \lambda); \lambda) + c_2)^{2^{67} - 1}$ 
5: else if  $\lambda = 256$  then
6:    $v_1 := (k + c_1)^{(2^5 - 1)^{-1} \bmod (2^\lambda - 1)}$ 
7:    $v_2 := (\text{BitsToField}(\mathbf{L}_0 \cdot \text{FieldToBits}(k; \lambda); \lambda) + c_2)^{2^{53} - 1}$ 
8: else if  $\lambda = 384$  then
9:    $v_1 := (k + c_1)^{(2^{13} - 1)^{-1} \bmod (2^\lambda - 1)}$ 
10:   $v_2 := (\text{BitsToField}(\mathbf{L}_0 \cdot \text{FieldToBits}(k; \lambda); \lambda) + c_2)^{2^{60} - 2^{30} + 1}$ 
11: else  $\triangleright \lambda = 512$ 
12:   $v_1 := (k + c_1)^{(2^{12} - 2^6 + 1)^{-1} \bmod (2^\lambda - 1)}$ 
13:   $v_2 := (\text{BitsToField}(\mathbf{L}_0 \cdot \text{FieldToBits}(k; \lambda); \lambda) + c_2)^{2^{180} - 2^{90} + 1}$ 
14: end if
15: return  $(v_1, v_2)$ 

```

5.5.3 Lynx Output Evaluation

The output-evaluation algorithm computes the public OWF output $t \in \mathbb{F}_{2^\lambda}$ from the extended witness (k, v_1, v_2) . It first applies the public linear layers $\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3$ to v_1, v_2 , and $k + v_1 + v_2$, respectively, using the same `FieldToBits` and `BitsToField` convention as in witness extension. For $\lambda = 160$, the output is the product of the first two transformed values plus the third transformed value. For $\lambda \in \{256, 384, 512\}$, the design additionally multiplies by the inversion of $k + c_3$, again written explicitly as exponentiation by $2^\lambda - 2$.

Lynx.EvalOutput $(\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3, c_1, c_2, c_3, k, v_1, v_2)$

Input: $\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3 \in \mathbb{F}_2^{\lambda \times \lambda}, c_1, c_2, c_3, k, v_1, v_2 \in \mathbb{F}_{2^\lambda}$ $\triangleright c_3$ is only required if $\lambda \in \{256, 384, 512\}$

Output: $t \in \mathbb{F}_{2^\lambda}$

```

1: Extract  $\lambda$  from param
2:  $w_1 := \text{BitsToField}(\mathbf{L}_1 \cdot \text{FieldToBits}(v_1; \lambda); \lambda)$ 
3:  $w_2 := \text{BitsToField}(\mathbf{L}_2 \cdot \text{FieldToBits}(v_2; \lambda); \lambda)$ 
4:  $w_3 := \text{BitsToField}(\mathbf{L}_3 \cdot \text{FieldToBits}(k + v_1 + v_2; \lambda); \lambda)$ 
5: if  $\lambda = 160$  then
6:    $t := w_1 \cdot w_2 + w_3$ 
7: else  $\triangleright \lambda \in \{256, 384, 512\}$ 
8:    $t := w_1 \cdot w_2 \cdot (k + c_3)^{2^\lambda - 2} + w_3$ 
9: end if
10: return  $t$ 

```

5.6 QuickSilver Proof System

This subsection instantiates the QuickSilver proof layer for the Lynx witness constraints. For Lynx, the extended witness is $\mathbf{w} = [k, v_1, v_2]$ with $\ell_{\text{wit}} = 3\lambda$. Lynx is designed so that given the bit-representations k, v_1, v_2 and public parameters $\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3, c_1, c_2, c_3, t$, their relations can be verified using three degree-2 polynomials. Therefore, the total number of QuickSilver constraints is three.

For the sake of conciseness, we use the IT-MAC notations in [QS.LynxProve](#), [QS.LynxVerify](#) functions. This implies that if the signing algorithm invokes this function, the operations are done over the signer/prover's IT-MAC tags; while if the verification algorithm invokes this function, the operations are done over the verifier's IT-MAC local keys.

As a slight abuse of notation for a matrix $\mathbf{L} \in \mathbb{F}_2^{\lambda \times \lambda}$ and $v \in \mathbb{F}_{2^\lambda}$, we use the notation $L(v) \in \mathbb{F}_{2^\lambda}$ such that $L(v) = \text{BitsToField}(\mathbf{L} \cdot \text{FieldToBits}(v; \lambda); \lambda)$. It contains the following algorithms.

- [QS.LynxProve](#): computes the prover-side QuickSilver response from the IT-MAC tags of the committed Lynx witness.
- [QS.LynxVerify](#): computes the verifier-side QuickSilver value from the IT-MAC local keys and the masked witness.

5.6.1 QuickSilver Proving

The proving algorithm starts from the committed extended witness $\mathbf{w}$ and the prover-side VOLE values $(\mathbf{u}, \mathbf{V})$. Each row of $\mathbf{V}$ is parsed as an element of $\mathbb{F}_{2^\lambda}$, so that a witness bit and its corresponding tag define a degree-1 IT-MAC polynomial. The algorithm then evaluates the Lynx constraint polynomials on these IT-MACs. Since the constraints have degree at most two and are satisfied by a valid witness, each of the three resulting IT-MACs has degree 2 and has zero degree-2 coefficient. Therefore, the prover only tracks its degree-1 and degree-0 coefficients. The vectors $\mathbf{a}_1$ and $\mathbf{a}_0$ collect these coefficients for the three Lynx OWF constraints.

Finally, the last λ VOLE rows are aggregated into one field element. They provide the random zero IT-MAC used by [ZKHash](#) to batch and mask all QuickSilver checks. The output (a_1, a_0) is the prover-side opening information for the batched zero-checking, with the degree-2 coefficient fixed to zero.

[QS.LynxProve](#)($\mathbf{w}, \mathbf{u}, \mathbf{V}, \mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3, c_1, c_2, c_3, t, \text{chall}_2; \text{param}$)

Input: $\left(\begin{array}{l} \mathbf{w} \in \mathbb{F}_2^{\ell_{\text{wit}}}, \mathbf{u} \in \mathbb{F}_2^{\ell_{\text{wit}} + \lambda}, \mathbf{V} \in \mathbb{F}_2^{(\ell_{\text{wit}} + \lambda) \times \lambda}, \\ \mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3 \in \mathbb{F}_2^{\lambda \times \lambda}, \\ c_1, c_2, c_3, t \in \mathbb{F}_{2^\lambda}, \text{chall}_2 \in \{0, 1\}^{3\lambda + 64} \end{array} \right)$

Output: $(a_1 \in \mathbb{F}_{2^\lambda}, a_0 \in \mathbb{F}_{2^\lambda})$

- 1: Extract $\lambda, \ell_{\text{wit}}$ from **param**
- 2: **for** $i \in [0, 3\lambda)$ **do** $v[i] := \text{BitsToField}(\mathbf{V}[i, :]; \lambda)$
- 3: **end for**
- 4: Let $\mathbf{w}[0, 3\lambda), \mathbf{v}[0, \ell_{\text{wit}})$ define degree-1 IT-MACs $\llbracket \mathbf{w} \rrbracket^1$
- 5: Parse $\llbracket \mathbf{w} \rrbracket^1 := [\llbracket \mathbf{k} \rrbracket^1, \llbracket \mathbf{v}_1 \rrbracket^1, \llbracket \mathbf{v}_2 \rrbracket^1]$ with $\mathbf{k}, \mathbf{v}_1, \mathbf{v}_2 \in \mathbb{F}_2^\lambda$
- 6: Initialize $\mathbf{a}_1, \mathbf{a}_0 \in \mathbb{F}_{2^\lambda}^3$.
- 7: **if** $\lambda = 160$ **then**
- 8: $\llbracket v_1 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket v_1[i] \rrbracket^1 \cdot X^i, \llbracket k \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket k[i] \rrbracket^1 \cdot X^i$
- 9: $\llbracket z_1 \rrbracket^2 := \llbracket v_1 \rrbracket^1 \cdot (\llbracket k \rrbracket^1 + c_1) + 1$
- 10: Parse $f_{z_1}(x) := z_{1,1} \cdot x + z_{1,0}$
- 11: $\mathbf{a}_1[0] := z_{1,1}, \mathbf{a}_0[0] := z_{1,0}$
- 12: $\llbracket L_0(k) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$
- 13: $\llbracket v_2 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket v_2[i] \rrbracket^1 \cdot X^i$
- 14: $\llbracket L_0(k)^{2^{67}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{67} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$
- 15: $\llbracket z_2 \rrbracket^2 := (\llbracket L_0(k) \rrbracket^1 + c_2) \cdot \llbracket v_2 \rrbracket^1 + \llbracket L_0(k)^{2^{67}} \rrbracket^1 + c_2^{2^{67}}$
- 16: Parse $f_{z_2}(x) := z_{2,1} \cdot x + z_{2,0}$
- 17: $\mathbf{a}_1[1] := z_{2,1}, \mathbf{a}_0[1] := z_{2,0}$

```

18:  $\llbracket L_1(v_1) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_1 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
19:  $\llbracket L_2(v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_2 \cdot \llbracket \mathbf{v}_2 \rrbracket^1)[i] \cdot X^i$ 
20:  $\llbracket L_3(k + v_1 + v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_3 \cdot (\llbracket \mathbf{k} \rrbracket^1 + \llbracket \mathbf{v}_1 \rrbracket^1 + \llbracket \mathbf{v}_2 \rrbracket^1))[i] \cdot X^i$ 
21:  $\llbracket z_3 \rrbracket^2 := \llbracket L_1(v_1) \rrbracket^1 \cdot \llbracket L_2(v_2) \rrbracket^1 + \llbracket L_3(k + v_1 + v_2) \rrbracket^1 + t$ 
22:  $\text{Parse } f_{z_3}(x) := z_{3,1} \cdot x + z_{3,0}$ 
23:  $\mathbf{a}_1[2] := z_{3,1}, \mathbf{a}_0[2] := z_{3,0}$ 
24: else if  $\lambda = 256$  then
25:  $\llbracket v_1 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_1[i] \rrbracket^1 \cdot X^i, \llbracket k \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{k}[i] \rrbracket^1 \cdot X^i$ 
26:  $\llbracket v_1^{25} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_5 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
27:  $\llbracket z_1 \rrbracket^2 := \llbracket v_1 \rrbracket^1 \cdot (\llbracket k \rrbracket^1 + c_1) + \llbracket v_1^{25} \rrbracket^1$ 
28:  $\text{Parse } f_{z_1}(x) := z_{1,1} \cdot x + z_{1,0}$ 
29:  $\mathbf{a}_1[0] := z_{1,1}, \mathbf{a}_0[0] := z_{1,0}$ 
30:  $\llbracket L_0(k) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
31:  $\llbracket v_2 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_2[i] \rrbracket^1 \cdot X^i$ 
32:  $\llbracket L_0(k)^{253} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{53} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
33:  $\llbracket z_2 \rrbracket^2 := \llbracket L_0(k)^{253} \rrbracket^1 + c_2^{253} + (\llbracket L_0(k) \rrbracket^1 + c_2) \cdot \llbracket v_2 \rrbracket^1$ 
34:  $\text{Parse } f_{z_2}(x) := z_{2,1} \cdot x + z_{2,0}$ 
35:  $\mathbf{a}_1[1] := z_{2,1}, \mathbf{a}_0[1] := z_{2,0}$ 
36:  $\llbracket L_1(v_1) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_1 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
37:  $\llbracket L_2(v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_2 \cdot \llbracket \mathbf{v}_2 \rrbracket^1)[i] \cdot X^i$ 
38:  $\llbracket L_3(k + v_1 + v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_3 \cdot (\llbracket \mathbf{k} \rrbracket^1 + \llbracket \mathbf{v}_1 \rrbracket^1 + \llbracket \mathbf{v}_2 \rrbracket^1))[i] \cdot X^i$ 
39:  $\llbracket z_3 \rrbracket^2 := \llbracket L_1(v_1) \rrbracket^1 \cdot \llbracket L_2(v_2) \rrbracket^1 + (\llbracket L_3(k + v_1 + v_2) \rrbracket^1 + t) \cdot (\llbracket k \rrbracket^1 + c_3)$ 
40:  $\text{Parse } f_{z_3}(x) := z_{3,1} \cdot x + z_{3,0}$ 
41:  $\mathbf{a}_1[2] := z_{3,1}, \mathbf{a}_0[2] := z_{3,0}$ 
42: else if  $\lambda = 384$  then
43:  $\llbracket v_1 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_1[i] \rrbracket^1 \cdot X^i, \llbracket k \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{k}[i] \rrbracket^1 \cdot X^i$ 
44:  $\llbracket v_1^{213} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{13} \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
45:  $\llbracket z_1 \rrbracket^2 := \llbracket v_1 \rrbracket^1 \cdot (\llbracket k \rrbracket^1 + c_1) + \llbracket v_1^{213} \rrbracket^1$ 
46:  $\text{Parse } f_{z_1}(x) := z_{1,1} \cdot x + z_{1,0}$ 
47:  $\mathbf{a}_1[0] := z_{1,1}, \mathbf{a}_0[0] := z_{1,0}$ 
48:  $\llbracket L_0(k) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
49:  $\llbracket v_2 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_2[i] \rrbracket^1 \cdot X^i$ 
50:  $\llbracket L_0(k)^{260} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{60} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
51:  $\llbracket L_0(k)^{230} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{30} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
52:  $\llbracket z_2 \rrbracket^2 := (\llbracket L_0(k)^{260} \rrbracket^1 + c_2^{260}) \cdot (\llbracket L_0(k) \rrbracket^1 + c_2) + (\llbracket L_0(k)^{230} \rrbracket^1 + c_2^{230}) \cdot \llbracket v_2 \rrbracket^1$ 
53:  $\text{Parse } f_{z_2}(x) := z_{2,1} \cdot x + z_{2,0}$ 
54:  $\mathbf{a}_1[1] := z_{2,1}, \mathbf{a}_0[1] := z_{2,0}$ 
55:  $\llbracket L_1(v_1) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_1 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
56:  $\llbracket L_2(v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_2 \cdot \llbracket \mathbf{v}_2 \rrbracket^1)[i] \cdot X^i$ 
57:  $\llbracket L_3(k + v_1 + v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_3 \cdot (\llbracket \mathbf{k} \rrbracket^1 + \llbracket \mathbf{v}_1 \rrbracket^1 + \llbracket \mathbf{v}_2 \rrbracket^1))[i] \cdot X^i$ 
58:  $\llbracket z_3 \rrbracket^2 := \llbracket L_1(v_1) \rrbracket^1 \cdot \llbracket L_2(v_2) \rrbracket^1 + (\llbracket L_3(k + v_1 + v_2) \rrbracket^1 + t) \cdot (\llbracket k \rrbracket^1 + c_3)$ 
59:  $\text{Parse } f_{z_3}(x) := z_{3,1} \cdot x + z_{3,0}$ 
60:  $\mathbf{a}_1[2] := z_{3,1}, \mathbf{a}_0[2] := z_{3,0}$ 
61: else
62:  $\llbracket v_1 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_1[i] \rrbracket^1 \cdot X^i, \llbracket k \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{k}[i] \rrbracket^1 \cdot X^i$ 
63:  $\llbracket v_1^{26} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_6 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
64:  $\llbracket v_1^{212} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{12} \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 

```

```

65:  $\llbracket z_1 \rrbracket^2 := \llbracket v_1^{2^6} \rrbracket^1 \cdot (\llbracket k \rrbracket^1 + c_1) + \llbracket v_1^{2^{12}} \rrbracket^1 \cdot \llbracket v_1 \rrbracket^1$ 
66:  $\text{Parse } f_{z_1}(x) := z_{1,1} \cdot x + z_{1,0}$ 
67:  $\mathbf{a}_1[0] := z_{1,1}, \mathbf{a}_0[0] := z_{1,0}$ 
68:  $\llbracket L_0(k) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_0 \cdot \llbracket k \rrbracket^1)[i] \cdot X^i$ 
69:  $\llbracket v_2 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket v_2[i] \rrbracket^1 \cdot X^i$ 
70:  $\llbracket L_0(k)^{2^{180}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{180} \cdot \mathbf{L}_0 \cdot \llbracket k \rrbracket^1)[i] \cdot X^i$ 
71:  $\llbracket L_0(k)^{2^{90}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{90} \cdot \mathbf{L}_0 \cdot \llbracket k \rrbracket^1)[i] \cdot X^i$ 
72:  $\llbracket z_2 \rrbracket^2 := (\llbracket L_0(k)^{2^{180}} \rrbracket^1 + c_2^{2^{180}}) \cdot (\llbracket L_0(k) \rrbracket^1 + c_2) + (\llbracket L_0(k)^{2^{90}} \rrbracket^1 + c_2^{2^{90}}) \cdot \llbracket v_2 \rrbracket^1$ 
73:  $\text{Parse } f_{z_2}(x) := z_{2,1} \cdot x + z_{2,0}$ 
74:  $\mathbf{a}_1[1] := z_{2,1}, \mathbf{a}_0[1] := z_{2,0}$ 
75:  $\llbracket L_1(v_1) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_1 \cdot \llbracket v_1 \rrbracket^1)[i] \cdot X^i$ 
76:  $\llbracket L_2(v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_2 \cdot \llbracket v_2 \rrbracket^1)[i] \cdot X^i$ 
77:  $\llbracket L_3(k + v_1 + v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_3 \cdot (\llbracket k \rrbracket^1 + \llbracket v_1 \rrbracket^1 + \llbracket v_2 \rrbracket^1))[i] \cdot X^i$ 
78:  $\llbracket z_3 \rrbracket^2 := \llbracket L_1(v_1) \rrbracket^1 \cdot \llbracket L_2(v_2) \rrbracket^1 + (\llbracket L_3(k + v_1 + v_2) \rrbracket^1 + t) \cdot (\llbracket k \rrbracket^1 + c_3)$ 
79:  $\text{Parse } f_{z_3}(x) := z_{3,1} \cdot x + z_{3,0}$ 
80:  $\mathbf{a}_1[2] := z_{3,1}, \mathbf{a}_0[2] := z_{3,0}$ 
81: end if
82:  $\tilde{u} := \text{BitsToField}(\mathbf{u}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda]; \lambda)$ 
83: for  $i \in [0, \lambda)$  do
84:    $\mathbf{v}[i] := \text{BitsToField}(\mathbf{V}[\ell_{\text{wit}} + i, :]; \lambda)$ 
85: end for
86:  $\tilde{v} := \sum_{i=0}^{\lambda-1} \mathbf{v}[i] X^i$   $\triangleright X$  is the generator of  $\mathbb{F}_{2^\lambda}$ .
87:  $a_1 := \text{ZKHash}(\text{chall}_2, [\mathbf{a}_1, \tilde{u}])$ 
88:  $a_0 := \text{ZKHash}(\text{chall}_2, [\mathbf{a}_0, \tilde{v}])$ 
89: return  $(a_1, a_0)$ 

```

5.6.2 QuickSilver Verification

The verification algorithm is the verifier-side analogue of [QS.LynxProve](#). It first applies [VOLE.Fix](#) to the verifier's local key matrix using the masked witness $\mathbf{d}$ and the global IT-MAC key derived from chall_3 . This converts local keys for the random committed vector into local keys for the actual witness. The rows are then parsed as $\mathbb{F}_{2^\lambda}$ elements and interpreted as degree-1 IT-MAC local-keys for $\mathbf{k}, \mathbf{v}_1, \mathbf{v}_2$.

The verifier evaluates the same three Lynx constraint expressions as in the prover, with the corresponding IT-MAC local keys collected in $\mathbf{b}$. The last λ rows of $\mathbf{Q}$ provide the local key for the random zero IT-MAC in QuickSilver. After hashing these IT-MACs with [ZKHash](#), the verifier recomputes the a_0 value from the purported zero degree-2 coefficient, the received degree-1 coefficient $\tilde{a}_1$, and the polynomials's evaluation result b at the global key Δ . The reconstructed a_0 value is later checked by the third Fiat-Shamir hash.

[QS.LynxVerify](#)($\mathbf{d}, \mathbf{Q}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3, c_1, c_2, [c_3], t; \text{param}$)

Input: $\left(\begin{array}{l} \mathbf{d} \in \mathbb{F}_2^{\ell_{\text{wit}}}, \mathbf{Q} \in \mathbb{F}_2^{(\ell_{\text{wit}} + \lambda) \times \lambda}, \text{chall}_2 \in \{0, 1\}^{3\lambda + 64}, \text{chall}_3 \in \{0, 1\}^\lambda, \\ \tilde{a}_1 \in \mathbb{F}_{2^\lambda}, \mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3 \in \mathbb{F}_2^{\lambda \times \lambda}, \\ c_1, c_2, [c_3], t \in \mathbb{F}_{2^\lambda} \end{array} \right)$

Output: $a_0 \in \mathbb{F}_{2^\lambda}$

- 1: Extract $\lambda, \ell_{\text{wit}}$ from param
- 2: $\Delta := \text{BitsToField}(\text{chall}_3; \lambda)$

3: Parse $\mathbf{Q} := \begin{bmatrix} \mathbf{Q}_0 \\ \mathbf{Q}_1 \end{bmatrix}$ $\triangleright \mathbf{Q}_0$ has ℓ_{wit} rows and $\mathbf{Q}_1$ has λ rows.

4: $\hat{\mathbf{Q}}_0 := \text{VOLE.Fix}(\mathbf{Q}_0, \mathbf{d}, \Delta; \lambda, \ell_{\text{wit}})$

5: $\hat{\mathbf{Q}} := \begin{bmatrix} \hat{\mathbf{Q}}_0 \\ \mathbf{Q}_1 \end{bmatrix}$

6: **for** $i \in [0, 3\lambda)$ **do**

7: $\mathbf{q}[i] := \text{BitsToField}(\hat{\mathbf{Q}}[i, :]; \lambda)$

8: **end for**

9: Let $\mathbf{q}[0, 3\lambda)$ define degree-1 IT-MACs $\llbracket \mathbf{w} \rrbracket^1$

10: Parse $\llbracket \mathbf{w} \rrbracket^1 := [\llbracket \mathbf{k} \rrbracket^1, \llbracket \mathbf{v}_1 \rrbracket^1, \llbracket \mathbf{v}_2 \rrbracket^1]$ with $\mathbf{k}, \mathbf{v}_1, \mathbf{v}_2 \in \mathbb{F}_2^\lambda$

11: Initialize $\mathbf{b} \in \mathbb{F}_{2^{3\lambda}}^3$.

12: **if** $\lambda = 160$ **then**

13: $\llbracket v_1 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_1[i] \rrbracket^1 \cdot X^i, \llbracket k \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{k}[i] \rrbracket^1 \cdot X^i$

14: $\llbracket z_1 \rrbracket^2 := \llbracket v_1 \rrbracket^1 \cdot (\llbracket k \rrbracket^1 + c_1) + 1$

15: Let $k_{z_1} \in \mathbb{F}_{2^\lambda}$ be the IT-MAC local key.

16: $\mathbf{b}[0] := k_{z_1}$

17: $\llbracket L_0(k) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$

18: $\llbracket v_2 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_2[i] \rrbracket^1 \cdot X^i$

19: $\llbracket L_0(k)^{2^{67}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{67} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$

20: $\llbracket z_2 \rrbracket^2 := (\llbracket L_0(k) \rrbracket^1 + c_2) \cdot \llbracket v_2 \rrbracket^1 + \llbracket L_0(k)^{2^{67}} \rrbracket^1 + c_2^{67}$

21: Let $k_{z_2} \in \mathbb{F}_{2^\lambda}$ be the IT-MAC local key.

22: $\mathbf{b}[1] := k_{z_2}$

23: $\llbracket L_1(v_1) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_1 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$

24: $\llbracket L_2(v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_2 \cdot \llbracket \mathbf{v}_2 \rrbracket^1)[i] \cdot X^i$

25: $\llbracket L_3(k + v_1 + v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_3 \cdot (\llbracket \mathbf{k} \rrbracket^1 + \llbracket \mathbf{v}_1 \rrbracket^1 + \llbracket \mathbf{v}_2 \rrbracket^1))[i] \cdot X^i$

26: $\llbracket z_3 \rrbracket^2 := \llbracket L_1(v_1) \rrbracket^1 \cdot \llbracket L_2(v_2) \rrbracket^1 + \llbracket L_3(k + v_1 + v_2) \rrbracket^1 + t$

27: Let $k_{z_3} \in \mathbb{F}_{2^\lambda}$ be the IT-MAC local key.

28: $\mathbf{b}[2] := k_{z_3}$

29: **else if** $\lambda = 256$ **then**

30: $\llbracket v_1 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_1[i] \rrbracket^1 \cdot X^i, \llbracket k \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{k}[i] \rrbracket^1 \cdot X^i$

31: $\llbracket v_1^{2^5} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_5 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$

32: $\llbracket z_1 \rrbracket^2 := \llbracket v_1 \rrbracket^1 \cdot (\llbracket k \rrbracket^1 + c_1) + \llbracket v_1^{2^5} \rrbracket^1$

33: Let $k_{z_1} \in \mathbb{F}_{2^\lambda}$ be the IT-MAC local key.

34: $\mathbf{b}[0] := k_{z_1}$

35: $\llbracket L_0(k) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$

36: $\llbracket v_2 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_2[i] \rrbracket^1 \cdot X^i$

37: $\llbracket L_0(k)^{2^{53}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{53} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$

38: $\llbracket z_2 \rrbracket^2 := \llbracket L_0(k)^{2^{53}} \rrbracket^1 + c_2^{2^{53}} + (\llbracket L_0(k) \rrbracket^1 + c_2) \cdot \llbracket v_2 \rrbracket^1$

39: Let $k_{z_2} \in \mathbb{F}_{2^\lambda}$ be the IT-MAC local key.

40: $\mathbf{b}[1] := k_{z_2}$

41: $\llbracket L_1(v_1) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_1 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$

42: $\llbracket L_2(v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_2 \cdot \llbracket \mathbf{v}_2 \rrbracket^1)[i] \cdot X^i$

43: $\llbracket L_3(k + v_1 + v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_3 \cdot (\llbracket \mathbf{k} \rrbracket^1 + \llbracket \mathbf{v}_1 \rrbracket^1 + \llbracket \mathbf{v}_2 \rrbracket^1))[i] \cdot X^i$

44: $\llbracket z_3 \rrbracket^2 := \llbracket L_1(v_1) \rrbracket^1 \cdot \llbracket L_2(v_2) \rrbracket^1 + (\llbracket L_3(k + v_1 + v_2) \rrbracket^1 + t) \cdot (\llbracket k \rrbracket^1 + c_3)$

45: Let $k_{z_3} \in \mathbb{F}_{2^\lambda}$ be the IT-MAC local key.

46: $\mathbf{b}[2] := k_{z_3}$

47: **else if** $\lambda = 384$ **then**

48: $\llbracket v_1 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_1[i] \rrbracket^1 \cdot X^i, \llbracket k \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{k}[i] \rrbracket^1 \cdot X^i$

```

49:  $\llbracket v_1^{2^{13}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{13} \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
50:  $\llbracket z_1 \rrbracket^2 := \llbracket v_1 \rrbracket^1 \cdot (\llbracket k \rrbracket^1 + c_1) + \llbracket v_1^{2^{13}} \rrbracket^1$ 
51: Let  $k_{z_1} \in \mathbb{F}_{2^\lambda}$  be the IT-MAC local key.
52:  $\mathbf{b}[0] := k_{z_1}$ 
53:  $\llbracket L_0(k) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
54:  $\llbracket v_2 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_2[i] \rrbracket^1 \cdot X^i$ 
55:  $\llbracket L_0(k)^{2^{60}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{60} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
56:  $\llbracket L_0(k)^{2^{30}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{30} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
57:  $\llbracket z_2 \rrbracket^2 := (\llbracket L_0(k)^{2^{60}} \rrbracket^1 + c_2^{2^{60}}) \cdot (\llbracket L_0(k) \rrbracket^1 + c_2) + (\llbracket L_0(k)^{2^{30}} \rrbracket^1 + c_2^{2^{30}}) \cdot \llbracket v_2 \rrbracket^1$ 
58: Let  $k_{z_2} \in \mathbb{F}_{2^\lambda}$  be the IT-MAC local key.
59:  $\mathbf{b}[1] := k_{z_2}$ 
60:  $\llbracket L_1(v_1) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_1 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
61:  $\llbracket L_2(v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_2 \cdot \llbracket \mathbf{v}_2 \rrbracket^1)[i] \cdot X^i$ 
62:  $\llbracket L_3(k + v_1 + v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_3 \cdot (\llbracket \mathbf{k} \rrbracket^1 + \llbracket \mathbf{v}_1 \rrbracket^1 + \llbracket \mathbf{v}_2 \rrbracket^1))[i] \cdot X^i$ 
63:  $\llbracket z_3 \rrbracket^2 := \llbracket L_1(v_1) \rrbracket^1 \cdot \llbracket L_2(v_2) \rrbracket^1 + (\llbracket L_3(k + v_1 + v_2) \rrbracket^1 + t) \cdot (\llbracket k \rrbracket^1 + c_3)$ 
64: Let  $k_{z_3} \in \mathbb{F}_{2^\lambda}$  be the IT-MAC local key.
65:  $\mathbf{b}[2] := k_{z_3}$ 
66: else
67:  $\llbracket v_1 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_1[i] \rrbracket^1 \cdot X^i$ ,  $\llbracket k \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{k}[i] \rrbracket^1 \cdot X^i$ 
68:  $\llbracket v_1^{2^6} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_6 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
69:  $\llbracket v_1^{2^{12}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{12} \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
70:  $\llbracket z_1 \rrbracket^2 := \llbracket v_1^{2^6} \rrbracket^1 \cdot (\llbracket k \rrbracket^1 + c_1) + \llbracket v_1^{2^{12}} \rrbracket^1 \cdot \llbracket v_1 \rrbracket^1$ 
71: Let  $k_{z_1} \in \mathbb{F}_{2^\lambda}$  be the IT-MAC local key.
72:  $\mathbf{b}[0] := k_{z_1}$ 
73:  $\llbracket L_0(k) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
74:  $\llbracket v_2 \rrbracket^1 := \sum_{i=0}^{\lambda-1} \llbracket \mathbf{v}_2[i] \rrbracket^1 \cdot X^i$ 
75:  $\llbracket L_0(k)^{2^{180}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{180} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
76:  $\llbracket L_0(k)^{2^{90}} \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{F}_{90} \cdot \mathbf{L}_0 \cdot \llbracket \mathbf{k} \rrbracket^1)[i] \cdot X^i$ 
77:  $\llbracket z_2 \rrbracket^2 := (\llbracket L_0(k)^{2^{180}} \rrbracket^1 + c_2^{2^{180}}) \cdot (\llbracket L_0(k) \rrbracket^1 + c_2) + (\llbracket L_0(k)^{2^{90}} \rrbracket^1 + c_2^{2^{90}}) \cdot \llbracket v_2 \rrbracket^1$ 
78: Let  $k_{z_2} \in \mathbb{F}_{2^\lambda}$  be the IT-MAC local key.
79:  $\mathbf{b}[1] := k_{z_2}$ 
80:  $\llbracket L_1(v_1) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_1 \cdot \llbracket \mathbf{v}_1 \rrbracket^1)[i] \cdot X^i$ 
81:  $\llbracket L_2(v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_2 \cdot \llbracket \mathbf{v}_2 \rrbracket^1)[i] \cdot X^i$ 
82:  $\llbracket L_3(k + v_1 + v_2) \rrbracket^1 := \sum_{i=0}^{\lambda-1} (\mathbf{L}_3 \cdot (\llbracket \mathbf{k} \rrbracket^1 + \llbracket \mathbf{v}_1 \rrbracket^1 + \llbracket \mathbf{v}_2 \rrbracket^1))[i] \cdot X^i$ 
83:  $\llbracket z_3 \rrbracket^2 := \llbracket L_1(v_1) \rrbracket^1 \cdot \llbracket L_2(v_2) \rrbracket^1 + (\llbracket L_3(k + v_1 + v_2) \rrbracket^1 + t) \cdot (\llbracket k \rrbracket^1 + c_3)$ 
84: Let  $k_{z_3} \in \mathbb{F}_{2^\lambda}$  be the IT-MAC local key.
85:  $\mathbf{b}[2] := k_{z_3}$ 
86: end if
87: for  $i \in [0, \lambda)$  do
88:    $\mathbf{q}[i] := \text{BitsToField}(\mathbf{Q}_1[i, :]; \lambda)$ 
89: end for
90:  $\tilde{q} := \sum_{i=0}^{\lambda-1} \mathbf{q}[i] X^i$   $\triangleright X$  is the generator of  $\mathbb{F}_{2^\lambda}$ .
91:  $b := \text{ZKHash}(\text{chall}_2, [\mathbf{b}, \tilde{q}])$ 
92: return  $a_0 := b + \tilde{a}_1 \cdot \Delta$ 

```

5.7 Universal Hash Functions

This subsection defines the linear universal hash functions. Both hash functions batch IT-MAC/VOLE checks by compressing many authenticated values into a short challenge-dependent value with linear operations. The difference is the underlying linear space. **VOLEHash** is linear over $\mathbb{F}_2$ and is used for checking consistency of the VOLE correlations generated from the small VOLE instances. **ZKHash** is linear over $\mathbb{F}_{2^\lambda}$ and is used for batched zero-checking in QuickSilver. In both cases, the prover applies the hash to committed messages or prover-side tags, while the verifier applies the same linear map to local keys and then performs the corresponding IT-MAC opening check. It contains the following algorithms.

- **VOLEHash**: compresses VOLE vectors over $\mathbb{F}_2$ for the consistency check.
- **ZKHash**: compresses QuickSilver check vectors over $\mathbb{F}_{2^\lambda}$.

5.7.1 VOLE Linear Universal Hash

The **VOLEHash** algorithm batches checks on degree-1 IT-MACs over $\mathbb{F}_2$. After **VOLE.Commit**, the prover has τ small VOLE instances. After applying the published correction vectors, all instances should authenticate the same committed message $\mathbf{u}$. Rather than checking the full VOLE vectors and tag matrices, the signer computes the compressed values $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{V}}$ with **VOLEHash** and binds them into the Fiat-Shamir transcript. The verifier applies the same linear map to the verifier-side local keys and then uses **VOLE.Fix** with $\tilde{\mathbf{u}}$ and the IT-MAC global key defined by chall_3 . This reconstructs the compressed local-key value that should match the signer's compressed tag in the transcript.

The input is split into a long part $\mathbf{x}_0$ and a short part $\mathbf{x}_1$. The long part is padded to a multiple of λ bits. It is parsed once as a vector over $\mathbb{F}_{2^\lambda}$ and once as a vector over $\mathbb{F}_{2^{64}}$. The algorithm evaluates a polynomial hash in each representation. The 64-bit result is embedded back into $\mathbb{F}_{2^\lambda}$, and two random linear combinations produce $\lambda + \mathbf{B}$ output bits. Finally, the short vector $\mathbf{x}_1$ is XOR-ed into the output. This final XOR gives the hiding property since the last $\lambda + \mathbf{B}$ input bits are uniformly random and are not utilized elsewhere in the signature scheme. When **VOLEHash** is applied to a matrix, it is applied column-wise with the same seed.

VOLEHash(seed, $(\mathbf{x}_0, \mathbf{x}_1)$)

(**VOLEHash** is linear over $\mathbb{F}_2$)

Input: $\left(\begin{array}{l} \text{seed} := \left(\begin{array}{l} \mathbf{r}_0 \in \{0, 1\}^\lambda, \mathbf{r}_1 \in \{0, 1\}^\lambda, \mathbf{r}_2 \in \{0, 1\}^\lambda, \\ \mathbf{r}_3 \in \{0, 1\}^\lambda, \mathbf{s} \in \{0, 1\}^\lambda, \mathbf{t} \in \{0, 1\}^{64} \end{array} \right), \\ (\mathbf{x}_0 \in \mathbb{F}_2^{\ell_{\text{wit}} + \lambda}, \mathbf{x}_1 \in \mathbb{F}_2^{\lambda + \mathbf{B}}) \end{array} \right)$

Output: $\mathbf{h} \in \{0, 1\}^{\lambda + \mathbf{B}}$

- 1: **for** $i \in [0, 3]$ **do**
- 2: $r_i := \text{BitsToField}(\mathbf{r}_i; \lambda)$
- 3: **end for**
- 4: $s := \text{BitsToField}(\mathbf{s}; \lambda)$
- 5: $t := \text{BitsToField}(\mathbf{t}; 64)$
- 6: $\ell' := \lambda \cdot \lceil \frac{\ell_{\text{wit}} + \lambda}{\lambda} \rceil$
- 7: $\hat{\mathbf{y}} := \text{BitsToField}(\mathbf{x}_0; \lambda)$
- 8: $\bar{\mathbf{y}} := \text{BitsToField}(\mathbf{x}_0; 64)$
- 9: $h_0 := \sum_{i=0}^{\ell'/\lambda - 1} s^{\ell'/\lambda - 1 - i} \cdot \hat{\mathbf{y}}[i]$ $\triangleright$ over $\mathbb{F}_{2^\lambda}$.
- 10: $h_1 := \sum_{i=0}^{\ell'/64 - 1} t^{\ell'/64 - 1 - i} \cdot \bar{\mathbf{y}}[i]$ $\triangleright$ over $\mathbb{F}_{2^{64}}$.
- 11: $h'_1 := \text{BitsToField}(\text{FieldToBits}(h_1; 64) \parallel 0^{\lambda - 64}; \lambda)$
- 12: $(h_2, h_3) := (r_0 \cdot h_0 + r_1 \cdot h'_1, r_2 \cdot h_0 + r_3 \cdot h'_1)$

```

13:  $\mathbf{h} := (\text{FieldToBits}(h_2; \lambda) \parallel \text{FieldToBits}(h_3; \lambda)[0, \mathbf{B}]) \oplus \mathbf{x}_1$ 
14: return  $\mathbf{h}$ 

```

5.7.2 ZK Linear Universal Hash

The **ZKHash** algorithm batches checks on IT-MACs over $\mathbb{F}_{2^\lambda}$. In **QS.LynxProve**, it is applied to the IT-MAC coefficient vectors $\mathbf{a}_1$ and $\mathbf{a}_0$, together with the extra field elements $\tilde{u}$ and $\tilde{v}$ for masking. In **QS.LynxVerify**, it is applied to the corresponding verifier-side local-key vector $\mathbf{b}$ and the extra local-key $\tilde{q}$ for $\tilde{u}$. The result is the compressed value used in the final IT-MAC opening check. Thus **ZKHash** plays the same batching role as **VOLEHash**, but for $\mathbb{F}_{2^\lambda}$ -linear IT-MAC checks rather than $\mathbb{F}_2$ -linear VOLE consistency checks.

The seed is parsed into two random coefficients r_0, r_1 and two evaluation points s, t in $\mathbb{F}_{2^\lambda}$. The vector $\mathbf{x}_0$ is viewed as the coefficient vector of a polynomial over $\mathbb{F}_{2^\lambda}$ and evaluated twice, once at s and once at t . The two evaluations are then combined as $r_0 h_0 + r_1 h_1$. Finally, the field element x_1 is added. As in **VOLEHash**, adding x_1 provides hiding.

ZKHash(seed, $(\mathbf{x}_0, x_1)$) (**ZKHash** is linear over $\mathbb{F}_{2^\lambda}$)

Input: $\left(\text{seed} := (r_0 \in \{0, 1\}^\lambda, r_1 \in \{0, 1\}^\lambda, s \in \{0, 1\}^\lambda, t \in \{0, 1\}^{64}), \right.$
 $\left. (\mathbf{x}_0 \in \mathbb{F}_{2^\lambda}^{\ell_{\text{wit}}}, x_1 \in \mathbb{F}_{2^\lambda}) \right)$

Output: $\mathbf{h} \in \{0, 1\}^\lambda$

```

1:  $r_0 := \text{BitsToField}(r_0; \lambda)$ 
2:  $r_1 := \text{BitsToField}(r_1; \lambda)$ 
3:  $s := \text{BitsToField}(s; \lambda)$ 
4:  $t := \text{BitsToField}(t \parallel 0^{\lambda-64}; \lambda)$ 
5:  $h_0 := \sum_{i=0}^{\ell_{\text{wit}}-1} s^{\ell_{\text{wit}}-1-i} \cdot \mathbf{x}_0[i]$ 
6:  $h_1 := \sum_{i=0}^{\ell_{\text{wit}}-1} t^{\ell_{\text{wit}}-1-i} \cdot \mathbf{x}_0[i]$ 
7:  $\mathbf{h} := \text{FieldToBits}(r_0 \cdot h_0 + r_1 \cdot h_1 + x_1; \lambda)$ 
8: return  $\mathbf{h}$ 

```

5.8 Utility Functions

This subsection collects the basic conversion routines used throughout the rest of the section. Since the construction moves repeatedly between integers, bit strings, and finite-field elements, these helpers make the serialization and parsing conventions explicit and keep the interfaces of the higher-level algorithms concise.

We use three representations for data: integers, finite-field elements, and bit strings. All bit strings are indexed from left to right as $\mathbf{x}[0], \mathbf{x}[1], \dots$ but interpreted in little-endian order for arithmetic conversions. Thus $\mathbf{x}[0]$ is the coefficient of 2^0 when converting to an integer, and it is the coefficient of X^0 when converting to a field element. The parameter $k \in \mathbb{N}^+$ fixes the bit length and the target field $\mathbb{F}_{2^k}$. These conventions are used for challenge decoding, counters, domain-separation bytes, field encodings in Lynx, and the universal-hash parsing routines above. The subsection contains the following algorithms.

- **BitsToInt**: interprets a little-endian bit string as an integer modulo 2^k .
- **IntToBits**: serializes an integer modulo 2^k as a little-endian bit string.
- **BitsToField**: interprets a bit string as a polynomial-basis element of $\mathbb{F}_{2^k}$.
- **FieldToBits**: serializes a field element by listing its polynomial-basis coefficients.

5.8.1 Bits to Integer

The [BitsToInt](#) algorithm interprets a length- k bit string as a little-endian integer modulo 2^k . It is used whenever a challenge substring or hash output must be read as an index, such as the BAVC leaf index in [VOLE.ChalDec](#).

[BitsToInt](#)($x; k$)

Input: $x \in \{0, 1\}^k$

Output: $x \in \mathbb{Z}_{2^k}$

1: $x := \sum_{i=0}^{k-1} x[i] \cdot 2^i$
 2: **return** x

5.8.2 Integer to Bits

The [IntToBits](#) algorithm is the inverse serialization map for integers modulo 2^k . It outputs the little-endian bit decomposition used throughout the hash inputs, domain-separation bytes, counters, and public constants. The input is viewed modulo 2^k , so the output always has exactly k bits. This fixed-length convention keeps XOF inputs unambiguous.

[IntToBits](#)($x; k$)

Input: $x \in \mathbb{Z}_{2^k}$

Output: $x \in \{0, 1\}^k$

1: **for** $i \in [0, k)$ **do**
 2: $x[i] := (x \gg i) \pmod{2}$
 3: **end for**
 4: **return** x

5.8.3 Bits to Field

The [BitsToField](#) algorithm interprets a length- k bit string as the polynomial-basis representation of an element of $\mathbb{F}_{2^k}$. The bit $x[i]$ is the coefficient of X^i , where X denotes the fixed generator used by the implementation of $\mathbb{F}_{2^k}$. This conversion is used to turn rows of VOLE tag matrices and local-key matrices into $\mathbb{F}_{2^k}$ elements, to derive intermediate field elements in Lynx from their serialized representations, and to parse universal-hash seeds as field coefficients.

[BitsToField](#)($x; k$)

Input: $x \in \{0, 1\}^k$

Output: $x \in \mathbb{F}_{2^k}$

1: $x := \sum_{i=0}^{k-1} x[i] \cdot X^i$ $\triangleright X$ is the generator of $\mathbb{F}_{2^k}$.
 2: **return** x

5.8.4 Field to Bits

The [FieldToBits](#) algorithm serializes a field element by listing its coefficients in the same polynomial basis. It is the inverse of [BitsToField](#) under the fixed basis convention. All hash outputs and Lynx values that are computed as field elements are serialized with this routine before they are concatenated into transcripts, public keys, or signatures.

FieldToBits($x; k$)

Input: $x \in \mathbb{F}_{2^k}$

Output: $\mathbf{x} \in \{0, 1\}^k$

- 1: Parse $x = x_0 + x_1X + \dots + x_{k-1}X^{k-1}$
- 2: $\mathbf{x} := [x_0, x_1, \dots, x_{k-1}]$
- 3: **return** $\mathbf{x}$

5.9 Symmetric Primitives

This subsection specifies the symmetric primitives on which the scheme is built. It includes the XOF-based hash functions used to derive Fiat-Shamir challenges and commitments, the TCCR hash used in the pcGGM tree, the PRG used for seed expansion and leaf commitment, and the underlying standardized interfaces for the XOF and block cipher. It contains the following algorithms.

- **MsgHash**, **CoinHash**, **BAVCHash**, and **MatVecHash**: domain-separated XOF calls for message binding, signing randomness, BAVC commitments, and Lynx public-parameter expansion.
- **FSHash₁**, **FSHash₂**, and **FSHash₃**: the three Fiat-Shamir random-oracle calls used in the signing transcript.
- **TCCR**: the tweakable circular correlation robust hash used in the pcGGM tree of BAVC.
- **PRG**: the pseudorandom generator used for leaf commitments and VOLE seed expansion.
- **XOF** and **Enc**: standardized primitive interfaces used to instantiate the random oracles and ideal ciphers.

For a hash function, whenever the security proof of [Section 9](#) requires it to be a random oracle, we realize it using **XOF** with a distinct domain separator. In contrast, **TCCR** and **PRG** are based on block ciphers.

5.9.1 Message Hash

The message hash binds the public key to the signed message and also derives the salt $\mathbf{s}$ used by the TCCR hash function. Including the public key prevents key-substitution attacks.

MsgHash(pk, msg)

Input: $\text{msg} \in \{0, 1\}^*$, $\text{pk} := (\text{iv}_{\text{owf}} \in \{0, 1\}^\lambda, t \in \mathbb{F}_{2^\lambda})$

Output: $(\mu \in \{0, 1\}^{2^\lambda}, \mathbf{s} \in \{0, 1\}^\lambda)$

- 1: $(\mu, \mathbf{s}) := \text{XOF}(\text{IntToBits}(0; 8) \parallel \text{iv}_{\text{owf}} \parallel \text{FieldToBits}(t; \lambda) \parallel \text{msg}; 3\lambda) \triangleright \text{IntToBits}(0; 8)$ is a domain separator.
- 2: **return** $(\mu, \mathbf{s})$

5.9.2 Signing-Coin Hash

The signing-coin hash deterministically derives the BAVC root seed and the 120-bit IV from the secret key, the message digest, and the per-signature randomness ρ . Setting $\rho = 0^\lambda$ gives deterministic signing, while random ρ gives randomized signatures.

CoinHash(sk, μ , ρ)

Input: $\text{sk} \in \{0, 1\}^{2\lambda}, \mu \in \{0, 1\}^{2\lambda}, \rho \in \{0, 1\}^\lambda$

Output: $(r, \text{iv}_{\text{sig}}) \in \{0, 1\}^\lambda \times \{0, 1\}^{120}$

- 1: $r \parallel \text{iv}_{\text{sig}} := \text{XOF}(\text{IntToBits}(1; 8) \parallel \text{sk} \parallel \mu \parallel \rho; \lambda + 120) \triangleright \text{IntToBits}(1; 8)$ is a domain separator.
- 2: **return** $(r, \text{iv}_{\text{sig}})$ $\triangleright$ BAVC seed and salt.

5.9.3 BAVC Commitment Hash

The BAVC commitment hash compresses all leaf commitments into the single commitment value com used in the Fiat-Shamir transcript. It first hashes the leaves in each small tree and then hashes the resulting per-tree digests together with μ , s , and iv_{sig} .

BAVCHash($\mu, s, \{\text{com}_i^\alpha\}_{\alpha \in [0, \tau], i \in [0, N_\alpha]}, \text{iv}_{\text{sig}}$)

Input: $\mu \in \{0, 1\}^{2\lambda}, s \in \{0, 1\}^\lambda, \{\text{com}\}_{\alpha \in [0, \tau], i \in [0, N_\alpha]} \in \{0, 1\}^{2L\lambda}, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}$

Output: $\text{com} \in \{0, 1\}^{2\lambda}$

- 1: **for** $\alpha \in [0, \tau]$ **do**
- 2: $\text{com}^\alpha := \text{XOF}((\text{com}_i^\alpha)_{i \in [0, N_\alpha]}; 2\lambda)$
- 3: **end for**
- 4: $\text{com} := \text{XOF}(\text{IntToBits}(2; 8) \parallel \mu \parallel s \parallel (\text{com}^\alpha)_{\alpha \in [0, \tau]} \parallel \text{iv}_{\text{sig}}; 2\lambda) \triangleright \text{IntToBits}(2; 8)$ is a domain separator.
- 5: **return** com

5.9.4 Matrix-and-Vector Expansion Hash

The matrix-and-vector hash expands the public-key seed into the public Lynx matrices and constants. Its output is parsed by `Lynx.SampleMatVec` into triangular matrices and field elements.

MatVecHash($\text{iv}_{\text{owf}}; \ell$)

Input: $\text{iv}_{\text{owf}} \in \{0, 1\}^\lambda$

Output: $h \in \mathbb{F}_2^\ell$

- 1: $h := \text{XOF}(\text{IntToBits}(3; 8) \parallel \text{iv}_{\text{owf}}; \ell) \triangleright \text{IntToBits}(3; 8)$ is a domain separator.
- 2: **return** h

5.9.5 First Fiat-Shamir Hash

The first Fiat-Shamir hash derives chall_1 from the BAVC commitment and the correction vectors. Its output serves as a seed for the VOLE consistency hash.

FSHash₁(x)

Input: $x \in \{0, 1\}^*$

Output: $h \in \{0, 1\}^{5\lambda+64}$

- 1: $h := \text{XOF}(\text{IntToBits}(8 + 1; 8) \parallel x; 5\lambda + 64) \triangleright \text{IntToBits}(8 + 1; 8)$ is a domain separator.
- 2: **return** h

5.9.6 Second Fiat-Shamir Hash

The second Fiat-Shamir hash derives chall_2 after the VOLE consistency values and masked witness have been fixed. This challenge seeds the QuickSilver batching hashes.

FSHash₂(x)

Input: $x \in \{0, 1\}^*$

Output: $h \in \{0, 1\}^{3\lambda+64}$

- 1: $h := \text{XOF}(\text{IntToBits}(8 + 2; 8) \| x; 3\lambda + 64)$ $\triangleright \text{IntToBits}(8 + 2; 8)$ is a domain separator.
- 2: **return** h

5.9.7 Third Fiat-Shamir Hash

The third Fiat-Shamir hash derives the final challenge chall_3 from the QuickSilver response and the grinding counter. The signer repeats this call until the proof-of-work suffix condition and the BAVC opening bound are both satisfied.

FSHash₃(x)

Input: $x \in \{0, 1\}^*$

Output: $h \in \{0, 1\}^\lambda$

- 1: $h := \text{XOF}(\text{IntToBits}(8 + 3; 8) \| x; \lambda)$ $\triangleright \text{IntToBits}(8 + 3; 8)$ is a domain separator.
- 2: **return** h

5.9.8 TCCR Hash Function

We define the following Tweakable Circular Correlation Robust (TCCR) hash function with 120-bit salt. The TCCR design is based on the block-cipher interface [Enc](#) with n_k -bit key and n_b -bit block size, satisfying $n_k \geq \lambda$, $\lceil \frac{\lambda}{n_b} \rceil \leq 2^8$, and $n_b \geq 128$. The [TCCR](#) algorithm hashes a λ -bit value with a salt-derived tweak and a public mask s . For $\lambda \leq n_b$, one block-cipher call suffices; for larger parameters, the construction derives a fixed key from the right part of the masked input and emits enough block-cipher outputs to cover λ bits.

TCCR($x, s, \text{iv}_{\text{sig}}$)

Input: ($x \in \{0, 1\}^\lambda, s \in \{0, 1\}^\lambda, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}$)

Output: $h \in \{0, 1\}^\lambda$

- 1: $\bar{x} := x \oplus s$
- 2: **if** $\lambda \leq n_b$ **then**
- 3: **return** $h := \text{Enc}(\text{iv}_{\text{sig}} \| 0^{n_k-120}, \sigma(\bar{x} \| 0^{n_b-\lambda}); \lambda) \oplus \sigma(\bar{x} \| 0^{n_b-\lambda})$
- 4: **end if**
- 5: $x_L := \bar{x}[0, n_b - 8)$
- 6: $x_R := \bar{x}[n_b - 8, \lambda)$
- 7: $N_{\text{block}} := \lceil \frac{\lambda}{n_b} \rceil$
- 8: **for** $i \in [0, N_{\text{block}})$ **do**
- 9: $\bar{i} := \text{IntToBits}(i; 8)$
- 10: $h_i := \text{Enc}(x_R \| \text{iv}_{\text{sig}} \| 0^{n_k+n_b-128-\lambda}, \bar{i} \| x_L; \lambda) \oplus (\bar{i} \| x_L)$
- 11: **end for**
- 12: **return** $h := h_0 \| \dots \| h_{N_{\text{block}}-1} \| 0, \lambda - n_b \cdot (N_{\text{block}} - 1)$

5.9.9 Pseudorandom Generator

Based on the off-the-shelf block cipher construction, we follow different strategies for building **PRG** under different security parameters. In particular, for $\lambda = 160, 256, 512$, there are suitable block ciphers with sufficient key lengths, namely AES-192, AES-256, and SHACAL-2. Therefore, the regular counter mode cipher is sufficient. For $\lambda = 384$, we re-use the existing **TCCR** infrastructure.

As for the counter range, since **PRG** is only called in **VOLE.ConvertToVOLE** and **LeafHash**, and in both cases the total output length for a fixed seed and iv_{sig} satisfies $\ell_{\text{vole}} + 2\lambda < 2^8 \cdot n_b$, using an 8-bit counter in counter mode is sufficient. The **PRG** algorithm therefore exposes a uniform interface over all parameter sets while switching internally between counter-mode encryption and the TCCR-based construction for $\lambda = 384$.

PRG($\mathbf{k}, \text{iv}_{\text{sig}}, t; \ell$)

Input: ($\mathbf{k} \in \{0, 1\}^\lambda, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}, t \in \mathbb{Z}_{2^8}; \ell \in \mathbb{N}^+$)

Output: $\mathbf{r} \in \{0, 1\}^\ell$

```

1: if  $\lambda \in \{160, 256, 512\}$  then
2:    $\bar{\mathbf{k}} := \mathbf{k} \| 0^{n_k - \lambda}$  ▷ Zero-pad the seed to  $n_k$  bits.
3:    $c := \lceil \frac{\ell}{n_b} \rceil$ 
4:   for  $i \in [0, c)$  do
5:      $\mathbf{m} := \text{iv}_{\text{sig}} \| \text{IntToBits}(t + i; 8) \| 0^{n_b - 128}$  ▷ Zero-pad  $\mathbf{m}$  to  $n_b$  bits.
6:      $\mathbf{r}_i := \text{Enc}(\bar{\mathbf{k}}, \mathbf{m}; \lambda)$ 
7:   end for
8:   return  $\mathbf{r} := \mathbf{r}_0 \| \mathbf{r}_1 \| \dots \| \mathbf{r}_{c-1} [0, \ell - (c - 1) \cdot n_b]$ 
9: else ▷  $\lambda = 384$ 
10:   $c := \lceil \frac{\ell}{384} \rceil$ 
11:  for  $i \in [0, c)$  do
12:     $\mathbf{s} := \text{IntToBits}(t + i; 8) \| 0^{\lambda - 8}$ 
13:     $\mathbf{r}_i := \text{TCCR}(\mathbf{k}, \mathbf{s}, \text{iv}_{\text{sig}})$ 
14:  end for
15:  return  $\mathbf{r} := \mathbf{r}_0 \| \mathbf{r}_1 \| \dots \| \mathbf{r}_{c-1} [0, \ell - (c - 1) \cdot \lambda]$ 
16: end if
```

5.9.10 Extendable-Output Function

The **XOF** algorithm is the common random-oracle interface used by all domain-separated hash calls. It is modeled as a random oracle in the security analysis. We provide two versions of this interface. The first one is the SM3-based “pseudoXOF” function provided by the contest, and the second one is a standardized Keccak-based instantiation. The concrete choice for each security parameter is given in [Table 6](#).

XOF($\mathbf{x}; \ell$)

Input: $\mathbf{x} \in \{0, 1\}^*$

Output: $\mathbf{h} \in \{0, 1\}^\ell$

```

1:  $\mathbf{h} := \text{PseudoXOF}(\mathbf{x}; \ell)$  or  $\mathbf{h} := \text{Keccak}[2\lambda](\mathbf{x}; \ell)$  ▷  $\text{Keccak}[2\lambda](\cdot; \ell)$  denotes Keccak with
    $2\lambda$ -bit capacity and  $\ell$ -bit output.
2: return  $\mathbf{h}$ 
```

5.9.11 Block-Cipher Interface

The **Enc** algorithm abstracts the concrete block cipher used by **TCCR** and **PRG**. It has n_b -bit block size and n_k -bit key size, and it is modeled as an ideal cipher in the security analysis. In the current implementation, the selected primitive depends on the target security parameter as specified in Table 7. Future standardized symmetric cryptographic algorithms with suitable key and block lengths could serve as direct drop-in replacements for the current choices.

Enc($k, x; \lambda$)

Input: ($k \in \{0, 1\}^{n_k}, x \in \{0, 1\}^{n_b}$)

$\triangleright \lambda \in \{160, 256, 384, 512\}$

Output: $c \in \{0, 1\}^{n_b}$

```

1: if  $\lambda = 160$  then
2:   return  $c := \text{AES-192}(k, x)$ 
3: else if  $\lambda = 256$  then
4:   return  $c := \text{AES-256}(k, x)$ 
5: else if  $\lambda = 384$  then
6:   return  $c := \text{Rijndael-256}(k, x)$ 
7: else
8:   return  $c := \text{SHACAL-2}(k, x)$ 
9: end if
```

$\triangleright \lambda = 512$

6 Parameter Selection of VOLE-in-the-Head

6.1 Classical and Quantum Security Levels

The submission requirements of the NGCC competition [Ins25] define three mandatory security levels and one optional security level: 128-bit classical (80-bit quantum), 256-bit classical (128-bit quantum), 384-bit classical (192-bit quantum), and 512-bit classical (256-bit quantum) security levels. Accordingly, this document provides four parameter sets, i.e., Lynxer-160, Lynxer-256, Lynxer-384, and Lynxer-512, corresponding to the 80-bit, 128-bit, 192-bit, and 256-bit quantum security, respectively.

We consider the Grover algorithm with unlimited depth of quantum circuits to obtain a conservative security estimate. In this case, for the λ -bit quantum security level, one should use symmetric-key primitives with at least 2λ -bit classical security. Similarly, we argue that the statistical security level of the proof systems should also be at least 2λ bits, because we can imagine a trivial forgery strategy of enumerating the random coins until running into one that fails the check’s soundness. A 2λ -bit soundness level guarantees that the density of such “bad” coins is upper-bounded by $1/2^{2\lambda}$. Therefore, even with Grover’s search, the expected work of such a strategy is bounded by 2^λ , satisfying the λ -bit security requirement.

All required parameters satisfy the above pattern except the first one, which requires 128-bit classical security and 80-bit quantum security. Following the above argument, we conservatively target 160-bit classical security in the first parameter set, namely, Lynxer-160. If the maximum depth of a quantum circuit is limited to, e.g., 2^{40} or 2^{64} , we can use a parameter set Lynxer-128 and the symmetric-key primitive Lynx with 128-bit classical security level, as shown in our work [JYC⁺26]. We do not adopt Lynxer-128 for the 128-bit classical and 80-bit quantum security levels in the current document, but it is a potential candidate to achieve a better efficiency for the lowest security level.

6.2 Selection Approach of VOLEith Parameters

As the design of Lynx fixes the extended witness length, the remaining concrete efficiency trade-offs are determined by the VOLE-in-the-Head parameters, i.e., the repetition parameter τ , the proof-of-work parameter w_{grind} , and the BAVC opening-size threshold T_{open} . These parameters jointly determine the signature size and the expected signing time. A larger τ means more VOLE instances with shallower per-instance GGM trees, which reduces the total number of BAVC leaves L and thus speeds up signing (since BAVC tree operations dominate the running time), but increases the signature size due to the additional $\tau - 1$ correction vectors $\mathbf{c}_i$. The proof-of-work parameter w_{grind} reduces the effective challenge space from λ to $\lambda - w_{\text{grind}}$ bits, forcing the signer to perform roughly $2^{w_{\text{grind}}}$ extra hash iterations to find a valid chall_3 ; this reduces the BAVC tree size at the cost of increased signing work. T_{open} controls the space-time trade-off within each parameter band: a tighter T_{open} reduces the signature size but raises the probability that the BAVC opening exceeds the threshold, requiring the signer to retry.

Our selection methodology follows the structural patterns observed across the 12 FAEST and FAEST-EM parameter sets [BBB⁺26]. Analysis of these variants reveals that the GGM tree depth $k_0 = \lceil (\lambda - w_{\text{grind}})/\tau \rceil$ consistently falls into one of two bands: fast variants use $k_0 \in \{7, 8\}$, while short variants use $k_0 \in \{11, 12\}$. We therefore adopt the same bands and, for each of the four Next Generation Commercial Cryptography (NGCC) security levels ($\lambda = 160, 256, 384, 512$) [Ins25], enumerate all feasible $(\tau, w_{\text{grind}}, T_{\text{open}})$ triples within the corresponding k_0 range. From the candidates, we select those minimizing the combined size-speed cost metric $L \cdot (T_{\text{open}} + 2\tau)$, where L reflects the signing time (proportional to the number of BAVC leaf commitments and tree PRG calls) and $T_{\text{open}} + 2\tau$ reflects the signature-size contribution of the BAVC component. The BAVC opening-size distribution, which determines the acceptance probability for a given T_{open} , is computed exactly using the divide-and-conquer dynamic programming method described in Section 6.3.

For the *fast* variant, we additionally require $w_{\text{grind}} \geq 8$ and $2^{w_{\text{grind}}} \leq L/10$, ensuring that the proof-of-work grinding dominates the total signing attempts. We set T_{open} generously such that the BAVC rejection probability is at most $1/3$, so the expected number of attempts stays close to the grinding floor $2^{w_{\text{grind}}}$. For the *small* variant, we set T_{open} more tightly, targeting an expected number of attempts in the range $[L/6, L/4]$, which yields a shorter signature at the cost of more retries during signing. The resulting parameters are shown in Table 5 of Section 6.4.

6.3 Failure Probability Analysis

As described in Section 6.1, the rejection sampling threshold T_{open} directly affects both the signature size and the expected signing time. Choosing T_{open} requires knowledge of the distribution of the BAVC opening size, i.e., the number of internal tree nodes that must be revealed when opening all-but- τ leaves of a binary tree with L leaves.

In the Lynxer signature scheme, the τ small binary trees are mapped to a single large tree by interleaving their leaves: the j -th leaf of the large tree corresponds to the $\lfloor j/\tau \rfloor$ -th leaf of the $(j \bmod \tau)$ -th small tree. This layout is convenient for implementation. However, this interleaved layout does not decompose cleanly into independent subproblems, making it difficult to compute the opening-size distribution analytically. To address this, we consider a broader random model in which the τ punctured leaves are chosen uniformly at random from all L leaves. This model covers the interleaved layout as a special case, and crucially, it admits a recursive decomposition: the sampling problem on a tree with L leaves and τ punctured leaves can be split into independent subproblems on the left and right subtrees.

The following algorithm exploits this recursive structure to compute the exact distribution of the opening size. Dynamic programming with memoization on (L, τ) avoids recomputation of subproblems.

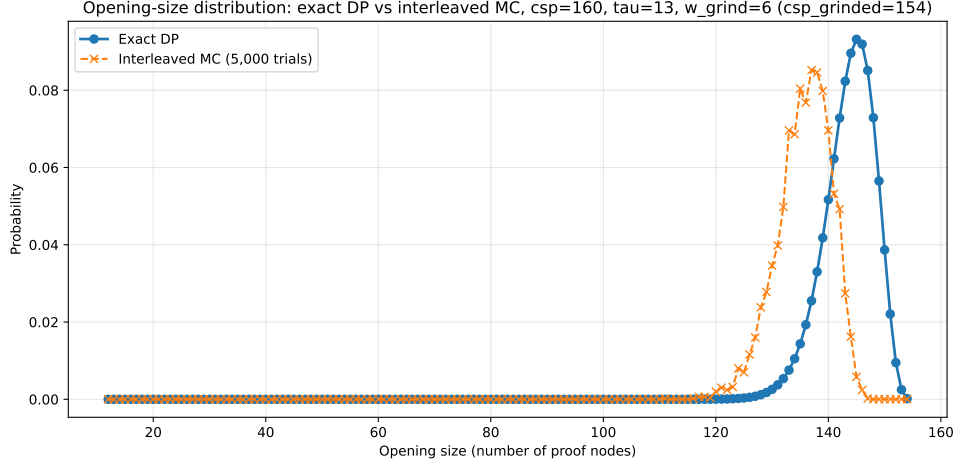


Figure 5: An example of BAVC opening size with $\lambda = 160$, $w_{\text{grind}} = 6$, $\tau = 13$, illustrating that the random sampling model is a conservative one in terms of size estimation.

RandOneTreeSample(L, τ)

Input: L, τ

▷ Number of leaves and number of punctured leaves.

Output: D

▷ Distribution of opening sizes.

```

1: if  $\tau = 0$  then
2:   return  $\{1 : 0\}$ 
3: else if  $L \leq \tau$  then
4:   return  $\{0 : 1\}$ 
5: end if
6:  $D := \{\}$                                 ▷ Format: {opening size : probability}.
7:  $L_L :=$  number of leaves in left subtree
8:  $L_R :=$  number of leaves in right subtree
9: for  $i \in [0, \tau]$  do
10:   $D_L := \text{RandOneTreeSample}(L_L, i)$ 
11:   $D_R := \text{RandOneTreeSample}(L_R, \tau - i)$ 
12:   $D' := D_L * D_R$                         ▷ Cartesian product of the two distributions.
13:   $D := D + \frac{\binom{L_L}{i} \cdot \binom{L_R}{\tau-i}}{\binom{L}{\tau}} \cdot D'$                 ▷ Weighted sum.
14: end for
15: return  $D$ 

```

Empirical validation confirms that the random sampling model used by **RandOneTreeSample** provides a conservative estimate of the opening size: it consistently yields slightly larger opening sizes than the actual interleaved layout, which leads to a lower rejection rate in practice and is therefore favorable for parameter selection. For example, in Figure 5, the opening size distribution of the random sampling model, which is exactly computed using the dynamic programming method, is juxtaposed with the distribution of the actual interleaving method, which is heuristically obtained with the Monte-Carlo method with 5,000 trials.

6.4 VOLE-in-the-Head Parameters

We follow the approach outlined in the previous subsections, and select two variants of parameters for each security level—a small variant and a fast variant. The rejection probability is estimated using the **RandOneTreeSample** algorithm as described in Section 6.3. The parameters,

together with signature sizes, are listed in Table 5.

Table 5: VOLE-in-the-Head parameters for Lynxer. ℓ_{vole} , τ , w_{grind} , and T_{open} denote the VOLE commitment length, the number of VOLE instances, the proof-of-work level, and the threshold for the number of opened BAVC nodes, respectively. $|\text{pk}|$, $|\text{sk}|$, and $|\sigma|$ denote the sizes of public key, secret key, and signature, respectively.

Parameter Set	VOLEitH Parameters				Sizes (Bytes)			
	ℓ_{vole}	τ	w_{grind}	T_{open}	$ \text{pk} $	$ \text{sk} $	$ \sigma $	$ \text{pk} + \sigma $
Lynxer-160s	816	14	6	129	40	40	4607	4647
Lynxer-160f	816	21	8	139	40	40	5801	5841
Lynxer-256s	1296	22	12	224	64	64	12191	12255
Lynxer-256f	1296	35	8	223	64	64	15097	15161
Lynxer-384s	1936	34	10	332	96	96	27495	27591
Lynxer-384f	1936	53	9	336	96	96	34109	34205
Lynxer-512s	2576	46	6	439	128	128	48879	49007
Lynxer-512f	2576	72	8	447	128	128	61091	61219

7 Performance Evaluation

We provide two implementations of Lynxer as part of our proposal. The first one is a reference implementation, which is written in standard ANSI C, targeting close adherence to the specification and maximum portability. The second one is an optimized C++ implementation for x86 platforms, using platform-specific instruction sets such as AES-NI, SHA-NI, AVX2, AVX-512, and VAES. The two implementations are functionally equivalent and provide the same API. We benchmark both implementations with two XOF backends: the NGCC SM3-based pseudoXOF backend specified in the `API_PKC.zip` package and a standard Keccak backend.

7.1 Instantiations of Standard Symmetric-key Primitives

Except for Lynx, the Lynxer signature scheme depends on the standard symmetric-key primitives: *block ciphers* for PRG and TCCR, and *cryptographic hash functions* for Fiat-Shamir and sampling public randomness. Below, we show how to instantiate these symmetric-key primitives with AES [Nat01], SHACAL-2 [KKL⁺05] and Keccak [ND15] whose security has been extensively studied over a long period of time. In the future, we are able to replace these symmetric-key primitives with future standardized primitives.

Cryptographic hash functions. The NGCC competition provides an SM3-based pseudoXOF function (in the `API_PKC.zip` package) for the evaluation of the submitted algorithms’ correctness and functionality. To comply with this evaluation framework, we include a version of Lynxer that uses pseudoXOF as its underlying XOF. For a standardized alternative, we also offer a version that employs Keccak, which is the winner of the NIST SHA-3 cryptographic hash competition [ND15] and has withstood extensive public scrutiny, providing high confidence in its security. In this version, for a given security parameter λ , we instantiate the XOF as Keccak[2λ] with a capacity of 2λ bits, offering λ -bit collision resistant, which is indeed a SHAKE variant. The XOF choices for each parameter set are summarized in Table 6.

Symmetric-key primitives from block ciphers. Aside from cryptographic hash functions, we use the following standardized primitives in various components of Lynxer:

Table 6: XOF used in each parameter set

Parameter Set	Quantum security	pseudoXOF backend	Keccak backend	Reference
Lynxer-160	80	pseudoXOF	Keccak[320]	[ND15]
Lynxer-256	128	pseudoXOF	Keccak[512]	[ND15]
Lynxer-384	192	pseudoXOF	Keccak[768]	[ND15]
Lynxer-512	256	pseudoXOF	Keccak[1024]	[ND15]

- The [TCCR](#) hash function requires a suitable block cipher modeled as an ideal cipher;
- The [LeafHash](#) algorithm in [BAVC.Commit](#) and [BAVC.Verify](#) invokes a [PRG](#);
- The [VOLE.ConvertToVOLE](#) algorithm utilizes a [PRG](#).

We use the standardized block ciphers to instantiate the above components, with the concrete choice depending on the target security level. The general principles are that:

- For the [TCCR](#) construction in [Section 2.3](#), the combined key length and block length $n_k + n_b$ should be no less than $\lambda + 128$, where λ is the security parameter;
- To instantiate [PRG](#) with a block cipher in the counter mode, we require $n_k \geq \lambda$.

The design choices are summarized in [Table 7](#). For the 160-bit and 256-bit parameter sets, we use AES-192 and AES-256, with 192/256-bit key and 128-bit block, to instantiate both [TCCR](#) and [PRG](#). For the 384-bit parameter set, we use Rijndael-256 with 256-bit key and 256-bit block to construct a TCCR hash function following the approach outlined in [Section 2.3](#), and then use this TCCR construction to build the [PRG](#). For the 512-bit parameter set, we use SHACAL-2 with 512-bit key and 256-bit block to instantiate the TCCR hash function, and use its counter mode to realize the [PRG](#).

Table 7: Symmetric-key primitives based on block ciphers for each parameter set

Parameter Set	Quantum security	TCCR Hash	PRG	Reference
Lynxer-160	80	AES-192	AES-192	[Nat01]
Lynxer-256	128	AES-256	AES-256	[Nat01]
Lynxer-384	192	Rijndael-256	Rijndael-256-based TCCR	[DR00]
Lynxer-512	256	SHACAL-2	SHACAL-2	[KKL ⁺ 05]

The block ciphers AES-128, AES-192, and AES-256 are the Rijndael variants selected by NIST as the Advanced Encryption Standard (AES) [Nat01]. The Rijndael-256 is now promoted as an officially recognized new standard variant by NIST [Nat24]. Modern processors provide dedicated instruction sets such as AES-NI, which offer substantial acceleration for AES or Rijndael computations. SHACAL-2 reuses the compression function of SHA-256 standard [Nat15]. It was chosen as a standard block cipher by the NESSIE project [Eur03] in 2003 and remains secure for more than 20 years. Moreover, the evaluation of SHACAL-2 can be accelerated using the SHA-NI instruction set.

7.2 Implementation Platform

We tested both implementations on a laptop computer equipped with an AMD Ryzen 9 9955HX3D CPU (16 cores, 32 threads, x86_64) and 30 GiB memory. The CPU clock frequency ranges from 1.22 GHz to 5.46 GHz. The compiler is GCC 15.2.0 and the operating system is Ubuntu 26.04 LTS, Linux 7.0.0-15-generic. [Table 8](#) summarizes the implementation platforms used for benchmarking.

Table 8: Benchmark platforms for the reference and optimized implementations.

Item	Reference implementation	Optimized implementation
Language	ANSI C	C++
Instruction set	Portable C only	AES-NI, AVX2, SHA-NI, AVX-512, VAES, VPCLMULQDQ
Cryptographic library	XKCP for Keccak	XKCP for Keccak

7.3 Software Performance

Tables 9 to 12 report the wall-clock time as well as cycle counts for key generation, signing, and verification across all parameter sets. Each entry is the mean over 100 trials. We report separate tables for each combination of implementation and XOF backend. In all tables, time is reported in ms and cycle counts are reported as processor cycles.

Table 9: Software performance benchmarks for the reference implementation using the NGCC pseudoXOF backend.

Parameter set	Time in ms; processor cycles					
	KeyGen		Sign		Verify	
	ms	cycles	ms	cycles	ms	cycles
Lynxer-160s	4.485	11,193,073	119.515	298,223,203	107.462	268,146,993
Lynxer-160f	4.485	11,192,790	26.777	66,815,847	17.512	43,698,671
Lynxer-256s	32.320	80,648,028	382.171	953,617,576	306.205	764,062,613
Lynxer-256f	32.255	80,486,250	126.263	315,060,748	61.334	153,045,542
Lynxer-384s	137.541	343,202,554	1233.990	3,079,135,372	911.013	2,273,219,402
Lynxer-384f	137.593	343,332,965	501.732	1,251,954,581	224.051	559,068,163
Lynxer-512s	399.011	995,638,604	1713.128	4,274,712,003	852.546	2,127,331,127
Lynxer-512f	399.079	995,809,393	1255.562	3,132,962,358	455.705	1,137,106,054

Table 10: Software performance benchmarks for the reference implementation using the Keccak backend.

Parameter set	Time in ms; processor cycles					
	KeyGen		Sign		Verify	
	ms	cycles	ms	cycles	ms	cycles
Lynxer-160s	0.316	789,897	98.680	246,232,129	96.539	240,891,287
Lynxer-160f	0.313	783,100	13.297	33,180,796	12.482	31,147,235
Lynxer-256s	0.328	819,676	258.279	644,474,352	256.593	640,267,948
Lynxer-256f	0.328	818,377	28.083	70,075,662	27.297	68,115,108
Lynxer-384s	0.948	2,366,195	719.413	1,795,126,894	712.514	1,777,913,939
Lynxer-384f	0.954	2,381,213	82.981	207,059,186	80.039	199,720,269
Lynxer-512s	1.851	4,620,356	317.077	791,192,223	308.148	768,911,377
Lynxer-512f	1.855	4,630,352	45.839	114,379,675	42.362	105,704,733

Table 11: Software performance benchmarks for the optimized AVX implementation using the NGCC pseudoXOF backend.

Parameter set	Time in ms; processor cycles					
	KeyGen		Sign		Verify	
	ms	cycles	ms	cycles	ms	cycles
Lynxer-160s	0.184	454,400	13.843	33,083,912	10.618	26,460,812
Lynxer-160f	0.183	455,575	1.726	4,122,425	1.522	3,775,100
Lynxer-256s	0.562	1,402,275	35.654	81,779,237	27.491	68,386,212
Lynxer-256f	0.561	1,397,487	3.830	9,336,362	3.514	8,765,750
Lynxer-384s	1.218	3,023,800	151.327	316,473,675	95.237	237,861,462
Lynxer-384f	1.210	3,015,150	14.314	33,669,712	12.207	30,450,375
Lynxer-512s	2.072	5,150,400	282.767	657,923,850	226.129	566,499,912
Lynxer-512f	2.062	5,167,487	27.944	68,778,637	26.724	66,334,175

Table 12: Software performance benchmarks for the optimized AVX implementation using the Keccak backend.

Parameter set	Time in ms; processor cycles					
	KeyGen		Sign		Verify	
	ms	cycles	ms	cycles	ms	cycles
Lynxer-160s	0.080	199,562	2.414	5,979,637	2.044	5,102,100
Lynxer-160f	0.081	202,300	0.378	935,287	0.342	850,187
Lynxer-256s	0.111	275,500	4.737	11,498,050	4.127	10,326,887
Lynxer-256f	0.111	275,675	0.588	1,464,837	0.553	1,379,937
Lynxer-384s	0.287	716,625	18.474	45,808,050	17.608	43,779,612
Lynxer-384f	0.287	718,287	2.551	6,346,762	2.447	6,093,825
Lynxer-512s	0.618	1,568,225	47.901	116,016,975	45.204	112,670,675
Lynxer-512f	0.619	1,553,762	5.874	14,648,275	5.737	14,534,637

7.4 Performance Comparison

We compare the NIST Level V parameter sets of Lynxer with AIMER [KCH⁺26], Rainier [DKR⁺22], FAEST and FAEST-EM [BBB⁺26, BBB⁺25], and SLH-DSA [HBD⁺22]. The corresponding benchmark code is taken from the public submission repositories or packages of AIMER, Rainier, FAEST, and SLH-DSA [Sam26, Ins26, FAE25, SPH20]. All timings in Table 13 were collected on the same platform as Table 8. For Lynxer, we use the optimized AVX implementation with the Keccak backend from Table 12. For Rainier, FAEST, and SLH-DSA, we use the benchmark programs shipped with the public code bases. We use 100 iterations or samples where the benchmark accepts this setting; the SLH-DSA benchmark is run with its default 10 iterations. The public AIMER package does not include a benchmark target; for AIMER only, we time the public signing API with a minimal wrapper over 100 iterations.

Table 13: Efficiency comparison of symmetric-based post-quantum signature schemes at NIST Level V.

Scheme	Running time (ms)		Communication (Bytes)	
	Signing	Verification	Public key	Signature
Lynxer-256s	4.737	4.127	64	12,191
Lynxer-256f	0.588	0.553	64	15,097
AIMer-256s	43.908	43.601	64	17,056
AIMer-256f	5.646	5.210	64	25,120
Rainier-3-L5	3.389	3.323	64	25,504
Rainier-4-L5	3.695	3.610	64	28,320
FAEST-256s	9.121	9.002	48	20,696
FAEST-256f	1.373	1.291	48	26,548
FAEST-EM-256s	7.156	6.889	64	17,984
FAEST-EM-256f	1.122	1.036	64	23,476
SLH-DSA-SHAKE-256s	821.108	1.096	64	29,792
SLH-DSA-SHAKE-256f	89.495	2.374	64	49,856

7.5 Discussion

The performance bottlenecks of Lynxer can be read off directly from the pseudocode in Sections 5.2, 5.4 and 5.9. The dominant costs are not isolated expensive primitives, but rather a few symmetric-key functions that are executed a very large number of times.

Key generation. The `Lynxer.KeyGen` algorithm invokes the `Lynx.SampleMatVec` procedure, which samples the L_0, L_1, L_2 matrices. To ensure that the sampled matrices are invertible, we sample two triangular matrices and multiply them together. This incurs extra cost compared to the FAEST scheme. As a computation-communication trade-off, the matrix values can be cached into the public key, speeding up `Lynxer.KeyGen` performance at the cost of bloating up the public key.

BAVC tree generation and leaf processing. In signing, the first major hotspot is `BAVC.Commit`. The loops over $\text{id} \in [1, L/2)$ and $\text{id} \in [L/2, L)$ evaluate `TCCR` for essentially every internal tree node, while the loop over $\text{id} \in [L, 2L)$ calls `LeafHash` for every leaf. Since `LeafHash` itself expands each leaf by $\text{PRG}(\mathbf{x}, \text{iv}_{\text{sig}}, 0^8; 2\lambda)$, the total cost scales with the whole interleaved BAVC forest size as reflected by L . A higher τ value results in a smaller L value. This is the most computation-heavy part of signing, and it is also mirrored during verification in `BAVC.Verify`, where all opened internal nodes are expanded again and the remaining leaves are recomputed.

VOLE expansion. The second hotspot is [VOLE.ConvertToVOLE](#). In [VOLE.Commit](#), this conversion is executed once for every $\alpha \in [0, \tau)$, and each call first expands all seeds with [PRG](#)(seed_i, iv_{sig}, ·; ℓ_{vole}) before performing the XOR folding that derives $(\mathbf{u}, \mathbf{V})$. Thus the cost is driven by repeated seed expansion over all leaves of all VOLE instances. For the 160/256/512-bit parameter sets, this directly translates into a large number of block-cipher invocations in counter mode; for the 384-bit set, the same step is implemented through repeated [TCCR](#) calls.

Fiat-Shamir hashing, rejection sampling, and proof-of-work. The top-level signing algorithm computes [MsgHash](#), [CoinHash](#), [FSHash₁](#), and [FSHash₂](#) once, but the real hashing bottleneck is the while-loop that repeatedly computes [FSHash₃](#)(chall₂, $\tilde{a}_1, \tilde{a}_0$, ctr) until both the proof-of-work condition and the opening-size condition are met. This loop combines proof-of-work with rejection sampling, so the cost grows with the expected number of signing attempts. Consequently, the wall-clock signing time is much more sensitive to $(\tau, w_{\text{grind}}, T_{\text{open}})$ than the verification time.

Benefit of instruction-set acceleration. The optimized implementation benefits most from accelerating exactly the symmetric-key primitives above. AES-NI directly speeds up both the counter-mode [PRG](#) and the block-cipher calls inside [TCCR](#) at the 160, 256, and 384 security levels, as they use ciphers in the Rijndael family. This affects tree expansion, leaf-node commitments, and VOLE seed expansion simultaneously, so one low-level optimization improves several top-level algorithms at once. For the 512-bit security level, the SHA256-based SHACAL-2 cipher can be accelerated with SHA-NI instructions.

8 Concrete Security Analysis

We analyze the security of Lynxer with respect to known attacks in [Section 8.2](#). In [Section 8.1](#), we give a detailed analysis of the concrete security of the Lynx one-way function. The cryptographic primitives used in Lynxer are the Lynx one-way function (OWF), the block-cipher-based [PRG](#) and [TCCR](#), and the hash functions (XOF) instantiated as random oracles. The symmetric primitives other than Lynx used in Lynxer are instantiated from well-studied block ciphers and standardized hash-function interfaces. In our security proof, we model the hash functions as random oracles. This is a standard practice in security analysis of cryptography.

8.1 Concrete Security of the Lynx One-Way Function

The security analysis of Lynx follows the standard security model for VOLEitH-based signatures: each randomly sampled secret k is uniquely bound to a randomly sampled public seed iv_{owf} , and together they produce exactly one output t . We consider two core security goals: preimage-resistance and indistinguishability. Given a $(\text{iv}_{\text{owf}}, t)$ pair, the adversary aims to find any k satisfying $\text{Lynx}(\text{iv}_{\text{owf}}, k) = t$ (not necessarily the original one) for preimage-resistance; for indistinguishability, it aims to distinguish t from a uniformly random λ -bit string. This naturally extends to the multi-user setting, where the adversary is given multiple independent $\{(\text{iv}_{\text{owf}}^{(i)}, t^{(i)})\}_i$ pairs and succeeds if it breaks either goal for any instance.

8.1.1 Algebraic Attacks

Given the attack model outlined above, our analysis focuses on algebraic attacks. First, we derive the equation systems from Lynx. Then the key recovery attack on Lynx reduces to solving those systems.

For $y := f(x) = x^d$, the checking polynomial is

$$\begin{cases} x \cdot y = 1, & \text{if } d = -1, \\ x \cdot y = x^{2^i}, & \text{if } d = 2^i - 1, \\ y \cdot x = y^{2^i}, & \text{if } d = (2^i - 1)^{-1} \bmod 2^\lambda - 1 \\ x^{2^i} \cdot y = x^{2^{2i}} \cdot x, & \text{if } d = 2^{2i} - 2^i + 1, \\ y^{2^i} \cdot x = y^{2^{2i}} \cdot y, & \text{if } d = (2^{2i} - 2^i + 1)^{-1} \bmod 2^\lambda - 1. \end{cases} \quad (8)$$

For $z := f(x, y) = x \cdot y$, the checking polynomial is $x \cdot y = z$; for $w := f(x, y, z) = x \cdot y \cdot z^{-1}$, it is $x \cdot y = z \cdot w$.

Then Eq.(5) is represented by substituting $x = k + c_1$ and $y = v_1$ into Eq.(8); Eq.(6) by substituting $x = L_0(k) + c_2$ and $y = v_2$. Eq.(7) is represented by

$$L_1(v_1) \cdot L_2(v_2) = \delta_\lambda(t + L_3(k + v_1 + v_2), k + c_3). \quad (9)$$

Thus, we derive the checking polynomial system over $\mathbb{F}_{2^\lambda}$ for **Lynx** in variables k , v_1 and v_2 .

A polynomial in $\mathbb{F}_{2^\lambda}$ corresponds to a set of multivariate Boolean equations once a basis is fixed. Since $x \rightarrow x^{2^i}$ is a Frobenius automorphism over $\mathbb{F}_{2^\lambda}$, the resulting system is quadratic over $\mathbb{F}_2$.

- For $x \cdot y = 1$ over $\mathbb{F}_{2^\lambda}$, it yields 5λ linearly independent quadratic equations over $\mathbb{F}_2$ [CL04], derived from

$$\begin{cases} x \cdot y = 1, \\ x^2 \cdot y = x, \\ x \cdot y^2 = y, \\ x^4 \cdot y = x^3, \\ x \cdot y^4 = y^3. \end{cases} \quad (10)$$

- For $x \cdot y = x^{2^i}$, or $y \cdot x = y^{2^i}$ over $\mathbb{F}_{2^\lambda}$, it yields 3λ linearly independent quadratic equations over $\mathbb{F}_2$ [KHS⁺23], derived from

$$\begin{cases} x^{2^i} = x \cdot y, \\ x^{2^i} \cdot x = x^2 \cdot y, \\ x^{2^i} \cdot y = x \cdot y^2 \end{cases} \quad \text{or} \quad \begin{cases} y^{2^i} = y \cdot x, \\ y^{2^i} \cdot y = y^2 \cdot x, \\ y^{2^i} \cdot x = y \cdot x^2 \end{cases} \quad (11)$$

- For $x^{2^i} \cdot y = x^{2^{2i}} \cdot x$, or $y^{2^i} \cdot x = y^{2^{2i}} \cdot y$ over $\mathbb{F}_{2^\lambda}$, it yields λ linearly independent quadratic equations over $\mathbb{F}_2$ [CL04]⁵ by itself.
- For $x \cdot y = z$ over $\mathbb{F}_{2^\lambda}$, it yields 3λ linearly independent quadratic equations over $\mathbb{F}_2$, derived from

$$\begin{cases} x \cdot y = z, \\ x^2 \cdot y = x \cdot z, \\ x \cdot y^2 = y \cdot z. \end{cases} \quad (12)$$

- For $x \cdot y = z \cdot w$, where $w = L(x + y + z) + c$ is a linearized polynomial with constant term over $\mathbb{F}_{2^\lambda}$, this yields λ linearly independent quadratic equations over $\mathbb{F}_2$ by itself. This follows from expanding $x \cdot y = z \cdot L(x) + z \cdot L(y) + z \cdot L(z) + c \cdot z$ and observing that it remains quadratic as the density of L : the $x \cdot y$ term on the left prohibits multiplication by any non-constant polynomial in z , and the $z \cdot L(z)$ term on the right prohibits multiplication by any non-constant polynomial in x or y .

⁵The proof remains valid even when $\gcd(i, \lambda) \neq 1$ for Kasami functions.

Table 14: Complexity against Gröbner Basis and XL Attacks

Scheme	m_e	n_e	Gröbner Basis ($\omega = 2$)		XL Attack ($\omega = 2.37$)	
			D_{reg}	complex.	D	complex.
Lynx-160	11 λ	3 λ	29	235.4	12	184.6
Lynx-256	7 λ	3 λ	42	468.7	19	296.4
Lynx-384	5 λ	3 λ	77	821.7	27	429.1
Lynx-512	3 λ	3 λ	1537	6133.8	39	612.2

† All complexities are reported in $\log_2$ unless otherwise noted.

Moreover, the settings of L_0 and L_3 prevent variable k elimination via equivalent representation, ensuring no additional independent quadratic equations in v_1 and v_2 can be derived. Therefore, we derive the system of equations $\{\eta_i\}_i$ over $\mathbb{F}_2$ for Lynx. The numbers of equations m_e and variables n_e are given in Table 14.

Gröbner basis attack. The Gröbner basis attack on the system of m_e equations $\{\eta_i\}_i$ in n_e variables defined over $\mathbb{F}_2$ proceeds as [SS21]: compute a grevlex Gröbner basis, then convert it to lex order; extract and solve a univariate polynomial from the basis; substitute the root back and repeat until all variables are recovered. The complexity is governed by the degree of regularity D_{reg} [MBS04] as $\mathcal{O}\left(\binom{n_e + D_{\text{reg}}}{D_{\text{reg}}}\omega\right)$, where ω is the linear algebra constant for Gaussian elimination, and especially $\omega = 2$ in practice for sparse Macaulay matrices (e.g., in F4/F5) [BFP09, ACG⁺19].

- For semi-regular systems ($m_e > n_e$), D_{reg} is the smallest d such that the coefficient of z^d in the Hilbert series [Frö85]

$$\text{HS}(z) = \frac{\prod_{i=1}^{m_e+n_e} (1 - z^{d_i})}{(1 - z)^{n_e}}$$

is non-positive. Here, $d_i = \deg(\eta_i)$ for $i = 1, 2, \dots, m_e$, and $d_{m_e+i} = 2$ for the field equations $x_i^2 = x_i$ ($i = 1, \dots, n_e$), which are included to restrict solutions to $\mathbb{F}_2$.

- For regular systems ($m_e = n_e$), $D_{\text{reg}} = 1 + \sum_{i=1}^{n_e} (\deg(\eta_i) - 1)$.

In line with standard Gröbner basis attacks, we estimate the complexity of Lynx. When $\lambda = 512$, the equation system is regular; otherwise, the equation systems are semi-regular. Corresponding D_{reg} values and attack complexities appear in Table 14, which confirms that Gröbner basis attacks do not threaten Lynx.

XL attack. We analyze the XL attack against Lynx on the quadratic system of m_e equations $\{\eta_i\}_i$ in n_e variables over $\mathbb{F}_2$. When m_e is less than the number of distinct monomials in the system, trivial linearization that recovers the secret by substituting high-degree terms with new variables fails. The XL algorithm [CKPS00] extends linearization by multiplying each original equation with all monomials of degree at most $D - 2$, for some $D > 2$, yielding an expanded linear system with:

- $E_D = m_e \sum_{i=0}^{D-2} \binom{n_e}{i}$ equations of degree at most D ,
- involving at most $T_D = \sum_{i=0}^D \binom{n_e}{i}$ monomials.

Field equations are excluded from XL, as they are already accounted for in variable counting and expansion. Solving becomes feasible via Gaussian elimination once $E_D \geq T_D$, under the

Table 15: Complexity against Dinur’s Method

Scheme	case1			case2				
	λ	d	complex.	λ	$\deg_2 S_1$	$\deg_2 S_2$	d	complex.
Lynx-160	3λ	2	409.0	λ	159	67	160	174.3
Lynx-256	3λ	2	644.9	λ	205	53	256	271.6
Lynx-384	3λ	2	959.0	λ	325	31	356	400.8
Lynx-512	3λ	2	1272.7	λ	257	91	348	529.5

standard assumption that the expanded equations are linearly independent. The time complexity is then $\mathcal{O}(T_D^\omega)$, where $\omega \approx 2.8$ using Strassen’s algorithm [Str69], or $\omega \approx 2.37$ with Galactic algorithms [AW20] (though with huge constant factors). Corresponding m_e , n_e , D values and attack complexities appear in Table 14, which shows that XL attacks are ineffective against Lynx.

Dinur’s method. Dinur [Din21] proposed a generic polynomial-method-based algorithm for solving multivariate equation systems over $\mathbb{F}_2$. The algorithm achieves time complexity $\lambda^2 \cdot 2^{(1-1/(2.7d))\lambda}$ and memory complexity $\lambda^2 \cdot 2^{(1-1/(1.35d))\lambda}$, where n is the number of variables and d is the degree of the system, assuming that the number of equations m is at least λ and $d \ll \lambda$ for practical applicability.

- We can directly evaluate Lynx’s resistance to Dinur’s method by solving the quadratic equation system $\{\eta_i\}_i$ in n_e variables;
- Alternatively, bypassing auxiliary variables v_1 and v_2 , we derive a high-degree algebraic system solely in variables k as

$$\begin{aligned} &L_1(S_1(k + c_1)) \cdot L_2(S_2(L_0(k) + c_2)) \\ &= \delta_\lambda(t + L_3(k + S_1(k + c_1) + S_2(L_0(k) + c_2)), k + c_3). \end{aligned} \quad (13)$$

As d increases and approaches n , Dinur’s method rapidly loses its advantage and ultimately collapses to exhaustive search.

Corresponding λ , d values, and attack complexities appear in Table 15. The analysis applies to other fast exhaustive search attacks over $\mathbb{F}_2$ [BCC⁺10, Bou22]. Lynx is secure against these attacks.

Linearization and G&D attacks. We evaluate Lynx against the guess-and-determine (G&D) attack based on S-box linearization [ZWY⁺23], which first broke two-round RAIN and AIM. Each S-box S_i satisfies $x^{d_i} = (x^u)^b \cdot x^{2^a}$. Guessing $x^u = \alpha$ reduces S_i to a linear map $\alpha^b \cdot Mx$ over $\mathbb{F}_2$. The guessing complexity equals the order of x^u , i.e., $\frac{2\lambda-1}{\gcd(2\lambda-1, u)}$. All d_i chosen in Lynx ensure that linearizing any S_i requires guessing complexity exceeding $2^{\lambda/2}$. This attack is infeasible against constructions employing three nonlinear components with different inputs—such as Lynx.

- To derive a solvable quadratic equation system in k over $\mathbb{F}_2$, an attacker must linearize both S_1 and S_2 simultaneously. Since their inputs are different, the total guessing complexity exceeds 2^λ .
- Linearizing only S_1 or S_2 and deriving linear equations from $x^u = \alpha$ enables the attacker to express k as an affine function of r free variables. Substituting into Eq.(13) yields a system of high-degree equations in those r variables over $\mathbb{F}_2$, and solving it requires exhaustive search over all 2^r candidates. For each α , the equation $x^u = \alpha$ has exactly $\gcd(2^\lambda - 1, u)$ solutions

in $\mathbb{F}_{2^\lambda}$. Since the linear system captures only partial information about $x^u = \alpha$, its solution space strictly contains those $\gcd(2^\lambda - 1, u)$ values. Hence $2^r > \gcd(2^\lambda - 1, u)$, and the total cost exceeds 2^λ .

Resultant attack. We now solve polynomial systems over $\mathbb{F}_{2^\lambda}$. The resultant enables solving such systems by detecting common roots [YZY⁺24]. Let $f(\mathbf{x}, y), g(\mathbf{x}, y) \in \mathbb{F}_q[\mathbf{x}, y]$, where $\mathbf{x} = (x_1, \dots, x_s)$. Their resultant $R(f, g, y)$ is the determinant of Sylvester matrix of f and g as univariate polynomials in y .

Definition 8.1 (Resultant). Let $f(\mathbf{x}, y), g(\mathbf{x}, y) \in \mathbb{F}_q[\mathbf{x}, y]$, where $\mathbf{x} = (x_1, \dots, x_s)$. If $f(\mathbf{x}, y) = \sum_{i=0}^m f_i y^i$ and $g(\mathbf{x}, y) = \sum_{i=0}^n g_i y^i$ with $f_i, g_i \in \mathbb{F}_q[\mathbf{x}]$, then the resultant of $f(\mathbf{x}, y)$ and $g(\mathbf{x}, y)$ is defined as

$$R(f, g, y) = \begin{vmatrix} f_m & f_{m-1} & \cdots & f_0 & & & \\ & f_m & f_{m-1} & \cdots & f_0 & & \\ & & \ddots & \ddots & \ddots & \ddots & \\ & & & f_m & f_{m-1} & \cdots & f_0 \\ g_n & g_{n-1} & \cdots & g_0 & & & \\ & g_n & g_{n-1} & \cdots & g_0 & & \\ & & \ddots & \ddots & \ddots & \ddots & \\ & & & g_n & g_{n-1} & \cdots & g_0 \end{vmatrix}.$$

The resultant vanishes iff f and g share a non-constant common factor. Crucially, $R(f, g, y) = h(\mathbf{x}) \in \mathbb{F}_q[\mathbf{x}]$ eliminates y . Iteratively applying resultants to the given polynomials in s variables yields a univariate polynomial; its roots, substituted back stepwise, give all common solutions.

For Lynx, the resultant elimination path is:

$$g_1 := R(\text{Eq.}(5), \text{Eq.}(7), v_1), \quad g_2 := R(\text{Eq.}(6), g_1, v_2).$$

This yields a univariate polynomial $g_2(k)$; solving it recovers the key. Let $\deg_x g$ be the maximal degree of variable x in g . Computing g_1 costs $(\deg_{v_1} \text{Eq.}(5) + \deg_{v_1} \text{Eq.}(7))^\omega \cdot M(D_1)$; computing g_2 costs $(\deg_{v_2} \text{Eq.}(6) + \deg_{v_2} g_1)^\omega \cdot M(D_2)$. Here, D_1 is the product of the maximal k - and v_2 -degrees in $\text{Eq.}(5)$ and $\text{Eq.}(7)$; D_2 is the maximal k -degree in $\text{Eq.}(6)$ and g_1 ; $M(d) = d \log d \log \log d$ is typically the cost of degree- d polynomial multiplication; and ω denotes the exponent used in computing the Sylvester determinant for the resultant. Solving $g_2(k)$ requires at least $M(\deg(g_2))$ operations. Since the degree reaches maximum in each resultant elimination step, the total cost exceeds 2^λ , rendering resultant elimination infeasible against Lynx.

Linearized polynomial decomposition. For a linearized polynomial $L(x)$, it is highly likely to find two linearized polynomials $L'(x)$ and $L''(x)$ of degree $2^{\lambda/2}$ such that $L(x) = L''^{-1} \circ L'(x)$ [YZY25]. This enables meet-in-the-middle modeling, yielding equations $L''(y) = L'(x)$ of degree $2^{\lambda/2}$ given the output y of L , achieving substantial degree reduction. The method applies to schemes with a single (or effectively reduced) linear layer—e.g., AIM, 2-round RAIN, or 3-round RAIN under S-box linearization. However, Lynx interleaves four linearized polynomials with nonlinear operations, blocking composite decomposition. Consider Lynx-128 as an example. Let w_1 and w_2 be the outputs of L_1 and L_2 , and suppose $L_i(x) = L_i''^{-1} \circ L_i'(x)$ for $i = 1, 2, 3$. This yields the system

$$\begin{cases} L_1'(\frac{1}{k+c_1}) = L_1''(w_1), \\ L_2'(\frac{1}{L_0(k)+c_2}) = L_2''(w_2), \\ L_3'(\frac{1}{k+c_1} + \frac{1}{L_0(k)+c_2} + k) = L_3''(w_1 \cdot w_2 + t), \end{cases}$$

which contains the product of $w_1 \cdot w_2$. Thus, efficiently eliminating intermediate variables to derive a low-degree univariate equation is infeasible—even eliminating any intermediate variable via resultants incurs complexity exceeding 2^λ . The complexity presented in Table 16 shows that these two resultant methods fail to attack Lynx.

Magic pot. The Magic Pot method [BFU26], which broke full AIM2, eliminates variables by fixing one variable— y —rewriting $f(\mathbf{x}, y) \in \mathbb{F}_{2^\lambda}[y][\mathbf{x}]$ as $f = \sum_{u \in U} \alpha_u(y) \mathbf{x}^u$, where $\mathbf{x} = (x_1, \dots, x_s)$, $\alpha_u(y) \in \mathbb{F}_{2^\lambda}[y]$, and U is a set of monomials in $\mathbf{x}$. Squaring (an $\mathbb{F}_2$ -linear operation) then yields extra linearly independent equations in $\mathbb{F}_{2^\lambda}[y]$. When these exceed $|U|$, variable elimination reduces to computing the left kernel of a coefficient matrix over $\mathbb{F}_{2^\lambda}[y]$. Using approximate-basis iteration and HalfGCD-based degree minimization, the attacker efficiently computes this kernel and recovers a low-degree polynomial in y ; its roots reveal the secret key. The attack targets primitives with variables missing high-degree terms.

We analyze Lynx using the Magic Pot method. Eq.(5)-(7) show that Lynx includes all pairwise cross-terms among k , v_1 , and v_2 , as well as high-degree monomials in each variable, guaranteed by the configuration of linearized polynomials. Consequently, no variable can be isolated to form a sparse polynomial ring representation, the complexity presented in Table 16 shows that the Magic Pot method fails to attack Lynx.

Table 16: Complexities of magic pot, resultant and linearized polynomial decomposition attacks on Lynx.

Scheme	λ	Magic Pot	Resultant	Linearized polynomial decomposition
			$(\omega = 2.37)$	$(\omega = 2.37)$
Lynx-160	160	$2^{346.72}$	$\mathcal{O}(2^{699.2})$	$\mathcal{O}(2^{699.2})$
Lynx-256	256	$2^{471.92}$	$\mathcal{O}(2^{1118.7})$	$\mathcal{O}(2^{1118.7})$
Lynx-384	384	$2^{680.63}$	$\mathcal{O}(2^{1678.1})$	$\mathcal{O}(2^{1678.1})$
Lynx-512	512	$2^{993.29}$	$\mathcal{O}(2^{2237.4})$	$\mathcal{O}(2^{2237.4})$

Other algebraic attacks. The Gröbner basis attack over $\mathbb{F}_{2^\lambda}$ is infeasible against both the single-variable equation Eq.(13) and the multivariate system Eq.(5)–(7), as Lynx’s polynomial systems almost surely achieve maximum degree. [LMØM23] converts the equation into a single-variable polynomial via equivalent representation, then into a low-degree Boolean equation system for solving. However, in Lynx, multiple $L(\frac{1}{k+c})$ -like terms in Eq.(13) block further expansion, thus making the attack impossible. [LMM24] observes that RAIN’s special round structure—lacking linear layers at both input and output—introduces extra algebraic relations, enabling construction of more overdetermined quadratic systems and thus drastically lowering the complexity of Gröbner basis attacks, thereby pointing that using 3-round RAIN with the 256-bit key is risky. Lynx lacks this structure entirely.

8.1.2 Statistical Attacks

We consider only attackers with access to a single input-output pair under each key; thus, multi-pair attacks—including differential, linear, and integral attacks—are inapplicable. For completeness, we briefly note them below.

Differential and linear attacks. To prevent unexpected variants of differential and linear cryptanalysis, we derive a lower bound on the differential and correlation trails in Lynx.

For function $f : \mathbb{F}_{2^\lambda}^m \rightarrow \mathbb{F}_{2^\lambda}$, the differential uniformity and correlation of f are defined as:

$$\begin{aligned} \text{diff}_f &= 2^{-\lambda m} \cdot \max_{\Delta_x \neq 0, \Delta_f} \#\{x \in \mathbb{F}_{2^\lambda}^m : f(x) \oplus f(x \oplus \Delta_x) = \Delta_f\}, \\ \text{cor}_f &= 2^{-\lambda m} \cdot \max_{\Gamma_f \neq 0, \Gamma_x} \left| \sum_{x \in \mathbb{F}_{2^\lambda}^m} (-1)^{\text{Tr}(\Gamma_f \cdot f(x) + \Gamma_x \cdot x)} \right|, \end{aligned}$$

where $\text{Tr} : \mathbb{F}_{2^\lambda} \rightarrow \mathbb{F}_2$ is the trace function, i.e., $\text{Tr}(x) = x + x^2 + x^{2^2} + \dots + x^{2^{\lambda-1}}$.

In **Lynx**, activating any nonlinear component implies activating all nonlinear components. Hence, the lower bound of the differential (resp. linear) trail equals the product of their individual diff (resp. cor^2) values. As shown in Table 2, these bounds are strictly less than $2^{-\lambda}$.

Randomness testing. To guarantee that $\{t_i\}_i$ is indistinguishable from a uniformly random string of equal length—even in the multi-user setting—we subject $\geq 10^{10}$ -bit samples of $\{t_i\}_i$ to the NIST SP 800-22 test suite [BRS⁺10], including the frequency, block-frequency, runs, overlapping template, and random walk tests, among others. All variants of **Lynx** pass all required tests at all targeted security levels.

8.1.3 Quantum Attacks

Grover’s algorithm [Gro96b] provides a quadratic speedup over classical brute-force key search. In this section, we analyze **Lynx**’s resistance to Grover’s preimage attack and examine whether any specialized quantum algorithm for **Lynx** could outperform it.

Grover’s preimage attack. We adopt NIST’s quantum resource metric [NIS22]:

$$(\text{Grover’s cost}) = (\text{circuit size}) \times (\text{circuit depth}),$$

in Clifford/T gates. For a one-way function $f : \{0, 1\}^\lambda \rightarrow \{0, 1\}^m$, Grover’s preimage search requires [JBK⁺22]

$$(\text{Grover’s circuit size/depth}) = (\text{size/depth of } f) \times 2 \times \left\lfloor \frac{\pi}{4} \sqrt{2^\lambda} \right\rfloor.$$

When applying Grover’s algorithm to **Lynx** given t , it is more efficient to compute v_1 and v_2 via Eq.(5)-(6) and verify Eq.(9) per k candidate than to directly check $\text{Lynx}(\text{iv}_{\text{owf}}, k) = t$. **Lynx**’s quantum implementation breaks down into three basic operations: finite-field multiplication (FFM), finite-field squaring (FFS), and random matrix multiplication (MM), enabling direct cost citations for each.

Table 17 reports the count and depth of basic operations per **Lynx**’s component, with S-box evaluations based on the best addition chains we found. Table 17 also gives the costs of Grover’s preimage attack for **Lynx**, where **Lynx**-256 meet NIST’s PQC security levels V—defined as the quantum-resistant equivalents of AES-256. Grover’s cost may be further reduced—for example, via improved circuit representation or more efficient S-box addition chains. However, such optimizations are unlikely to bring the cost below that of AES at the corresponding security level, as finite-field multiplications alone already exceed AES’s total cost.

The common claim that Grover’s algorithm halves the bit security of symmetric cryptography is only technically true in a narrow theoretical sense and highly misleading in practice. Grover’s quadratic speedup is real, but the algorithm is inherently sequential (cannot be split across multiple quantum processors the way classical computation can). To capture this real-world limit, NIST uses a parameter called “MAXDEPTH”, plausible values range from 2^{40} (roughly the serial throughput of near-future architectures over one year) through 2^{64} , up to a theoretical ceiling of 2^{96} . Without taking “MAXDEPTH” into account, the quantum security of **Lynx** meets all quantum security strength levels. When factoring in NIST’s quantum

Table 17: Total cost of Grover’s algorithm on Lynx for each level of security.

λ	#FFM $d_1, d_2, \times$	FFM depth	#FFS d_1, d_2	FFS depth	#MM	Gate	Depth	Grover’s cost	Security level
160	12, 8, 1	13	159, 67	159	4	101.6	95.4	197.0	N/A
256	10, 8, 2	11	255, 52	255	4	150.6	144.7	295.3	$V(\geq 285)$
384	13, 7, 2	14	383, 59	383	4	216.0	209.9	425.8	N/A
512	14, 9, 2	15	511, 179	511	4	280.6	274.7	555.3	N/A

† d_1, d_2 and $\times$ correspond to the number and depth of basic operations required to implement the S-boxes as defined in Table 3 and the final multiplication.

security requirements, the security margin will be higher based on “Grover’s cost divided by MAXDEPTH”.

Other quantum attacks. GroverXL [BY18] and QuantumBooleanSolve [FHK⁺17] are hybrid quantum-classical attacks on MQ systems over $\mathbb{F}_2$. Both exploit Grover’s quadratic speedup to reduce brute-force guessing cost, then solve the residual low-dimensional system classically via FXL or BooleanSolve, and numerically optimize the fraction of guessed variables to balance quantum and classical effort. Their asymptotic complexities are nearly identical; only the classical solver differs. For Lynx, the complexity exceeds Grover’s generic bound of $2^{0.5\lambda}$ across all security levels, e.g., $\mathcal{O}(2^{1.008\lambda})$ at a 59% for $\lambda = 256$. This degradation stems from introducing 2λ intermediate variables during algebraic solving, nullifying Grover’s quadratic advantage.

A quantum algorithm based on the HHL algorithm [CG22] solves sparse linear systems with exponential speedup under restrictive conditions when the input’s (i.e., k ’s) Hamming weight is below $\mathcal{O}(\log \lambda)$; otherwise, it is slower than Grover’s algorithm.

Quantum generic attacks exploit no special property of the underlying primitives and treat them as public random permutations or functions. Most rely on Simon’s period-finding algorithm (Q2) or Grover/offline Simon variants (Q1). Since Simon’s algorithm requires multiple queries, these attacks do not apply to Lynx.

8.2 Known Attacks

8.2.1 Brute-Force the Public Key

The simplest attack strategy is to recover the signing key by inverting the OWF output contained in the public key. In Lynx, this exactly corresponds to the Lynx one-wayness game, and the complexity of this attack is analyzed in the Lynx cryptanalysis summarized in Section 8.1.

8.2.2 Brute-Force the TCCR Tree Generation

Another way to try to recover the signing key is to attack the TCCR construction used in BAVC.Commit and BAVC.Verify to generate the correlated GGM tree. This is the analogue of the PRG attack discussed in FAEST, except that our tree expansion in BAVC.Commit is driven by a tweakable circular-correlation-robust hash rather than a length-doubling PRG. At each internal node of the tree, a signature reveals all-but-one of the relevant descendants along the opened paths. An attacker may therefore try to guess the hidden node value and test the guess by evaluating TCCR and checking consistency with the revealed tree structure. This is a multi-target attack surface, because many tree nodes across one or more signatures are expanded under the same construction. However, the relevant security notion here is precisely the multi-target TCCR security of the block-cipher-based construction. Since the per-signature salt iv_{sig} is chosen uniformly at random and independently for every signature, and the tree computations are domain-separated by their explicit inputs (including $\text{const}_0, \text{const}_1$ in the

lower half of the tree), we are not aware of any attack that performs better than attacking a single target of the underlying block-cipher-based TCCR primitive.

Moreover, the proof in [Section 9.4.2](#) guarantees information-theoretic security in the ideal cipher model. Therefore, the advantage of any attack against TCCR translates to a distinguishing attack against the underlying block cipher.

8.2.3 Brute-Force the PRG

The PRG in Lynxer is used in [LeafHash](#) and [VOLE.ConvertToVOLE](#). In [LeafHash](#), a hidden leaf seed can be tested by expanding it to a candidate commitment. In [VOLE.ConvertToVOLE](#), all-but-one seeds are known after opening, so an attacker may try to guess the missing seed and verify the resulting VOLE correlation. This is similar in spirit to the corresponding FAEST attack on the PRG used in leaf commitments and VOLE conversion. Nevertheless, the calls are separated by the random per-signature iv_{sig} and by distinct counter values, so we are not aware of any effective multi-target attack that combines these invocations in a way that is meaningfully better than attacking a single target of the underlying block-cipher-based PRG.

Given multiple signatures. Suppose the attacker is given up to Q_{sig} signatures. Each signature uses an independent random $iv_{\text{sig}} \in \{0, 1\}^{120}$, so IV collisions are expected to remain negligible as long as $Q_{\text{sig}} \ll 2^{60}$. If the IVs are unique, then both the TCCR-based tree generation and the PRG-based expansions are independent across signatures, and the best known attacks reduce to the corresponding single-signature attacks described above. Even if a small number of IV collisions were to occur accidentally, they are not under the control of the attacker and do not appear to create a practical attack vector that competes with directly attacking the Lynx instance.

8.2.4 Attack Soundness of the ZK Protocol

Instead of recovering the signing key, an attacker may attempt to forge a signature by violating the soundness of the underlying VOLE-in-the-Head proof system. This possibility is covered by the EUF-KO security proof in [Theorem 9.4](#). As in FAEST, the concrete meaning of this bound is that an attacker must either break the random-oracle layer, violate the binding and hiding guarantees of the commitment layer, or produce a false witness that passes the QuickSilver checks. We are not aware of any attack strategy on these components that is more effective than exhaustive search against the underlying primitives.

8.2.5 Multi-User Attacks

We also consider the setting where the attacker is given many public keys and signatures under independently generated keys. Since each public key uses an independently sampled Lynx instance, there is no obvious way to aggregate users into a single multi-target inversion attack on the OWF. Moreover, the fresh per-signature IV randomizes the TCCR and PRG instances used inside the commitment and VOLE layers, so the auxiliary symmetric-key computations are also isolated across users. Finally, key-substitution attacks [\[MS04\]](#) are prevented because the public key is hashed together with the message in [MsgHash](#), so each signature is bound to its originating public key.

9 Security Proof

We first prove functional correctness and then present the security proofs against any quantum/classical adversary. For simplicity, we omit the public parameters (e.g., param) of functions and leave them implicit in this section.

9.1 Correctness Analysis

Correctness of VOLE commitment. We first prove the correctness of the subroutine [VOLE.ConvertToVOLE](#) used in the VOLE commitment scheme, and then derive the correctness of the full VOLE commitment scheme from it.

Lemma 9.1 (Correctness of [VOLE.ConvertToVOLE](#)). *Given any $k \in \mathbb{N}^+$, $N = 2^k$, and $\Delta \in [0, N)$, let $(\text{seed}_0, \dots, \text{seed}_{N-1})$ and $(\overline{\text{seed}}_0, \dots, \overline{\text{seed}}_{N-1})$ satisfy*

$$\begin{aligned} \overline{\text{seed}}_0 &= \perp, \\ \forall i \in [1, N) : \overline{\text{seed}}_i &= \text{seed}_{i \oplus \Delta}. \end{aligned}$$

For any iv_{sig} , it holds that

$$\Pr \left[\begin{array}{l} (\delta_0, \dots, \delta_{k-1}) \leftarrow (\Delta[0], \dots, \Delta[k-1]), \\ (\mathbf{u}, [\mathbf{v}_0, \dots, \mathbf{v}_{k-1}]) \leftarrow \text{VOLE.ConvertToVOLE}((\text{seed}_i)_{i \in [0, N)}, \text{iv}_{\text{sig}}), \\ (\mathbf{u}', [\mathbf{q}_0, \dots, \mathbf{q}_{k-1}]) \leftarrow \text{VOLE.ConvertToVOLE}((\overline{\text{seed}}_i)_{i \in [0, N)}, \text{iv}_{\text{sig}}) \\ : \forall j \in [0, k) : \mathbf{q}_j = \mathbf{v}_j \oplus \delta_j \cdot \mathbf{u} \end{array} \right] = 1.$$

Proof. Since [PRG](#) is deterministic, the [PRG](#) outputs $(\mathbf{r}_{0,i})_{i \in [0, N)}$ computed from $(\text{seed}_i)_{i \in [0, N)}$ (i.e., the outputs of the prover) and the [PRG](#) outputs $(\overline{\mathbf{r}}_{0,i})_{i \in [0, N)}$ computed from $(\overline{\text{seed}}_i)_{i \in [0, N)}$ (i.e., the outputs of the verifier) satisfy the basic equality:

$$\forall i \in [1, N) : \overline{\mathbf{r}}_{0,i} = \mathbf{r}_{0,i \oplus \Delta}. \quad (14)$$

According to [VOLE.ConvertToVOLE](#), it holds for the prover that

$$\begin{aligned} \mathbf{r}_{j,i} &= \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \mathbf{r}_{0,d}, \\ \mathbf{u} = \mathbf{r}_{k,0} &= \sum_{d \in [0, N)} \mathbf{r}_{0,d} \\ \mathbf{v}_j &= \sum_{i \in [0, \frac{N}{2^j+1})} \mathbf{r}_{j,2i+1}. \end{aligned} \quad (15)$$

Meanwhile, it holds for the verifier that

$$\begin{aligned} \overline{\mathbf{r}}_{j,i} &= \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \overline{\mathbf{r}}_{0,d}, \\ \mathbf{q}_j &= \sum_{i \in [0, \frac{N}{2^j+1})} \overline{\mathbf{r}}_{j,2i+1}. \end{aligned} \quad (16)$$

We use an induction to prove $\overline{\mathbf{r}}_{j,i} = \mathbf{r}_{j,i \oplus \Delta[j:k-1]}$ for any $j \in [0, k)$, $i \in [1, \frac{N}{2^j})$, and the $(k-j)$ -bit prefix $\Delta[j:k-1] = (\Delta[j], \dots, \Delta[k-1]) = (\delta_j, \dots, \delta_{k-1})$. For the base case $j = 0$, this target equality holds due to eq.(14). Then, we assume that the target equality holds for $j-1 \in [0, k-1)$ and consider the case $j \in [1, k)$ for induction. Following from eq.(15) and

eq.(16), it holds that

$$\begin{aligned} \mathbf{r}_{j,i} &= \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \mathbf{r}_{0,d} = \sum_{d=(2i) \cdot 2^{j-1}}^{(2i+2) \cdot 2^{j-1} - 1} \mathbf{r}_{0,d} \\ &= \left(\sum_{d=(2i) \cdot 2^{j-1}}^{(2i+1) \cdot 2^{j-1} - 1} \mathbf{r}_{0,d} \right) \oplus \left(\sum_{d=(2i+1) \cdot 2^{j-1}}^{(2i+2) \cdot 2^{j-1} - 1} \mathbf{r}_{0,d} \right) = \mathbf{r}_{j-1,2i} \oplus \mathbf{r}_{j-1,2i+1}, \end{aligned} \quad (17)$$

$$\begin{aligned} \bar{\mathbf{r}}_{j,i} &= \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \bar{\mathbf{r}}_{0,d} = \sum_{d=(2i) \cdot 2^{j-1}}^{(2i+2) \cdot 2^{j-1} - 1} \bar{\mathbf{r}}_{0,d} \\ &= \left(\sum_{d=(2i) \cdot 2^{j-1}}^{(2i+1) \cdot 2^{j-1} - 1} \bar{\mathbf{r}}_{0,d} \right) \oplus \left(\sum_{d=(2i+1) \cdot 2^{j-1}}^{(2i+2) \cdot 2^{j-1} - 1} \bar{\mathbf{r}}_{0,d} \right) = \bar{\mathbf{r}}_{j-1,2i} \oplus \bar{\mathbf{r}}_{j-1,2i+1}. \end{aligned} \quad (18)$$

Combining eq.(17), eq.(18), and the induction assumption, we have for any $i \in [1, \frac{N}{2^j})$,

$$\begin{aligned} \bar{\mathbf{r}}_{j,i} &= \bar{\mathbf{r}}_{j-1,2i} \oplus \bar{\mathbf{r}}_{j-1,2i+1} && \text{(By eq.(18))} \\ &= \mathbf{r}_{j-1,(2i) \oplus \Delta_{[j-1:k-1]}} \oplus \mathbf{r}_{j-1,(2i+1) \oplus \Delta_{[j-1:k-1]}} && \text{(By the induction assumption)} \\ &= \mathbf{r}_{j-1,2(i \oplus \Delta_{[j:k-1]})} \oplus \mathbf{r}_{j-1,2(i \oplus \Delta_{[j:k-1]})+1} && \text{(For any } \delta_{j-1} \in \{0, 1\}) \\ &= \mathbf{r}_{j,i \oplus \Delta_{[j:k-1]}} && \text{(By eq.(17))} \end{aligned}$$

showing that the target equality holds and concluding the induction.

For any $j \in [0, k)$, if $\delta_j = 0$ (i.e., $\Delta_{[j:k-1]}$ is even), it holds that $\mathbf{q}_j = \mathbf{v}_j$ since the permutation $i \mapsto (i \oplus \Delta_{[j:k-1]})$ does not change the oddness of the second subscripts of the terms summed in $\mathbf{q}_j$ and $\mathbf{v}_j$ so that the two sums are equivalent. Otherwise, if $\delta_j = 1$ (i.e., $\Delta_{[j:k-1]}$ is odd), the permutation will flip the parity of the second subscripts of the summed terms. By using the proved target equality for all $\bar{\mathbf{r}}_{j,i}$'s with odd i 's (thus, $i \in [1, \frac{N}{2^j})$) in $\mathbf{q}_j$, we have that

$$\begin{aligned} \mathbf{q}_j \oplus \mathbf{v}_j &= \left(\sum_{i \in [0, \frac{N}{2^{j+1}})} \bar{\mathbf{r}}_{j,2i+1} \right) \oplus \left(\sum_{i \in [0, \frac{N}{2^{j+1}})} \mathbf{r}_{j,2i+1} \right) \\ &= \left(\sum_{i \in [0, \frac{N}{2^{j+1}})} \mathbf{r}_{j,2i} \right) \oplus \left(\sum_{i \in [0, \frac{N}{2^{j+1}})} \mathbf{r}_{j,2i+1} \right) \\ &= \sum_{i \in [0, \frac{N}{2^j})} \mathbf{r}_{j,i} = \sum_{i \in [0, \frac{N}{2^j})} \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \mathbf{r}_{0,d} = \sum_{d \in [0, N)} \mathbf{r}_{0,d} = \mathbf{u}. \end{aligned}$$

Putting the two cases together, we have that $\mathbf{q}_j = \mathbf{v}_j \oplus \delta_j \cdot \mathbf{u}$ for any $j \in [0, k)$. $\square$

Definition 9.1 (Correctness of VOLE Commitment). *The correctness of the VOLE commitment scheme $\text{VOLE} = (\text{VOLE.Commit}, \text{VOLE.Reconstruct})$ holds if:*

- $\forall \mathbf{r} \in \{0, 1\}^\lambda, \text{iv}_{\text{sig}} \in \{0, 1\}^{120}, \mu \in \{0, 1\}^\lambda, \mathbf{s} \in \{0, 1\}^\lambda$.
- $(\text{com}, \text{decom}, (\mathbf{c}_\alpha)_{\alpha \in [1, \tau]}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLE.Commit}(\mathbf{r}, \text{iv}_{\text{sig}}, \mu, \mathbf{s})$.
- $\forall \text{chall} \in \{0, 1\}^\lambda, I = (I_0, \dots, I_{\tau-1}) \leftarrow \text{VOLE.ChalDecAll}(\text{chall})$.
- $\text{decom}_I \leftarrow \text{BAVC.Open}(\text{decom}, I, \text{iv}_{\text{sig}})$.

- One of the following cases holds:
 - **VOLE.Reconstruct** outputs $\perp$, or
 - $(\overline{\text{com}}, Q) \leftarrow \text{VOLE.Reconstruct}(\text{chall}, \text{decom}_I, (\mathbf{c}_\alpha)_{\alpha \in [1, \tau]}, \text{iv}_{\text{sig}}, \mu, \mathbf{s})$ such that $\overline{\text{com}} = \text{com}$ and $Q = V \oplus \text{chall} \cdot \mathbf{u}$.

Lemma 9.2 (Correctness of VOLE Commitment). *The VOLE commitment scheme $\text{VOLE} = (\text{VOLE.Commit}, \text{VOLE.Reconstruct})$ satisfies the correctness defined in Definition 9.1.*

Proof. **VOLE.Reconstruct** returns $\perp$ if and only if **BAVC.Verify** returns $\perp$, and the latter happens if and only if $|\text{id-list}| > T_{\text{open}}$.

Otherwise, the partial decommitment decom_I gives the binary-tree nodes in **BAVC.Commit** indexed by id-list that span all leaves except the τ punctured leaves in the set I , together with the commitment values corresponding to these punctured leaves. Using the binary-tree nodes in id-list , **BAVC.Verify** reconstructs all seed values and commitment values except those at the punctured leaves. It then uses these commitment values together with the commitment values of the punctured leaves to recover the same com as computed by **BAVC.Commit**.

Moreover, for each $\alpha \in [0, \tau)$, the seed values returned by **BAVC.Verify** agree with those computed by **BAVC.Commit** at every position in the block of size N_α except at the punctured index Δ_α . By Lemma 9.1, we obtain at step 16 of **VOLE.Reconstruct** that

$$Q'_\alpha = V_\alpha \oplus [\delta_{\alpha,0} \cdot \mathbf{u}_\alpha, \dots, \delta_{\alpha,n_\alpha-1} \cdot \mathbf{u}_\alpha],$$

where $(\delta_{\alpha,0}, \dots, \delta_{\alpha,n_\alpha-1}) = (\Delta_\alpha[0], \dots, \Delta_\alpha[n_\alpha - 1])$. Since $\mathbf{c}_\alpha = \mathbf{u} \oplus \mathbf{u}_\alpha$ for each $\alpha \in [1, \tau)$, the final output of **VOLE.Reconstruct** satisfies

$$\begin{aligned} \forall \alpha \in [0, \tau) : Q_\alpha &= V_\alpha \oplus [\delta_{\alpha,0} \cdot \mathbf{u}, \dots, \delta_{\alpha,n_\alpha-1} \cdot \mathbf{u}], \\ Q &= V \oplus [\delta_{0,0} \cdot \mathbf{u}, \dots, \delta_{0,n_0-1} \cdot \mathbf{u}, \dots, \delta_{\tau-1,0} \cdot \mathbf{u}, \dots, \delta_{\tau-1,n_{\tau-1}-1} \cdot \mathbf{u}]. \end{aligned}$$

This proves correctness. $\square$

Correctness of QuickSilver. Given correctly generated VOLE correlations, we prove the correctness of QuickSilver, the zero-knowledge proof protocol used in our signature scheme.

Lemma 9.3 (Correctness of the QuickSilver Proof System). *For any $\text{chall}_3 \in \{0, 1\}^{\lambda - w_{\text{grind}}}$, let a set of VOLE correlations $(\mathbf{u}, V, Q, \Delta) \in \mathbb{F}_2^{\ell_{\text{wit}} + \lambda} \times \mathbb{F}_2^{(\ell_{\text{wit}} + \lambda) \times \lambda} \times \mathbb{F}_2^{(\ell_{\text{wit}} + \lambda) \times \lambda} \times \mathbb{F}_2^\lambda$ satisfy*

$$Q = V \oplus [\delta_{0,0} \cdot \mathbf{u}, \dots, \delta_{0,n_0-1} \cdot \mathbf{u}, \dots, \delta_{\tau-1,0} \cdot \mathbf{u}, \dots, \delta_{\tau-1,n_{\tau-1}-1} \cdot \mathbf{u}],$$

where $\Delta = (\Delta_0, \dots, \Delta_{\tau-1}) \leftarrow \text{VOLE.ChalDecAll}(\text{chall}_3 \| 0_{\text{grind}}^w; \text{param})$, and for each $i \in [0, \tau)$ we have $(\delta_{i,0}, \dots, \delta_{i,n_i-1}) \leftarrow (\Delta_i[0], \dots, \Delta_i[n_i - 1])$. For any chall_2 , we have

$$\Pr \left[\begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{Lynxer.KeyGen}(\text{param}), \\ \text{pk} := (\text{iv}_{\text{owf}}, t) \\ \text{sk} := (\text{iv}_{\text{owf}}, k) \\ (v_1, v_2) \leftarrow \text{Lynx.ExtendWitness}(\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, c_1, c_2, c_3, k; \text{param}) \\ \mathbf{w} \leftarrow \text{FieldToBits}(k; \lambda) \| \text{FieldToBits}(v_1; \lambda) \| \text{FieldToBits}(v_2; \lambda) \\ d \leftarrow \mathbf{w} \oplus \mathbf{u}[0, \ell), : a_0 = \tilde{a}_0 \\ (a_1, a_0) \leftarrow \text{QS.LynxProve} \left(\begin{array}{l} \mathbf{w}, \mathbf{u}, V, \mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_3, \\ c_1, c_2, c_3, t, \text{chall}_2; \text{param} \end{array} \right), \\ \tilde{a}_0 \leftarrow \text{QS.LynxVerify} \left(\begin{array}{l} d, Q, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \mathbf{L}_0, \mathbf{L}_1, \\ \mathbf{L}_2, \mathbf{L}_3, c_1, c_2, c_3, t; \text{param} \end{array} \right) \end{array} \right] = 1.$$

Proof. The VOLE correlations are converted into a single VOLE relation over the extension field. Because the extended witness $\mathbf{w}$ is legitimate (i.e., it satisfies all constraint polynomials that captures the Lynx computation), the QuickSilver verifier's polynomial evaluates to a quadratic whose constant term equals a_0 . $\square$

Correctness of Lynxer. Based on the correctness of QuickSilver, we can prove the correctness of our signature scheme.

Theorem 9.1 (Correctness of Lynxer). *For the digital signature scheme Lynxer with public parameters $\mathbf{param}$ and any message $\mathbf{msg} \in \{0, 1\}^*$, we have*

$$\Pr \left[\begin{array}{l} (\mathbf{pk}, \mathbf{sk}) \leftarrow \text{Lynxer.KeyGen}(\mathbf{param}), \\ \sigma \leftarrow \text{Lynxer.Sign}(\mathbf{sk}, \mathbf{msg}; \mathbf{param}), : b = 1 \\ b \leftarrow \text{Lynxer.Verify}(\mathbf{pk}, \mathbf{msg}, \sigma; \mathbf{param}) \end{array} \right] = 1.$$

Proof. For a given signature σ , up to Step 3 of `Lynxer.Verify`, the correctness of the VOLE commitment scheme (Lemma 9.2) implies that the value `com` computed in Step 9 of `Lynxer.Sign` equals the value computed in Step 3 of `Lynxer.Verify`, and that

$$\begin{aligned} \mathbf{Q} &= \mathbf{V} \oplus [\delta_{0,0} \cdot \mathbf{u}, \dots, \delta_{0,n_0-1} \cdot \mathbf{u}, \dots, \delta_{\tau-1,0} \cdot \mathbf{u}, \dots, \delta_{\tau-1,n_{\tau-1}-1} \cdot \mathbf{u}], \\ \tilde{\mathbf{Q}} &= \tilde{\mathbf{V}}, \end{aligned}$$

where $(\Delta_0, \dots, \Delta_{\tau-1}) \leftarrow \text{VOLE.ChalDecAll}(\text{chall}_3)$ and, for each $i \in [0, \tau)$, $(\delta_{i,0}, \dots, \delta_{i,n_i-1}) \leftarrow (\Delta_i[0], \dots, \Delta_i[n_i-1])$. These equalities imply that `Lynxer.Sign` and `Lynxer.Verify` compute the same values of `chall1` and `chall2`.

By the correctness of the QuickSilver proof system (Lemma 9.3), the value $\tilde{a}_0$ computed in `QS.LynxVerify` equals the value $\tilde{a}_0$ implicitly determined by the signer through the call to `QS.LynxProve` in `Lynxer.Sign`. Together with the above equalities and the definition of `ctr` in a valid signature, it follows that the value `chall'3` computed by `Lynxer.Verify` satisfies the check in Step 13, so `Lynxer.Verify` outputs 1. Hence the theorem follows. $\square$

9.2 Security Proof in the QROM and QICM

Proof overview. Our proof uses a similar blueprint to FAESTv2 [BBB⁺26] in the round 2 NIST submission. In the QROM, we would like to reduce the EUF-KO security of Lynxer to the soundness of its underlying (public-coin) IP system. To do so, we first view the random-oracle calls in Lynxer (including those at very low-level subroutine `BAVC.Commit`) as the generation of Fiat-Shamir challenges. From the view of Fiat-Shamir transformation, we can reversely define the underlying IP system and analyze its RBR soundness. Then, using the implication lemma of [BBB⁺25] from RBR soundness to soundness, we prove the soundness of the IP system. Finally, by switching a valid public key in the EUF-KO security game into a random one, which would be invalid with a non-negligible probability (depending on the concrete parameter choices), we obtain a lossy-key argument [KW03] to show that an efficient EUF-KO adversary can break the soundness of the IP system when the public key is invalid. This proof strategy does not require the knowledge soundness of the IP system since the witness extraction is difficult in the QROM.

Moreover, we can reduce the EUF-CMA security to the EUF-KO security. The high-level idea is that we use a hybrid argument to replace the signature oracle $\text{Osig}_{\mathbf{sk}}$ by a simulated oracle that never requires the secret key $\mathbf{sk}$. In this hybrid argument, we use the quantum unpredictability of the commitment scheme and the adaptive reprogramming lemma [LGX25]

5. It does not query any random oracle but instead interacts with the verifier $\mathcal{V}_{\text{Lynxer-IP}}$ to receive the challenges (see Fig. 6) as the outputs of random oracle.

The verifier $\mathcal{V}_{\text{Lynxer-IP}}$ runs [Lynxer.Verify](#) but has the following modifications:

1. It does not call [Lynx.SampleMatVec](#) to compute $(\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, c_1, c_2, c_3)$ but uses these values in pk' instead. It never uses iv_{owf} .
2. It replaces pk by pk' in [MsgHash](#) at the step 3.
3. It ignores the computation of challenges at the step 3 of [Lynxer.Verify](#), the step 27 of [BAVC.Verify](#) and the step 6, 9, and 12 of [Lynxer.Verify](#), but sends uniformly sampled challenges (with their specified lengths) to the prover $\mathcal{P}_{\text{Lynxer-IP}}$.
4. Given $\text{pmsg}_0 = (\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha]}, \text{iv}_{\text{sig}})$, it outputs **reject** finally if there exists $\alpha \in [0, \tau)$ such that (i) there exists $i \in [0, N_\alpha)$ such that com_i^α has at least two pre-images, or (ii) there exist two distinct $i, i' \in [0, N_\alpha)$ such that both com_i^α and $\text{com}_{i'}^\alpha$ have no pre-images. The verifier needs a brute force for all possible pre-images and is exponential-time. However, the soundness of Lynxer-IP does not require an efficient verifier so that this runtime has no effect.
5. Given $\text{pmsg}_2 = (\tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d})$, if $\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}}$ at the step 8, it outputs **reject** finally.
6. Given $\text{pmsg}_3 = (\tilde{a}_1, \tilde{a}_0, \text{ctr})$, if $\tilde{a}_0 \neq \tilde{a}_0$ at the step 11, it outputs **reject** finally. Moreover, it ignores the condition “ $\text{chall}_3 = \text{chall}'_3$ ” at the step 13.
7. Given $\text{pmsg}_4 = \text{decom}_I$, it outputs **reject** finally if the commitments $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}$ reconstructed from decom_I (as per [BAVC.Verify](#)) do not equal $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}$.

The above IP system Lynxer-IP does not use any random oracle. We will see that replacing pk by pk' in [MsgHash](#) does not affect the soundness since $(\mu, \mathbf{s})$ is identically distributed. To see this, we apply the Fiat-Shamir transformation [FS87] to Lynxer-IP to obtain a signature scheme FS[Lynxer-IP]. More concretely, the transformation is as follows:

- The prover (i.e., the signing algorithm) invokes random oracles to non-interactively produce the challenges of the verifier:
 1. $\text{chall}_{-1} = (\mu, \mathbf{s}) = \text{MsgHash}(\text{pk}', \text{pmsg}_{-1}) = \text{MsgHash}(\text{pk}', \text{msg})$.
 2. $\text{chall}_0 = \text{com} = \text{BAVCHash}(\text{chall}_{-1}, \text{pmsg}_0) = \text{BAVCHash}(\mu, \mathbf{s}, \{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}, \text{iv}_{\text{sig}})$.
 3. $\text{chall}_1 = \text{FSHash}_1(\text{chall}_0, \text{pmsg}_1) = \text{FSHash}_1(\text{com}, \{\mathbf{c}_\alpha\}_{\alpha \in [1, \tau)})$.
 4. $\text{chall}_2 = \text{FSHash}_2(\text{chall}_1, \text{pmsg}_2) = \text{FSHash}_2(\text{chall}_1, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d})$.
 5. $\text{chall}_3 = \text{FSHash}_3(\text{chall}_2, \text{pmsg}_3) = \text{FSHash}_3(\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$.

Finally, the prover sends the message $\text{msg} = \text{pmsg}_{-1}$ and its signature σ' to the verifier:

$$\begin{aligned} \sigma' &= (\text{pmsg}_0, \text{pmsg}_1, \text{pmsg}_2, \text{pmsg}_3, \text{pmsg}_4) \\ &= (\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}, \text{iv}_{\text{sig}}, \{\mathbf{c}_\alpha\}_{\alpha \in [1, \tau)}, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d}, \tilde{a}_1, \tilde{a}_0, \text{ctr}, \text{decom}_I). \end{aligned} \quad (19)$$

- The verifier (i.e., the verification algorithm) will use $\mathbf{x}$, σ' , and the given random oracles to reproduce the above challenges if needed.

Note that the signature scheme FS[Lynxer-IP] uses uniformly sampled $(r, \text{iv}_{\text{sig}})$ due to [Item 3](#).

RBR soundness of Lynxer-IP. Lynxer-IP does not use any random oracle. The following theorem comes from pure probability analysis based on the uniformly random challenges.

Theorem 9.2 (RBR Soundness of Lynxer-IP). *In Lynxer-IP, consider the following ideal-model oracles and their respective numbers of queries:*

- E is an ideal cipher realizing **PRG** in **LeafHash**, with Q_E queries,
- E' is an ideal cipher realizing **TCCR** in **BAVC.Commit**, with $Q_{E'}$ queries.

Let $\mathcal{O}$ be a list of the above ideal-model oracles and $Q_{\mathcal{O}}$ be a list of their respective numbers of queries. Let $\mathcal{R}$ be the Lynx relation and $\mathcal{L}_{\mathcal{R}}$ be its language. For the language $\mathcal{L}_{\mathcal{R}}$, Lynxer-IP is ε_{RBR} -RBR-sound in the QICM parameterized by $(\mathcal{O}, Q_{\mathcal{O}})$, where

$$\begin{aligned}\varepsilon_{\text{RBR}} &= \max(\varepsilon_{-1}, \varepsilon_0, \varepsilon_1, \varepsilon_2, \varepsilon_3), \\ \varepsilon_{-1} &= \varepsilon_0 = 0, \\ \varepsilon_1 &= \varepsilon_{\text{vole}} \cdot \binom{\tau}{2}, \varepsilon_2 = \varepsilon_{\text{zk}}, \varepsilon_3 = \frac{2}{2^\lambda}.\end{aligned}$$

Proof. The RBR soundness is taken over the random challenges and independent of the ideal ciphers. We construct the following *non-polynomial-time* algorithm **state**.

- Initial case. On input an instance $\mathbb{x}$ and an empty transcript $\text{tr}_0 = \emptyset$, **state** enumerates all possible witness to check whether $\mathbb{x} \in \mathcal{L}_{\mathcal{R}}$. If and only if $\mathbb{x} \notin \mathcal{L}_{\mathcal{R}}$, **state** outputs **reject**.
- On input $\mathbb{x}$ and $\text{tr}_2 = (\text{msg}, \mu, s)$, **state** outputs $\text{state}(\mathbb{x}, \text{tr}_0)$. We have

$$\Pr_{(\mu, s)} [\text{state}(\mathbb{x}, \text{tr}_2) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_0) = \text{reject}] = 0 = \varepsilon_{-1}.$$

- On input $\mathbb{x}$ and $\text{tr}_4 = (\text{tr}_2, \{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}, \text{iv}_{\text{sig}}, \text{com})$, **state** outputs $\text{state}(\mathbb{x}, \text{tr}_2)$.

$$\Pr_{\text{com}} [\text{state}(\mathbb{x}, \text{tr}_4) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_2) = \text{reject}] = 0 = \varepsilon_0.$$

- On input $\mathbb{x}$ and $\text{tr}_6 = (\text{tr}_4, \{c_\alpha\}_{\alpha \in [1, \tau)}, \text{chall}_1)$, if $\text{state}(\mathbb{x}, \text{tr}_4) = \text{accept}$, **state** outputs **accept**; otherwise, **state** enumerates all possible pre-images and distinguishes the following cases:
 - Case **BadPre**: There exists $\alpha \in [0, \tau)$ such that (i) there exists $i \in [0, N_\alpha)$ such that com_i^α has at least two pre-images, or (ii) there exist two distinct $i, i' \in [0, N_\alpha)$ such that both com_i^α and $\text{com}_{i'}^\alpha$ have no pre-images. In this case, **state** outputs **reject**.
Conditioned on that **BadPre** does not happen (denoted by an event $\neg \text{BadPre}$), there must be one of the following two cases for each $\alpha \in [0, \tau)$.
 - Case **OneNoPre** $_\alpha$: There exists only one $i \in [0, N_\alpha)$ such that com_i^α has no pre-image (i.e., for any $i' \neq i$, $\text{com}_{i'}^\alpha$ has a unique pre-image $\text{seed}_{i'}^\alpha$). **state** defines $\Delta_\alpha = i$ and extracts

$$\begin{aligned}(\mathbf{u}'_\alpha, \mathbf{Q}'_\alpha) &= \text{VOLE.ConvertToVOLE}((\perp, \text{seed}_{1 \oplus \Delta_\alpha}^\alpha, \dots, \text{seed}_{(N_\alpha - 1) \oplus \Delta_\alpha}^\alpha), \text{iv}_{\text{sig}}), \\ (\mathbf{u}_\alpha, \mathbf{V}_\alpha) &= \begin{cases} (\mathbf{u}'_\alpha, \text{VOLE.Fix}(\mathbf{Q}'_\alpha, \mathbf{u}'_\alpha, \Delta_\alpha)), & \alpha = 0 \\ (\mathbf{u}'_\alpha \oplus c_\alpha, \text{VOLE.Fix}(\mathbf{Q}'_\alpha, \mathbf{u}'_\alpha, \Delta_\alpha)), & \alpha \in [1, \tau) \end{cases} \end{aligned} \quad (20)$$

- Case **UniquePre** $_\alpha$: For any $i \in [0, N_\alpha)$, com_i^α has a unique pre-image seed_i^α . **state** extracts

$$(\mathbf{u}_\alpha, \mathbf{V}_\alpha) = \text{VOLE.ConvertToVOLE}(\{\text{seed}_i^\alpha\}_{i \in [0, N_\alpha)}, \text{iv}_{\text{sig}}). \quad (21)$$

That is, conditioned on $\neg\text{BadPre}$, state can extract $(\mathbf{u}_\alpha, \mathbf{V}_\alpha)$ for each $\alpha \in [0, \tau)$. We define, conditioned on $\neg\text{BadPre}$, an event BadCh_1 that there exist two distinct $\alpha, \alpha' \in [0, \tau)$ such that $\mathbf{u}_\alpha \neq \mathbf{u}_{\alpha'}$ and $\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) = \text{VOLEHash}(\text{chall}_1, \mathbf{u}_{\alpha'})$. If and only if BadCh_1 occurs, state additionally outputs accept . We use the following lemma to bound the probability.

Lemma 9.4. *Given an $\varepsilon_{\text{vole}}$ -universal linear family VOLEHash of hash functions, we have*

$$\Pr_{\text{chall}_1} \left[\text{BadCh}_1 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg\text{BadPre} \end{array} \right] \leq \varepsilon_{\text{vole}} \cdot \binom{\tau}{2}.$$

Proof. Conditioned on $\neg\text{BadPre}$, either OneNoPre_α or UniquePre_α occurs for each $\alpha \in [0, \tau)$. All $\{\mathbf{u}_\alpha\}_{\alpha \in [0, \tau)}$'s are well-defined (more precisely, syntactically identical due to Lemma 9.1). For any two *fixed* distinct $\alpha, \alpha' \in [0, \tau)$, as the randomness chall_1 is uniform and independent of $\{\mathbf{u}_\alpha\}_{\alpha \in [0, \tau)}$, the event that $\mathbf{u}_\alpha \neq \mathbf{u}_{\alpha'}$ and $\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) = \text{VOLEHash}(\text{chall}_1, \mathbf{u}_{\alpha'})$ can only happen with a probability at most $\varepsilon_{\text{vole}}$ due to the universality and linearity. The target bound follows from a simple union bound over all possible distinct $\alpha, \alpha' \in [0, \tau)$. This lemma is a special case of the Theorem 2 in [BBD⁺23b]. $\square$

As a result, we have that

$$\begin{aligned} & \Pr_{\text{chall}_1} \left[\text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \right] \\ &= \Pr_{\text{chall}_1} \left[\text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \text{BadPre} \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_1} \left[\text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \right] \\ & \quad + \Pr_{\text{chall}_1} \left[\text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg\text{BadPre} \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_1} \left[\neg\text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \right] \\ &= \Pr_{\text{chall}_1} \left[\text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg\text{BadPre} \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_1} \left[\neg\text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \right] \\ &\leq \Pr_{\text{chall}_1} \left[\text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg\text{BadPre} \end{array} \right] \\ &= \Pr_{\text{chall}_1} \left[\text{BadCh}_1 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg\text{BadPre} \end{array} \right] \leq \varepsilon_{\text{vole}} \cdot \binom{\tau}{2} = \varepsilon_1. \end{aligned}$$

- On input $\mathbb{x}$ and $\text{tr}_8 = (\text{tr}_6, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d}, \text{chall}_2)$, if $\text{state}(\mathbb{x}, \text{tr}_6) = \text{accept}$, state outputs accept ; otherwise, the condition $\text{state}(\mathbb{x}, \text{tr}_6) = \text{reject}$ happens if and only if BadPre or $\neg\text{BadPre} \wedge \neg\text{BadCh}_1$ occurs, where the latter event is that all $\{(\mathbf{u}_\alpha, \mathbf{V}_\alpha)\}_{\alpha \in [0, \tau)}$ can be extracted and, for any two distinct $\alpha, \alpha' \in [0, \tau)$, $\mathbf{u}_\alpha = \mathbf{u}_{\alpha'}$ or $\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) \neq \text{VOLEHash}(\text{chall}_1, \mathbf{u}_{\alpha'})$. state distinguishes the following mutually exclusive and exhaustive cases of the condition:

- Case BadPre . In this case, state still outputs reject .
- Case $\neg\text{BadPre} \wedge \neg\text{BadCh}_1$. In this case, state defines $\mathbf{u} \leftarrow \mathbf{u}_0$ and extracts the extended witness $\mathbf{w} \leftarrow \mathbf{d} \oplus \mathbf{u}[0, \ell_{\text{wit}})$. In QuickSilver , the relation to be proved is parsed as degree-2 polynomials of $\mathbf{w}$. Let $\mathbf{a}$ be a vector of the degree-2 terms of the polynomials evaluated

on w . Note that the witness k (which is the first λ bits of w) satisfies the Lynx relation if and only if $\mathbf{a} = \mathbf{0}$. We define an event BadCh_2 that $\mathbf{a} \neq \mathbf{0}$ and $\text{ZKHash}(\text{chall}_2, \mathbf{a}) = 0$. If and only if BadCh_2 happens, state additionally outputs accept .

Lemma 9.5. *Given an ε_{zk} -universal linear family ZKHash of hash functions, we have*

$$\Pr_{\text{chall}_2} \left[\text{BadCh}_2 \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \leq \varepsilon_{\text{zk}}.$$

Proof. From the law of conditional probability, we have

$$\begin{aligned} & \Pr_{\text{chall}_2} \left[\text{BadCh}_2 \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\ &= \Pr_{\text{chall}_2} \left[\mathbf{a} \neq \mathbf{0} \wedge \text{ZKHash}(\text{chall}_2, \mathbf{a}) = 0 \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\ &= \Pr_{\text{chall}_2} \left[\begin{array}{l} \text{ZKHash}(\text{chall}_2, \mathbf{a}) = 0 \\ \wedge \mathbf{a} \neq \mathbf{0} \end{array} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\ &\quad \cdot \Pr_{\text{chall}_2} \left[\mathbf{a} \neq \mathbf{0} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\ &\leq \Pr_{\text{chall}_2} \left[\begin{array}{l} \text{ZKHash}(\text{chall}_2, \mathbf{a}) = 0 \\ \wedge \mathbf{a} \neq \mathbf{0} \end{array} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\ &\leq \varepsilon_{\text{zk}}, \quad \text{(By uniform } \text{chall}_2 \text{ and the universality)} \end{aligned}$$

arriving at the target bound. $\square$

From this lemma and the cases, we can prove that

$$\begin{aligned} & \Pr_{\text{chall}_2} [\text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject}] \\ &= \Pr_{\text{chall}_2} \left[\begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \\ \wedge \text{BadPre} \end{array} \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \right] \\ &\quad \cdot \Pr_{\text{chall}_2} [\text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject}] \\ &\quad + \Pr_{\text{chall}_2} \left[\begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \right] \\ &\quad \cdot \Pr_{\text{chall}_2} [\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject}] \\ &= \Pr_{\text{chall}_2} \left[\begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \right] \\ &\quad \cdot \Pr_{\text{chall}_2} [\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject}] \\ &\leq \Pr_{\text{chall}_2} \left[\begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \right] \\ &= \Pr_{\text{chall}_2} \left[\text{BadCh}_2 \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \leq \varepsilon_{\text{zk}} = \varepsilon_2. \end{aligned}$$

- On input $\mathbb{x}$ and $\text{tr}_{10} = (\text{tr}_8, \tilde{a}_1, \tilde{a}_0, \text{ctr}, \text{chall}_3)$, if $\text{state}(\mathbb{x}, \text{tr}_8) = \text{accept}$, state outputs **accept**; otherwise, the condition $\text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}$ happens if and only if BadPre or $\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2$ occurs. state distinguishes the mutually exclusive and exhaustive cases of the condition:

- Case BadPre . In this case, state still outputs **reject**.
- Case $\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2$. In this case, state will use the previously extracted $\{(\mathbf{u}_\alpha, \mathbf{V}_\alpha)\}_{\alpha \in [0, \tau]}$ and $\text{chall}_3 \in \{0, 1\}^\lambda$ to compute

$$\begin{aligned} \Delta &= (\Delta_0, \dots, \Delta_{\tau-1}) \leftarrow \text{VOLE.ChalDecAll}(\text{chall}_3) \in \mathbb{F}_2^\lambda, \\ \forall \alpha \in [0, \tau) : (\delta_{\alpha,0}, \dots, \delta_{\alpha,k_\alpha-1}) &\leftarrow (\Delta_\alpha[0], \dots, \Delta_\alpha[k_\alpha-1]) \in \mathbb{F}_2^{k_\alpha}, \\ \forall \alpha \in [0, \tau) : \mathbf{Q}_\alpha &\leftarrow \mathbf{V}_\alpha \oplus [\delta_{\alpha,0} \cdot \mathbf{u}_\alpha, \dots, \delta_{\alpha,k_\alpha-1} \cdot \mathbf{u}_\alpha] \in \mathbb{F}_2^{\ell \times k_\alpha}, \\ \mathbf{Q} &\leftarrow [\mathbf{Q}_0, \dots, \mathbf{Q}_{\tau-1}] \in \mathbb{F}_2^{\ell \times \lambda}. \end{aligned} \tag{22}$$

Then, state uses $\mathbf{Q}$ to compute $\tilde{\mathbf{Q}}$ (until the step 8) and $\tilde{a}_0$ (until the step 11) according to [Lynx.Verify](#). We define an event BadQS that k in the extracted extended witness $\mathbf{w}$ does not satisfy the Lynx relation and $\tilde{a}_0 = \tilde{a}_0$, and an event $\text{BadCh}_3 \leftarrow (\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}) \wedge \text{BadQS}$. If and only if BadCh_3 happens, state additionally outputs **accept**.

Lemma 9.6. *We have*

$$\Pr_{\text{chall}_3} \left[\text{BadCh}_3 \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \leq \frac{2}{2^\lambda}.$$

Proof. Conditioned on $\text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2)$, we have

- For $\tilde{\mathbf{V}} = [\tilde{\mathbf{V}}_0, \dots, \tilde{\mathbf{V}}_{\tau-1}] \in \mathbb{F}_2^{(\lambda+B) \times \lambda}$, the checked equality $\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}$ is equivalent to

$$\begin{aligned} \tilde{\mathbf{Q}} &= \tilde{\mathbf{V}} \in \mathbb{F}_2^{(\lambda+B) \times \lambda} \\ \Leftrightarrow \forall \alpha \in [0, \tau) : \text{VOLEHash}(\text{chall}_1, \mathbf{Q}_\alpha) \oplus [\delta_{\alpha,0} \cdot \tilde{\mathbf{u}}, \dots, \delta_{\alpha,k_\alpha-1} \cdot \tilde{\mathbf{u}}] &= \tilde{\mathbf{V}}_\alpha \in \mathbb{F}_2^{(\lambda+B) \times k_\alpha} \\ \Leftrightarrow \forall \alpha \in [0, \tau) : \\ &[\delta_{\alpha,0} \cdot (\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) \oplus \tilde{\mathbf{u}}), \dots, \delta_{\alpha,k_\alpha-1} \cdot (\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) \oplus \tilde{\mathbf{u}})] \\ &= \text{VOLEHash}(\text{chall}_1, \mathbf{V}_\alpha) \oplus \tilde{\mathbf{V}}_\alpha \in \mathbb{F}_2^{(\lambda+B) \times k_\alpha}. \end{aligned} \tag{23}$$

Conditioned on $\neg \text{BadPre} \wedge \neg \text{BadCh}_1$, we define a subset from $(\{(\mathbf{u}_\alpha, \mathbf{V}_\alpha)\}_{\alpha \in [0, \tau)}, \tilde{\mathbf{u}}, \tilde{\mathbf{V}})$:

$$S = \left\{ \Delta = [\delta_{0,0}, \dots, \delta_{0,k_0-1}, \dots, \delta_{\tau-1,0}, \dots, \delta_{\tau-1,k_{\tau-1}-1}] \in \{0, 1\}^\lambda \mid \text{such that eq.(23) holds} \right\} \subseteq \{0, 1\}^\lambda,$$

which corresponds to the prover's guess on chall_3 (i.e., Δ) by the time of its sending pmsg_2 . It is clear that $\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}$ if and only if the uniformly sampled $\Delta \in S \neq \emptyset$. Therefore,

$$\Pr_{\text{chall}_3} \left[\tilde{\mathbf{Q}} = \tilde{\mathbf{V}} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \leq \frac{|S|}{2^\lambda}.$$

- Conditioned on $\text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \wedge \neg \text{BadCh}_2$, no valid extended witness can satisfy the Lynx relation (otherwise, $\text{state}(\mathbb{x}, \text{tr}_0) = \text{accept}$ further guarantees $\text{state}(\mathbb{x}, \text{tr}_8) = \text{accept}$). Also, $\text{ZKHash}(\text{chall}_2, \mathbf{a}) \neq 0$ for the vector of degree-2 coefficients $\mathbf{a} \neq \mathbf{0}$ evaluated on any invalid extended witness. Additionally conditioned on the prover's correct guess on Δ (i.e., $\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}$), [QS.LynxVerify](#) algorithm ensures that $\tilde{a}_0 - \tilde{a}_0$ is a degree-2 non-zero polynomial

of Δ . With this additional condition, Δ is uniformly sampled from S and independent of the polynomial. We can use the Schwartz-Zippel lemma to show that

$$\Pr_{\text{chall}_3} \left[\text{BadQS} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \\ \wedge (\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}) \end{array} \right] \leq \frac{2}{|S|}.$$

This lemma follows from the above two upper bounds. $\square$

Using the law of total probability, we have

$$\begin{aligned} & \Pr_{\text{chall}_3} [\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}] \\ &= \Pr_{\text{chall}_3} \left[\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge \text{BadPre} \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_3} [\text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}] \\ & \quad + \Pr_{\text{chall}_3} \left[\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_3} [\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2 \mid \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}] \\ &= \Pr_{\text{chall}_3} \left[\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_3} [\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2 \mid \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}] \\ &\leq \Pr_{\text{chall}_3} \left[\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \\ &= \Pr_{\text{chall}_3} \left[\text{BadCh}_3 \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \leq \frac{2}{2^\lambda} = \varepsilon_3. \end{aligned}$$

- Finally, on input $\mathbb{x}$ and a full transcript $\text{tr}_{11} = (\text{tr}_{10}, \text{decom}_I)$, state outputs reject if and only if $\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{reject}$ or the commitments $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}$ reconstructed from decom_I (as per [BAVC.Verify](#)) do not equal $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}$. $\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{reject}$ occurs if and only if $\mathbb{x} \notin \mathcal{L}_{\mathcal{R}}$, and either BadPre or $\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2 \wedge \neg \text{BadCh}_3$ occurs.

- If $\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge \text{BadPre}$ occurs, the verifier $\mathcal{V}_{\text{Lynxer-IP}}$ outputs reject given pmsg_0 .
- If $\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge \neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2 \wedge \neg \text{BadCh}_3$ occurs, the sub-event

$$\begin{aligned} \mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge \neg \text{BadCh}_3 &= \mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge ((\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}}) \vee \neg \text{BadQS}) \\ &= (\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge (\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}})) \vee (\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge \neg \text{BadQS}) \\ &= (\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge (\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}})) \vee (\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge (\tilde{a}_0 \neq \tilde{a}_0)) \\ &= \mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge ((\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}}) \vee (\tilde{a}_0 \neq \tilde{a}_0)) \end{aligned}$$

must hold, which further implies that the event $(\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}}) \vee (\tilde{a}_0 \neq \tilde{a}_0)$ occurs. According to eq.(20), eq.(21), eq.(22), and [Lemma 9.1](#), the value of $\mathbf{Q}$ computed in state is identically defined as its counterpart in [Lynxer.Verify](#) run by the verifier $\mathcal{V}_{\text{Lynxer-IP}}$. Thus, so is the $(\tilde{\mathbf{Q}}, \tilde{a}_0)$ identically computed from the same $\mathbf{Q}$. This indicates that the event $(\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}}) \vee (\tilde{a}_0 \neq \tilde{a}_0)$ is identically defined for state and [Lynxer.Verify](#) run by the verifier. If this event occurs, the verifier $\mathcal{V}_{\text{Lynxer-IP}}$ outputs reject given pmsg_2 or pmsg_3 .

- If the commitments $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}$ reconstructed from decom_I (as per [BAVC.Verify](#)) do not equal $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}$, the verifier $\mathcal{V}_{\text{Lynxer-IP}}$ outputs reject given pmsg_4 .

Therefore, for the constructed algorithm state , if $\text{state}(\mathbb{x}, \text{tr}_{11}) = \text{reject}$ for any full transcript tr_{11} , then $\mathcal{V}_{\text{Lynxer-IP}}(\mathbb{x}, \text{tr}_{11}) = \text{reject}$ as required.

The above round-by-round analysis concludes this proof. $\square$

RBR soundness of Lynxer-IP $\Rightarrow$ soundness of FS[Lynxer-IP]. To achieve this reduction, we use the lemma of [\[BBB+25\]](#), which compiles RBR soundness into soundness in the QROM.

Definition 9.2 (Collision-Resistance-Preserving and Sensitive Function [\[BBB+25\]](#)). *A function $f^H : \mathcal{X} \rightarrow \mathcal{Y}$ that can be computed using N queries to a random oracle $H : \mathcal{X}' \rightarrow \mathcal{Y}'$ is collision-resistance-preserving if, for any $x_0, x_1 \in \mathcal{X}$ such that $x_0 \neq x_1$ and $f^H(x_0) = f^H(x_1)$, there exist $x'_0, x'_1 \in \mathcal{X}'$ such that $x'_0 \neq x'_1$ and*

- H is queried on input x'_i when $f^H(x_i)$ is computed for each $i \in \{0, 1\}$.
- $H(x'_0) = H(x'_1)$.

Moreover, the function f^H is sensitive if (i) $|\{f^{H^{x' \mapsto u}}(x) \mid u \in \mathcal{Y}'\}| \in \{1, |\mathcal{Y}'|\}$, and (ii) for any $x, x' \in \mathcal{X}$ such that $x \neq x'$, $f^H(x)$ does not make the same queries to H as $f^H(x')$.

Lemma 9.7 (RBR Soundness of IP $\Rightarrow$ Soundness of IP [\[BBB+25\]](#)). *Let IP be an IP system with ℓ challenges $\{\text{chall}_i\}_{i \in [\ell]}$. Let $S \subset [\ell]$ and $|S| = \ell_1$ such that IP has the RBR soundness with an error 0 for the challenges $\{\text{chall}_i\}_{i \in S}$ and an error ε else. Let f^{H_1} be a collision-resistance-preserving and sensitive function making N queries to the random oracle H_1 and $\text{FS}[\text{IP}]$ be the Fiat-Shamir transformation of IP, where the challenges $\{\text{chall}_i\}_{i \in S}$ are derived using f^{H_1} and the challenge chall_i is derived using the random oracle H_2^i for each $i \in [\ell] \setminus S$. Let Q_{H_1} be the number of queries to H_1 and $Q_{H_2^i}$ be the number of queries to H_2^i for each $i \in [\ell] \setminus S$. Let $\mathcal{O} = (H_1, \{H_2^i\}_{i \in [\ell] \setminus S})$ and $Q_{\mathcal{O}} = (Q_{H_1}, \{Q_{H_2^i}\}_{i \in [\ell] \setminus S})$. For any quantum adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ quantum queries respectively to the oracles in $\mathcal{O}$,*

$$\text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}, \text{FS}[\text{IP}]}(Q_{\mathcal{O}}) \leq 30 \cdot Q^3 \cdot 2^{-\ell_{\text{out}}} + 10 \cdot Q^2 \cdot \varepsilon$$

where ℓ_{out} is the output length of H_1 and $Q = (Q_{H_1} + \sum_{i \in [\ell] \setminus S} Q_{H_2^i}) + 2\ell + 2(N - 1)\ell_1$.

[\[BBB+25\]](#) has argued that $f^H(x_0, \dots, x_{N-1}) = (H(x_0), \dots, H(x_{N-1}))$ meets [Definition 9.2](#). Using this result, we have the following theorem.

Theorem 9.3 (Soundness of FS[Lynxer-IP]). *In FS[Lynxer-IP], consider the following ideal-model oracles and their respective numbers of queries:*

- E is an ideal cipher realizing [PRG](#) in [LeafHash](#), with Q_E queries,
- E' is an ideal cipher realizing [TCCR](#) in [BAVC.Commit](#), with $Q_{E'}$ queries,
- [MsgHash](#) is a random oracle in [Lynxer.Sign](#), with Q_{MsgHash} queries,
- [BAVCHash](#) is a random oracle in [BAVC.Commit](#), with Q_{BAVCHash} queries,
- [FSHash₁](#) is a random oracle in [Lynxer.Sign](#), with Q_{FSHash_1} queries,
- [FSHash₂](#) is a random oracle in [Lynxer.Sign](#), with Q_{FSHash_2} queries,
- [FSHash₃](#) is a random oracle in [Lynxer.Sign](#), with Q_{FSHash_3} queries.

Let $\mathcal{O}$ be a list of the above ideal-model oracles (where E and E' are inherited from Lynxer-IP and do not affect RBR soundness) and $Q_{\mathcal{O}}$ be a list of their respective numbers of queries. For any quantum adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ quantum queries respectively to the oracles in $\mathcal{O}$,

$$\text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}, \text{FS}[\text{Lynxer-IP}]}(Q_{\mathcal{O}}) \leq 30 \cdot Q^3 \cdot 2^{-2\lambda} + 10 \cdot Q^2 \cdot \varepsilon_{\text{RBR}},$$

where $Q = Q_{\text{MsgHash}} + Q_{\text{BAVCHash}} + Q_{\text{FSHash}_1} + Q_{\text{FSHash}_2} + Q_{\text{FSHash}_3} + 10$ and ε_{RBR} is from [Theorem 9.2](#).

Proof. Due to [Theorem 9.2](#), in $\text{FS}[\text{Lynxer-IP}]$ (where the underlying IP system Lynxer-IP proceeds in the QICM), we can define $S = \{0\}$ and $\ell = 5$ (where we slightly abuse the notation since the subscript of challenges starts from -1). For this S , chall_0 is derived with an RBR soundness error 0 from one call to the random oracle [BAVCHash](#). Therefore, this theorem follows from [Lemma 9.7](#) by using $S = \{0\}$, $\ell = 5$, $\ell_1 = 1$, $N = 1$, $\ell_{\text{out}} = 2\lambda$, and the given Q . $\square$

Soundness of $\text{FS}[\text{Lynxer-IP}] \Rightarrow \text{EUF-KO}$ security of Lynxer. We recall a lemma used in the lossy-key argument and provide the proof of the EUF-KO security.

Lemma 9.8 ([\[BBB⁺25\]](#)). *Let $G : \mathcal{X} \rightarrow \mathcal{Y}$ be a PRG. For any (potentially unbounded) distinguisher $\mathcal{A}$,*

$$\Pr_{y \leftarrow \text{im}(G)} [\mathcal{A}(G, y) = 1] \leq \frac{|\text{dom}(G)|}{|\text{im}(G)|} \cdot \Pr_{x \leftarrow \text{dom}(G)} [\mathcal{A}(G, G(x)) = 1].$$

In our EUF-KO proof against the quantum adversary, we artificially require the first two bits of a Lynx key are not both 1 so that the domain size of Lynx (essentially a PRG) is only $\frac{3}{4}$ of its image size to apply the above lemma. This is an additional restriction on the proposed signature scheme. It is reasonable that if the signature scheme with such a lower-entropy key is EUF-KO secure against the quantum adversary, then the scheme with a fully random key (which is actually implemented in our construction) should be EUF-KO secure. This artificial treatment is only for the security proof in the quantum model and can be removed in the proof against the classical adversary.

Theorem 9.4 (EUF-KO Security of Lynxer). *In Lynxer, consider the following ideal-model oracles and their respective numbers of queries:*

- E is an ideal cipher realizing [PRG](#) in [LeafHash](#), with Q_E queries,
- E' is an ideal cipher realizing [TCCR](#) in [BAVC.Commit](#), with $Q_{E'}$ queries,
- [MatVecHash](#) is a random oracle in [Lynx.SampleMatVec](#), with $Q_{\text{MatVecHash}}$ queries,
- [MsgHash](#) is a random oracle in [Lynx.Sign](#), with Q_{MsgHash} queries,
- [CoinHash](#) is a random oracle in [Lynx.Sign](#), with Q_{CoinHash} queries,
- [BAVCHash](#) is a random oracle in [BAVC.Commit](#), with Q_{BAVCHash} queries,
- [FSHash₁](#) is a random oracle in [Lynx.Sign](#), with Q_{FSHash_1} queries,
- [FSHash₂](#) is a random oracle in [Lynx.Sign](#), with Q_{FSHash_2} queries,
- [FSHash₃](#) is a random oracle in [Lynx.Sign](#), with Q_{FSHash_3} queries.

Let $\mathcal{O}$ be a list of the above ideal-model oracles and $Q_{\mathcal{O}}$ be a list of their respective numbers of queries. Let $\mathcal{O}' = \mathcal{O} \setminus \{\text{MatVecHash}, \text{CoinHash}\}$ and $Q_{\mathcal{O}'} = Q_{\mathcal{O}} \setminus \{Q_{\text{MatVecHash}}, Q_{\text{CoinHash}}\}$. For any quantum adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ quantum queries respectively to the oracles in $\mathcal{O}$,

$$\begin{aligned} & \text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{Lynxer}}(Q_{\mathcal{O}}) \\ & \leq 4 \cdot \left(\text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}', \text{FS}[\text{Lynxer-IP}]}(Q_{\mathcal{O}'}) + \text{AdvInj}_{\mathcal{A}_{\text{Inj}}}^{\text{E}, \text{PRG}'}(Q_{\text{E}} + L, L) + \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{\text{Lynx}}() + 3 \cdot \varepsilon_{\text{SR}}\left(\frac{\lambda}{2}\right) \right), \end{aligned}$$

where $\text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}', \text{FS}[\text{Lynxer-IP}]}(Q_{\mathcal{O}'})$ is from [Theorem 9.3](#), $\varepsilon_{\text{SR}}\left(\frac{\lambda}{2}\right)$ is from [Lemma 9.12](#),

- $\text{PRG}' : (\mathbf{x}, \text{iv}_{\text{sig}}, t) \in \{0, 1\}^{\lambda} \times \{0, 1\}^{120} \times \{0, 1\}^8 \mapsto \text{PRG}(\mathbf{x}, \text{iv}_{\text{sig}}, t; 2\lambda) \in \{0, 1\}^{2\lambda}$ is an E -based almost injective PRG ([Conjecture 9.1](#)).

Proof. We prove this theorem using the following hybrid argument.

- G_0 . This is the real EUF-KO security game.

$$\Pr [G_0(Q_{\mathcal{O}}) = 1] = \text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{Lynxer}}(Q_{\mathcal{O}}). \quad (24)$$

- G_1 . This game is identical to the previous one, except that at the very beginning, it samples uniform $(\text{iv}_{\text{owf}}, \text{buf})$, uses the compressed oracle technique to locally emulate the random oracle [MatVecHash](#) in [Lynx.SampleMatVec](#) for $\mathcal{A}$ at the only position iv_{owf} so that

$$(\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, c_1, c_2, c_3] = \text{Lynx.SampleMatVec}(\text{iv}_{\text{owf}}),$$

and sends $\text{pk} = (\text{iv}_{\text{owf}}, t)$ to $\mathcal{A}$. These two games are perfectly indistinguishable due to the compressed oracle technique.

$$\Pr [G_0(Q_{\mathcal{O}}) = 1] = \Pr [G_1(Q_{\mathcal{O}}) = 1]. \quad (25)$$

- G_2 . This game is identical to the previous one, except that it sends uniform pk to $\mathcal{A}$. We use a reduction to the PRG security against an adversary $\mathcal{A}_{\text{PRG}}$ to prove

$$|\Pr [G_1(Q_{\mathcal{O}}) = 1] - \Pr [G_2(Q_{\mathcal{O}}) = 1]| \leq \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{\text{Lynx}}() + 2 \cdot \varepsilon_{\text{SR}}\left(\frac{\lambda}{2}\right). \quad (26)$$

Assume that eq.(26) does not hold for contradiction. We can construct the quantum adversary $\mathcal{A}_{\text{PRG}}$ that internally runs $\mathcal{A}$ to break the PRG security of [Lynx](#) parameterized by randomly sampled $R = (\mathbf{L}_0, \mathbf{L}_1, \mathbf{L}_2, c_1, c_2, c_3]$ as per [Lynx.SampleMatVec](#):

1. Given some PRG challenge (Lynx_R, t) , $\mathcal{A}_{\text{PRG}}$ samples uniform iv_{owf} , uses the compressed oracle technique to locally emulate [MatVecHash](#) for $\mathcal{A}$ at the only position iv_{owf} so that

$$R = \text{Lynx.SampleMatVec}(\text{iv}_{\text{owf}}),$$

and sends $\text{pk} = (\text{iv}_{\text{owf}}, t)$ to $\mathcal{A}$.

2. $\mathcal{A}_{\text{PRG}}$ locally emulates the oracles in $\mathcal{O}'$ for $\mathcal{A}$. Let BadOra be the event that this emulation is not perfect.
3. $\mathcal{A}_{\text{PRG}}$ receives (msg, σ) from $\mathcal{A}$ and outputs [Lynx.Verify](#)($\text{pk}, \text{msg}, \sigma$) $\stackrel{?}{=}$ accept.

If $\mathcal{A}_{\text{PRG}}$ is given $t = \text{Lynx}_R(k)$, the execution is G_1 conditioned on $\neg \text{BadOra}$:

$$\Pr_{k \leftarrow \text{dom}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, \text{Lynx}_R(k)) = 1 \mid \neg \text{BadOra}] = \Pr [G_1(Q_{\mathcal{O}}) = 1].$$

Otherwise (i.e., t is uniform), the execution is G_2 conditioned on $\neg\text{BadOra}$:

$$\Pr_{t \leftarrow \text{codom}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, t) = 1 \mid \neg\text{BadOra}] = \Pr[G_2(Q_{\mathcal{O}}) = 1].$$

From the emulation loss of [Lemma 9.12](#) for E and E' (each of which has $|\mathcal{X}| \geq \frac{\lambda}{2}$ due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have $\Pr[\text{BadOra}] \leq 2 \cdot \varepsilon_{\text{SR}}(\frac{\lambda}{2})$. Using the contradiction assumption and the above results, we have that $\mathcal{A}_{\text{PRG}}$ breaks the PRG security conditioned on $\neg\text{BadOra}$, leading to a contradiction and proving [eq.\(26\)](#).

- G_3 . This game is identical to the previous one, except that it in addition returns 0 if $\text{pk} = (\text{iv}_{\text{owf}}, t) \in \mathcal{L}_{\mathcal{R}}$ for the language $\mathcal{L}_{\mathcal{R}}$ of the relation

$$\mathcal{R} = \left\{ (t, k) : t = \text{Lynx.EvalOutput}(R, k, v_1, v_2) \mid \begin{array}{l} R = \text{Lynx.SampleMatVec}(\text{iv}_{\text{owf}}), \\ (v_1, v_2) = \text{Lynx.ExtendWitness}(R, k) \end{array} \right\},$$

which is simply written as $t = \text{Lynx}_R(k)$. By definition, we have

$$\Pr[G_3(Q_{\mathcal{O}}) = 1] = \Pr[G_2(Q_{\mathcal{O}}) = 1 \wedge \text{pk} \notin \mathcal{L}_{\mathcal{R}}]. \quad (27)$$

Using the PRG adversary $\mathcal{A}_{\text{PRG}}$ distinguishing between G_1 and G_2 , we have

$$\begin{aligned} & \Pr[G_2(Q_{\mathcal{O}}) = 1] \\ &= \Pr_{t \leftarrow \text{codom}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, t) = 1 \mid \neg\text{BadOra}] \quad (\text{By the definition of } G_2) \\ &= \Pr_{t \leftarrow \text{codom}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, t) = 1 \wedge \text{pk} \in \mathcal{L}_{\mathcal{R}} \mid \neg\text{BadOra}] \\ &\quad + \Pr_{t \leftarrow \text{codom}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, t) = 1 \wedge \text{pk} \notin \mathcal{L}_{\mathcal{R}} \mid \neg\text{BadOra}] \\ &\quad (\text{By the law of total probability}) \\ &= \Pr_{t \leftarrow \text{codom}(\text{Lynx}_R)} [\text{pk} \in \mathcal{L}_{\mathcal{R}}] \cdot \Pr_{t \leftarrow \text{codom}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, t) = 1 \mid \neg\text{BadOra} \wedge \text{pk} \in \mathcal{L}_{\mathcal{R}}] \\ &\quad (\text{By the independence between } \neg\text{BadOra} \text{ and } \text{pk} \in \mathcal{L}_{\mathcal{R}}) \\ &\quad + \Pr[G_2(Q_{\mathcal{O}}) = 1 \wedge \text{pk} \notin \mathcal{L}_{\mathcal{R}}] \quad (\text{By the definition of } G_2) \\ &= \frac{|\text{im}(\text{Lynx}_R)|}{|\text{codom}(\text{Lynx}_R)|} \cdot \Pr_{t \leftarrow \text{codom}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, t) = 1 \mid \neg\text{BadOra} \wedge \text{pk} \in \mathcal{L}_{\mathcal{R}}] \\ &\quad + \Pr[G_3(Q_{\mathcal{O}}) = 1] \quad (\text{By eq.(27)}) \\ &= \frac{|\text{im}(\text{Lynx}_R)|}{|\text{codom}(\text{Lynx}_R)|} \cdot \Pr_{t \leftarrow \text{im}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, t) = 1 \mid \neg\text{BadOra}] + \Pr[G_3(Q_{\mathcal{O}}) = 1] \\ &\leq \frac{|\text{im}(\text{Lynx}_R)|}{|\text{codom}(\text{Lynx}_R)|} \cdot \frac{|\text{dom}(\text{Lynx}_R)|}{|\text{im}(\text{Lynx}_R)|} \cdot \Pr_{k \leftarrow \text{dom}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, \text{Lynx}_R(k)) = 1 \mid \neg\text{BadOra}] \\ &\quad (\text{By Lemma 9.8 for such a PRG adversary that } \neg\text{BadOra} \text{ must hold}) \\ &\quad + \Pr[G_3(Q_{\mathcal{O}}) = 1] \\ &= \frac{3}{4} \cdot \Pr_{k \leftarrow \text{dom}(\text{Lynx}_R)} [\mathcal{A}_{\text{PRG}}(\text{Lynx}_R, \text{Lynx}_R(k)) = 1 \mid \neg\text{BadOra}] + \Pr[G_3(Q_{\mathcal{O}}) = 1] \\ &= \frac{3}{4} \cdot \Pr[G_1(Q_{\mathcal{O}}) = 1] + \Pr[G_3(Q_{\mathcal{O}}) = 1]. \quad (\text{By the definition of } G_1) \end{aligned}$$

Combining the above inequality with [eq.\(26\)](#), we have an upper bound

$$\Pr[G_1(Q_{\mathcal{O}}) = 1] \leq 4 \cdot \left(\Pr[G_3(Q_{\mathcal{O}}) = 1] + \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{\text{Lynx}}() + 2 \cdot \varepsilon_{\text{SR}}\left(\frac{\lambda}{2}\right) \right). \quad (28)$$

- G_4 . This game is identical to the previous one, except that it receives (msg, σ) from $\mathcal{A}$, runs $\text{Lynxer.Verify}(\text{pk}, \text{msg}, \sigma)$ but uses its sampled (R, t) therein without using pk , uses the internal results in this verification to construct a signature σ' of the form (as per eq.(19)) in $\text{FS}[\text{Lynxer-IP}]$, and outputs whatever the following modified verifier of $\text{FS}[\text{Lynxer-IP}]$ outputs on $\text{pk}' = (R, t)$, msg , and σ' . This game works as follows:

– It parses the signature of $\mathcal{A}$ as:

$$\sigma = (\text{iv}_{\text{sig}}, \{c_\alpha\}_{\alpha \in [1, \tau)}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \text{ctr}, \text{decom}_I, \text{chall}_3)$$

and uses these values in the constructed σ' .

- It uses $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}$ reconstructed from decom_I (as per BAVC.Verify) for the commitments $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}$ in the constructed σ' .
- It uses the value $\tilde{Q}$ computed at the step 8 of Lynxer.Verify for the value $\tilde{V}$ in the constructed σ' .
- It uses the value $\tilde{a}_0$ computed at the step 11 of Lynxer.Verify for the value $\tilde{a}_0$ in the constructed σ' .
- The modified verifier is identical to the verifier of $\text{FS}[\text{Lynxer-IP}]$, except that it will not output **reject** for the two cases for the message pmsg_0 (see Item 4).

By the above $(\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}, \tilde{V}, \tilde{a}_0)$, it is trivial that this modified verifier never outputs **reject** for pmsg_2 , pmsg_3 , and pmsg_4 . As a result, this game (i.e., the modified verifier with the given $(\text{pk}', \text{msg}, \sigma')$) outputs 1 if and only if $\text{Lynxer.Verify}(\text{pk}, \text{msg}, \sigma) \stackrel{?}{=} \text{accept}$ (i.e., the previous game outputs 1). We have

$$\Pr [G_3(Q_{\mathcal{O}}) = 1] = \Pr [G_4(Q_{\mathcal{O}}) = 1]. \quad (29)$$

Note that, in the games subsequent to (including) this game, the modifier verifier need not call MatVecHash in Lynx.SampleMatVec to spawn R , which is locally sampled by the games.

- G_5 . This game is identical to the previous one, except that its modified verifier additionally outputs **reject** for the case (i) in the message pmsg_0 (see Item 4). This modification makes this game return 0 for the case (i) in pmsg_0 . We use a reduction to the almost injectivity of PRG' (in LeafHash to compute $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau), i \in [0, N_\alpha)}$) against an adversary $\mathcal{A}_{\text{Inj}}$ to prove

$$|\Pr [G_4(Q_{\mathcal{O}}) = 1] - \Pr [G_5(Q_{\mathcal{O}}) = 1]| \leq \text{AdvInj}_{\mathcal{A}_{\text{Inj}}}^{\text{E}, \text{PRG}'}(Q_{\text{E}} + L, L) + \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right). \quad (30)$$

Due to the definition of the case (i), we define an event CollPre that there exist $\alpha \in [0, \tau)$ and $i \in [0, N_\alpha)$ such that com_i^α has at least two pre-images. We observe that G_4 and G_5 are in the same probability space, and the two games proceed identically conditioned on the event $\neg \text{CollPre}$. Using the well-known difference lemma [Sho04], we have

$$|\Pr [G_4(Q_{\mathcal{O}}) = 1] - \Pr [G_5(Q_{\mathcal{O}}) = 1]| \leq \Pr [\text{CollPre}].$$

Assume that eq.(30) does not hold for contradiction and construct the following quantum adversary $\mathcal{A}_{\text{Inj}}$ that internally runs $\mathcal{A}$ to break the almost injectivity:

1. Given the quantum oracle access to E , $\mathcal{A}_{\text{Inj}}$ uses E for $\mathcal{A}$'s quantum queries to E .
2. $\mathcal{A}_{\text{Inj}}$ locally emulates the oracles in $\mathcal{O} \setminus \{\text{E}\}$ for $\mathcal{A}$. Let BadOra be the event that this emulation is not perfect.

3. $\mathcal{A}_{\text{Inj}}$ follows G_5 with the adversary $\mathcal{A}$ to reconstruct $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau], i \in [0, N_\alpha]}$ and sends

$$(\text{iv}_{\text{sig}}, \{(0, \text{com}_i^\alpha)\}_{\alpha \in [0, \tau], i \in [0, N_\alpha]}),$$

where iv_{sig} is included in $\mathcal{A}$'s signature σ , to the challenger of the almost injectivity game.

During this reconstruction, $\mathcal{A}_{\text{Inj}}$ can make at most L more quantum queries to E so that there are at most $Q_E + L$ quantum queries to E . Conditioned on $\neg \text{BadOra}$, such a game is the almost injective game so that $\Pr[\text{CollPre} \mid \neg \text{BadOra}] = \text{AdvInj}_{\mathcal{A}_{\text{Inj}}}^{E, \text{PRG}'}(Q_E + L, L)$. From the emulation loss of Lemma 9.12 for E' (which has $|\mathcal{X}| \geq \frac{\lambda}{2}$ due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have $\Pr[\text{BadOra}] \leq \varepsilon_{\text{SR}}(\frac{\lambda}{2})$. Using the above results, we prove a contradiction so that eq.(30) holds.

- G_6 . This game is identical to the previous one, except that its modified verifier additionally outputs reject for the case (ii) in the message pmsg_0 (see Item 4). Note that this hybrid is perfectly indistinguishable from the previous one as the reconstructed $\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau], i \in [0, N_\alpha]}$ follow BAVC.Verify and have at most one $\text{com}_{i_\alpha}^\alpha$ (which is given in the adversary's decom_I) with no pre-image for each $\alpha \in [0, \tau]$. Therefore, the case (ii) never happens so that

$$\Pr[G_5(Q_{\mathcal{O}}) = 1] = \Pr[G_6(Q_{\mathcal{O}}) = 1]. \quad (31)$$

- G_7 . This game is identical to the previous one, except that it locally emulates the random oracle CoinHash (i.e., returning uniform $(r, \text{iv}_{\text{sig}})$) using the compressed oracle technique for $\mathcal{A}$. Since the modified verifier does not invoke CoinHash , this game is perfectly indistinguishable from the previous one. Notice that the verifier run by this game is identical to the $\text{FS}[\text{Lynxer-IP}]$ verifier, which is explicitly given R and does not call MatVecHash in Lynx.SampleMatVec . This game is the soundness game of $\text{FS}[\text{Lynxer-IP}]$ given the oracles in $\mathcal{O}'$. Therefore, we have

$$\Pr[G_6(Q_{\mathcal{O}}) = 1] = \Pr[G_7(Q_{\mathcal{O}}) = 1] = \text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}', \text{FS}[\text{Lynxer-IP}]}(Q_{\mathcal{O}'}). \quad (32)$$

Based on eq.(24), eq.(25), eq.(28), eq.(29), eq.(30), eq.(31), and eq.(32), we can prove this theorem. $\square$

Quantum-list unpredictability of VOLE.Commit . This quantum-list unpredictability is to ensure that the quantum adversary, given the access to QROMs, cannot predict the honestly generated commitment, based on which the Fiat-Shamir challenge chall_1 will be computed. It will be used to prove the EUF-CMA security.

Lemma 9.9 (Adaptive Reprogramming in QROM [LGX25]). *Let X and Y be two finite sets. Let $\mathcal{A}$ be any stateful quantum adversary making at most Q quantum queries to $\mathcal{O}_b$ as per the adaptive reprogramming game $\text{GameRP}_{\mathcal{A}}^b(Q, R)$ in Fig. 7 for $b \in \{0, 1\}$. Let p_X^i be the i -th marginal distribution over X parsed by Orac , $p_{X, \max}^i = \mathbb{E}(\max_x p_X^i(x))$ be the expectation taken over $\mathcal{A}$'s behavior until the i -th query to Orac , and $p_{X, \max} \geq p_{X, \max}^i$ for any $i \in [R]$. We have*

$$|\Pr[\text{GameRP}_{\mathcal{A}}^0(Q, R) = 1] - \Pr[\text{GameRP}_{\mathcal{A}}^1(Q, R) = 1]| \leq 2\sqrt{2} \cdot \sqrt{R \cdot Q \cdot p_{X, \max}}.$$

Definition 9.3 (Quantum-List Unpredictability [BBB⁺25]). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $\mathcal{B}^{\mathcal{O}}$ be an algorithm using these oracles. $\mathcal{B}^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, Q, \varepsilon)$ -quantum-list-unpredictable if for any stateful quantum adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$,*

$$\text{AdvUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}}(Q_{\mathcal{O}}, Q) = \left| \Pr[\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, 0}(Q_{\mathcal{O}}, Q) = 1] - \Pr[\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, 1}(Q_{\mathcal{O}}, Q) = 1] \right| \leq \varepsilon,$$

where the quantum-list unpredictability game $\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, b}$ for $b \in \{0, 1\}$ is defined in Fig. 8.

Game $\text{GameRP}_{\mathcal{A}}^b(Q, R)$	$\text{Orac}(y')$
1 : $\mathcal{O}_0 \leftarrow \$ \mathcal{F}_{X,Y}, \mathcal{O}_1 \leftarrow \mathcal{O}_0$	1 : Parse y' as a distribution p over $X \times \{0,1\}^*$
2 : $y' \leftarrow \$ \mathcal{A}^{\mathcal{O}_b}()$	2 : $(x, x') \leftarrow \$ p$
3 : for $i \in [R]$:	3 : $y \leftarrow \$ Y$
4 : $(z, x, y) \leftarrow \$ \text{Orac}(y')$	4 : return $(x, x'), x, y$
5 : $\mathcal{O}_1 \leftarrow \$ \mathcal{O}_1[x \mapsto y]$	
6 : $y' \leftarrow \$ \mathcal{A}^{\mathcal{O}_b}(z)$	
7 : return $b' \leftarrow (y' = 1)$	

Figure 7: The adaptive reprogramming game in QROM.

Game $\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, b}(Q_{\mathcal{O}}, Q)$
1 : $ x\rangle \leftarrow \$ \mathcal{A}^{\mathcal{O}}()$, where $ x\rangle$ is a quantum register holding a superposition of a Q -element list $x = \{x_i\}_{i \in [Q]}$, which guesses the output of $\mathcal{B}$
2 : $c \leftarrow \$ \mathcal{B}^{\mathcal{O}}()$
3 : $b' \leftarrow 1$
4 : if $b = 1$:
5 : Apply a binary measurement to $ x\rangle$ to test the membership $b'' \leftarrow (c \in x)$ in superposition
6 : if $b'' = 1$: $b' \leftarrow 0$
7 : if $b' = 1$: $b' \leftarrow \$ \mathcal{A}^{\mathcal{O}}(x\rangle, c)$
8 : return b'

Figure 8: The quantum-list unpredictability game.

Lemma 9.10 (Quantum-List Unpredictability of [VOLE.Commit](#)). *Consider the following ideal-model oracles and their respective numbers of queries:*

- $\mathbf{E}$ is an ideal cipher realizing [PRG](#) in [LeafHash](#), with $Q_{\mathbf{E}}$ queries,
- $\mathbf{E}'$ is an ideal cipher realizing [TCCR](#) in [BAVC.Commit](#), with $Q_{\mathbf{E}'}$ queries,
- [BAVCHash](#) is a random oracle in [BAVC.Commit](#), with Q_{BAVCHash} queries.

Let $\mathcal{O}$ be a list of the above ideal-model oracles and $Q_{\mathcal{O}}$ be a list of their respective numbers of queries. Let χ be a uniform distribution over $\{0,1\}^\lambda$. For any quantum adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ quantum queries respectively to the oracles in $\mathcal{O}$ and having Q guesses in $|x\rangle$,

$$\begin{aligned}
& \text{AdvUP}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}}(Q_{\mathcal{O}}, Q) \\
& \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathbf{E}', \text{CCR}} \left(Q_{\mathbf{E}'} + \frac{3L}{2} + 2\lceil \log L \rceil + 5, 2\lceil \log L \rceil + 5, \chi \right) + \frac{(2\lceil \log L \rceil + 5)^2}{2^{\lambda-1}} \\
& \quad + \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{\mathbf{E}, \text{PRG}'}(Q_{\mathbf{E}}) + 2\sqrt{2} \cdot \frac{\sqrt{Q_{\text{BAVCHash}} + 1}}{2^\lambda} + \frac{2\sqrt{Q}}{2^\lambda} + \frac{Q}{2^{2\lambda}} + 4 \cdot \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right),
\end{aligned}$$

where $\varepsilon_{\text{SR}}(\frac{\lambda}{2})$ is from [Lemma 9.12](#),

- $\text{PRG}' : \mathbf{x} \in \{0,1\}^\lambda \mapsto \text{PRG}(\mathbf{x}, \text{iv}_{\text{sig}}, 0^8; 2\lambda) \in \{0,1\}^{2\lambda}$ is an $\mathbf{E}$ -based PRG for some fixed iv_{sig} ([Theorem 9.8](#)), and
- $\text{CCR} : \mathbf{x} \in \{0,1\}^\lambda \mapsto \text{TCCR}(\mathbf{x}, \mathbf{s}, \text{iv}_{\text{sig}}) \in \{0,1\}^\lambda$ is an $\mathbf{E}'$ -based CCR hash function for some fixed $\mathbf{s}$ and (uniform) iv_{sig} ([Theorem 9.13](#)).

Proof. We introduce the following hybrid argument.

- G_0 . This game is the real quantum-list unpredictability game.

$$\Pr [G_0(Q_{\mathcal{O}}, Q) = 1] = \Pr [\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, 0}(Q_{\mathcal{O}}, Q) = 1] . \quad (33)$$

- G_1 . This game is identical to the previous one, except that in [VOLE.Commit](#), it replaces the leftmost leaf seed seed_0^0 (which lies in the leftmost punctured path 0^h in a possibly not full pseudorandom correlated GGM tree [\[GYW⁺23\]](#) of the height $h = \lceil \log L \rceil$) and the at most h off-path nodes adjacent to the path 0^h by random values in $\{0, 1\}^\lambda$. We use a reduction to the CCR for NNQs (see [Lemma 3.1](#)) against an adversary $\mathcal{A}_{\text{NNQ}}$ to prove

$$\begin{aligned} & |\Pr [G_0(Q_{\mathcal{O}}, Q) = 1] - \Pr [G_1(Q_{\mathcal{O}}, Q) = 1]| \\ & \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{E}', \text{CCR}} \left(Q_{\mathcal{E}'} + \frac{3L}{2} + 2\lceil \log L \rceil + 5, 2\lceil \log L \rceil + 5, \chi \right) + \frac{(2\lceil \log L \rceil + 5)^2}{2^{\lambda-1}} + \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right). \end{aligned} \quad (34)$$

Assume that eq.(34) does not hold for contradiction. We can construct the quantum adversary $\mathcal{A}_{\text{NNQ}}$ that internally runs $\mathcal{A}$ to break the CCR for NNQs:

1. Given the quantum oracle access to $\mathcal{E}'$, $\mathcal{A}_{\text{NNQ}}$ uses $\mathcal{E}'$ for $\mathcal{A}$'s quantum queries to $\mathcal{E}'$.
2. $\mathcal{A}_{\text{NNQ}}$ locally emulates the oracles in $\{\mathcal{E}, \text{BAVCHash}\}$ for $\mathcal{A}$. Let BadOra be the event that this emulation is not perfect.
3. Given the access to the NNQ oracle $\mathcal{O}_{\text{NNQ}}$, $\mathcal{A}_{\text{NNQ}}$ proceeds as follows:
 - $\mathcal{A}_{\text{NNQ}}$ queries $\mathcal{O}_{\text{NNQ}}$ with $q_0 : \text{sum}_0 \leftarrow 0^\lambda$, $q'_0 : l \leftarrow \text{const}_0$, and $q''_0 : r \leftarrow \text{const}_1$.
 - For each $i \in [h-1]$, $\mathcal{A}_{\text{NNQ}}$ queries $\mathcal{O}_{\text{NNQ}}$ with $q_{i+1} : \text{sum}_{i+1} \leftarrow \mathcal{Q}(\text{sum}_i, 1)$ and $q'_{i+1} : \text{off}_{i+1} \leftarrow \text{sum}_{i+1} \oplus \text{sum}_i$.
 - $\mathcal{A}_{\text{NNQ}}$ queries $\mathcal{O}_{\text{NNQ}}$ with $q_h : x \leftarrow \text{sum}_{h-1} \oplus l$ and $q'_h : x' \leftarrow \text{sum}_{h-1} \oplus r$.
 - $\mathcal{A}_{\text{NNQ}}$ queries $\mathcal{O}_{\text{NNQ}}$ with $q_{h+1} : \text{seed}_0^0 \leftarrow \mathcal{Q}(x, 0)$ and $q'_{h+1} : \text{off}_h \leftarrow \mathcal{Q}(x', 0)$.
4. $\mathcal{A}_{\text{NNQ}}$ uses the returned seed_0^0 and $\{\text{off}_i\}_{i \in [1, h]}$ to reconstruct all $\{\text{seed}_i^\alpha\}_{i \in [0, N_\alpha]}$ as per the tree expansion in [VOLE.Commit](#) and honestly follows the rest of computation to obtain c as the output of [VOLE.Commit](#).
5. $\mathcal{A}_{\text{NNQ}}$ outputs whatever the quantum-list unpredictability game outputs.

$\mathcal{A}_{\text{NNQ}}$ makes $2h + 5$ NNQs to its given NNQ oracle and, in the step 4 as above, at most $\frac{3L}{2}$ more queries to $\mathcal{E}'$ to run [CCR](#) to obtain the leaf seeds $\{\text{seed}_i^\alpha\}_{\alpha \in [0, \tau], i \in [0, N_\alpha]} \setminus \{\text{seed}_0^0\}$ rooted with the off-path nodes $\{\text{off}_i\}_{i \in [1, h]}$ adjacent to the path 0^h .

If $\mathcal{O}_{\text{NNQ}} = \text{Real}_{\text{CCR}, \Delta}^{\mathcal{E}'}$, the distribution of c (for $\mathbf{r} = \Delta \leftarrow \chi$) is that in the previous game conditioned on $\neg \text{BadOra}$:

$$\Pr [\mathcal{A}_{\text{NNQ}}^{\mathcal{E}', \text{Real}_{\text{CCR}, \Delta}^{\mathcal{E}'}}() = 1 \mid \neg \text{BadOra}] = \Pr [G_0(Q_{\mathcal{O}}, Q) = 1] .$$

If $\mathcal{O}_{\text{NNQ}} = \text{Ideal}_{\text{CCR}}^{\mathcal{E}'}$, the leftmost leaf seed and the at most h off-path nodes are as uniform as in this game, leading to the distribution of c in this game conditioned on $\neg \text{BadOra}$:

$$\Pr [\mathcal{A}_{\text{NNQ}}^{\mathcal{E}', \text{Ideal}_{\text{CCR}}^{\mathcal{E}'}}() = 1 \mid \neg \text{BadOra}] = \Pr [G_1(Q_{\mathcal{O}}, Q) = 1] .$$

From the emulation loss of [Lemma 9.12](#) for $\mathcal{E}$ (which has $|\mathcal{X}| \geq \frac{\lambda}{2}$ due to the implementation) and the perfect emulation of [BAVCHash](#) from the compressed oracle technique, we have $\Pr [\text{BadOra}] \leq \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right)$. Using the contradiction assumption and the above results, we prove a contradiction against [Lemma 3.1](#) for the specified $\mathcal{A}_{\text{NNQ}}$, showing that eq.(34) holds.

- G_2 . This game is identical to the previous one, except that in [VOLE.Commit](#), it replaces the leftmost commitment com_0^0 by a random value in $\{0,1\}^{2\lambda}$. We use a reduction to the PRG security of PRG' (called in [LeafHash](#)) against an adversary $\mathcal{A}_{\text{PRG}}$ to prove that

$$|\Pr[G_1(Q_{\mathcal{O}}, Q) = 1] - \Pr[G_2(Q_{\mathcal{O}}, Q) = 1]| \leq \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{\text{E}, \text{PRG}'}(Q_E) + \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right). \quad (35)$$

Assume that eq.(35) does not hold for contradiction. We can construct the quantum adversary $\mathcal{A}_{\text{PRG}}$ that invokes $\mathcal{A}$ to break the PRG security:

1. Given the quantum oracle access to E , $\mathcal{A}_{\text{PRG}}$ uses E for $\mathcal{A}$'s quantum queries to E .
2. $\mathcal{A}_{\text{PRG}}$ locally emulates the oracles in $\{E', \text{BAVCHash}\}$ for $\mathcal{A}$. Let BadOra be the event that this emulation is not perfect.
3. Given some PRG challenge y , $\mathcal{A}_{\text{PRG}}$ uses y as com_0^0 to proceed as in G_1 .
4. $\mathcal{A}_{\text{PRG}}$ outputs whatever the quantum-list unpredictability game outputs.

$\mathcal{A}_{\text{PRG}}$ makes at most Q_E queries to E . If $\mathcal{A}_{\text{PRG}}$ is given $y = \text{PRG}'(\text{seed}_0^0)$ for some uniform seed_0^0 as in the real PRG game, the execution is G_1 conditioned on $\neg \text{BadOra}$:

$$\Pr_{\text{seed}_0^0} \left[\mathcal{A}_{\text{PRG}}^E(\text{PRG}'(\text{seed}_0^0)) = 1 \mid \neg \text{BadOra} \right] = \Pr[G_1(Q_{\mathcal{O}}, Q) = 1].$$

If $\mathcal{A}_{\text{PRG}}$ is given some uniform y , the execution is G_2 conditioned on $\neg \text{BadOra}$:

$$\Pr_y \left[\mathcal{A}_{\text{PRG}}^E(y) = 1 \mid \neg \text{BadOra} \right] = \Pr[G_2(Q_{\mathcal{O}}, Q) = 1].$$

From the emulation loss of [Lemma 9.12](#) for E' (which has $|\mathcal{X}| \geq \frac{\lambda}{2}$ due to the implementation) and the perfect emulation of [BAVCHash](#) from the compressed oracle technique, we have $\Pr[\text{BadOra}] \leq \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right)$. Using the contradiction assumption and the above results, we have that $\mathcal{A}_{\text{PRG}}$ breaks the PRG security conditioned on $\neg \text{BadOra}$ so that eq.(35) holds.

- G_3 . This game is identical to the previous one, except that in [VOLE.Commit](#), it replaces the output commitment com by a random value in $\{0,1\}^{2\lambda}$. We use a reduction to the adaptive reprogramming game ([Fig. 7](#)) against an adversary $\mathcal{A}_{\text{RP}}$ to prove

$$|\Pr[G_2(Q_{\mathcal{O}}, Q) = 1] - \Pr[G_3(Q_{\mathcal{O}}, Q) = 1]| \leq 2\sqrt{2} \cdot \frac{\sqrt{Q_{\text{BAVCHash}} + 1}}{2^\lambda} + 2 \cdot \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right). \quad (36)$$

Assume that eq.(36) does not hold for contradiction. We can construct the quantum adversary $\mathcal{A}_{\text{RP}}$ that internally runs $\mathcal{A}$ as a subroutine (where $\mathcal{O}_0 = \text{BAVCHash}$ in the game):

1. Given the quantum oracle access to $\mathcal{O}_b$, $\mathcal{A}_{\text{RP}}$ uses $\mathcal{O}_b$ for $\mathcal{A}$'s quantum queries to [BAVCHash](#).
2. $\mathcal{A}_{\text{RP}}$ locally emulates the oracles in $\{E, E'\}$ for $\mathcal{A}$. Let BadOra be the event that this emulation is not perfect.
3. When it is time to compute the output c of [VOLE.Commit](#), $\mathcal{A}_{\text{RP}}$ defines a distribution y' over $X = \{0,1\}^{2\lambda L} \times \{0,1\}^{120}$ such that the first 2λ bits (i.e., com_0^0) is uniform while the other part is fixed to $(\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau], i \in [0, N_\alpha]} \setminus \{\text{com}_0^0\}, \text{iv}_{\text{sig}})$, receives

$$x = (\{\text{com}_i^\alpha\}_{\alpha \in [0, \tau], i \in [0, N_\alpha]}, \text{iv}_{\text{sig}}),$$

and queries $\mathcal{O}_b$ with x to obtain com as a part of c .

4. $\mathcal{A}_{\text{RP}}$ outputs whatever the quantum-list unpredictability game outputs.

$\mathcal{A}_{\text{RP}}$ makes at most $Q_{\text{BAVCHash}} + 1$ adaptive quantum queries to $\mathcal{O}_b$. If $b = 0$, $\mathcal{O}_0 = \text{BAVCHash}$ implies that the view of $\mathcal{A}$ is identically distributed as in G_2 conditioned on $\neg \text{BadOra}$:

$$\Pr \left[\text{GameRP}_{\mathcal{A}_{\text{RP}}}^0(Q_{\text{BAVCHash}} + 1, 1) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[\text{G}_2(Q_{\mathcal{O}}, Q) = 1 \right].$$

If $b = 1$, $\mathcal{O}_1$ is identically distributed as in G_3 conditioned on $\neg \text{BadOra}$:

$$\Pr \left[\text{GameRP}_{\mathcal{A}_{\text{RP}}}^1(Q_{\text{BAVCHash}} + 1, 1) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[\text{G}_3(Q_{\mathcal{O}}, Q) = 1 \right].$$

From the emulation loss of [Lemma 9.12](#) for E and E' (each of which has $|\mathcal{X}| \geq \frac{\lambda}{2}$ due to the implementation), we have $\Pr [\text{BadOra}] \leq 2 \cdot \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right)$. Using the contradiction assumption and the above results, we prove a contradiction against [Lemma 9.9](#) for $Q = Q_{\text{BAVCHash}} + 1$, $R = 1$, and $p_{X, \max} \leq \frac{1}{2^{2\lambda}}$, showing that [eq.\(36\)](#) holds.

- G_4 . This game is identical to the previous one, except that it applies the binary measurement at the step 5 to test the membership $b'' \leftarrow (\text{VOLE.Commit}(r, s, \text{iv}_{\text{sig}}) \in x)$ in superposition but does not perform the step 6. Let ε be the probability that this measurement returns $b'' = 1$ (which will be analyzed in the end of this proof). From the gentle measurement lemma, the original state and post-measurement state of the quantum register $|x\rangle$ have a disturbance at most $2\sqrt{\varepsilon}$, which upper bounds the advantage of $\mathcal{A}$. As a result, we have

$$|\Pr [\text{G}_3(Q_{\mathcal{O}}, Q) = 1] - \Pr [\text{G}_4(Q_{\mathcal{O}}, Q) = 1]| \leq 2\sqrt{\varepsilon}. \quad (37)$$

- G_5 . This game is identical to the previous one, except that it further performs the step 6. It is the ideal quantum-list unpredictability game. Moreover, this game proceeds in the same probability space as the previous game so that ε is equivalently defined and quantitatively identical. Since this game and the previous game are identical unless the binary measurement returns $b'' = 1$, the difference lemma [[Sho04](#)] gives

$$\begin{aligned} & |\Pr [\text{G}_4(Q_{\mathcal{O}}, Q) = 1] - \Pr [\text{G}_5(Q_{\mathcal{O}}, Q) = 1]| \\ &= \left| \Pr [\text{G}_4(Q_{\mathcal{O}}, Q) = 1] - \Pr [\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, 1}(Q_{\mathcal{O}}, Q) = 1] \right| \leq \varepsilon. \end{aligned} \quad (38)$$

Finally, we consider how to bound ε in the probability space underlying G_5 . To maximize ε , we without loss of generality assume that each of the Q guesses in x is stored in its own quantum sub-register so that $|x\rangle = \otimes_{i \in [Q]} |x_i\rangle$ (where each $|x_i\rangle$ is a computational basis state). For any fixed output c of [VOLE.Commit](#), the projector P_c^{No} meaning c not in any sub-register is

$$P_c^{\text{No}} = \bigotimes_{i \in [Q]} (I - |c\rangle\langle c|)_i$$

and the projector $P_c^{\text{Yes}} = I^{\otimes Q} - P_c^{\text{No}}$ corresponds to c in some sub-register. Applying the binary measurement to $|x\rangle$, we have for any fixed c ,

$$\begin{aligned} \Pr [b'' = 1 \mid c] &= \langle x | P_c^{\text{Yes}} | x \rangle \\ &= 1 - \langle x | P_c^{\text{No}} | x \rangle \\ &= 1 - \prod_{i \in [Q]} \langle x_i | (I - |c\rangle\langle c|) | x_i \rangle \\ &= 1 - \prod_{i \in [Q]} (1 - |\langle x_i | c \rangle|^2) \\ &= 1 - \prod_{i \in [Q]} (1 - \delta_{x_i, c}) \quad (\text{Since } |x_i\rangle \text{ and } |c\rangle \text{ are computational basis states}) \\ &= \mathbb{1}(c \in x) \end{aligned}$$

where the Kronecker delta $\delta_{x_i,c} = 1$ if $x_i = c$ and $\delta_{x_i,c} = 0$ otherwise. Therefore,

$$\begin{aligned}
\varepsilon &= \Pr[b'' = 1] = \sum_c \Pr[b'' = 1 \mid c] \cdot \Pr[c] \\
&\leq \sum_c \Pr[b'' = 1 \mid c] \cdot \frac{1}{2^{2\lambda}} \quad (\text{Since the uniform } \text{com} \in \{0,1\}^{2\lambda} \text{ is a part of } c) \\
&= \frac{1}{2^{2\lambda}} \cdot \sum_c \mathbb{1}(c \in x) \\
&\leq \frac{Q}{2^{2\lambda}}. \quad (\text{Since the sum counts the distinct elements in } x)
\end{aligned}$$

Using this bound with eq.(33), eq.(34), eq.(35), eq.(36), eq.(37), and eq.(38), we can prove the target bound. $\square$

EUf-KO security of Lynxer $\Rightarrow$ EUf-CMA security of Lynxer. We finish the picture by proving the multi-hiding property of the commitment scheme and putting all things established before together to prove the EUf-CMA security.

Lemma 9.11 (Multi-Hiding of [VOLE.Commit](#)). *Consider the following ideal-model oracles and their respective numbers of queries:*

- E is an ideal cipher realizing [PRG](#) in [LeafHash](#) and [VOLE.ConvertToVOLE](#), with Q_E queries,
- E' is an ideal cipher realizing [TCCR](#) in [BAVC.Commit](#), with $Q_{E'}$ queries.

Let $\mathcal{O}$ be a list of the above ideal-model oracles and $Q_{\mathcal{O}}$ be a list of their respective numbers of queries. Let χ be a uniform distribution over $\{0,1\}^\lambda$. For any stateful quantum adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ quantum queries respectively to the oracles in $\mathcal{O}$,

$$\begin{aligned}
&\text{AdvMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}}(Q_{\mathcal{O}}, Q) \\
&= \left| \Pr \left[\text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, 0}(Q_{\mathcal{O}}, Q) = 1 \right] - \Pr \left[\text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, 1}(Q_{\mathcal{O}}, Q) = 1 \right] \right| \\
&\leq Q \cdot \left(2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{E', \text{CCR}} \left(Q_{E'} + \frac{3L}{2} + (2\lceil \log L \rceil + 5) \cdot \tau, (2\lceil \log L \rceil + 5) \cdot \tau, \chi \right) \right. \\
&\quad \left. + \frac{(2\lceil \log L \rceil + 5)^2 \cdot \tau^2}{2^{\lambda-1}} + \tau \cdot \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{E, \text{PRG}'}(Q_E) + 2 \cdot \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right) \right),
\end{aligned}$$

where the multi-hiding game $\text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, b}(Q_{\mathcal{O}}, Q)$ for $b \in \{0,1\}$ is defined in [Fig. 9](#), $\varepsilon_{\text{SR}}(\frac{\lambda}{2})$ is from [Lemma 9.12](#),

- $\text{PRG}' : \mathbf{x} \in \{0,1\}^\lambda \mapsto \text{PRG}(\mathbf{x}, \text{iv}_{\text{sig}}, 0^8; 2\lambda) \parallel \text{PRG}(\mathbf{x}, \text{iv}_{\text{sig}}, t; \ell_{\text{vole}}) \in \{0,1\}^{2\lambda} \times \mathbb{F}_2^{\ell_{\text{vole}}}$ is an E -based PRG for some fixed iv_{sig} and t ([Theorem 9.8](#)), and
- $\text{CCR} : \mathbf{x} \in \{0,1\}^\lambda \mapsto \text{TCCR}(\mathbf{x}, \mathbf{s}, \text{iv}_{\text{sig}}) \in \{0,1\}^\lambda$ is an E' -based CCR hash function for some fixed $\mathbf{s}$ and (uniform) iv_{sig} ([Theorem 9.13](#)).

Proof. For clarity, we introduce the following hybrid argument for $Q = 1$ and omit the superscript i in these hybrids. We will see how to generalize it for any $Q \geq 1$ in the end.

- G_0 . This is the real multi-hiding game.

$$\Pr[G_0(Q_{\mathcal{O}}, Q) = 1] = \Pr \left[\text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, 0}(Q_{\mathcal{O}}, Q) = 1 \right]. \quad (39)$$

Game $\text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, b}(Q_{\mathcal{O}}, Q)$	
1 :	for $i \in [Q]$:
2 :	$(\mu^i, \mathbf{s}^i) \leftarrow \{0, 1\}^{2\lambda} \times \{0, 1\}^{\lambda}$
3 :	$(r^i, \text{iv}_{\text{sig}}^i) \leftarrow \{0, 1\}^{\lambda} \times \{0, 1\}^{120}$
4 :	$(\text{com}^i, \text{decom}^i, \{\mathbf{c}_{\alpha}^i\}_{\alpha \in [1, \tau)}, \mathbf{u}^i, \mathbf{V}^i) \leftarrow \text{VOLE.Commit}(r^i, \mu^i, \mathbf{s}^i, \text{iv}_{\text{sig}}^i)$
5 :	$\text{chall}_3^i \leftarrow \{0, 1\}^{\lambda - w_{\text{grind}}} \parallel 0_{\text{grind}}^w$ such that $I^i \leftarrow \text{VOLE.ChalDecAll}(\text{chall}_3^i)$ does not abort BAVC.Open
6 :	$\text{decom}_{I^i}^i \leftarrow \text{BAVC.Open}(\text{decom}^i, I^i)$
7 :	if $b = 1$:
8 :	$(\{\mathbf{c}_{\alpha}^i\}_{\alpha \in [1, \tau)}, \mathbf{u}^i) \leftarrow \mathbb{F}_2^{\ell_{\text{vole}}}{}^{\tau}$
9 :	$\mathcal{A}^{\mathcal{O}}(\text{com}^i, \{\mathbf{c}_{\alpha}^i\}_{\alpha \in [1, \tau)}, \mathbf{u}^i, \text{chall}_3^i, \text{decom}_{I^i}^i)$
10 :	$b' \leftarrow \mathcal{A}^{\mathcal{O}}()$
11 :	return b'

Figure 9: The multi-hiding game of VOLE.Commit .

- G_1 . This game is identical to the previous one, except that for each $\alpha \in [0, \tau)$, it replaces the punctured leaf seed $\text{seed}_{\Delta_{\alpha}}^{\alpha}$ by a random value in $\{0, 1\}^{\lambda}$. Similar to the game hop from G_0 to G_1 in the proof of Lemma 9.10, we use a reduction to the CCR for NNQs against an adversary $\mathcal{A}_{\text{NNQ}}$, which makes at most $(2\lceil \log L \rceil + 5) \cdot \tau$ NNQs to its given NNQ oracle and at most $\frac{3L}{2}$ more queries to E' to emulate the output of VOLE.Commit for its internally used $\mathcal{A}$ against the multi-hiding game. As a result,

$$\begin{aligned}
& |\Pr[G_0(Q_{\mathcal{O}}, Q) = 1] - \Pr[G_1(Q_{\mathcal{O}}, Q) = 1]| \\
& \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{E', \text{CCR}} \left(Q_{E'} + \frac{3L}{2} + (2\lceil \log L \rceil + 5) \cdot \tau, (2\lceil \log L \rceil + 5) \cdot \tau, \chi \right) \\
& \quad + \frac{(2\lceil \log L \rceil + 5)^2 \cdot \tau^2}{2^{\lambda-1}} + \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right).
\end{aligned} \tag{40}$$

- G_2 . This game is identical to the previous one, except that for each $\alpha \in [0, \tau)$, it replaces the punctured commitment $\text{com}_{\Delta_{\alpha}}^{\alpha}$ along with the VOLE payload $\mathbf{u}_{\alpha}$ by a random value in $\{0, 1\}^{2\lambda} \times \mathbb{F}_2^{\ell_{\text{vole}}}$. As the pre-image $\text{seed}_{\Delta_{\alpha}}^{\alpha}$ of $\text{com}_{\Delta_{\alpha}}^{\alpha}$ (modulo PRG in LeafHash) and the $i = \Delta_{\alpha}$ additive term (modulo PRG in $\text{VOLE.ConvertToVOLE}$) in $\mathbf{u}_{\alpha} = \sum_{i \in [N_{\alpha}]} \text{PRG}(\text{seed}_i^{\alpha}, \text{iv}_{\text{sig}}, i)$, which follows from $\text{VOLE.ConvertToVOLE}$, is hidden from the adversary and independent of other pre-images, a reduction to the PRG security of PRG' shows that

$$\begin{aligned}
& |\Pr[G_1(Q_{\mathcal{O}}, Q) = 1] - \Pr[G_2(Q_{\mathcal{O}}, Q) = 1]| \\
& = \left| \Pr[G_1(Q_{\mathcal{O}}, Q) = 1] - \Pr[\text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, 1}(Q_{\mathcal{O}}, Q) = 1] \right| \\
& \leq \tau \cdot \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{E, \text{PRG}'}(Q_E) + \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right),
\end{aligned} \tag{41}$$

where the first equality comes from that $\mathbf{c}_{\alpha} = \mathbf{u}_0 \oplus \mathbf{u}_{\alpha}$ for each $\alpha \in [1, \tau)$ is uniform in this game so that $(\{\mathbf{c}_{\alpha}^i\}_{\alpha \in [1, \tau)}, \mathbf{u}^i)$ is as uniform as in the ideal multi-hiding game when $Q = 1$.

To generalize the above hybrid argument for any $Q \geq 1$, we can use it for each $i \in [Q]$ as each execution has independent randomness, arriving at the target bound by a factor of Q . $\square$

Theorem 9.5 (EUF-CMA Security of Lynxer). *In Lynxer, consider the following ideal-model oracles and their respective numbers of queries:*

- E is an ideal cipher realizing PRG in LeafHash and $\text{VOLE.ConvertToVOLE}$, with Q_E queries,

- E' is an ideal cipher realizing **TCCR** in **BAVC.Commit**, with $Q_{E'}$ queries,
- **MatVecHash** is a random oracle in **Lynx.SampleMatVec**, with $Q_{\text{MatVecHash}}$ queries,
- **MsgHash** is a random oracle in **Lynx.Sign**, with Q_{MsgHash} queries,
- **CoinHash** is a random oracle in **Lynx.Sign**, with Q_{CoinHash} queries,
- **BAVCHash** is a random oracle in **BAVC.Commit**, with Q_{BAVCHash} queries,
- **FSHash₁** is a random oracle in **Lynx.Sign**, with Q_{FSHash_1} queries,
- **FSHash₂** is a random oracle in **Lynx.Sign**, with Q_{FSHash_2} queries,
- **FSHash₃** is a random oracle in **Lynx.Sign**, with Q_{FSHash_3} queries.

Let $\mathcal{O}$ be a list of the above ideal-model oracles and $Q_{\mathcal{O}}$ be a list of their respective numbers of queries. Let $\mathcal{O}' = \{E, E', \text{BAVCHash}\}$ and $Q_{\mathcal{O}'} = \{Q_E + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil, Q_{E'} + \frac{Q_{\text{sig}} \cdot 3L}{2}, Q_{\text{BAVCHash}} + Q_{\text{sig}}\}$. Let $\mathcal{O}'' = \{E, E'\}$ and $Q_{\mathcal{O}''} = \{Q_E + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil, Q_{E'} + \frac{Q_{\text{sig}} \cdot 3L}{2}\}$. For any quantum adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ quantum queries respectively to the oracles in $\mathcal{O}$ and at most Q_{sig} queries to the signature oracle OSig_{sk} ,

$$\begin{aligned}
& \text{AdvEUFMA}_{\mathcal{A}}^{\mathcal{O}, \text{Lynx}}(Q_{\mathcal{O}}, Q_{\text{sig}}) \\
& \leq \frac{Q_{\text{sig}}^2}{2^\lambda} + Q_{\text{sig}} \cdot \text{AdvUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{FSHash}_1}) + 2\sqrt{2} \cdot \sqrt{\frac{Q_{\text{sig}} \cdot (Q_{\text{FSHash}_2} + Q_{\text{sig}})}{2^{5\lambda+64}}} \\
& \quad + 2\sqrt{2} \cdot \sqrt{\frac{Q_{\text{sig}} \cdot (Q_{\text{FSHash}_3} + Q_{\text{sig}})}{2^{3\lambda+64}}} + \text{AdvMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}}(Q_{\mathcal{O}''}, Q_{\text{sig}}) \\
& \quad + 4 \cdot \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right) + \text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{Lynx}}(Q_{\mathcal{O}}),
\end{aligned}$$

where $\text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{Lynx}}(Q_{\mathcal{O}})$ is from [Theorem 9.4](#), $\text{AdvUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{FSHash}_1})$ is from [Lemma 9.10](#), $\text{AdvMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}}(Q_{\mathcal{O}''}, Q_{\text{sig}})$ is from [Lemma 9.11](#), and $\varepsilon_{\text{SR}}(\frac{\lambda}{2})$ is from [Lemma 9.12](#).

Proof. We introduce the following hybrid argument.

- G_0 . This is the real EUF-CMA security game.

$$\Pr [G_0(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] = \text{AdvEUFMA}_{\mathcal{A}}^{\mathcal{O}, \text{Lynx}}(Q_{\mathcal{O}}, Q_{\text{sig}}). \quad (42)$$

- G_1 . This game is identical to the previous one, except that the signature oracle OSig_{sk} uses random $(r, \text{iv}_{\text{sig}})$ instead of those from **CoinHash**. The indistinguishability between this game and the previous game follows from the random oracle **CoinHash** for independently uniform ρ 's. Taking a union bound over all possible pairwise collision of $\rho \in \{0, 1\}^\lambda$, we have

$$|\Pr [G_0(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] - \Pr [G_1(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1]| \leq \frac{Q_{\text{sig}}^2}{2^\lambda}. \quad (43)$$

- G_2 . This game is identical to the previous one, except that
 - All random oracles are emulated by the compressed oracle technique [[Zha19](#)].
 - When OSig_{sk} queries the random oracle **FSHash₁** on $(\text{com}, \{c_\alpha\}_{\alpha \in [1, \tau]})$, this game will apply a binary measurement to the compressed oracle database to test whether this query has been recorded in the database. If true, this game outputs 0; otherwise, it programs **FSHash₁** to output a uniformly sampled **chall₁**.

We consider the following subroutine hybrid argument for the games $\{G_{1,i}\}_{i \in [Q_{\text{sig}}+1]}$, where the game $G_{1,i}$ applies the binary measurement and runs according to the measurement result as in G_2 for the first i queries to OSig_{sk} . It is obvious that $G_{1,0} = G_1$ and $G_{1,Q_{\text{sig}}} = G_2$. Thus, $G_{1,i}$ and $G_{1,i+1}$ differ only for the $(i+1)$ -th query. We use a reduction to the quantum-list unpredictability game (Fig. 8) against an adversary $\mathcal{A}_{\text{UP}}$ to prove

$$\begin{aligned} & \left| \Pr [G_{1,i}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] - \Pr [G_{1,i+1}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] \right| \\ & \leq \text{AdvUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1}). \end{aligned} \quad (44)$$

Assume that eq.(44) does not hold for contradiction. We can construct the quantum adversary $\mathcal{A}_{\text{UP}}$ that uses $\mathcal{A}$ to break the quantum-list unpredictability:

1. Given the quantum oracle access to $\mathcal{O}'$, $\mathcal{A}_{\text{UP}}$ uses $\mathcal{O}'$ for $\mathcal{A}$'s quantum queries to $\mathcal{O}'$.
2. $\mathcal{A}_{\text{UP}}$ locally emulates the oracles in $\mathcal{O} \setminus \mathcal{O}'$ (which includes FHash_1 using the compressed oracle technique) for $\mathcal{A}$. Note that this emulation is perfect due to the compressed oracle technique.
3. $\mathcal{A}_{\text{UP}}$ emulates the execution of $G_{1,i}$ for $\mathcal{A}$ except that for the $(i+1)$ -th query to OSig_{sk} , it outputs the quantum register $|x\rangle$ consisting of all sub-registers for the compressed oracle database inputs of FHash_1 . When $\mathcal{A}_{\text{UP}}$ is given c , it uses this value for the $(i+1)$ -th signature output by OSig_{sk} , and honestly computes the remaining $Q_{\text{sig}} - i - 1$ signatures for $\mathcal{A}$ as in $G_{1,i}$.
4. $\mathcal{A}_{\text{UP}}$ outputs whatever its emulated game against $\mathcal{A}$ outputs.

Here, $\mathcal{A}_{\text{UP}}$ makes at most $Q_{\text{E}} + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil$ queries to E (by the construction of PRG), at most $Q_{\text{E}'} + Q_{\text{sig}} \cdot \frac{3L}{2}$ queries to E' , and at most $Q_{\text{BAVCHash}} + Q_{\text{sig}}$ queries to BAVCHash . Moreover, the number of sub-registers in $|x\rangle$ is at most Q_{FHash_1} . If $b = 0$, $\mathcal{A}_{\text{UP}}$ proceeds in the game $\text{GameUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}, 0}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1})$ and $\mathcal{A}$ works as in the game $G_{1,i}$ (where no binary measurement or programming for the $(i+1)$ -th query is needed in either the emulated game of $\mathcal{A}_{\text{UP}}$ or the game $G_{1,i}$):

$$\Pr \left[\text{GameUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}, 0}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1}) = 1 \right] = \Pr [G_{1,i}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1].$$

If $b = 1$, $\mathcal{A}_{\text{UP}}$ is in the game $\text{GameUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}, 1}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1})$. Conditioned on $b'' = 0$, the programming in $G_{1,i+1}$ must succeed and result in chall_1 as uniform as the one from the compressed oracle technique in the emulated game of $\mathcal{A}_{\text{UP}}$. That is, the emulated game of $\mathcal{A}_{\text{UP}}$ is identically distributed as $G_{1,i+1}$ under this condition. Conditioned on $b'' = 1$, both the quantum-list unpredictability game and the game $G_{1,i+1}$ output 0. Thus,

$$\Pr \left[\text{GameUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}, 1}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1}) = 1 \right] = \Pr [G_{1,i+1}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1].$$

Using the contradiction assumption and the above results, we prove a contradiction against Lemma 9.10 parameterized by $(Q_{\mathcal{O}'}, Q_{\text{FHash}_1})$ so that eq.(44) holds. Taking a union bound over all i 's, it follows from $G_{1,0} = G_1$ and $G_{1,Q_{\text{sig}}} = G_2$ that

$$\begin{aligned} & \left| \Pr [G_1(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] - \Pr [G_2(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] \right| \\ & \leq Q_{\text{sig}} \cdot \text{AdvUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1}). \end{aligned} \quad (45)$$

- G_3 . This game is identical to the previous one, except that when OSig_{sk} queries the random oracle FHash_2 , it is reprogrammed to output a uniform chall_2 . We use a reduction to the adaptive reprogramming game (Fig. 7) against an adversary $\mathcal{A}_{\text{RP}}$ to prove

$$\left| \Pr [G_2(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] - \Pr [G_3(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] \right| \leq 2\sqrt{2} \cdot \sqrt{\frac{Q_{\text{sig}} \cdot (Q_{\text{FHash}_2} + Q_{\text{sig}})}{2^{5\lambda+64}}} + 2 \cdot \varepsilon_{\text{SR}} \left(\frac{\lambda}{2} \right). \quad (46)$$

Assume that eq.(46) does not hold for contradiction. We can construct the quantum adversary $\mathcal{A}_{\text{RP}}$ that internally runs $\mathcal{A}$ as a subroutine (where $\mathcal{O}_0 = \text{FSHash}_2$):

1. Given the quantum oracle access to $\mathcal{O}_b$, $\mathcal{A}_{\text{RP}}$ directly uses $\mathcal{O}_b$ for $\mathcal{A}$'s quantum queries to FSHash_2 .
2. $\mathcal{A}_{\text{RP}}$ locally emulates the oracles in $\mathcal{O} \setminus \{\text{FSHash}_2\}$ for $\mathcal{A}$. Let BadOra be the event that this emulation is not perfect.
3. In each of the OSig_{sk} calls, when it is time to compute chall_2 , $\mathcal{A}_{\text{RP}}$ defines a distribution y' over $X = \{0, 1\}^{5\lambda+64} \times \{0, 1\}^{\lambda+B} \times \{0, 1\}^{(\lambda+B)\lambda} \times \mathbb{F}_2^{\ell_{\text{wit}}}$ and with the parameter $(\mathbf{u}, \mathbf{V}, \mathbf{d})$ according to VOLE.Commit , such that the first $5\lambda + 64$ bits (i.e., chall_1) is uniform while the other part is fixed to

$$\begin{aligned}\tilde{\mathbf{u}} &\leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{u}), \\ \tilde{\mathbf{V}} &\leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{V}), \\ \mathbf{d} &\leftarrow \mathbf{d},\end{aligned}$$

receives $x = (\text{chall}_1, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d})$, and queries $\mathcal{O}_b$ with x to obtain chall_2 .

4. $\mathcal{A}_{\text{RP}}$ outputs whatever its emulated game against $\mathcal{A}$ outputs.

$\mathcal{A}_{\text{RP}}$ can make at most $Q_{\text{FSHash}_2} + Q_{\text{sig}}$ adaptive quantum queries to $\mathcal{O}_b$. If $b = 0$, the oracle $\mathcal{O}_0 = \text{FSHash}_2$ shows that the view of $\mathcal{A}$ is identically distributed as in G_2 conditioned on $\neg \text{BadOra}$

$$\Pr \left[\text{GameRP}_{\mathcal{A}_{\text{RP}}}^0(Q_{\text{FSHash}_2} + Q_{\text{sig}}, Q_{\text{sig}}) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[\text{G}_2(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

If $b = 1$, $\mathcal{O}_1$ is reprogrammed as in G_3 conditioned on $\neg \text{BadOra}$:

$$\Pr \left[\text{GameRP}_{\mathcal{A}_{\text{RP}}}^1(Q_{\text{FSHash}_2} + Q_{\text{sig}}, Q_{\text{sig}}) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[\text{G}_3(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

From the emulation loss of Lemma 9.12 for E and E' (each of which has $|\mathcal{X}| \geq \frac{\lambda}{2}$ due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have $\Pr[\text{BadOra}] \leq 2 \cdot \varepsilon_{\text{SR}}(\frac{\lambda}{2})$. Using the contradiction assumption and the above results, we prove a contradiction against Lemma 9.9 for $Q = Q_{\text{FSHash}_2} + Q_{\text{sig}}$, $R = Q_{\text{sig}}$, and $p_{X, \max} \leq \frac{1}{2^{5\lambda+64}}$ conditioned on $\neg \text{BadOra}$.

- G_4 . This game is identical to the previous one, except that when OSig_{sk} queries the random oracle FSHash_3 , it is reprogrammed to output a uniform chall_3 . We use a reduction to the adaptive reprogramming game (Fig. 7) against an adversary $\mathcal{A}_{\text{RP}}$ to prove

$$\left| \Pr \left[\text{G}_3(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] - \Pr \left[\text{G}_4(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] \right| \leq 2\sqrt{2} \cdot \sqrt{\frac{Q_{\text{sig}} \cdot (Q_{\text{FSHash}_3} + Q_{\text{sig}})}{2^{3\lambda+64}}} + 2 \cdot \varepsilon_{\text{SR}}\left(\frac{\lambda}{2}\right). \quad (47)$$

Assume that eq.(47) does not hold for contradiction. We can construct the quantum adversary $\mathcal{A}_{\text{RP}}$ that internally runs $\mathcal{A}$ as a subroutine (where $\mathcal{O}_0 = \text{FSHash}_3$):

1. Given the quantum oracle access to $\mathcal{O}_b$, $\mathcal{A}_{\text{RP}}$ directly uses $\mathcal{O}_b$ for $\mathcal{A}$'s quantum queries to FSHash_3 .
2. $\mathcal{A}_{\text{RP}}$ locally emulates the oracles in $\mathcal{O} \setminus \{\text{FSHash}_3\}$ for $\mathcal{A}$. Let BadOra be the event that this emulation is not perfect.
3. In each of the OSig_{sk} calls, when it is time to compute chall_3 , $\mathcal{A}_{\text{RP}}$ defines a distribution y' over $X = \{0, 1\}^{3\lambda+64} \times \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda} \times \{0, 1\}^{32}$ and with the parameter $(\mathbf{w}, \mathbf{u}, \mathbf{V}, R, t, \text{ctr})$ such that the first $3\lambda + 64$ bits (i.e., chall_2) is uniform, the value of $(\tilde{a}_1, \tilde{a}_0)$ is distributed as in $\text{QS.LynxProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, R, t, \text{chall}_2)$, and $\text{ctr} \leftarrow \text{ctr}$, receives $x = (\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$, and queries $\mathcal{O}_b$ with x to obtain chall_3 .

4. $\mathcal{A}_{\text{RP}}$ outputs whatever its emulated game against $\mathcal{A}$ outputs.

$\mathcal{A}_{\text{RP}}$ can make at most $Q_{\text{FHash}_3} + Q_{\text{sig}}$ adaptive quantum queries to $\mathcal{O}_b$. If $b = 0$, the oracle $\mathcal{O}_0 = \text{FHash}_3$ shows that the view of $\mathcal{A}$ is identically distributed as in G_3 conditioned on $\neg \text{BadOra}$:

$$\Pr \left[\text{GameRP}_{\mathcal{A}_{\text{RP}}}^0(Q_{\text{FHash}_3} + Q_{\text{sig}}, Q_{\text{sig}}) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[G_3(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

If $b = 1$, $\mathcal{O}_1$ is reprogrammed as in G_4 conditioned on $\neg \text{BadOra}$:

$$\Pr \left[\text{GameRP}_{\mathcal{A}_{\text{RP}}}^1(Q_{\text{FHash}_3} + Q_{\text{sig}}, Q_{\text{sig}}) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[G_4(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

From the emulation loss of Lemma 9.12 for E and E' (each of which has $|\mathcal{X}| \geq \frac{\lambda}{2}$ due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have $\Pr[\text{BadOra}] \leq 2 \cdot \varepsilon_{\text{SR}}(\frac{\lambda}{2})$. Using the contradiction assumption and the above results, we prove a contradiction against Lemma 9.9 for $Q = Q_{\text{FHash}_3} + Q_{\text{sig}}$, $R = Q_{\text{sig}}$, and $p_{X,\max} \leq \frac{1}{2^{3\lambda+64}}$ conditioned on $\neg \text{BadOra}$.

- G_5 . This game is identical to the previous one, except that for each OSig_{sk} query, it samples uniform $(\{c_\alpha\}_{\alpha \in [1,\tau]}, \mathbf{u})$, reconstructs $(\Delta, \mathbf{Q})$ from uniform chall_3 (until $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda] = 0_{\text{grind}}^w$) and decom_I , and reversely computes $\mathbf{V} \leftarrow \mathbf{Q} \oplus \Delta \cdot \mathbf{u}$ for the output signature. We use a reduction to the multi-hiding game (Fig. 9) against an adversary $\mathcal{A}_{\text{MH}}$ to prove that

$$\left| \Pr \left[G_4(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] - \Pr \left[G_5(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] \right| \leq \text{AdvMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}}(Q_{\mathcal{O}''}, Q_{\text{sig}}). \quad (48)$$

Assume that eq.(48) does not hold for contradiction. We can construct the quantum adversary $\mathcal{A}_{\text{MH}}$ that uses $\mathcal{A}$ to break the multi-hiding of VOLE.Commit :

1. Given the quantum oracle access to $\mathcal{O}''$, $\mathcal{A}_{\text{MH}}$ uses $\mathcal{O}''$ for $\mathcal{A}$'s quantum queries to $\mathcal{O}''$.
2. $\mathcal{A}_{\text{MH}}$ locally emulates the oracles in $\mathcal{O} \setminus \mathcal{O}''$ for $\mathcal{A}$. Note that this emulation is perfect due to the compressed oracle technique.
3. For each OSig_{sk} query, $\mathcal{A}_{\text{MH}}$ receives $(\text{com}, \{c_\alpha\}_{\alpha \in [1,\tau]}, \mathbf{u}, \text{chall}_3, \text{decom}_I)$ from the multi-hiding game, reconstructs $(\Delta, \mathbf{Q})$ using chall_3 and decom_I , and reversely computes $\mathbf{V} \leftarrow \mathbf{Q} \oplus \Delta \cdot \mathbf{u}$ for the output signature.
4. $\mathcal{A}_{\text{MH}}$ outputs whatever its emulated game against $\mathcal{A}$ outputs.

$\mathcal{A}_{\text{MH}}$ makes at most $Q_E + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil$ queries to E (by the construction of PRG) and at most $Q_{E'} + Q_{\text{sig}} \cdot \frac{3L}{2}$ queries to E' . If $b = 0$, the emulated game of $\mathcal{A}_{\text{MH}}$ runs as G_4 since in each OSig_{sk} query, (i) $(\mu, \mathbf{s})$ is independently uniform over $\{0, 1\}^{2\lambda} \times \{0, 1\}^\lambda$ for the unique signed message (without loss of generality), (ii) chall_3 is uniformly sampled until $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda] = 0_{\text{grind}}^w$, and (iii) $\mathbf{V}$ is correctly defined for the VOLE correlation by Lemma 9.1 for each of its columns. So,

$$\Pr \left[\text{GameMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}, 0}(Q_{\mathcal{O}''}, Q_{\text{sig}}) = 1 \right] = \Pr \left[G_4(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

If $b = 1$, the ideal multi-hiding game leads to the same distribution of G_5 :

$$\Pr \left[\text{GameMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}, 1}(Q_{\mathcal{O}''}, Q_{\text{sig}}) = 1 \right] = \Pr \left[G_5(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

Using the contradiction assumption and the above results, we prove a contradiction against Lemma 9.11 for $Q_{\mathcal{O}''}$ and $Q = Q_{\text{sig}}$.

- G_6 . This game is identical to the previous one, except that for each OSig_{sk} query, it samples uniform $\tilde{\mathbf{u}}$, computes $\tilde{\mathbf{V}} \leftarrow \tilde{\mathbf{Q}} \oplus \Delta \cdot \tilde{\mathbf{u}}$ for the reconstructed $(\Delta, \mathbf{Q})$, and uniformly resamples $\mathbf{u}[\ell_{\text{wit}} + \lambda, \ell_{\text{wit}} + 2\lambda + \text{B})$ and $\mathbf{V}[\ell_{\text{wit}} + \lambda, \ell_{\text{wit}} + 2\lambda + \text{B})$ to be consistent with $\tilde{\mathbf{u}} = \text{VOLEHash}(\text{chall}_1, \mathbf{u})$ and $\tilde{\mathbf{V}} = \text{VOLEHash}(\text{chall}_1, \mathbf{V})$.

This game has the same distribution as the previous one since (i) $\mathbf{u}[\ell_{\text{wit}} + \lambda, \ell_{\text{wit}} + 2\lambda + \text{B})$ and $\mathbf{V}[\ell_{\text{wit}} + \lambda, \ell_{\text{wit}} + 2\lambda + \text{B})$ in both games are uniform, and (ii) VOLEHash is $\mathbb{F}_2^{\ell_{\text{wit}} + \lambda}$ -hiding.

$$\Pr [G_5(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] = \Pr [G_6(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] . \quad (49)$$

- G_7 . This game is identical to the previous one, except that for each OSig_{sk} query, it samples uniform $\tilde{a}_1$ and $\tilde{a}_0$, and uniformly resamples $\mathbf{u}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda)$ and $\mathbf{V}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda)$ to be consistent with $\tilde{\mathbf{u}} = \text{VOLEHash}(\text{chall}_1, \mathbf{u})$, $\tilde{\mathbf{V}} = \text{VOLEHash}(\text{chall}_1, \mathbf{V})$, $\tilde{a}_1 = \text{ZKHash}(\text{chall}_2, [\mathbf{a}_1, \tilde{\mathbf{u}}])$, and $\tilde{a}_0 = \text{ZKHash}(\text{chall}_2, [\mathbf{a}_0, \tilde{\mathbf{v}}])$.

This game has the same distribution as the previous one since (i) $\mathbf{u}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda)$ and $\mathbf{V}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda)$ in both games are uniform, and (ii) ZKHash is $\mathbb{F}_2^{\ell_{\text{wit}}}$ -hiding.

$$\Pr [G_6(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] = \Pr [G_7(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] . \quad (50)$$

- G_8 . This game is identical to the previous one, except that for each OSig_{sk} query, it samples uniform $\mathbf{d}$. This game has the same distribution as the previous one since $\mathbf{u}[0, \ell_{\text{wit}})$ is already uniform in G_5 and independent of any other value in the signature computation. Thus,

$$\begin{aligned} \Pr [G_7(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] &= \Pr [G_8(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] \\ &= \text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{Lynxer}}(Q_{\mathcal{O}}), \end{aligned} \quad (51)$$

where the last equality follows from the fact that in G_8 , for each OSig_{sk} query, the output signature can be simulated without sk as in the EUF-KO security game.

According to eq.(42), eq.(43), eq.(45), eq.(46), eq.(47), eq.(48), eq.(49), eq.(50), and eq.(51), we obtain this theorem. $\square$

9.3 Security Proof in the ROM and ICM

Although the classical security is trivially implied by the quantum security, we can have more dedicated proofs against a classical adversary. In this setting, we use the following proof ideas of [BBD⁺23a], the version 1.1 of FAEST, which is not based on the lossy-key argument.

Theorem 9.6 (EUF-KO Security of Lynxer against Classical Adversary). *In Lynxer, consider the following ideal-model oracles and their respective numbers of queries:*

- E is an ideal cipher realizing PRG in LeafHash , with Q_E queries,
- E' is an ideal cipher realizing TCCR in BAVC.Commit , with $Q_{E'}$ queries,
- MsgHash is a random oracle in Lynxer.Sign , with Q_{MsgHash} queries,
- CoinHash is a random oracle in Lynxer.Sign , with Q_{CoinHash} queries,
- BAVCHash is a random oracle in BAVC.Commit , with Q_{BAVCHash} queries,
- FSHash_1 is a random oracle in Lynxer.Sign , with Q_{FSHash_1} queries,
- FSHash_2 is a random oracle in Lynxer.Sign , with Q_{FSHash_2} queries,
- FSHash_3 is a random oracle in Lynxer.Sign , with Q_{FSHash_3} queries.

Game $\text{GameVOLEExtBind}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}})$
1 : $(\text{com}, (\mathbf{c}_{\alpha})_{\alpha \in [1, \tau]}, \text{iv}_{\text{sig}}) \leftarrow \mathcal{A}^{\mathcal{O}}()$ 2 : $(\mathbf{u}_{\alpha}^*, \mathbf{V}_{\alpha}^*)_{\alpha \in [0, \tau)} \leftarrow \text{Ext}(\mathcal{L}_{\mathcal{O}}, \text{com}, (\mathbf{c}_{\alpha})_{\alpha \in [1, \tau)}, \text{iv}_{\text{sig}})$ 3 : $(\text{chall}_3, \text{decom}_I) \leftarrow \mathcal{A}^{\mathcal{O}}()$ such that $I \leftarrow \text{VOLE.ChalDecAll}(\text{chall}_3)$ 4 : $\forall \alpha \in [0, \tau) : (\delta_{\alpha,0}, \dots, \delta_{\alpha, n_{\alpha}-1}) \leftarrow (\Delta_{\alpha}[0], \dots, \Delta_{\alpha}[n_{\alpha}-1])$ 5 : $(\overline{\text{com}}, \mathbf{Q}_0, \dots, \mathbf{Q}_{\tau-1}) \leftarrow \text{VOLE.Reconstruct}(\text{chall}, \text{decom}_I, (\mathbf{c}_{\alpha})_{\alpha \in [1, \tau)}, \text{iv}_{\text{sig}}, \mu, \mathbf{s})$ 6 : return 0 if 7 : (a) VOLE.Reconstruct returns $\perp$ or $\overline{\text{com}} \neq \text{com}$, or 8 : (b) $\forall \alpha \in [0, \tau) : \mathbf{Q}_{\alpha} = \mathbf{V}_{\alpha}^* \oplus [\delta_{\alpha,0} \cdot (\mathbf{u}_{\alpha}^* \oplus \mathbf{c}_{\alpha}), \dots, \delta_{\alpha, n_{\alpha}-1} \cdot (\mathbf{u}_{\alpha}^* \oplus \mathbf{c}_{\alpha})]$, where $\mathbf{c}_0 = 0$ 9 : return 1

Figure 10: The extractable binding game of $\text{VOLE} = (\text{VOLE.Commit}, \text{VOLE.Reconstruct})$.

Let $\mathcal{O}$ be a list of the above ideal-model oracles and $Q_{\mathcal{O}}$ be a list of their respective numbers of queries. Let $\mathcal{O}' = \{\mathbf{E}, \mathbf{E}', \text{BAVCHash}\}$ and $Q_{\mathcal{O}'} = \{Q_{\mathbf{E}}, Q_{\mathbf{E}'}, Q_{\text{BAVCHash}}\}$. For any classical adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ classical queries respectively to the oracles in $\mathcal{O}$,

$$\begin{aligned}
\text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{Lynxer}}(Q_{\mathcal{O}}) &\leq \frac{(Q_{\text{BAVCHash}} + Q_{\text{FSHash}_1})^2}{2^{2\lambda}} + \frac{(Q_{\text{FSHash}_1} + Q_{\text{FSHash}_2})^2}{2^{5\lambda+64}} \\
&\quad + \frac{(Q_{\text{FSHash}_2} + Q_{\text{FSHash}_3})^2}{2^{3\lambda+64}} + \frac{Q_{\text{FSHash}_3}^2}{2^{\lambda}} \\
&\quad + Q_{\text{FSHash}_1} \cdot \text{AdvVOLEExtBind}_{\mathcal{A}_{\text{VOLE}}}^{\mathcal{O}', \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}'}) \\
&\quad + Q_{\text{FSHash}_1} \cdot Q_{\text{FSHash}_2} \cdot \varepsilon_{\text{vole}} \cdot \binom{\tau}{2} + Q_{\text{FSHash}_2} \cdot \varepsilon_{\text{zk}} + Q_{\text{FSHash}_3} \cdot \frac{2}{2^{\lambda}} \\
&\quad + \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{\text{Lynx}}(),
\end{aligned}$$

where the extractable binding game $\text{GameVOLEExtBind}_{\mathcal{A}_{\text{VOLE}}}^{\mathcal{O}', \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}'})$ is defined in Fig. 10, and the advantage

$$\text{AdvVOLEExtBind}_{\mathcal{A}_{\text{VOLE}}}^{\mathcal{O}', \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}'}) = \Pr \left[\text{GameVOLEExtBind}_{\mathcal{A}_{\text{VOLE}}}^{\mathcal{O}', \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}'}) = 1 \right].$$

Theorem 9.7 (EUF-CMA Security of Lynxer against Classical Adversary). *In Lynxer, consider the following ideal-model oracles and their respective numbers of queries:*

- $\mathbf{E}$ is an ideal cipher realizing PRG in LeafHash and $\text{VOLE.ConvertToVOLE}$, with $Q_{\mathbf{E}}$ queries,
- $\mathbf{E}'$ is an ideal cipher realizing TCCR in BAVC.Commit , with $Q_{\mathbf{E}'}$ queries,
- MsgHash is a random oracle in Lynxer.Sign , with Q_{MsgHash} queries,
- CoinHash is a random oracle in Lynxer.Sign , with Q_{CoinHash} queries,
- BAVCHash is a random oracle in BAVC.Commit , with Q_{BAVCHash} queries,
- FSHash_1 is a random oracle in Lynxer.Sign , with Q_{FSHash_1} queries,
- FSHash_2 is a random oracle in Lynxer.Sign , with Q_{FSHash_2} queries,
- FSHash_3 is a random oracle in Lynxer.Sign , with Q_{FSHash_3} queries.

Let $\mathcal{O}$ be a list of the above ideal-model oracles and $Q_{\mathcal{O}}$ be a list of their respective numbers of queries. Let $\mathcal{O}' = \{E, E'\}$ and $Q_{\mathcal{O}'} = \{Q_E + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil, Q_{E'} + \frac{Q_{\text{sig}} \cdot 3L}{2}\}$. For any classical adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q_{sig} queries to the signature oracle OSig_{sk} ,

$$\begin{aligned} & \text{AdvEUF-CMA}_{\mathcal{A}}^{\mathcal{O}, \text{Lynxer}}(Q_{\mathcal{O}}, Q_{\text{sig}}) \\ & \leq Q_{\text{sig}} \cdot \left(\frac{Q_{\text{sig}} + Q_{\text{CoinHash}}}{2^\lambda} + \frac{Q_{\text{sig}} + Q_{\text{FSHash}_1}}{2^{2\lambda+128}} + \frac{Q_{\text{sig}} + Q_{\text{BAVCHash}}}{2^{2\lambda}} \right. \\ & \quad + 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{E', \text{CCR}} \left(Q_{E'} + \frac{3L}{2} + 2\lceil \log L \rceil + 5, 2\lceil \log L \rceil + 5, \chi \right) + \frac{(2\lceil \log L \rceil + 5)^2}{2^{\lambda-1}} \\ & \quad + \frac{Q_{\text{sig}} + Q_{\text{FSHash}_2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q_{\text{FSHash}_3}}{2^{3\lambda+64}} \Bigg) \\ & \quad + \text{AdvMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}', \text{Vole.Commit}}(Q_{\mathcal{O}'}, Q_{\text{sig}}) \\ & \quad + \text{AdvEUF-KO}_{\mathcal{A}}^{\mathcal{O}, \text{Lynxer}}(Q_{\mathcal{O}}), \end{aligned}$$

where $\text{AdvEUF-KO}_{\mathcal{A}}^{\mathcal{O}, \text{Lynxer}}(Q_{\mathcal{O}})$ is from [Theorem 9.6](#), and

- **CCR** : $x \in \{0, 1\}^\lambda \mapsto \text{TCCR}(x, s, \text{iv}_{\text{sig}}) \in \{0, 1\}^\lambda$ is an E' -based CCR hash function for some fixed s and (uniform) iv_{sig} ([Theorem 9.12](#)).

Proof sketch. For the EUF-KO bound in [Theorem 9.6](#), we need the analysis of the collision between random-oracle inputs and the extractable binding property of the VOLE commitment ([Fig. 10](#)). This extractable binding property can only exist in the classical setting, where it is feasible to observe all oracle queries. Assuming that all random-oracle inputs have no collision, we can prove that ([Vole.Commit](#), [Vole.Reconstruct](#)) satisfies the property by defining Ext to use these non-colluding inputs to reconstruct the VOLE outputs of the prover as in our proof against the quantum adversary, but no longer use the inefficient enumeration of all pre-images. Then, we have the same probability analysis of the RBR soundness in the quantum proof since these bad events occur regardless of the classical or quantum adversary. However, we do not use the reduction from RBR soundness to soundness but take a simple union bound over all random-oracle queries for these bad events. Putting these arguments with the underlying Lynx security, we arrive at the EUF-KO bound against the classical adversary.

For the EUF-CMA bound in [Theorem 9.7](#), the proof is similar to its counterpart in the QROM and QICM in that it replaces the real signature oracle by a simulated oracle without the secret key to reduce the EUF-CMA security to the EUF-KO security. The main difference between the quantum and classical security is whether the local emulation of (Q)ICM can be perfect or not due to the quantum/classical access. Furthermore, we can directly analyze the successful programming and do not rely on the reprogramming lemma. Based on the above strategies, we have the following security theorems in the classical setting. We stress that, in the classical setting, we do not rely on [Conjecture 9.1](#) as in the quantum setting but can provably construct an almost injective PRG in the ideal cipher model as in [Theorem 9.9](#).

9.4 Proofs of Subroutine Lemmas and Theorems

In this subsection, we provide formal proofs for the lemmas and theorems used in the previous proofs for completeness.

9.4.1 Local Emulation of Ideal Cipher

Lemma 9.12 (Simulating a Quantum SPRP). *Let $\mathcal{X}$ and $\mathcal{K}$ be finite sets, and let $E : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{X}$ be an ideal cipher. Let q be a query bound. Then there exists a construction $\widehat{E}_r$ built*

from a random string r and $\Theta(\log_2^2 |\mathcal{X}|)$ rounds of the Morris-Rogaway sometimes-recurse (SR) shuffle [MR14], each round using a $2q$ -wise independent hash function $\mathcal{K} \times \mathcal{X} \rightarrow \mathcal{X}$, such that:

1. (Resource.) Implementing the $\Theta(\log_2^2 |\mathcal{X}|)$ round functions requires a random string r of length

$$\Theta(q \log_2^2 |\mathcal{X}| \cdot \log_2(|\mathcal{X}||\mathcal{K}|)) \text{ bits.}$$

2. (Classical security.) For any (classical) (p, q) -bounded distinguisher D ,

$$\left| \Pr[D^{E, E^{-1}} \Rightarrow 1] - \Pr[D^{\hat{E}_r, \hat{E}_r^{-1}} \Rightarrow 1] \right| \leq \frac{1}{|\mathcal{X}|}.$$

3. (Quantum security.) For any quantum distinguisher D making q superposition queries,

$$\left| \Pr[D^{|E\rangle} \Rightarrow 1] - \Pr[D^{\hat{E}_r} \Rightarrow 1] \right| \leq \varepsilon_{\text{SR}}(|\mathcal{X}|),$$

where $\varepsilon_{\text{SR}}(|\mathcal{X}|)$ is the quantum strong PRP (SPRP) advantage of the SR shuffle of Morris and Rogaway [MR14], as lifted to the quantum setting by [Zha25].

Proof. Step 1: SR construction and classical gap. By [MR14, Theorem 4], the sometimes-recurse (SR) shuffle can convert $t = \Theta(\log_2^2 |\mathcal{X}| - \log_2 |\mathcal{X}| \cdot \log_2 \varepsilon)$ independent random functions $F_1, \dots, F_t : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{X}$ and t round-keys into a t -round block cipher construction $\hat{E}_r$ such that the classical SPRP advantage against E is at most ε , independently of the adversary's computational power. Setting $\varepsilon = 1/|\mathcal{X}|$ yields $t = \Theta(\log_2^2 |\mathcal{X}|)$ rounds. The total number of round-key bits is $\Theta(\log_2^3 |\mathcal{X}|)$.

Step 2: Implementing the round functions. In the classical setting, each $F_i : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{X}$ is implemented as a q -wise independent hash function (sufficient to fool any q -query distinguisher [Zha21, Sect. 5.4]). In the quantum setting, each F_i must be a $2q$ -wise independent hash function instead [Zha12]. In both cases, the change in degree has no asymptotic effect on the $\Theta(\cdot)$ notation for the resource count.

Storing $\Theta(\log_2^2 |\mathcal{X}|)$ such functions, each requiring $\Theta(q \cdot (\log_2 |\mathcal{K}| + \log_2 |\mathcal{X}|))$ random bits to describe, the total length of the shared random string is

$$|r| = \Theta(q \log_2^2 |\mathcal{X}| \cdot \log_2(|\mathcal{X}||\mathcal{K}|)).$$

Step 3: Classical security (Item 2). By the SR theorem [MR14, Theorem 4], with $\varepsilon = 1/|\mathcal{X}|$, the classical SPRP advantage between $\hat{E}_r$ and the ideal cipher E is at most $1/|\mathcal{X}|$, establishing Item 2.

Step 4: Quantum security (Item 3). By [Zha25], a classically secure full-domain SPRP (such as the SR construction with quantum-degree-doubled round functions from Step 2) can be lifted to a quantum-secure SPRP. Concretely, [Zha25] guarantees the existence of a quantum SPRP advantage bound, which we denote $\varepsilon_{\text{SR}}(|\mathcal{X}|)$, for the SR construction. As noted in [Zha25], the explicit form of $\varepsilon_{\text{SR}}(|\mathcal{X}|)$ cannot be extracted from the current version of [MR14]; we therefore leave it as a named quantity. This establishes Item 3. $\square$

9.4.2 Proof of Lemma 3.1

Lemma 3.1 (CCR for Natural and Non-trivial Queries). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $H^{\mathcal{O}} : \{0, 1\}^{\ell} \rightarrow \{0, 1\}^{\ell}$ be a $(Q'_{\mathcal{O}}, Q, \chi, \varepsilon)$ -CCR hash function using these oracles, where $Q'_{\mathcal{O}} = Q_{\mathcal{O}} + (Q, \dots, Q)$ corresponds to Q more queries for each oracle in $\mathcal{O}$. For any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q NNQs to its second oracle (the real-world oracle*

$\text{Real}_{H_S, \Delta}^{\mathcal{O}}$ or the ideal-world oracle $\text{Ideal}_{H_S}^{\mathcal{O}}$), where each NNQ only depends on the results of the previous NNQs (with the initial NNQ only depending on an empty result),

$$\left| \Pr_{\Delta, S \leftarrow \chi} \left[\mathcal{A}^{\mathcal{O}, \text{Real}_{H_S, \Delta}^{\mathcal{O}}}() = 1 \right] - \Pr_{S \leftarrow \chi, c \leftarrow \mathcal{U}} \left[\mathcal{A}^{\mathcal{O}, \text{Ideal}_{H_S}^{\mathcal{O}}}() = 1 \right] \right| \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q_{\mathcal{O}}, Q, \chi) + \frac{Q^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}},$$

where $\mathcal{U}$ is the uniform distribution of some proper unspecified length, and c is the random coin used for Type 4 NNQs in $\text{Ideal}_{H_S}^{\mathcal{O}}$.

Proof. We prove this lemma from the following two lemmas.

Lemma 9.13. *Let $\text{coll}_{\text{real}}$ be the event that, during the interaction between $\mathcal{A}$ and its given oracles, there exists two defined results $x, x' \in \{0, 1\}^{\ell}$ such that $x = x'$. It holds that*

$$\Pr_{\Delta, S \leftarrow \chi} [\text{coll}_{\text{real}}] \leq \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q_{\mathcal{O}}, Q, \chi) + \frac{Q^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}}.$$

Proof. For some $f' \leftarrow \mathcal{F}_{\ell, \ell}$, define a hybrid oracle $\text{Hyb}_{f', \Delta}^{\mathcal{O}}$ that is identical to $\text{Real}_{H_S, \Delta}^{\mathcal{O}}$ except that it executes a Type 3 NNQ by defining $x_i \leftarrow f'(x_j)$ and executes a Type 4 NNQ by defining $x_i \leftarrow f'(x_j \oplus \Delta) \oplus b_j \cdot \Delta$. In other words, $\text{Hyb}_{f', \Delta}^{\mathcal{O}}$ replaces the $H_S^{\mathcal{O}}$ calls in $\text{Real}_{H_S, \Delta}^{\mathcal{O}}$ by f' calls. Let coll_{hyb} be the event that, during the interaction between $\mathcal{A}$ and $(\mathcal{O}, \text{Hyb}_{f', \Delta}^{\mathcal{O}})$, there exist two defined results $x, x' \in \{0, 1\}^{\ell}$ such that $x = x'$.

First, we proceed to prove that

$$\left| \Pr_{\Delta, S \leftarrow \chi} [\text{coll}_{\text{real}}] - \Pr_{\Delta \leftarrow \chi, f' \leftarrow \mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}] \right| \leq \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q_{\mathcal{O}}, Q, \chi). \quad (52)$$

For contradiction, we assume that eq.(52) does not hold for a deterministic polynomial-time $\mathcal{A}$. We show that the following adversary $\mathcal{A}_{\text{CCR}}$ can use $\mathcal{A}$ to break the CCR security of $H^{\mathcal{O}}$:

1. Given the quantum oracle access to $\mathcal{O}$, $\mathcal{A}_{\text{CCR}}$ uses $\mathcal{O}$ for $\mathcal{A}$'s quantum queries to $\mathcal{O}$.
2. $\mathcal{A}_{\text{CCR}}$ samples $\Delta \leftarrow \chi$. Upon receiving an NNQ q_i from $\mathcal{A}$, $\mathcal{A}_{\text{CCR}}$ proceeds as follows:
 - If q_i is a Type 1 or Type 2 NNQ, $\mathcal{A}_{\text{CCR}}$ executes it as required.
 - If q_i is a Type 3 NNQ with input x_j , $\mathcal{A}_{\text{CCR}}$ queries the CCR oracle with $(x_j, 0)$, receives a response r , and defines the result of q_i to r .
 - If q_i is a Type 4 NNQ with input (x_j, b_j) , $\mathcal{A}_{\text{CCR}}$ queries the CCR oracle with $(x_j \oplus \Delta, 0)$, receives a response r , and defines the result of q_i to $r \oplus b_j \cdot \Delta$.
3. $\mathcal{A}_{\text{CCR}}$ terminates when $\mathcal{A}$ terminates. It outputs 1 if there exist two defined results x, x' such that $x = x'$; otherwise it outputs 0.

$\mathcal{A}_{\text{CCR}}$ makes at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ due to at most $Q_{\mathcal{O}}$ queries of $\mathcal{A}$, and at most Q queries to the CCR oracle due to at most Q NNQs. Furthermore, $\mathcal{A}_{\text{CCR}}$ is polynomial-time since $\mathcal{A}$ is polynomial-time.

If $\mathcal{A}_{\text{CCR}}$ is given $\mathcal{O}_{H, S}^{\mathcal{O}, \text{ccr}}$ as the second oracle for $S \leftarrow \chi$, $\mathcal{A}$ works as if it is given $\text{Real}_{H_S, \Delta}^{\mathcal{O}}$ as the second oracle. Thus, during this emulated interaction between $\mathcal{A}$ and its given oracles, $\text{coll}_{\text{real}}$ happens with the same probability as in the case where $\mathcal{A}$ interacts with $(\mathcal{O}, \text{Real}_{H_S, \Delta}^{\mathcal{O}})$. Although $\mathcal{A}$ does not know the key S in the emulation (in contrast to a public S in $\text{Real}_{H_S, \Delta}^{\mathcal{O}}$), this does not affect the probability of $\text{coll}_{\text{real}}$ since every NNQ only depends on the previous NNQ responses. One can consider the event $\text{coll}_{\text{real}}$ in the emulated and the real cases under the same probability space, i.e., the identical literal values of Δ and (unknown) S in both two

cases. Since $\mathcal{A}$ is deterministic, an induction from the first NNQ queried by $\mathcal{A}$ shows that the responses are the *same* in the two cases given the same Δ and S . Depending on these responses, the occurrence of $\text{coll}_{\text{real}}$ is the same. Thus,

$$\Pr_{S \leftarrow \mathbb{S}_\chi} \left[\mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{\text{H}, S}^{\text{ccr}}} () = 1 \right] = \Pr_{\Delta, S \leftarrow \mathbb{S}_\chi} [\text{coll}_{\text{real}}].$$

If $\mathcal{A}_{\text{CCR}}$ is given $f^* \leftarrow \mathbb{S}_{\mathcal{F}_{\ell+1, \ell}}$ as the second oracle, $\mathcal{A}$ will interact with an emulated second oracle that perfectly works as $\text{Hyb}_{f', \Delta}^{\mathcal{O}}$ for the random function $f'(\cdot) \leftarrow f^*(\cdot, 0)$. Thus,

$$\Pr_{f^* \leftarrow \mathbb{S}_{\mathcal{F}_{\ell+1, \ell}}} \left[\mathcal{A}_{\text{CCR}}^{\mathcal{O}, f^*} () = 1 \right] = \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}].$$

By the contradiction assumption, $\mathcal{A}_{\text{CCR}}$ breaks the CCR property of $\text{H}^{\mathcal{O}}$.

Second, we prove in the following induction that

$$\Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}] \leq \frac{Q^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}}. \quad (53)$$

As the base case, eq.(53) trivially holds for only one NNQ. Then, assume that eq.(53) holds for exact $1 \leq Q' - 1 \leq Q - 1$ NNQs and consider the Q' -th NNQ that defines a result $x \in \{0, 1\}^\ell$. Let $\text{coll}_{\text{hyb}}(i)$ be the event that, during the interaction between $\mathcal{A}$ and its given oracles, there exist two results $x, x' \in \{0, 1\}^\ell$ such that $x = x'$ in the first i NNQs. $\text{coll}_{\text{hyb}} = \text{coll}_{\text{hyb}}(Q)$ and

$$\begin{aligned} & \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}(Q')] \\ &= \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}(Q') \mid \text{coll}_{\text{hyb}}(Q' - 1)] \cdot \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}(Q' - 1)] \\ & \quad + \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1)] \cdot \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\neg \text{coll}_{\text{hyb}}(Q' - 1)] \\ &= \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}(Q' - 1)] \\ & \quad + \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1)] \cdot \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\neg \text{coll}_{\text{hyb}}(Q' - 1)] \\ &\leq \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}(Q' - 1)] + \Pr_{\Delta \leftarrow \mathbb{S}_\chi, f' \leftarrow \mathbb{S}_{\mathcal{F}_{\ell, \ell}}} [\text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1)]. \end{aligned}$$

For each result $x' \in \{0, 1\}^\ell$ defined in the previous $Q' - 1$ NNQs, it *syntactically* holds that

$$x \oplus x' = \left(\bigoplus_{j \in \mathcal{J}} f'(x_j) \right) \oplus \left(\bigoplus_{k \in \mathcal{K}} f'(x_k \oplus \Delta) \right) \oplus b \cdot \Delta \oplus \bigoplus_{\tau \in \mathcal{T}} b_\tau \cdot \tau$$

as x, x' can be written as a linear combination of NNQs. Note that the non-triviality of NNQ ensures that the case “ $\mathcal{J} = \mathcal{K} = \emptyset$ and $b = 0$ and each $b_\tau = 0$ ” is impossible. Conditioned on $\neg \text{coll}_{\text{hyb}}(Q' - 1)$, there are two cases for such x and x' :

- $\mathcal{J} = \mathcal{K} = \emptyset$ and $b = 0$ and at least one $b_\tau \neq 0$. By the definition of $\mathcal{T}$, $x \oplus x' \neq 0$.
- At least one $\mathcal{J}, \mathcal{K} \neq \emptyset$ or $b \neq 0$. This case is essentially identical to the case that at least one $\mathcal{J}, \mathcal{K} \neq \emptyset$ since $b \neq 0$ implies that $\mathcal{K}$ is not trivially empty, given that (i) $b \neq 0$ implies at least one syntactical term of the form $f'(x_k \oplus \Delta)$ for some $k \in \mathcal{K}$, and (ii) any x_k for $k \in \mathcal{K}$ is distinct conditioned on $\neg \text{coll}_{\text{hyb}}(Q' - 1)$ (i.e., any additive term $f'(x_k \oplus \Delta)$ in $\mathcal{K}$ cannot be canceled due to previous collisions).

The probability of $x = x'$ in this case equals the probability of that a random string (from at least one non-empty $\mathcal{J}, \mathcal{K}$) lies in the space spanned by linearly independent strings in $\mathcal{T}$ (for any Boolean coefficients $\{b_\tau\}_{\tau \in \mathcal{T}}$). This probability is bounded by $2^{|\mathcal{T}|}/2^\ell$.

Combining the two cases, we can see that $x = x'$ occurs with the probability at most $2^{|\mathcal{T}|}/2^\ell$. Taking a union bound over all x' 's in the previous $Q' - 1$ NNQs,

$$\Pr_{\Delta \leftarrow \mathcal{S}, f' \leftarrow \mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1)] \leq \frac{(Q' - 1) \cdot 2^{|\mathcal{T}|}}{2^\ell}.$$

Using the induction assumption, we have that

$$\begin{aligned} \Pr_{\Delta \leftarrow \mathcal{S}, f' \leftarrow \mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}(Q')] &\leq \Pr_{\Delta \leftarrow \mathcal{S}, f' \leftarrow \mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}(Q' - 1)] \\ &\quad + \Pr_{\Delta \leftarrow \mathcal{S}, f' \leftarrow \mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1)] \\ &\leq \frac{(Q' - 1)^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}} + \frac{(Q' - 1) \cdot 2^{|\mathcal{T}|}}{2^\ell} < \frac{Q'^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}} \end{aligned}$$

This induction shows that eq.(53) holds. This lemma follows from eq.(52) and eq.(53). $\square$

Lemma 9.14. *Let $\mathcal{U}$ and c be defined as per Lemma 3.1. Let $\text{coll}_{\text{real}}$ be the event defined as per Lemma 9.13. It holds that*

$$\begin{aligned} &\left| \Pr_{\Delta, S \leftarrow \mathcal{S}} [\mathcal{A}^{\mathcal{O}, \text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}}() = 1] - \Pr_{S \leftarrow \mathcal{S}, c \leftarrow \mathcal{U}} [\mathcal{A}^{\mathcal{O}, \text{Ideal}_{\text{H}_S}^{\mathcal{O}}}() = 1] \right| \\ &\leq \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, \text{H}}(Q'_{\mathcal{O}}, Q, \chi) + \Pr_{\Delta, S \leftarrow \mathcal{S}} [\text{coll}_{\text{real}}]. \end{aligned}$$

Proof. Assume that this lemma does not hold for some deterministic polynomial-time $\mathcal{A}$. The following adversary $\mathcal{A}_{\text{CCR}}$ can use $\mathcal{A}$ to break the CCR security of $\text{H}^{\mathcal{O}}$.

1. Given the quantum oracle access to $\mathcal{O}$, $\mathcal{A}_{\text{CCR}}$ uses $\mathcal{O}$ for $\mathcal{A}$'s quantum queries to $\mathcal{O}$.
2. $\mathcal{A}_{\text{CCR}}$ samples $S \leftarrow \mathcal{S}$. Upon receiving an NNQ q_i from $\mathcal{A}$, $\mathcal{A}_{\text{CCR}}$ proceeds as follows:
 - If q_i is a Type 1/2/3 NNQ, $\mathcal{A}_{\text{CCR}}$ executes it as required.
 - If q_i is a Type 4 NNQ with input (x_j, b_j) and x_j was not shown in previous NNQs, $\mathcal{A}_{\text{CCR}}$ queries the CCR oracle with $(x_j \oplus S, b_j)$, receives a response r , and defines the result of q_i to r . If q_i is a Type 4 NNQ with input (x_j, b_j) and x_j was shown in some previous NNQ, $\mathcal{A}_{\text{CCR}}$ samples $s \leftarrow \mathcal{S}$ and defines the result of q_i to s .
3. $\mathcal{A}_{\text{CCR}}$ terminates when $\mathcal{A}$ terminates and outputs whatever $\mathcal{A}$ outputs.

$\mathcal{A}_{\text{CCR}}$ makes at most $Q'_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ due to at most $Q_{\mathcal{O}}$ queries of $\mathcal{A}$ plus at most Q queries per oracle in $\mathcal{O}$ resulting from Type 3 NNQs, and at most Q queries to the CCR oracle due to at most Q NNQs. $\mathcal{A}_{\text{CCR}}$ is polynomial-time.

Consider that $\mathcal{A}_{\text{CCR}}$ is given $\mathcal{O}_{\text{H}, \Delta}^{\mathcal{O}, \text{CCR}}$ as the second oracle for some $\Delta \leftarrow \mathcal{S}$. We analyze the event $\text{coll}_{\text{real}}$ during the interaction between $\mathcal{A}$ and $\mathcal{O}$ plus the second oracle either emulated by $\mathcal{A}_{\text{CCR}}$ or the oracle $\text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}$ under the same probability space. That is, the randomness Δ and S take the *same* values in both cases, respectively (note that, in the emulated case, $\text{coll}_{\text{real}}$ does not depend on the randomness s in the presence of any colliding $x_j \in \{0, 1\}^\ell$; otherwise, there will be a circularity).

For any given deterministic $\mathcal{A}$, the first NNQ is the same regardless of the oracle given to it. Assume that the event $\text{coll}_{\text{real}}$ does not occur. By the assumption, an induction based on the same values of Δ and S shows that every defined result and every queried NNQ are the *same* in the two cases. Thus, for any fixed Δ and S , the event $(\mathcal{A}^{\mathcal{O}, \text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}}() = 1) \wedge \neg \text{coll}_{\text{real}}$ occurs if and only if the event $(\mathcal{A}^{\mathcal{A}_{\text{CCR}}}() = 1) \wedge \neg \text{coll}_{\text{real}}$ occurs. Since, in the emulation, $\mathcal{A}_{\text{CCR}}$

outputs 1 if and only if $\mathcal{A}$ outputs 1, the event $(\mathcal{A}^{\mathcal{A}_{\text{CCR}}}() = 1) \wedge \neg \text{coll}_{\text{real}}$ occurs if and only if $(\mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{\text{H}, \Delta}^{\text{ccr}}}() = 1) \wedge \neg \text{coll}_{\text{real}}$ occurs. It follows from the difference lemma [Sho04] that

$$\left| \Pr_{\Delta \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{\text{H}, \Delta}^{\text{ccr}}}() = 1 \right] - \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}^{\mathcal{O}, \text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}}() = 1 \right] \right| \leq \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} [\text{coll}_{\text{real}}]. \quad (54)$$

If $\mathcal{A}_{\text{CCR}}$ is given $f^* \leftarrow \mathcal{F}_{\ell+1, \ell}$, we have

$$\Pr_{f^* \leftarrow \mathcal{F}_{\ell+1, \ell}} \left[\mathcal{A}_{\text{CCR}}^{\mathcal{O}, f^*}() = 1 \right] = \Pr_{S \leftarrow \mathcal{S}_\chi, c \leftarrow \mathcal{U}} \left[\mathcal{A}^{\mathcal{O}, \text{Ideal}_{\text{H}_S}^{\mathcal{O}}}() = 1 \right]. \quad (55)$$

for the following reasons. First, if there is no collision in the execution of Type 4 NNQs, the unique x_j 's ensure that the outputs of the random function f^* (when $\mathcal{A}$ is given the second oracle emulated by $\mathcal{A}_{\text{CCR}}$) are uniform and pairwise independent as those uniformly sampled (when $\mathcal{A}$ is given $\text{Ideal}_{\text{H}_S}^{\mathcal{O}}$ as the second oracle). Second, if there exists at least one collision, the result defined by the first NNQ with the conflicting x_j is still uniform as per f^* . For the subsequent results with respect to the same x_j , their uniformness and pairwise independence are ensured by the fresh sampling of $\mathcal{A}_{\text{CCR}}$.

Using the contradiction assumption and eq.(54), eq.(55), we have

$$\begin{aligned} & \left| \Pr_{\Delta \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{\text{H}, \Delta}^{\text{ccr}}}() = 1 \right] - \Pr_{f^* \leftarrow \mathcal{F}_{\ell+1, \ell}} \left[\mathcal{A}_{\text{CCR}}^{\mathcal{O}, f^*}() = 1 \right] \right| \\ &= \left| \left(\Pr_{\Delta \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{\text{H}, \Delta}^{\text{ccr}}}() = 1 \right] - \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}^{\mathcal{O}, \text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}}() = 1 \right] \right) \right. \\ & \quad \left. + \left(\Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}^{\mathcal{O}, \text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}}() = 1 \right] - \Pr_{S \leftarrow \mathcal{S}_\chi, c \leftarrow \mathcal{U}} \left[\mathcal{A}^{\mathcal{O}, \text{Ideal}_{\text{H}_S}^{\mathcal{O}}}() = 1 \right] \right) \right| \\ &\geq \left| \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}^{\mathcal{O}, \text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}}() = 1 \right] - \Pr_{S \leftarrow \mathcal{S}_\chi, c \leftarrow \mathcal{U}} \left[\mathcal{A}^{\mathcal{O}, \text{Ideal}_{\text{H}_S}^{\mathcal{O}}}() = 1 \right] \right| \\ & \quad - \left| \Pr_{\Delta \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{\text{H}, \Delta}^{\text{ccr}}}() = 1 \right] - \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}^{\mathcal{O}, \text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}}() = 1 \right] \right| > \text{Adv}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, \text{H}}(Q'_\mathcal{O}, Q, \chi), \end{aligned}$$

which contradicts the CCR security of $\text{H}^{\mathcal{O}}$. $\square$

This lemma follows from Lemma 9.13, Lemma 9.14, and the trivial fact that

$$\text{Adv}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, \text{H}}(Q'_\mathcal{O}, Q, \chi) \geq \text{Adv}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, \text{H}}(Q_\mathcal{O}, Q, \chi)$$

since $\mathcal{A}_{\text{CCR}}$ can make more queries to the oracles in $\mathcal{O}$. $\square$

9.4.3 (Quantum) Security of the CTR-based PRG

Quantum pseudorandomness of CTR-based PRG. We first show the quantum pseudorandomness security bound of $G^E(k, \text{iv}_{\text{sig}}, t) := E(k, \text{iv}_{\text{sig}} \| t) \| E(k, (\text{iv}_{\text{sig}} \| t) + 1) \| \dots \| E(k, (\text{iv}_{\text{sig}} \| t) + \ell - 1)$, in the quantum ideal cipher model.

Theorem 9.8 (Quantum pseudorandomness security bound of CTR-based PRG). *Fix iv_{sig} and t in arbitrary. For any quantum adversary $\mathcal{A}^E$ making T queries to the quantum ideal cipher $E : \{0, 1\}^\lambda \times \{0, 1\}^n \mapsto \{0, 1\}^n$, its advantage on distinguishing $G^E(k, \text{iv}_{\text{sig}}, t)$ output and random strings is bounded by*

$$\left| \Pr[k \leftarrow \mathcal{S} : \{0, 1\}^\lambda : \mathcal{A}^E(G^E(k, \text{iv}_{\text{sig}}, t)) = 1] - \Pr[z \leftarrow \mathcal{S} : \{0, 1\}^{\ell n} : \mathcal{A}^E(y) = 1] \right| \leq O\left(\frac{T}{2^{\lambda/2}} + \frac{\ell^3}{2^n}\right). \quad (56)$$

Proof. We introduce a random permutation $\mathbf{RP} : \{0, 1\}^n \mapsto \{0, 1\}^n$ and define $\mathbf{RP}$ -based “idealized” PRG variant $G^{\mathbf{RP}}(k, \text{iv}_{\text{sig}}, t) := \mathbf{RP}(\text{iv}_{\text{sig}} \| t) \| \mathbf{RP}((\text{iv}_{\text{sig}} \| t) + 1) \| \dots \| \mathbf{RP}((\text{iv}_{\text{sig}} \| t) + \ell - 1)$. It is well-known that

$$\left| \Pr[k \xleftarrow{\$} \{0, 1\}^\lambda : \mathcal{A}^E(G^E(k, \text{iv}_{\text{sig}}, t)) = 1] - \Pr[k \xleftarrow{\$} \{0, 1\}^\lambda : \mathcal{A}^E(G^{\mathbf{RP}}(k, \text{iv}_{\text{sig}}, t)) = 1] \right| \leq O\left(\frac{T}{2^{\lambda/2}}\right). \quad (57)$$

Further introduce a random function $\mathbf{RF} : \{0, 1\}^n \mapsto \{0, 1\}^n$ and consider $\mathbf{RF}$ -based PRG $G^{\mathbf{RF}}(k, \text{iv}_{\text{sig}}, t) := \mathbf{RF}(\text{iv}_{\text{sig}} \| t) \| \mathbf{RF}((\text{iv}_{\text{sig}} \| t) + 1) \| \dots \| \mathbf{RF}((\text{iv}_{\text{sig}} \| t) + \ell - 1)$. Since $G^{\mathbf{RP}}(k, \text{iv}_{\text{sig}}, t)$, resp. $G^{\mathbf{RF}}(k, \text{iv}_{\text{sig}}, t)$, makes ℓ calls to $\mathbf{RP}$, resp. $\mathbf{RF}$, the quantum PRF-PRP switch lemma indicates that

$$\left| \Pr[k \xleftarrow{\$} \{0, 1\}^\lambda : \mathcal{A}^E(G^{\mathbf{RP}}(k, \text{iv}_{\text{sig}}, t)) = 1] - \Pr[k \xleftarrow{\$} \{0, 1\}^\lambda : \mathcal{A}^E(G^{\mathbf{RF}}(k, \text{iv}_{\text{sig}}, t)) = 1] \right| \leq O\left(\frac{\ell^3}{2^n}\right). \quad (58)$$

Summing over Eqs. (57) and (58) yields the claim. $\square$

Classical and quantum almost injectivity of CTR-based PRG. The security of the scheme relies on the so-called almost injectivity of the PRG, meaning informally that an adversary cannot produce a PRG output that has multiple preimages. In this section, we formalize this notion and prove it in the ideal cipher model. Since the proof relies on the quantum ideal cipher model, the definition is also presented only in this model.

Proof for classical almost injectivity. The security of the scheme relies on the so-called almost injectivity of the PRG, meaning informally that an adversary cannot produce a PRG output that has multiple preimages. In this section, we formalize this notion and prove it in the ideal cipher model. Since the proof relies on the quantum ideal cipher model, the definition is also presented only in this model.

Definition 9.4. Consider an IV-based tweakable PRG $G^E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \times \{0, 1\}^\lambda \mapsto \{0, 1\}^{\ell\lambda}$. For any adversary $\mathcal{A}^E$, we define the injectivity game $\text{Inj}_{G, \mathcal{A}}$, which runs in two phases:

1. The adversary $\mathcal{A}^E$ interacts with the ideal cipher E , and outputs an IV $\text{iv}_{\text{sig}} \in \{0, 1\}^\lambda$ and D tuples $(t_1, y_1), \dots, (t_D, y_D)$.
2. The game $\text{Inj}_{G, \mathcal{A}}$ outputs 1 if there exists at least one tuple (t_j, y_j) among the D tuples such that there exist two distinct PRG seeds $k, k' \in \{0, 1\}^\lambda$ both producing y_j under t_j , i.e., $G^E(k, \text{iv}_{\text{sig}}, t_j) = G^E(k', \text{iv}_{\text{sig}}, t_j) = y_j$. Otherwise, the game $\text{Inj}_{G, \mathcal{A}}$ outputs 0.

The advantage of adversary $\mathcal{A}$ in breaking the almost injectivity of G is defined as

$$\text{Adv}_G^{\text{Inj}}(\mathcal{A}) := \Pr[\text{Inj}_{G, \mathcal{A}} = 1]. \quad (59)$$

We then prove that $G^E(k, \text{iv}_{\text{sig}}, t) := E(k, \text{iv}_{\text{sig}} \| t) \| E(k, (\text{iv}_{\text{sig}} \| t) + 1) \| \dots \| E(k, (\text{iv}_{\text{sig}} \| t) + \ell - 1)$ is quantum almost injective in the quantum ideal cipher model.

Theorem 9.9 (Almost Injectivity Bound for CTR-based PRG). Consider any adversary $\mathcal{A}^E$ that makes T queries to the ideal cipher $E : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ and outputs D tuples $(t_1, y_1), \dots, (t_D, y_D)$ in the first phase of $\text{Inj}_{G, \mathcal{A}}$. Then

$$\text{Adv}_G^{\text{Inj}}(\mathcal{A}) \leq \frac{2T^2(\ell^\ell + 1) + \ell T 2^\lambda + D 2^\lambda}{2^{\ell(n-1)}}. \quad (60)$$

For the case $\ell n = 2\lambda$, the above bound simplifies to

$$\mathbf{Adv}_G^{\text{Inj}}(\mathcal{A}) \leq \frac{2^{\ell+1}T^2(\ell+1)}{2^{2\lambda}} + \frac{2^\ell(\ell T + D)}{2^\lambda}. \quad (61)$$

Proof. Given an adversary $\mathcal{A}$ for the game $\text{Inj}_{G,\mathcal{A}}$, we construct an adversary $\mathcal{B}^E$ that maintains a query list $\mathcal{Q}_E$ and runs $\mathcal{A}$ as follows. When $\mathcal{A}$ queries $E(L, u)$, $\mathcal{B}^E$ queries its own oracle $E(L, u) \rightarrow v$, forwards the answer v to $\mathcal{A}$, and adds the query record (L, u, v) to $\mathcal{Q}_E$. $\square$

Proof. Given an adversary $\mathcal{A}$ for the game $\text{Inj}_{G,\mathcal{A}}$, we construct an adversary $\mathcal{B}^E$ that maintains a query list $\mathcal{Q}_E$ and runs $\mathcal{A}$ as follows. When $\mathcal{A}$ queries $E(L, u)$, $\mathcal{B}^E$ queries its oracle $E(L, u) \rightarrow v$, forwards the reply v to $\mathcal{A}$, and adds the record (L, u, v) to $\mathcal{Q}_E$.

When $\mathcal{A}$ queries $E^{-1}(L, v)$, $\mathcal{B}^E$ queries its oracle $E^{-1}(L, v) \rightarrow u$, forwards the reply u to $\mathcal{A}$, and adds the record (L, u, v) to $\mathcal{Q}_E$.

Then, if the number of records under key L in $\mathcal{Q}_E$ reaches $2^\lambda/2$, $\mathcal{B}$ issues all remaining $2^\lambda/2$ unseen queries under L . That is, for all $(L, u) \in \{0, 1\}^\lambda \times \{0, 1\}^n$, if $\mathcal{Q}_E$ does not already contain a record of the form $(L, u, \star)$, $\mathcal{B}$ makes an additional query $E(L, u) \rightarrow v$ and appends (L, u, v) to $\mathcal{Q}_E$.

We now analyze the execution of the game $\text{Inj}_{G,\mathcal{B}}$. It is easy to see that if $\mathcal{A}$ makes T queries to E during phase one, then $\mathcal{B}$ makes at most $2T$ queries to E in phase one.⁶

An execution of $\text{Inj}_{G,\mathcal{A}}$ is called *bad* if, after completion, there exist 2ℓ triples in $\mathcal{Q}_E$ with identical plaintexts and ciphertexts but distinct keys, i.e., of the form $(L, u, v_1), (L', u, v_1), (L, u + 1, v_2), (L', u + 1, v_2), \dots, (L, u + \ell - 1, v_\ell), (L', u + \ell - 1, v_\ell)$ with $L \neq L'$.

Given two such starting records (L, u, v_1) and (L', u, v_1) , of which there are $\binom{2T}{2} \leq 2T^2$ possible pairs, the subsequent $2\ell - 2$ records are uniquely determined. For any pair $((L, u + i, v_i), (L', u + i, v_i))$, we consider the following cases:

- Case where $(L, u + i, v_i)$ appears later than $(L', u + i, v_i)$:
 - If $(L, u + i, v_i)$ is generated by $\mathcal{B}$ forwarding a query $E(L, u + i)$ from $\mathcal{A}$, then by construction of $\mathcal{B}$, at the time of the query, the number of records under key L in $\mathcal{Q}_E$ is at most $2^n/2$. Thus the response $E(L, u + i)$ is chosen uniformly from at least $2^n - 2^n/2 \geq 2^n/2$ values, so the probability that $E(L, u + i) = v_i$ is at most $2/2^n$.
 - If $(L, u + i, v_i)$ is generated by an additional query $E(L, u + i)$ from $\mathcal{B}$, then the response is again uniform over $2^n/2$ values, so the probability that $E(L, u + i) = v_i$ is at most $2/2^n$.
 - If $(L, u + i, v_i)$ is generated by $\mathcal{B}$ forwarding an inverse query $E^{-1}(L, v_i)$ from $\mathcal{A}$, then similarly, the number of records under L is at most $2^n/2$, so the response $E^{-1}(L, v_i)$ is uniform over at least $2^n/2$ values, and the probability that $E^{-1}(L, v_i) = u + i$ is at most $2/2^n$.
- The case where $(L', u + i, v_i)$ appears later than $(L, u + i, v_i)$ is symmetric.

In all cases, the probability that $\mathcal{B}$ successfully forms such a pair $((L, u + i, v_i), (L', u + i, v_i))$ is bounded by $1/2^{n-1}$. Taking the union bound over ℓ positions and summing over the $2T^2$ initial pairs yields

$$\Pr [\text{Inj}_{G,\mathcal{B}} \text{ is bad}] \leq \frac{2T^2}{2^{\ell(n-1)}}. \quad (62)$$

⁶The reason is that T queries by $\mathcal{A}$ can “fill” at most $\frac{T}{2^\lambda-1} = \frac{2T}{2^\lambda}$ keys L to the point where $\mathcal{B}$ must complete $2^\lambda/2$ queries. Thus $\mathcal{B}$ completes at most $\frac{2T}{2^\lambda} \times \frac{2^\lambda}{2} = T$ queries, which together with $\mathcal{A}$ ’s T queries sum to $2T$.

An execution of $\text{Inj}_{G,\mathcal{B}}$ is called *good* if $\mathcal{Q}_E$ contains no such families of triples. It follows that

$$\begin{aligned} \Pr [\text{Inj}_{G,\mathcal{B}} = 1] &\leq \Pr [\text{Inj}_{G,\mathcal{B}} \text{ is bad}] \\ &\quad + \Pr [\text{Inj}_{G,\mathcal{B}} = 1 \mid \text{Inj}_{G,\mathcal{B}} \text{ is good}]. \end{aligned} \quad (63)$$

In a good execution of $\text{Inj}_{G,\mathcal{B}}$, let $\mathcal{Q}_E = ((L_1, u_1, v_1), \dots, (L_T, u_T, v_T))$ be the list of queries after phase one. Under this condition, the ideal cipher E satisfies

$$E(L_1, u_1) = v_1 \wedge \dots \wedge E(L_T, u_T) = v_T.$$

We write $E \vdash \mathcal{Q}_E$ for this condition.

Let $\mathcal{S} = \{(t_1, y_1), \dots, (t_D, y_D)\}$ be the set output by $\mathcal{A}$. Then

$$\begin{aligned} &\Pr [\text{Inj}_{G,\mathcal{B}} = 1 \mid \text{Inj}_{G,\mathcal{B}} \text{ is good}] \\ &\leq \sum_{\substack{(k, k', (t, y)) \in \{0,1\}^\lambda \times \{0,1\}^\lambda \times \mathcal{S} \\ k \neq k'}} \Pr [G(k, \text{iv}_{\text{sig}}, t) = G(k', \text{iv}_{\text{sig}}, t) = y \mid \text{Inj}_{G,\mathcal{B}} \text{ is good} \wedge E \vdash \mathcal{Q}_E]. \end{aligned} \quad (64)$$

The sum contains at most $(D2^\lambda)^2$ distinct tuples $(k, k', (t, y))$.

For any such tuple, the event $G(k, \text{iv}_{\text{sig}}, t) = G(k', \text{iv}_{\text{sig}}, t) = y$ depends on 2ℓ entries of E :

$$\begin{aligned} &E(k, \text{iv}_{\text{sig}} \| t) = E(k', \text{iv}_{\text{sig}} \| t) = y[1] \\ &\wedge E(k, (\text{iv}_{\text{sig}} \| t) + 1) = E(k', (\text{iv}_{\text{sig}} \| t) + 1) = y[2] \\ &\wedge \dots \\ &\wedge E(k, (\text{iv}_{\text{sig}} \| t) + \ell - 1) = E(k', (\text{iv}_{\text{sig}} \| t) + \ell - 1) = y[\ell]. \end{aligned}$$

Under $E \vdash \mathcal{Q}_E$, some of these entries are fixed by $\mathcal{Q}_E$. However, under the goodness condition, the ℓ equations

$$\begin{aligned} &E(k, \text{iv}_{\text{sig}} \| t) = E(k', \text{iv}_{\text{sig}} \| t), \\ &E(k, (\text{iv}_{\text{sig}} \| t) + 1) = E(k', (\text{iv}_{\text{sig}} \| t) + 1), \\ &\dots, \\ &E(k, (\text{iv}_{\text{sig}} \| t) + \ell - 1) = E(k', (\text{iv}_{\text{sig}} \| t) + \ell - 1) \end{aligned} \quad (65)$$

cannot all hold simultaneously.

We classify tuples $(k, k', (\text{iv}_{\text{sig}}, t, y))$ into four types based on which entries are fixed:

- **Type 1 (Pair Conflict):** There exists an index $i \in \{0, \dots, \ell - 1\}$ such that $\mathcal{Q}_E$ contains two records $(k, (\text{iv}_{\text{sig}} \| t) + i, v)$ and $(k', (\text{iv}_{\text{sig}} \| t) + i, v')$ with $v \neq v'$. Then under $E \vdash \mathcal{Q}_E$, we have $E(k, (\text{iv}_{\text{sig}} \| t) + i) \neq E(k', (\text{iv}_{\text{sig}} \| t) + i)$, so (65) cannot hold. Thus

$$\Pr [\exists \text{ Type 1 tuple } (k, k', (t, y)) : G(k, \text{iv}_{\text{sig}}, t) = G(k', \text{iv}_{\text{sig}}, t) = y] = 0. \quad (66)$$

- **Type 2 (α -Paired), $\alpha = 1, \dots, \ell - 1$:** There exist α indices $i_1, \dots, i_\alpha \in \{0, \dots, \ell - 1\}$ such that:

- For each $i \in \{i_1, \dots, i_\alpha\}$, $\mathcal{Q}_E$ contains both $(k, (\text{iv}_{\text{sig}} \| t) + i, y[i+1])$ and $(k', (\text{iv}_{\text{sig}} \| t) + i, y[i+1])$, so under $E \vdash \mathcal{Q}_E$, $E(k, (\text{iv}_{\text{sig}} \| t) + i) = E(k', (\text{iv}_{\text{sig}} \| t) + i) = y[i+1]$.
- For each $i \in \{0, \dots, \ell - 1\} \setminus \{i_1, \dots, i_\alpha\}$, $\mathcal{Q}_E$ is missing at least one of $(k, (\text{iv}_{\text{sig}} \| t) + i, \star)$ or $(k', (\text{iv}_{\text{sig}} \| t) + i, \star)$, so at least one of $E(k, (\text{iv}_{\text{sig}} \| t) + i)$ or $E(k', (\text{iv}_{\text{sig}} \| t) + i)$ is unassigned.

Since at least $\ell - \alpha$ positions are unassigned,

$$\Pr \left[\bigwedge_{\substack{i \in \{0, \dots, \ell-1\} \\ i \notin \{i_1, \dots, i_\alpha\}}} E(k, (\text{iv}_{\text{sig}} \| t) + i) = y[i+1] \wedge E(k', (\text{iv}_{\text{sig}} \| t) + i) = y[i+1] \mid E \vdash \mathcal{Q}_E \right] \leq \left(\frac{1}{2^{n-1}} \right)^{\ell-\alpha}. \quad (67)$$

To count such α -paired tuples, we define an α -paired query sequence for a set $\{i_1, \dots, i_\alpha\}$ as 2α records of the form

$$\begin{aligned} & (L, u + i_1, v_1), (L', u + i_1, v_1), \\ & (L, u + i_2, v_2), (L', u + i_2, v_2), \\ & \dots, \\ & (L, u + i_\alpha, v_\alpha), (L', u + i_\alpha, v_\alpha). \end{aligned}$$

For any such index set, there are at most $\binom{2T}{2} \leq 2T^2$ pairs of queries among $\mathcal{B}$'s $2T$ queries that could form such a sequence. The probability that any pair actually forms such a sequence is at most $1/2^{\alpha(n-1)}$. By linearity of expectation,

$$\mathbb{E}[\text{number of } \{i_1, \dots, i_\alpha\}\text{-}\alpha\text{-paired sequences}] \leq \frac{2T^2}{2^{\alpha(n-1)}}. \quad (68)$$

There are at most $\binom{\ell}{\alpha} \leq \ell^\alpha$ index sets, so

$$\mathbb{E}[\text{number of } \alpha\text{-paired sequences}] \leq \frac{2T^2 \ell^\alpha}{2^{\alpha(n-1)}}. \quad (69)$$

The number of α -paired tuples is at most the number of such sequences. Thus

$$\begin{aligned} & \Pr [\exists \alpha\text{-paired tuple } (k, k', (t, y)) : G(k, \text{iv}_{\text{sig}}, t) = G(k', \text{iv}_{\text{sig}}, t) = y] \\ & \leq \sum_M \Pr [\text{number of } \alpha\text{-paired sequences} = M] \cdot M \cdot \left(\frac{1}{2^{n-1}} \right)^{\ell-\alpha} \\ & \leq \mathbb{E}[\text{number of } \alpha\text{-paired sequences}] \cdot \frac{1}{2^{(\ell-\alpha)(n-1)}} \\ & \leq \frac{2T^2 \ell^\alpha}{2^{\ell(n-1)}}. \end{aligned} \quad (70)$$

Summing over α yields

$$\begin{aligned} & \Pr [\exists \text{ Type 2 tuple } (k, k', (t, y)) : G(k, \text{iv}_{\text{sig}}, t) = G(k', \text{iv}_{\text{sig}}, t) = y] \\ & \leq \sum_{\alpha=1}^{\ell-1} \frac{2T^2 \ell^\alpha}{2^{\ell(n-1)}} \leq \frac{2T^2 \ell^\ell}{2^{\ell(n-1)}}. \end{aligned} \quad (71)$$

- **Type 3 (β -Partial), $\beta = 1, \dots, \ell - 1$:** There exist β indices $i_1, \dots, i_\beta \in \{0, \dots, \ell - 1\}$ such that:

- For each $i \in \{i_1, \dots, i_\beta\}$, $\mathcal{Q}_E$ contains exactly one of $(k, (\text{iv}_{\text{sig}} \| t) + i, y[i+1])$ or $(k', (\text{iv}_{\text{sig}} \| t) + i, y[i+1])$, so one value is fixed to $y[i+1]$ and the other is unassigned.
- For all other i , neither entry is in $\mathcal{Q}_E$, so both are unassigned.

There are $2\ell - \beta$ unassigned positions, so

$$\Pr \left[\bigwedge_{i=0}^{\ell-1} E(k, (\text{iv}_{\text{sig}} \| t) + i) = y[i+1] \wedge E(k', (\text{iv}_{\text{sig}} \| t) + i) = y[i+1] \mid E \vdash \mathcal{Q}_E \right] \leq \left(\frac{1}{2^{n-1}} \right)^{2\ell-\beta} \leq \frac{1}{2^{\ell(n-1)}}. \quad (72)$$

Each Type 3 tuple corresponds to at least one query in $\mathcal{Q}_E$, with up to ℓ possible positions and at most 2^λ choices for k' . Thus the number of Type 3 tuples is at most $\ell T 2^\lambda$, and

$$\Pr [\exists \text{ Type 3 tuple } (k, k', (t, y)) : G(k, \text{iv}_{\text{sig}}, t) = G(k', \text{iv}_{\text{sig}}, t) = y] \leq \frac{\ell T 2^\lambda}{2^{\ell(n-1)}}. \quad (73)$$

- **Type 4 (Blank):** For all $i \in \{0, \dots, \ell-1\}$, neither $(k, (\text{iv}_{\text{sig}} \| t) + i, \star)$ nor $(k', (\text{iv}_{\text{sig}} \| t) + i, \star)$ appears in $\mathcal{Q}_E$. For each such tuple and each i ,

$$\begin{aligned} \Pr [E(k, (\text{iv}_{\text{sig}} \| t) + i) = y[i+1] \mid E \vdash \mathcal{Q}_E] &\leq \frac{1}{2^{n-1}}, \\ \Pr [E(k', (\text{iv}_{\text{sig}} \| t) + i) = y[i+1] \mid E \vdash \mathcal{Q}_E] &\leq \frac{1}{2^{n-1}}. \end{aligned}$$

Thus the probability that $G(k, \text{iv}_{\text{sig}}, t) = G(k', \text{iv}_{\text{sig}}, t) = y$ is at most $1/2^{2\ell(n-1)}$. There are at most $(D 2^\lambda)^2$ such tuples, so

$$\begin{aligned} \Pr [\exists \text{ Type 4 tuple } (k, k', (t, y)) : G(k, \text{iv}_{\text{sig}}, t) = G(k', \text{iv}_{\text{sig}}, t) = y] \\ \leq \left(\frac{D 2^\lambda}{2^{\ell(n-1)}} \right)^2. \end{aligned} \quad (74)$$

Summing (66), (71), (73), and (74) gives

$$\begin{aligned} \Pr [\exists (k, k', (t, y)) : G(k, \text{iv}_{\text{sig}}, t) = G(k', \text{iv}_{\text{sig}}, t) = y \mid E \vdash \mathcal{Q}_E] \\ \leq 0 + \frac{2T^2 \ell^\ell}{2^{\ell(n-1)}} + \frac{\ell T 2^\lambda}{2^{\ell(n-1)}} + \frac{D^2 2^{2\lambda}}{2^{2\ell(n-1)}} \\ \leq \frac{2T^2 \ell^\ell + \ell T 2^\lambda + D^2 2^{2\lambda}}{2^{\ell(n-1)}}. \end{aligned} \quad (75)$$

Combining with the bad-event bound (62) yields the final bound (60). $\square$

$$\Pr [\text{Inj}_{G, \mathcal{B}} \text{ is bad}] \leq \frac{2T^2}{2^{\ell(n-1)}}. \quad (76)$$

Equation (76) bounds the probability that $\mathcal{A}$ can find records in its query list that violate injectivity. We thus obtain the following corollary.

Theorem 9.10. *Let $\mathcal{A}^E$ be any adversary that makes T queries to the ideal cipher $E : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ and produces a query list $\mathcal{Q}_E$. Then*

$$\Pr [\mathcal{A}^E \Rightarrow \mathcal{Q}_E, \exists ((L, u + i, v_i), (L', u + i, v_i))_{i=0}^{\ell-1} \in \mathcal{Q}_E^{2\ell}] \leq \frac{2T^2}{2^{\ell(n-1)}}. \quad (77)$$

For the case $\ell n = 2\lambda$, the bound simplifies to

$$\text{Adv}_G^{\text{Inj}}(\mathcal{A}) \leq \frac{2^{\ell+1} T^2}{2^{2\lambda}}. \quad (78)$$

Intuition for quantum almost injectivity. The quantum analogue of almost injectivity can be obtained by modifying Definition 9.4 to allow the adversary $\mathcal{A}^{[E]}$ having access to the ideal cipher implemented as a quantum oracle—but $\mathcal{A}^{[E]}$ still produce classical outputs $\text{iv}_{\text{sig}} \in \{0, 1\}^\lambda$ and D tuples $(t_1, y_1), \dots, (t_D, y_D)$. Denote this security game by $\text{QInj}_{G, \mathcal{A}}$, and define

$$\text{Adv}_G^{\text{QInj}}(\mathcal{A}) := \Pr[\text{QInj}_{G, \mathcal{A}} = 1]. \quad (79)$$

Regarding the PRG construction $G^E(k, \text{iv}_{\text{sig}}, t) := E(k, \text{iv}_{\text{sig}} \| t) \| E(k, (\text{iv}_{\text{sig}} \| t) + 1) \| \dots \| E(k, (\text{iv}_{\text{sig}} \| t) + \ell - 1)$, we have the following conjecture on its quantum almost injective bound.

Conjecture 9.1 (Quantum Almost Injectivity Bound for CTR-based PRG). *Consider any adversary $\mathcal{A}^{[E]}$ that makes T superposition queries to the quantum ideal cipher $E : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ and outputs D tuples $(t_1, y_1), \dots, (t_D, y_D)$ in the first phase of $\text{QInj}_{G, \mathcal{A}}$. Then*

$$\text{Adv}_G^{\text{QInj}}(\mathcal{A}) \leq O\left(\frac{T^4}{2^n} + \frac{\ell T 2^\lambda + D 2^\lambda}{2^{\ell(n-1)}}\right). \quad (80)$$

For the case $\ell n = c = 2\lambda$, the above bound simplifies to

$$\text{Adv}_G^{\text{QInj}}(\mathcal{A}) \leq O\left(\frac{T^4}{2^n} + \frac{2^\ell(\ell T + D)}{2^\lambda}\right). \quad (81)$$

A similar conjecture has been made in FAESTv2 [BBB⁺26]. However, in FAESTv2, the authors heuristically treat ideal-cipher queries and PRG queries as the same and ignore backward queries, which may potentially be used by the adversary to break the almost injective PRG. Based on this treatment, they give a concrete security bound for a specific attack (see [BBB⁺26, page 71]). This bound does not explicitly consider the quantum setting. In contrast, we provide more intuitions to justify this almost injectiveness in the QICM as follows.

We serve intuitions for this conjectured bound below. The classical bound given in Eq. (60) consists of two terms $\frac{2T^2(\ell+1)}{2^{\ell(n-1)}}$ and $\frac{\ell T 2^\lambda + D 2^\lambda}{2^{\ell(n-1)}}$: the former reflects the probability that the classical adversary $\mathcal{A}^E$ “finds” (and thus can output) such two keys k, k' using his T (classical) ideal cipher queries, while the latter reflects the probability that without enough knowledge, $\mathcal{A}^E$ succeeds in “guessing” such two keys k, k' . Intuitively, the latter only depends on the randomness of the ideal cipher and thus should be the same in the quantum setting.

To bound the former in the quantum setting, note that $\mathcal{A}^{[E]}$ outputs such two keys k, k' (using his T quantum ideal cipher queries) only if $\mathcal{A}^{[E]}$ can output two distinct inputs (k, x) and (k', x') such that $k \neq k', x = x'$ and $E(k, x) = E(k', x')$. The probability is easy to bound in the classical setting, and a recent work of Cojocaru et al. [CHL⁺25] proved theorem lifting evasiveness w.r.t. classical ideal ciphers to that w.r.t. quantum ideal ciphers.

Theorem 9.11 ([CHL⁺25], Theorem 4.1). *Let $\mathcal{A}^{E, E^{-1}}$ be a quantum algorithm that makes T quantum queries to an (invertible) ideal cipher oracle E and R is a relation on $(\{0, 1\}^b)^m \times (\{0, 1\}^b)^m$. Then there exists an algorithm $\mathcal{B}^{E, E^{-1}}$ making at most m classical queries such that*

$$\begin{aligned} \Pr_E[\mathcal{B}^E \Rightarrow ((k_1, x_1), \dots, (k_m, x_m)) : (((k_1, x_1), \dots, (k_m, x_m)), (E(k_1, x_1), \dots, E(k_m, x_m))) \in R] \\ \geq \frac{\left(1 - \frac{m^2}{2^b}\right)}{(8T + 1)^{2m}} \cdot \Pr_E[\mathcal{A}^{[E]} \Rightarrow ((k_1, x_1), \dots, (k_m, x_m)) : \\ (((k_1, x_1), \dots, (k_m, x_m)), (E(k_1, x_1), \dots, E(k_m, x_m))) \in R]. \end{aligned} \quad (82)$$

By these, let R_{pair} be the relation such that

$$(((k_1, x_1), (k_2, x_2)), (E(k_1, x_1), E(k_2, x_2))) \in R_{\text{pair}} \text{ iff. } k_1 \neq k_2 \wedge x_1 = x_2 \wedge E(k_1, x_1) = E(k_2, x_2).$$

With 2 queries, it clearly holds

$$\Pr_E[\mathcal{B}^E \Rightarrow ((k_1, x_1), (k_2, x_2)) : (((k_1, x_1), (k_2, x_2)), (E(k_1, x_1), E(k_2, x_2))) \in R_{\text{pair}}] = O\left(\frac{1}{2^n}\right).$$

It thus holds

$$\Pr_E[\mathcal{A}^{[E]} \Rightarrow ((k_1, x_1), (k_2, x_2)) : (((k_1, x_1), (k_2, x_2)), (E(k_1, x_1), E(k_2, x_2))) \in R_{\text{pair}}] = O\left(\frac{T^4}{2^n}\right).$$

Using this to replace the first term in Eq. (60) yields Eq. (80).

9.4.4 TCCR Security

Non-quantum TCCR security. We prove the following theorem.

Theorem 9.12 (Non-Quantum Single-User TCCR Security of TCCR). *In the ideal cipher model, the hash function H^E defined above is $(q_E, q_C, \rho, \varepsilon)$ -tweakable circular correlation robust (single-user), where*

$$\varepsilon \leq \frac{4q_E\mu}{2^\rho} + \frac{q_C(\mu - 1)}{2^n}, \quad (83)$$

and μ is the maximum number of construction queries on any fixed tweak (so $\mu \leq q_C$).

Proof. Fix a deterministic distinguisher $\mathcal{D}$ that makes at most q_E queries to the ideal cipher E/E^{-1} and at most q_C queries to the construction oracle. In the real world, the construction oracle is $\hat{\mathcal{O}}_\Gamma^{\text{tccr}}$ for $\Gamma \leftarrow \mathcal{R}$. In the ideal world, the construction oracle is a uniform random function $\hat{f} : \{0, 1\}^\lambda \times \{0, 1\}^{n-8} \times \{0, 1\} \rightarrow \{0, 1\}^{2n}$.

Transcript. Let Q_E be the transcript of ideal cipher queries, consisting of entries (k, x, y) recording $E(k, x) = y$. Let Q_O be the transcript of construction queries, consisting of entries (w, t, b, z) recording $\hat{\mathcal{O}}(w, t, b) = z$, where $z = (z_L, z_R) \in \{0, 1\}^{2n}$. Let Γ denote the key. The full transcript is $\tau = (Q_E, Q_O, \Gamma)$.

Bad Transcripts. We say a transcript $\tau = (Q_E, Q_O, \Gamma)$ is *bad* if either of the following conditions holds:

- **(B-1)** There exists $(w, t, b, z) \in Q_O$ and $(k, x, y) \in Q_E$ such that one of the following four sub-cases occurs:
 1. (a) $k = k(w_R, t)$ and $x = p_0(w_L)$;
 2. (b) $k = k(w_R, t)$ and $x = p_1(w_L)$;
 3. (c) $k = k(w_R, t)$ and $y = z_L \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus b \cdot \tilde{\Gamma}_L$;
 4. (d) $k = k(w_R, t)$ and $y = z_R \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus b \cdot \tilde{\Gamma}_R$.
- **(B-2)** There exist *distinct* $(w, t, b, z), (w', t', b', z') \in Q_O$ with $w_R = w'_R$ and $t = t'$ such that either:
 1. (a) $w_L \oplus b \cdot \tilde{\Gamma}_L \oplus z_L = w'_L \oplus b' \cdot \tilde{\Gamma}_L \oplus z'_L$;
 2. (b) $w_L \oplus b \cdot \tilde{\Gamma}_R \oplus z_R = w'_L \oplus b' \cdot \tilde{\Gamma}_R \oplus z'_R$ (in the $n < \lambda \leq 2n$ case).

Bounding $\Pr[(\text{B-1})]$ in the ideal world. We bound each case of (B-1) for every pair $((w, t, b, z), (k, x, y)) \in Q_O \times Q_E$.

Case (a): $k = k(w_R, t) \wedge x = p_0(w_L)$. This is equivalent to

$$(w_R \oplus \Gamma_R \parallel t \parallel 0^{2n-\lambda-8} = k) \wedge ([0]_8 \parallel (w_L \oplus \Gamma_L) = x). \quad (84)$$

From the first condition we recover $\Gamma_R = \text{left}_{\lambda-(n-8)}(k) \oplus w_R$. From the second condition, $w_L \oplus \Gamma_L = \text{right}_{n-8}(x)$, hence $\Gamma_L = \text{right}_{n-8}(x) \oplus w_L$. Thus Γ is fully determined by (k, x) , and the probability over $\Gamma \leftarrow \mathcal{R}$ is at most $2^{-\rho}$.

Case (b): $k = k(w_R, t) \wedge x = p_1(w_L)$. By an identical argument, substituting $[1]_8$ for $[0]_8$: Γ_R is recovered from the key equation, and $\Gamma_L = \text{right}_{n-8}(x) \oplus w_L$ from the plaintext equation. The probability is bounded by $2^{-\rho}$.

Case (c): $k = k(w_R, t) \wedge y = z_L \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus b \cdot \tilde{\Gamma}_L$. We consider two sub-cases based on b .

Sub-case $b = 0$: The condition becomes

$$(w_R \oplus \Gamma_R \parallel t \parallel 0^{2n-\lambda-8} = k) \wedge (y = z_L \oplus (0^8 \parallel (w_L \oplus \Gamma_L))).$$

From the first equation we get Γ_R . From the second:

$$(0^8 \parallel (w_L \oplus \Gamma_L)) = z_L \oplus y,$$

so $\Gamma_L = \text{right}_{n-8}(z_L \oplus y) \oplus w_L$. Hence Γ is fully determined and the probability is $\leq 2^{-\rho}$.

Sub-case $b = 1$: The condition becomes

$$(w_R \oplus \Gamma_R \parallel t \parallel 0^{2n-\lambda-8} = k) \wedge (y = z_L \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus \tilde{\Gamma}_L).$$

From the first equation we get $\Gamma_R = \text{left}_{\lambda-(n-8)}(k) \oplus w_R$. From the second:

$$(0^8 \parallel w_L) \oplus (0^8 \parallel \Gamma_L) \oplus \tilde{\Gamma}_L = z_L \oplus y.$$

Recall that $\tilde{\Gamma}_L = \text{left}_n(\Gamma \parallel 0^{2n-\lambda})$, which equals the first n bits of Γ . Whereas Γ_L provides the first $n-8$ bits of Γ . Therefore, the first $n-8$ bits of Γ can be recovered from this equation, by which Γ is fully determined as well. This case thus also holds with probability $\leq 2^{-\rho}$.

Case (d): $k = k(w_R, t) \wedge y = z_R \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus b \cdot \tilde{\Gamma}_R$.

The analysis is similar to Case (c) (yet simpler). When $b = 0$, Γ_R is determined from the key equation, and $\Gamma_L = \text{right}_{n-8}(z_R \oplus y) \oplus w_L$ from the output equation. When $b = 1$, Γ_R is again determined from the key equation, and the output equation

$$(0^8 \parallel (w_L \oplus \Gamma_L)) \oplus \tilde{\Gamma}_R = z_R \oplus y$$

uniquely determines Γ_L , because $\tilde{\Gamma}_R = \text{right}_n(\Gamma \parallel 0^{2n-\lambda})$ contains Γ_L -dependent bits as well as Γ_R -dependent bits (which are already known). In all sub-cases, the probability is $\leq 2^{-\rho}$.

We now sum over all pairs. For $t \in \{0, 1\}^{n-8}$, define

$$Q_E[t] := \{(k, x, y) \in Q_E \mid \text{the tweak field of } k \text{ equals } t\}.$$

Then by the union bound:

$$\begin{aligned} \Pr[(B-1)] &\leq \sum_{(w,t,b,z) \in Q_O} \frac{4 \cdot |Q_E[t]|}{2^\rho} \\ &= \sum_{t \in \{0,1\}^{n-8}} \sum_{(w,t,b,z) \in Q_O} \frac{4 \cdot |Q_E[t]|}{2^\rho} \\ &\leq \mu \sum_{t \in \{0,1\}^{n-8}} \frac{4 \cdot |Q_E[t]|}{2^\rho} = \frac{4\mu q_E}{2^\rho}. \end{aligned} \tag{85}$$

Bounding $\Pr[(\mathbf{B-2})]$ in the ideal world. Consider distinct $(w, t, b, z), (w', t', b', z') \in Q_O$ with $w_R = w'_R$ and $t = t'$. In the ideal world, z_L, z_R, z'_L, z'_R are uniform and independent random values.

Sub-case (a): We need

$$w_L \oplus b \cdot \tilde{\Gamma}_L \oplus z_L = w'_L \oplus b' \cdot \tilde{\Gamma}_L \oplus z'_L,$$

or equivalently $z_L \oplus z'_L = w_L \oplus w'_L \oplus (b \oplus b') \cdot \tilde{\Gamma}_L$. Conditioned on Γ , the values z_L and z'_L are independent and uniform over $\{0, 1\}^n$. Hence the probability that this equation holds is exactly 2^{-n} .

Sub-case (b): Similarly, for the R side,

$$w_L \oplus b \cdot \tilde{\Gamma}_R \oplus z_R = w'_L \oplus b' \cdot \tilde{\Gamma}_R \oplus z'_R.$$

Since z_R, z'_R are independent and uniform, the probability is 2^{-n} .

Let C_t denote the number of construction queries with fixed tweak t . By union bound:

$$\begin{aligned} \Pr[(\mathbf{B-2})] &\leq \sum_{t \in \{0,1\}^{n-8}} \binom{C_t}{2} \cdot \frac{2}{2^n} \\ &\leq \sum_{t \in \{0,1\}^{n-8}} \frac{C_t(\mu-1)}{2} \cdot \frac{2}{2^n} = \frac{q_C(\mu-1)}{2^n}. \end{aligned} \quad (86)$$

Analysis of Good Transcripts. Fix a good transcript $\tau = (Q_E, Q_O, \Gamma)$. We compare the probabilities that τ occurs in the real and ideal worlds.

Ideal world. The probability of obtaining τ is:

$$\Pr_{\text{id}}[\tau] = \frac{1}{2^\rho} \cdot \prod_{j=1}^m \prod_{\ell=1}^{q_{E,j}} \frac{1}{2^n - \ell + 1} \cdot \frac{1}{2^{2n \cdot q_C}},$$

where m is the number of distinct keys appearing in Q_E , and $q_{E,j}$ is the number of entries under the j -th key.

Real world. The probability is:

$$\Pr_{\text{re}}[\tau] = \frac{1}{2^\rho} \cdot \prod_{j=1}^m \prod_{\ell=1}^{q_{E,j}} \frac{1}{2^n - \ell + 1} \cdot p_O,$$

where

$$p_O := \Pr[\forall (w, t, b, z) \in Q_O : \hat{\mathcal{O}}_\Gamma^{\text{tccr}}(w, t, b) = z \mid E \vdash Q_E].$$

For each construction query (w_i, t_i, b_i, z_i) , define:

$$p_i := \Pr[\hat{\mathcal{O}}_\Gamma^{\text{tccr}}(w_i, t_i, b_i) = z_i \mid E \vdash Q_E \wedge \forall j < i : \hat{\mathcal{O}}_\Gamma^{\text{tccr}}(w_j, t_j, b_j) = z_j].$$

Recall that $\hat{\mathcal{O}}_\Gamma^{\text{tccr}}(w_i, t_i, b_i) = z_i$ requires:

$$z_{i,L} = E(k(w_{i,R}, t_i), p_0(w_{i,L})) \oplus (0^8 \parallel (w_{i,L} \oplus \Gamma_L)) \oplus b_i \cdot \tilde{\Gamma}_L, \quad (87)$$

$$z_{i,R} = E(k(w_{i,R}, t_i), p_1(w_{i,L})) \oplus (0^8 \parallel (w_{i,L} \oplus \Gamma_L)) \oplus b_i \cdot \tilde{\Gamma}_R. \quad (88)$$

Since the transcript is good, (B-1) does not occur, so neither $E(k(w_{i,R}, t_i), p_0(w_{i,L}))$ nor $E(k(w_{i,R}, t_i), p_1(w_{i,L}))$ is already in Q_E , nor are their inverses. Moreover, (B-2) does not occur, so these values are not determined by previous construction queries.

The ideal cipher E is a random permutation for each key. At the time of the i -th construction query, for each of the two E calls under k_i , there are at most $q_E + i - 1$ already-determined

input-output pairs. Hence each evaluation is uniform over at least $2^n - q_E - i + 1$ possible values, giving

$$p_i \geq \frac{1}{(2^n - q_E - i + 1)^2} \geq \frac{1}{2^{2n}}.$$

Thus $p_O = \prod_{i=1}^{q_C} p_i \geq 2^{-2n \cdot q_C}$, and we obtain

$$\frac{\Pr_{\text{re}}[\tau]}{\Pr_{\text{id}}[\tau]} \geq 1.$$

By the H-coefficient lemma [Pat09, CS14], the distinguishing advantage is bounded by

$$\text{Adv} \leq \Pr[(\text{B-1})] + \Pr[(\text{B-2})] \leq \frac{4\mu q_E}{2^\rho} + \frac{q_C(\mu - 1)}{2^n}.$$

This completes the proof. $\square$

Subroutine lemmas for the non-quantum proof. For completeness, we provide self-contained statements and proofs of all supporting lemmas that verify the recovery arguments used in the main proof.

Lemma 9.15 (H-Coefficient Lemma [Pat09, CS14]). *Let $\mathcal{D}$ be a deterministic distinguisher interacting with either the real world or the ideal world. Let T_{re} and T_{id} denote the probability distributions on transcripts induced by the real and ideal worlds, respectively. Partition the set of attainable transcripts into $\mathcal{T}_1$ (good) and $\mathcal{T}_2$ (bad). If there exists $\varepsilon_1 \in [0, 1)$ such that for all $\tau \in \mathcal{T}_1$,*

$$\frac{\Pr[T_{\text{re}} = \tau]}{\Pr[T_{\text{id}} = \tau]} \geq 1 - \varepsilon_1,$$

then the distinguishing advantage is bounded by

$$\Delta(T_{\text{re}}, T_{\text{id}}) \leq \varepsilon_1 + \Pr[T_{\text{id}} \in \mathcal{T}_2].$$

Proof. See [CS14, Section 3.3] for the standard proof. $\square$

Lemma 9.16 (Real-World Conditional Probability for Good Transcripts). *For a good transcript τ in the $n < \lambda \leq 2n$ case, for each construction query i ,*

$$p_i := \Pr[\hat{\mathcal{O}}_{\Gamma}^{\text{tccr}}(w_i, t_i, b_i) = z_i \mid E \vdash Q_E \wedge \forall j < i : \hat{\mathcal{O}}_{\Gamma}^{\text{tccr}}(w_j, t_j, b_j) = z_j] \geq \frac{1}{(2^n - q_E - i + 1)^2}.$$

Proof. The event $\hat{\mathcal{O}}_{\Gamma}^{\text{tccr}}(w_i, t_i, b_i) = z_i$ is equivalent to:

$$E(k_i, p_{0,i}) = z_{i,L} \oplus (0^8 \parallel (w_{i,L} \oplus \Gamma_L)) \oplus b_i \cdot \tilde{\Gamma}_L, \quad (89)$$

$$E(k_i, p_{1,i}) = z_{i,R} \oplus (0^8 \parallel (w_{i,L} \oplus \Gamma_L)) \oplus b_i \cdot \tilde{\Gamma}_R, \quad (90)$$

where $k_i := k(w_{i,R}, t_i)$, $p_{0,i} := p_0(w_{i,L})$, $p_{1,i} := p_1(w_{i,L})$.

Since the transcript is good, (B-1) does not occur. Hence neither $(k_i, p_{0,i})$ nor $(k_i, p_{1,i})$ appears as a forward query in Q_E , nor do the corresponding inverse entries appear. Furthermore, (B-2) does not occur, so these E values are not forced by previous construction queries.

The ideal cipher E is a random permutation for each key. At the time of the i -th construction query, for each of the two E calls, there are at most $q_E + i - 1$ already-determined input-output pairs under k_i . Hence each evaluation is uniform over at least $2^n - q_E - i + 1$ possible values, giving probability at least $1/(2^n - q_E - i + 1)$ for each E call. Since the two calls are on the same key k_i but with distinct inputs $p_{0,i} \neq p_{1,i}$ (they differ in the first 8 bits), they are independent uniform choices from the remaining permutation domain. Therefore,

$$p_i \geq \frac{1}{(2^n - q_E - i + 1)^2} \geq \frac{1}{2^{2n}} \quad (\text{assuming } q_E \ll 2^n).$$

$\square$

Corollary 9.1 (Overall Advantage Bound). *The overall distinguishing advantage for the single-user TCCR security of TCCR in the non-quantum ideal cipher model is*

$$\text{Adv}_{HE}^{\text{tccr}}(q_E, q_C) \leq \frac{4\mu q_E}{2^\rho} + \frac{q_C(\mu - 1)}{2^n}.$$

Quantum Q1 TCCR security. We will prove the following theorem step-by-step in the subsequent paragraphs.

Theorem 9.13 (Quantum Q1 TCCR Security Bound for TCCR). *The hash function TCCR is $(T, D, \chi, \varepsilon)$ -quantum TCCR, where*

$$\text{AdvTCCR}_{\mathcal{A}_{\text{TCCR}}}^{E, \text{TCCR}}(T, D, \chi) \leq O\left(\frac{D\sqrt{T}}{2^\rho} + \frac{T\sqrt{D}}{2^\rho} + \frac{D}{2^{\lambda-n}}\right), \quad (91)$$

where ρ is the min-entropy of the key distribution χ , D is the number of classical construction queries, and T is the total number of quantum ideal cipher queries.

Toolkit for quantum proofs. We recall the necessary quantum cryptography primitives, following the same presentation as in the original quantum proof.

Notations and definitions. We define the swap operator $\text{swap}_{a,b} : \mathcal{X} \rightarrow \mathcal{X}$ as

$$\text{swap}_{a,b}(x) = \begin{cases} a, & \text{if } x = b, \\ b, & \text{if } x = a, \\ x, & \text{otherwise.} \end{cases}$$

Given a function $F : \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$, we let $|F\rangle$ denote the appropriate quantum oracle $|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus F(x)\rangle$ for making quantum queries to F .

A reprogramming lemma. For a function $F : \{0, 1\}^\ell \rightarrow \{0, 1\}^n$ and a set $B \subset \{0, 1\}^\ell \times \{0, 1\}^n$ such that each $x \in \{0, 1\}^\ell$ is the first element of at most one tuple in B , define

$$F^{(B)}(x) := \begin{cases} y & \text{if } (x, y) \in B, \\ F(x) & \text{otherwise.} \end{cases}$$

Lemma 9.17 (Reprogramming Lemma [ABKM22]). *Let $\mathcal{D}$ be a quantum distinguisher in the following experiment:*

Phase 1: $\mathcal{D}$ outputs descriptions of a function $F_0 = F : \{0, 1\}^\ell \rightarrow \{0, 1\}^n$ and a randomized algorithm $\mathcal{B}$ whose output is a set $B \subset \{0, 1\}^\ell \times \{0, 1\}^n$ where each $x \in \{0, 1\}^\ell$ is the first element of at most one tuple in B . Let $B_1 = \{x \mid \exists y : (x, y) \in B\}$ and $\varepsilon = \max_{x \in \{0, 1\}^\ell} \{\Pr_{B \leftarrow \mathcal{B}}[x \in B_1]\}$.

Phase 2: $\mathcal{B}$ is run to obtain B . Let $F_1 = F^{(B)}$. A uniform bit b is chosen, and $\mathcal{D}$ is given quantum access to F_b .

Phase 3: $\mathcal{D}$ loses access to F_b , and receives the randomness r used to invoke $\mathcal{B}$ in phase 2. Then $\mathcal{D}$ outputs a guess b' .

For any $\mathcal{D}$ making q queries in expectation when its oracle is F_0 , it holds that

$$|\Pr[\mathcal{D} \text{ outputs } 1 \mid b = 1] - \Pr[\mathcal{D} \text{ outputs } 1 \mid b = 0]| \leq 2q \cdot \sqrt{\varepsilon}.$$

Quantum ideal cipher and resampling. Define $\mathcal{E}(m, n)$ to be the space of all keyed permutations where the key $k \in \{0, 1\}^m$ and the input $x \in \{0, 1\}^n$. In the quantum ideal cipher model (QICM), we treat the underlying block cipher as an ideal cipher $E \leftarrow \mathcal{E}(m, n)$. A quantum distinguisher in the QICM can apply the unitary operators

$$|k\rangle|x\rangle|y\rangle \mapsto |k\rangle|x\rangle|E_k(x) \oplus y\rangle, \quad (92)$$

$$|k\rangle|x\rangle|y\rangle \mapsto |k\rangle|x\rangle|E_k^{-1}(x) \oplus y\rangle. \quad (93)$$

Lemma 9.18 (Resampling Lemma [ABMS26]). *Let $\mathcal{D} = (\mathcal{D}_0, \mathcal{D}_1)$ be a quantum distinguisher interacting with the following experiment:*

Phase 1: *Choose a uniform ideal cipher $E \in \mathcal{E}(m, n)$ and give $\mathcal{D}$ quantum access to E and E^{-1} . Then $\mathcal{D}_0$ outputs a distribution D over $\{0, 1\}^{m+2n}$.*

Phase 2: *Sample $(k_0, s_0, s_1) \in \{0, 1\}^m \times \{0, 1\}^n \times \{0, 1\}^n$ according to D . Define $E^{(0)} = E$ and $E^{(1)}$ as*

$$E_{k^*}^{(1)}(x) = \begin{cases} E_{k^*}(x), & \text{if } k^* \neq k_0, \\ E_{k^*} \circ \text{swap}_{s_0, s_1}(x), & \text{if } k^* = k_0, \end{cases}$$

where swap_{s_0, s_1} is the transposition swapping s_0 and s_1 . A uniform bit $b \in \{0, 1\}$ is chosen, and $\mathcal{D}$ is given k_0, s_0, s_1 along with quantum access to $E^{(b)}$. Finally, $\mathcal{D}$ outputs a guess b' .

For a distribution D on $\{0, 1\}^{m+2n}$, define

$$\varepsilon = \max_{(k_0^*, s_0^*, s_1^*) \in \{0, 1\}^{m+2n}} D(k_0^*, s_0^*, s_1^*).$$

Then for any distinguisher $\mathcal{D}$ making at most q queries to E in Phase 1, it holds that

$$|\Pr[\mathcal{D} \text{ outputs } 1 \mid b = 1] - \Pr[\mathcal{D} \text{ outputs } 1 \mid b = 0]| \leq 4\sqrt{2^n \cdot q \cdot \varepsilon}.$$

Abstract model for the quantum proof. We consider the case $n < \lambda \leq 2n$ (the case $\lambda = n$ follows analogously with a single E -call per hash evaluation). For a key $\Delta \in \{0, 1\}^\lambda$, define $\Delta_L := \text{left}_{n-8}(\Delta)$ and $\Delta_R := \text{right}_{\lambda-(n-8)}(\Delta)$.

The TCCR construction oracle for TCCR returns:

$$\mathcal{O}_{H, \Delta}^{\text{tccr}}(w, \text{iv}_{\text{sig}}, b) = z_L \parallel z_R,$$

where, using the expanded $2n$ -bit formulation:

$$\begin{aligned} z_L &= E\left((w_R \oplus \Delta_R) \parallel \text{iv}_{\text{sig}} \parallel 0^{2n-\lambda-8}, [0]_8 \parallel (w_L \oplus \Delta_L)\right) \oplus (0^8 \parallel (w_L \oplus \Delta_L)) \oplus b \cdot \tilde{\Delta}_L, \\ z_R &= E\left((w_R \oplus \Delta_R) \parallel \text{iv}_{\text{sig}} \parallel 0^{2n-\lambda-8}, [1]_8 \parallel (w_L \oplus \Delta_L)\right) \oplus (0^8 \parallel (w_L \oplus \Delta_L)) \oplus b \cdot \tilde{\Delta}_R. \end{aligned} \quad (94)$$

Here $\tilde{\Delta}$ is the $2n$ -bit expansion of Δ , and $\tilde{\Delta}_L, \tilde{\Delta}_R$ are the left and right n -bit halves.

The two E calls share the *same* key:

$$k(w, \text{iv}_{\text{sig}}, \Delta) := (w_R \oplus \Delta_R) \parallel \text{iv}_{\text{sig}} \parallel 0^{2n-\lambda-8}.$$

They are differentiated by their *plaintexts*:

$$\mathbf{p}_0(w_L, \Delta_L) := [0]_8 \parallel (w_L \oplus \Delta_L), \quad (95)$$

$$\mathbf{p}_1(w_L, \Delta_L) := [1]_8 \parallel (w_L \oplus \Delta_L). \quad (96)$$

The modified cipher. For any list $T_j = \{(w_i, \text{iv}_i, b_i, z_i)\}_{i=1}^j$ of classical construction query/answer pairs and key Δ , we define the *modified cipher* $E_\Delta^{T_j}$ as follows.

Set $E_\Delta^{T_0} = E$. For $j \geq 1$:

$$E_\Delta^{T_j}(x) = \begin{cases} E_k(x) & \text{if } k \neq \mathbf{k}(w_i, \text{iv}_i, \Delta) \text{ for all } i \in [j], \\ E_{\Delta, k}^{T_j}(x) & \text{if } k = \mathbf{k}(w_i, \text{iv}_i, \Delta) \text{ for some } i \in [j], \end{cases} \quad (97)$$

where $E_{\Delta, k}^{T_j}$ is constructed recursively: $E_\Delta^{T_j}$ is obtained from $E_\Delta^{T_{j-1}}$ by performing two swap operations on the relevant key $k = \mathbf{k}(w_j, \text{iv}_j, \Delta)$ to ensure consistency with the j -th construction query/answer pair. Concretely,

$$E_\Delta^{T_j} = \text{swap}_{s_{j,0}, s'_{j,0}} \circ \text{swap}_{s_{j,1}, s'_{j,1}} \circ E_\Delta^{T_{j-1}}, \quad (98)$$

where the swap points $s_{j,0} = \mathbf{p}_0(w_{j,L}, \Delta_L)$ and $s_{j,1} = \mathbf{p}_1(w_{j,L}, \Delta_L)$ are the two plaintext inputs, and $s'_{j,0}, s'_{j,1}$ are chosen so that the modified cipher is consistent with the construction output z_j . Specifically:

$$s'_{j,0} := z_{j,L} \oplus (0^8 \parallel (w_{j,L} \oplus \Delta_L)) \oplus b_j \cdot \tilde{\Delta}_L, \quad (99)$$

$$s'_{j,1} := z_{j,R} \oplus (0^8 \parallel (w_{j,L} \oplus \Delta_L)) \oplus b_j \cdot \tilde{\Delta}_R. \quad (100)$$

Since $\mathbf{p}_0$ and $\mathbf{p}_1$ are distinct (they differ in the first 8 bits), the two swap operations are on distinct domain elements under the same key, and they commute. For compactness, we write E^j in place of $E^{T_j, \Delta}$ when T_j and Δ are clear from context.

Hybrid experiments. We define a sequence of hybrid experiments

$$H_0, H_0^1, H_0^2, H_0^3, H_0^4, H_1, H_1^1, \dots, H_{D-1}^4, H_D, \quad (101)$$

where each experiment begins by sampling uniform $E \in \mathcal{E}(\lambda, n)$ and key $\Delta \leftarrow \chi$.

Experiment H_j (for $j = 0, \dots, D$):

1. Run $\mathcal{A}$, answering its first j classical construction queries using f (a random function) and its quantum queries using $|E\rangle$. Stop immediately before the $(j+1)$ -st classical query. Let T_j be the transcript of the first j classical construction queries.
2. For the remainder of the execution, answer classical construction queries using $\mathcal{O}_{H^{E^{T_j, \Delta}}, \Delta}^{\text{tccr}}$, and quantum queries using $|E^{T_j, \Delta}\rangle$.

Experiment H_j^1 :

1. Run $\mathcal{A}$, answering its classical construction queries using f and quantum queries using $|E\rangle$, until $\mathcal{A}$ makes its $(j+1)$ -st classical construction query (assumed forward without loss of generality). Let T_j be the transcript so far.
2. Define

$$S := \{\mathbf{p}_0(w_{\ell,L}, \Delta_L), \mathbf{p}_1(w_{\ell,L}, \Delta_L) : \ell \in \{1, \dots, j\} \wedge (w_{\ell,R}, \text{iv}_\ell) = (w_{j+1,R}, \text{iv}_{j+1})\}. \quad (102)$$

Choose uniform $s \xleftarrow{\$} \{0, 1\}^n \setminus S$, and define $E^{(1)}$ as

$$E_{k^*}^{(1)}(x) := \begin{cases} E_{k^*}(x) & \text{if } k^* \neq \mathbf{k}(w_{j+1}, \text{iv}_{j+1}, \Delta), \\ E_{k^*} \circ \text{swap}_{\mathbf{p}_0(w_{j+1,L}, \Delta_L), s}(x) & \text{if } k^* = \mathbf{k}(w_{j+1}, \text{iv}_{j+1}, \Delta). \end{cases} \quad (103)$$

Continue running $\mathcal{A}$, answering its remaining classical queries (including the $(j+1)$ -st) using $\mathcal{O}_{H^{(E^{(1)})^{T_j}, \Delta}}^{\text{tccr}}$, and its quantum queries using $|(E^{(1)})^{T_j}\rangle$.

Experiment H_j^2 :

1. Run $\mathcal{A}$ as in H_j^1 step 1, up to the $(j+1)$ -st classical query.
2. Recall S from H_j^1 . Choose uniform $s \xleftarrow{\$} \{0, 1\}^n \setminus S$, and define $E^{(2)}$ as

$$E_{k^*}^{(2)}(x) := \begin{cases} E_{k^*}(x) & \text{if } k^* \neq k(w_{j+1}, \text{iv}_{j+1}, \Delta), \\ E_{k^*} \circ \text{swap}_{\mathbf{p}_1(w_{j+1}, L, \Delta_L), s}(x) & \text{if } k^* = k(w_{j+1}, \text{iv}_{j+1}, \Delta). \end{cases} \quad (104)$$

Continue running $\mathcal{A}$, answering its remaining classical queries using $\mathcal{O}_{\mathbf{H}(E^{(2)})^{T_j}, \Delta}^{\text{tccr}}$, and its quantum queries using $|E^{(2)}\rangle^{T_j}$.

Experiment H_j^3 :

1. Run $\mathcal{A}$ as above. Let T_{j+1} be the list of classical queries through the $(j+1)$ -st, and T_j the subset of the first j .
2. Construct $E_{j+1} := E_{\Delta}^{T_{j+1}}$ (the modified cipher consistent with T_{j+1}). Continue running $\mathcal{A}$, answering its remaining classical construction queries using $\mathcal{O}_{\mathbf{H}^{E_{j+1}}, \Delta}^{\text{tccr}}$ and quantum queries using $|E_{j+1}\rangle$.

Experiment H_j^4 :

1. Run $\mathcal{A}$ as above, through the $(j+1)$ -st classical query. Let T_{j+1} be the transcript.
2. For the remainder, answer classical queries using $\mathcal{O}_{\mathbf{H}^{E^{T_{j+1}}}, \Delta}^{\text{tccr}}$ and quantum queries using $|E^{T_{j+1}}\rangle$.

Distinguishability lemmas. We establish bounds on the distinguishability of consecutive hybrids.

Lemma 9.19. $|\Pr[\mathcal{A}(H_j) = 1] - \Pr[\mathcal{A}(H_j^1) = 1]| \leq 4\sqrt{\frac{T}{2^\rho}}.$

Lemma 9.20. $|\Pr[\mathcal{A}(H_j^1) = 1] - \Pr[\mathcal{A}(H_j^2) = 1]| \leq 4\sqrt{\frac{T}{2^\rho}}.$

Lemma 9.21. $H_j^2 = H_j^3.$

Lemma 9.22. $|\Pr[\mathcal{A}(H_j^3) = 1] - \Pr[\mathcal{A}(H_j^4) = 1]| \leq \frac{2}{2^{\lambda-n}}.$

Lemma 9.23. $|\Pr[\mathcal{A}(H_j^4) = 1] - \Pr[\mathcal{A}(H_{j+1}) = 1]| \leq 2\sqrt{2} \cdot T_{j+1} \cdot \sqrt{\frac{j+1}{2^\rho}},$ where T_{j+1} is the expected number of quantum queries $\mathcal{A}$ makes between the $(j+1)$ -st and $(j+2)$ -nd classical queries.

Proofs of the lemmas. We now provide *complete* proofs for all five lemmas.

Proof of Lemma 9.19.

Proof. The hybrids H_j and H_j^1 differ only in the quantum oracle provided after the j -th classical query: in H_j it is E , while in H_j^1 it is $E^{(1)}$, which resamples the first plaintext point ($\mathbf{p}_0$) relevant to the $(j+1)$ -st construction query by swapping values.

We construct from $\mathcal{A}$ a distinguisher $\mathcal{D}$ for the resampling experiment of Lemma 9.18. $\mathcal{D}$ operates as follows:

Phase 1: $\mathcal{D}$ is given quantum access to an ideal cipher E . It samples key $\Delta \leftarrow \chi$ and runs $\mathcal{A}$, answering its quantum queries with $|E\rangle$ and its classical construction queries using $\mathcal{O}_{H^E, \Delta}^{\text{tccr}}$ (computed via the ideal cipher), until $\mathcal{A}$ submits its $(j+1)$ -st classical construction query $(w_{j+1}, \text{iv}_{j+1}, b_{j+1})$.

At that point, $\mathcal{D}$ constructs a distribution D on $\{0, 1\}^\lambda \times \{0, 1\}^n \times \{0, 1\}^n$ as follows. To sample $(k, s_0, s_1) \leftarrow D$:

1. Compute $s_0 := \mathbf{p}_0(w_{j+1,L}, \Delta_L) = [0]_8 \parallel (w_{j+1,L} \oplus \Delta_L)$ and $k := \mathbf{k}(w_{j+1}, \text{iv}_{j+1}, \Delta) = (w_{j+1,R} \oplus \Delta_R) \parallel \text{iv}_{j+1} \parallel 0^{2n-\lambda-8}$.
2. Construct

$$S := \{\mathbf{p}_0(w_{\ell,L}, \Delta_L), \mathbf{p}_1(w_{\ell,L}, \Delta_L) : \ell \in \{1, \dots, j\} \wedge (w_{\ell,R}, \text{iv}_{\ell}) = (w_{j+1,R}, \text{iv}_{j+1})\}. \quad (105)$$

3. Sample uniform $s_1 \xleftarrow{\$} \{0, 1\}^n \setminus S$, and output (k, s_0, s_1) .

Phase 2: $\mathcal{D}$ is given $(k, s_0, s_1) \leftarrow D$ and quantum oracle access to $E^{(b)}$, $b \in \{0, 1\}$. $\mathcal{D}$ continues running $\mathcal{A}$, answering its remaining classical construction queries using $\mathcal{O}_{H^{(E^{(b)})_{\Delta}^{T_j}}, \Delta}^{\text{tccr}}$ and quantum queries using $|E^{(b)}\rangle_{\Delta}^{T_j}$. $\mathcal{D}$ outputs whatever $\mathcal{A}$ does.

Simulation correctness. In Phase 1, $\mathcal{D}$ perfectly simulates H_j and H_j^1 for $\mathcal{A}$ until the $(j+1)$ -st classical query. If $b = 0$, $\mathcal{D}$ receives $E^{(0)} = E$ (the original ideal cipher) and simulates H_j . If $b = 1$, $\mathcal{D}$ receives $E^{(1)}$ with the swap on $\mathbf{p}_0(w_{j+1,L}, \Delta_L)$ and simulates H_j^1 .

Parameter bound. The key Δ has min-entropy at least ρ , and s_1 is sampled from a set of size at least $2^n - 2j$ (since each of up to j prior queries with matching $(w_R, \text{iv}_{\text{sig}})$ contributes at most 2 elements to S). When $2j \leq 2^n/2$, the quantity ε for Lemma 9.18 satisfies

$$\varepsilon = \max_{(k^*, s_0^*, s_1^*)} \Pr_{(k, s_0, s_1) \leftarrow D}[(k, s_0, s_1) = (k^*, s_0^*, s_1^*)] \leq \frac{1}{2^\rho \cdot (2^n - 2j)} \leq \frac{2}{2^{\rho+n}}. \quad (106)$$

Applying Lemma 9.18 yields

$$|\Pr[\mathcal{A}(H_j) = 1] - \Pr[\mathcal{A}(H_j^1) = 1]| \leq 4\sqrt{2^n \cdot T_{j+1} \cdot \frac{2}{2^{\rho+n}}} = 4\sqrt{\frac{2 \cdot T_{j+1}}{2^\rho}} \leq 4\sqrt{\frac{T}{2^\rho}}. \quad (107)$$

This completes the proof. $\square$

Proof of Lemma 9.20.

Proof. This lemma is the symmetric counterpart of Lemma 9.19, where the second plaintext point $\mathbf{p}_1$ is resampled instead of $\mathbf{p}_0$. The proof follows the identical structure:

We construct a distinguisher $\mathcal{D}$ for Lemma 9.18 where:

1. In Phase 1, $\mathcal{D}$ runs $\mathcal{A}$ until the $(j+1)$ -st classical query $(w_{j+1}, \text{iv}_{j+1}, b_{j+1})$.
2. $\mathcal{D}$ sets $s_0 := \mathbf{p}_1(w_{j+1,L}, \Delta_L) = [1]_8 \parallel (w_{j+1,L} \oplus \Delta_L)$ and $k := \mathbf{k}(w_{j+1}, \text{iv}_{j+1}, \Delta)$.
3. The set S is identical to that in the previous proof.
4. $\mathcal{D}$ samples $s_1 \xleftarrow{\$} \{0, 1\}^n \setminus S$ and outputs (k, s_0, s_1) .
5. In Phase 2, $\mathcal{D}$ simulates $\mathcal{A}$ with the modified cipher.

The same bound applies: the key Δ has min-entropy ρ , and s_1 is sampled from a set of size at least $2^n - 2j$. Therefore

$$\varepsilon \leq \frac{2}{2^{\rho+n}}, \quad (108)$$

and applying Lemma 9.18 gives

$$|\Pr[\mathcal{A}(H_j^1) = 1] - \Pr[\mathcal{A}(H_j^2) = 1]| \leq 4\sqrt{2^n \cdot T_{j+1} \cdot \frac{2}{2^{\rho+n}}} \leq 4\sqrt{\frac{T}{2^\rho}}. \quad (109)$$

□

Proof of Lemma 9.21.

Proof. We show that H_j^2 and H_j^3 are identical experiments. In H_j^2 , the modified cipher $(E^{(2)})^{T_j}$ incorporates a single swap on $\mathbf{p}_1(w_{j+1,L}, \Delta_L)$ under key $\mathbf{k}(w_{j+1}, \mathbf{iv}_{j+1}, \Delta)$, and then the classical construction queries are answered using $\mathcal{O}_{H^{(E^{(2)})^{T_j}, \Delta}}^{\text{tccr}}$.

In H_j^3 , we directly construct $E_{\Delta}^{T_{j+1}}$, which incorporates both swaps (for $\mathbf{p}_0$ and $\mathbf{p}_1$) into the cipher. Crucially, in H_j^2 , the swap corresponding to $\mathbf{p}_1$ was already applied to obtain $(E^{(2)})^{T_j}$ in Phase 2, and the swap for $\mathbf{p}_0$ was applied in H_j^1 (which is already accounted for). The construction oracle in H_j^2 , when answering the $(j+1)$ -st query, effectively completes the remaining consistency requirements, which is equivalent to directly computing $E_{\Delta}^{T_{j+1}}$.

Formally, let $E' := (E^{(2)})^{T_j}$ be the cipher in H_j^2 after Phase 2. Then E' has swaps consistent with all entries in T_j plus the $\mathbf{p}_1$ swap from H_j^2 . When the $(j+1)$ -st construction query is answered in H_j^2 , the response z_{j+1} is computed using $\mathcal{O}_{H^{E'}, \Delta}^{\text{tccr}}$, which internally evaluates $E'(k, \mathbf{p}_0)$ and $E'(k, \mathbf{p}_1)$. Since E' already has the $\mathbf{p}_1$ -swap applied, both evaluations are consistent with the full transcript T_{j+1} . This is precisely the same state achieved by $E_{\Delta}^{T_{j+1}}$ in H_j^3 .

Therefore, for all subsequent classical and quantum queries, the oracle distributions are identical, establishing $H_j^2 = H_j^3$. □

Proof of Lemma 9.22.

Proof. H_j^3 and H_j^4 differ only in the response to the $(j+1)$ -st classical construction query. In H_j^4 , the $(j+1)$ -st response z_{j+1} is uniformly distributed in $\{0, 1\}^{2n}$ (expanded output). In H_j^3 , the response $z_{j+1} = (z_{j+1,L}, z_{j+1,R})$ satisfies:

$$z_{j+1,L} = y_0 \oplus (0^8 \parallel (w_{j+1,L} \oplus \Delta_L)) \oplus b_{j+1} \cdot \tilde{\Delta}_L, \quad (110)$$

$$z_{j+1,R} = y_1 \oplus (0^8 \parallel (w_{j+1,L} \oplus \Delta_L)) \oplus b_{j+1} \cdot \tilde{\Delta}_R, \quad (111)$$

where y_0 and y_1 are obtained from the modified cipher: $y_0 = (E^{(2)})^{T_j}(k, \mathbf{p}_0)$ and $y_1 = (E^{(2)})^{T_j}(k, \mathbf{p}_1)$ for $k = \mathbf{k}(w_{j+1}, \mathbf{iv}_{j+1}, \Delta)$.

In H_j^2 (and thus H_j^3), y_0 is uniformly distributed over a set of size at least $2^n - 2j$ (the $\mathbf{p}_0$ swap was applied in H_j^1 , and consistency with prior queries removes at most $2j$ values). Similarly, y_1 is uniform over a set of size at least $2^n - 2j$ after the H_j^2 swap. Since XOR with a constant is a bijection, each z -component is uniform over at least $2^n - 2j$ values.

The true uniform distribution in H_j^4 is over all 2^{2n} values. However, since the TCCR oracle returns λ bits (truncated from $2n$), the relevant statistical distance per query is between λ -bit strings. When $\lambda > n$, the left n bits contribute fully and the right $\lambda - n$ bits contribute partially. The worst-case distance is bounded by:

$$|\Pr[\mathcal{A}(H_j^3) = 1] - \Pr[\mathcal{A}(H_j^4) = 1]| \leq \frac{2}{2^{\lambda-n}}, \quad (112)$$

following the standard truncation analysis: the left n bits are nearly uniform for large n , and the truncated right $\lambda - n$ bits contribute at most $2^{-(\lambda-n)}$ distinguishing power from uniformity. □

Proof of Lemma 9.23.

Proof. H_j^4 and H_{j+1} differ in the quantum oracle: H_j^4 uses E_{j+1} (the modified cipher after $j+1$ queries), while H_{j+1} uses E (the original ideal cipher) for quantum queries between the $(j+1)$ -st and $(j+2)$ -nd classical queries.

We construct a distinguisher $\mathcal{D}$ for the reprogramming lemma (Lemma 9.17):

Phase 1: $\mathcal{D}$ samples uniform $E \in \mathcal{E}(\lambda, n)$ and key $\Delta \leftarrow \chi$. It runs $\mathcal{A}$, answering its quantum queries using $|E\rangle$ and its classical construction queries using random responses, until after it responds to $\mathcal{A}$'s $(j+1)$ -st classical query.

$\mathcal{D}$ defines $F : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ by $F(k, x) = E_k(x)$. It also defines a randomized algorithm $\mathcal{B}$ that, on input Δ (sampled afresh from χ), computes the set B of input/output pairs to be reprogrammed such that $F^{(B)}(k, x) = (E_{j+1})_k(x)$ for all keys relevant to the first $j+1$ construction queries.

For the i -th construction query ($i \leq j+1$), the key is

$$k_i := \mathbf{k}(w_i, \mathbf{iv}_i, \Delta) = (w_{i,R} \oplus \Delta_R) \parallel \mathbf{iv}_i \parallel 0^{2n-\lambda-8}.$$

The two plaintexts are $\mathbf{p}_0(w_{i,L}, \Delta_L)$ and $\mathbf{p}_1(w_{i,L}, \Delta_L)$. For each such pair $(k_i, \mathbf{p}_a)$ ($a \in \{0, 1\}$), reprogramming is needed to swap its image to the value consistent with the construction output z_i .

Phase 2: $\mathcal{B}$ is run to generate B , and $\mathcal{D}$ is given quantum access to F_b . $\mathcal{D}$ resumes running $\mathcal{A}$, answering its quantum queries using F_b . Phase 2 ends before $\mathcal{A}$ makes its $(j+2)$ -nd classical query.

If $b = 0$, $\mathcal{D}$ provides $F_0 = F = E$, simulating H_{j+1} . If $b = 1$, $\mathcal{D}$ provides $F_1 = F^{(B)} = E_{j+1}$, simulating H_j^4 .

Computing ε . The reprogramming set $B_1 = \{x \mid \exists y : (x, y) \in B\}$ contains at most $2(j+1)$ points (two per construction query). For a fixed (k^*, x^*) , the event $(k^*, x^*) \in B_1$ means there exists $i \in [j+1]$ such that:

$$k^* = (w_{i,R} \oplus \Delta_R) \parallel \mathbf{iv}_i \parallel 0^{2n-\lambda-8} \wedge x^* \in \{\mathbf{p}_0(w_{i,L}, \Delta_L), \mathbf{p}_1(w_{i,L}, \Delta_L)\}. \quad (113)$$

From the first equation, Δ_R is uniquely determined ($\Delta_R = \text{left}_{\lambda-(n-8)}(k^*) \oplus w_{i,R}$). From the second equation, Δ_L is uniquely determined ($\Delta_L = \text{right}_{n-8}(x^*) \oplus w_{i,L}$). Since Δ has min-entropy ρ , the probability that this occurs for a specific i is at most $2^{-\rho}$. By the union bound over $i \in [j+1]$:

$$\varepsilon = \max_{(k^*, x^*)} \Pr_{\Delta \leftarrow \chi} [(k^*, x^*) \in B_1] \leq \frac{2(j+1)}{2^\rho}. \quad (114)$$

Applying Lemma 9.17 yields:

$$|\Pr[\mathcal{A}(H_j^4) = 1] - \Pr[\mathcal{A}(H_{j+1}) = 1]| \leq 2T_{j+1} \cdot \sqrt{\frac{2(j+1)}{2^\rho}} = 2\sqrt{2} \cdot T_{j+1} \cdot \sqrt{\frac{j+1}{2^\rho}}. \quad (115)$$

This completes the proof. $\square$

Assembling the hybrid argument. Using the five lemmas above, we bound the total distinguishing advantage between H_0 and H_D :

$$\begin{aligned}
& |\Pr[\mathcal{A}(H_0) = 1] - \Pr[\mathcal{A}(H_D) = 1]| \\
& \leq \sum_{j=0}^{D-1} \left(4\sqrt{\frac{T}{2^\rho}} + 4\sqrt{\frac{T}{2^\rho}} + 0 + \frac{2}{2^{\lambda-n}} + 2\sqrt{2} \cdot T_{j+1} \cdot \sqrt{\frac{j+1}{2^\rho}} \right) \\
& \leq \sum_{j=0}^{D-1} \left(8\sqrt{\frac{T_{j+1}}{2^\rho}} + \frac{2}{2^{\lambda-n}} + 2\sqrt{2} \cdot T_{j+1} \cdot \sqrt{\frac{j+1}{2^\rho}} \right) \\
& \leq O\left(\frac{D\sqrt{T}}{2^{\rho/2}} + \frac{T\sqrt{D}}{2^{\rho/2}} + \frac{D}{2^{\lambda-n}} \right), \tag{116}
\end{aligned}$$

where $T = \sum_{j=0}^{D-1} T_{j+1}$ is the total number of quantum queries.

Simplifying to the standard asymptotic form, we obtain the final bound:

$$\text{AdvTCCR}_{\mathcal{A}_{\text{TCCR}}}^{E, \text{TCCR}}(T, D, \chi) \leq O\left(\frac{D\sqrt{T}}{2^\rho} + \frac{T\sqrt{D}}{2^\rho} + \frac{D}{2^{\lambda-n}} \right).$$

This completes the proof of Theorem 9.13.

Additional lemma: classical indistinguishability detail.

Lemma 9.24 (Classical Indistinguishability for the New Construction). *For $j = 0, \dots, D$, the statistical distance between the $(j+1)$ -st construction query response in H_j^3 and H_j^4 is bounded by $2/2^{\lambda-n}$.*

Proof. In H_j^3 , after the two resampling steps (H_j^1 and H_j^2) and the swap consistency step, the $(j+1)$ -st response z_{j+1} has its left component $z_{j+1,L}$ derived from a value y_0 that is uniform over a set of size at least $2^n - 2j$, and its right component $z_{j+1,R}$ derived from y_1 that is uniform over a set of size at least $2^n - 2j$.

Specifically, $y_0 = (E^{(1)})^{T_j}(k, \mathbf{p}_0(w_{j+1,L}, \Delta_L))$ and $y_1 = (E^{(2)})^{T_j}(k, \mathbf{p}_1(w_{j+1,L}, \Delta_L))$. After the two swaps:

- y_0 is uniform over $\{0, 1\}^n \setminus S_0$, where $|S_0| \leq 2j$.
- y_1 is uniform over $\{0, 1\}^n \setminus S_1$, where $|S_1| \leq 2j$.

The construction response is then:

$$z_{j+1,L} = y_0 \oplus (0^8 \parallel (w_{j+1,L} \oplus \Delta_L)) \oplus b_{j+1} \cdot \tilde{\Delta}_L, \tag{117}$$

$$z_{j+1,R} = y_1 \oplus (0^8 \parallel (w_{j+1,L} \oplus \Delta_L)) \oplus b_{j+1} \cdot \tilde{\Delta}_R. \tag{118}$$

In H_j^4 , z_{j+1} is uniform over $\{0, 1\}^{2n}$ (before truncation to λ bits). The statistical distance per component from uniformity is at most $2j/2^n$. Since we ultimately truncate to λ bits, the effective distinguishing power from the deviation in the right $\lambda-n$ bits is bounded by $2/2^{\lambda-n}$. $\square$

References

- [ABBS25] Amit Agarwal, Carsten Baum, Lennart Braun, and Peter Scholl. Low-bandwidth mixed arithmetic in VOLE-based ZK from low-degree PRGs. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 396–426, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.

- [ABKM22] Gorjan Alagic, Chen Bai, Jonathan Katz, and Christian Majenz. Post-quantum security of the Even-Mansour cipher. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 458–487, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- [ABMS26] Gorjan Alagic, Chen Bai, Christian Majenz, and Kaiyan Shi. Post-quantum security of block cipher constructions. Cryptology ePrint Archive, Paper 2026/209, 2026.
- [ACG⁺19] Martin R. Albrecht, Carlos Cid, Lorenzo Grassi, Dmitry Khovratovich, Reinhard Lüftenegger, Christian Rechberger, and Markus Schofnegger. Algebraic cryptanalysis of STARK-friendly designs: Application to MARVELlous and MiMC. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019, Part III*, volume 11923 of *Lecture Notes in Computer Science*, pages 371–397, Kobe, Japan, December 8–12, 2019. Springer, Cham, Switzerland.
- [ADI⁺17] Benny Applebaum, Ivan Damgård, Yuval Ishai, Michael Nielsen, and Lior Zichron. Secure arithmetic computation with constant computational overhead. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 223–254, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Cham, Switzerland.
- [AW20] Josh Alman and Virginia Vassilevska Williams. A refined laser method and faster matrix multiplication. *ArXiv*, abs/2010.05846, 2020.
- [BBB⁺25] Carsten Baum, Ward Beullens, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. Shorter, tighter, FAESTer: Optimizations and improved (QROM) analysis for VOLE-in-the-head signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part VI*, volume 16005 of *Lecture Notes in Computer Science*, pages 124–156, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.
- [BBB⁺26] Carsten Baum, Lennart Braun, Ward Beullens, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. FAESTv2. Technical report, National Institute of Standards and Technology, 2026. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-3-additional-signatures>.
- [BBD⁺23a] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. FAEST. Technical report, National Institute of Standards and Technology, 2023. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>.
- [BBD⁺23b] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 581–615, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.

- [BBM⁺24] Carsten Baum, Ward Beullens, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. One tree to rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part I*, volume 15484 of *Lecture Notes in Computer Science*, pages 463–493, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
- [BCC⁺10] Charles Bouillaguet, Hsieh-Chung Chen, Chen-Mou Cheng, Tung Chou, Ruben Niederhagen, Adi Shamir, and Bo-Yin Yang. Fast exhaustive search for polynomial systems in $\mathbb{F}_2$. In Stefan Mangard and François-Xavier Standaert, editors, *Cryptographic Hardware and Embedded Systems – CHES 2010*, volume 6225 of *Lecture Notes in Computer Science*, pages 203–218, Santa Barbara, CA, USA, August 17–20, 2010. Springer Berlin Heidelberg, Germany.
- [BFP09] Luk Bettale, Jean-Charles Faugère, and Ludovic Perret. Hybrid approach for solving multivariate systems over finite fields. *Journal of Mathematical Cryptology*, 3(3):177–197, 2009.
- [BFU26] Alex Biryukov, Pablo García Fernández, and Aleksei Udovenko. Magic pot: Cryptanalysis of full aim2 in the standard and related-/reused-key setting using new linearization framework. In *Advances in Cryptology – EUROCRYPT 2026, 45th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Rome, Italy, May 10-14, 2026, Proceedings*, Lecture Notes in Computer Science, Rome, Italy, May 2026. Springer. To appear.
- [Bou22] Charles Bouillaguet. Boolean polynomial evaluation for the masses. Cryptology ePrint Archive, Report 2022/1412, 2022.
- [BRS⁺10] Lawrence E. Bassham, Andrew L. Rukhin, Juan Soto, James Nechvatal, Miles E. Smid, Elaine B. Barker, Stefan Leigh, Mark S. Levenson, Mark G. Vangel, David Banks, N. Alan Heckert, James F. Dray, and San C. Vo. A statistical test suite for random and pseudorandom number generators for cryptographic applications. Tech. Rep. NIST Special Publication (SP) 800-22 Rev. 1a, National Institute of Standards and Technology, Gaithersburg, MD, 2010.
- [BY18] Daniel J. Bernstein and Bo-Yin Yang. Asymptotically faster quantum algorithms to solve multivariate quadratic equations. In *PQCrypto*, pages 487–506. Springer, 2018.
- [CCH⁺19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-Shamir: from practice to theory. In Moses Charikar and Edith Cohen, editors, *51st Annual ACM Symposium on Theory of Computing*, pages 1082–1090, Phoenix, AZ, USA, June 23–26, 2019. ACM Press.
- [CG22] Yu-Ao Chen and Xiao-Shan Gao. Quantum algorithm for boolean equation solving and quantum algebraic attack on cryptosystems. *Journal of Systems Science and Complexity*, 35(1):373–412, February 2022.
- [CGW⁺24] Hongrui Cui, Chun Guo, Xiao Wang, Chenkai Weng, Kang Yang, and Yu Yu. AES-based CCR hash with high security and its application to zero-knowledge proofs. Cryptology ePrint Archive, Report 2024/1271, 2024.
- [CHL⁺25] Alexandru Cojocaru, Minki Hhan, Qipeng Liu, Takashi Yamakawa, and Aaram Yun. Quantum lifting for invertible permutations and ideal ciphers. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*,

- Part II*, volume 16001 of *Lecture Notes in Computer Science*, pages 481–512, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.
- [CKPS00] Nicolas Courtois, Alexander Klimov, Jacques Patarin, and Adi Shamir. Efficient algorithms for solving overdefined systems of multivariate polynomial equations. In Bart Preneel, editor, *Advances in Cryptology – EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 392–407, Bruges, Belgium, May 14–18, 2000. Springer Berlin Heidelberg, Germany.
 - [CL04] Jung Hee Cheon and Dong Hoon Lee. Resistance of s-boxes against algebraic attacks. In Bimal Roy and Willi Meier, editors, *Fast Software Encryption*, pages 83–93, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
 - [CS14] Shan Chen and John P. Steinberger. Tight security bounds for key-alternating ciphers. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 327–350, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
 - [Din21] Itai Dinur. Cryptanalytic applications of the polynomial method for solving multivariate equation systems over $\text{GF}(2)$. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 374–403, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
 - [DKR⁺22] Christoph Dobraunig, Daniel Kales, Christian Rechberger, Markus Schofneger, and Greg Zaverucha. Shorter signatures based on tailor-made minimalist symmetric-key crypto. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 843–857, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.
 - [DR00] Joan Daemen and Vincent Rijmen. The block cipher rijndael. In Jean-Jacques Quisquater and Bruce Schneier, editors, *Smart Card Research and Applications*, volume 1820 of *Lecture Notes in Computer Science*, pages 277–284. Springer, 2000.
 - [Eur03] European NESSIE cryptography evaluation project. New european schemes for signatures, integrity, and encryption. <https://www.cryptoscam.info/old-projects/nessie/>, (2000–2003).
 - [FAE25] FAEST Team. FAEST avx2 implementation, 2025. GitHub repository. Verified on 2026-04-08. Public repository; GitHub organization page describes it as “FAEST implementation for AVX2 and other extensions on the x86-64 platform” and reports last update on Feb. 27, 2025.
 - [FHK⁺17] Jean-Charles Faugère, Selene Horan, Delaram Kahrobaei, Marc Kaplan, Elham Kashefi, and Ludovic Perret. Fast quantum algorithm for solving multivariate quadratic equations. *Cryptology ePrint Archive*, Paper 2017/1236, 2017.
 - [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. How to enhance the security of public-key encryption at minimum cost. In Hideki Imai and Yuliang Zheng, editors, *PKC’99: 2nd International Workshop on Theory and Practice in Public Key Cryptography*, volume 1560 of *Lecture Notes in Computer Science*, pages 53–68, Kamakura, Japan, March 1–3, 1999. Springer Berlin Heidelberg, Germany.
 - [Frö85] Ralf Fröberg. An inequality for hilbert series of graded algebras. *Mathematica Scandinavica*, 56(2), 1985.

- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO’86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany.
- [GKW⁺20] Chun Guo, Jonathan Katz, Xiao Wang, Chenkai Weng, and Yu Yu. Better concrete security for half-gates garbling (in the multi-instance setting). In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part II*, volume 12171 of *Lecture Notes in Computer Science*, pages 793–822, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.
- [GKWY20] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841, San Francisco, CA, USA, May 18–21, 2020. IEEE Computer Society Press.
- [GMR85] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *17th Annual ACM Symposium on Theory of Computing*, pages 291–304, Providence, RI, USA, May 6–8, 1985. ACM Press.
- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th Annual ACM Symposium on Theory of Computing*, pages 218–229, New York City, NY, USA, May 25–27, 1987. ACM Press.
- [GMW91] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM*, 38(3):691–729, July 1991.
- [Gol01] Oded Goldreich. *Foundations of Cryptography: Basic Tools*, volume 1. Cambridge University Press, Cambridge, UK, 2001.
- [Gro96a] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *28th Annual ACM Symposium on Theory of Computing*, pages 212–219, Philadelphia, PA, USA, May 22–24, 1996. ACM Press.
- [Gro96b] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC ’96)*, pages 212–219, New York, NY, USA, 1996. Association for Computing Machinery.
- [GWYY25] Chun Guo, Xiao Wang, Kang Yang, and Yu Yu. On tweakable correlation robust hashing against key leakages. *Designs, Codes and Cryptography*, 93(8):3375–3412, 2025.
- [GYW⁺23] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part I*, volume 14004 of *Lecture Notes in Computer Science*, pages 330–362, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [HBD⁺22] Andreas Hülsing, Daniel J. Bernstein, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Martin M. Lauridsen, Florian Mendel, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe, Jean-Philippe Aumasson, Bas Westerbaan,

- and Ward Beullens. SPHINCS⁺. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [Ins25] Institute of Commercial Cryptography Standards (ICCS). Call for proposals for the next-generation public-key cryptographic algorithms. Official Announcement of the Next-generation Commercial Cryptographic Algorithms Program (NGCC), October 2025. Accessed: 2026-04-09.
- [Ins26] Institute of Applied Information Processing and Communications, Graz University of Technology. Rainier signatures implementation, 2026. GitHub repository. Verified on 2026-04-08. Public repository for the Rainier signature scheme implementation.
- [JBK⁺22] Kyungbae Jang, Anubhab Baksi, Hyunji Kim, Hwajeong Seo, and Anupam Chattopadhyay. Improved quantum analysis of SPECK and LowMC. In Takanori Isobe and Santanu Sarkar, editors, *Progress in Cryptology - INDOCRYPT 2022: 23rd International Conference in Cryptology in India*, volume 13774 of *Lecture Notes in Computer Science*, pages 517–540, Kolkata, India, December 11–14, 2022. Springer, Cham, Switzerland.
- [JYC⁺26] Lin Jiao, Hongsen Yang, Hongrui Cui, Yituo He, Yonglin Hao, Xiaojie Guo, Qunxiong Zheng, Jiang Zhang, Yu Yu, and Kang Yang. Lynx: Symmetric primitive for shorter and faster VOLE-in-the-head signatures. Cryptology ePrint Archive, Paper 2026/1099, 2026.
- [KCH⁺26] Seongkwang Kim, Jihoon Cho, Jincheol Ha, Jihoon Kwon, Byeonghak Lee, Joohee Lee, Jooyoung Lee, Sangyub Lee, Dukjae Moon, Mincheol Son, and Hyojin Yoon. The AIMer signature scheme. Technical report, Samsung SDS, KAIST, CryptoLab, Sungshin Women’s University, January 2026. Official specification document for the AIMer post-quantum digital signature scheme.
- [KHS⁺23] Seongkwang Kim, Jincheol Ha, Mincheol Son, ByeongHak Lee, Dukjae Moon, Joohee Lee, Sangyub Lee, Jihoon Kwon, Jihoon Cho, Hyojin Yoon, and Jooyoung Lee. AIM: Symmetric primitive for shorter signatures with stronger security. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023: 30th Conference on Computer and Communications Security*, pages 401–415, Copenhagen, Denmark, November 26–30, 2023. ACM Press.
- [KKL⁺05] Jongsung Kim, Guil Kim, Sangjin Lee, Jongin Lim, and Junghwan Song. Related-key attacks on reduced rounds of shacal-2. In Anne Canteaut and Kapaleeswaran Viswanathan, editors, *Progress in Cryptology - INDOCRYPT 2004*, pages 175–190, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [Kpq26] KpqC Project. AIMer signature algorithm final specification. KpqC Algorithms Exhibit, January 2026. Korean Post-Quantum Cryptography Project, https://kpgc.or.kr/contents/03_exhibit/sub_03.html.
- [KSW19] Daniel J. Katz, K. U. Schmidt, and A. Winterhof. Weil sums of binomials: Properties applications and open problems. In *Combinatorics and Finite Fields: Difference Sets, Polynomials, Pseudorandomness and Applications*, volume 23, pages 109–134. De Gruyter, 2019.
- [KW03] Jonathan Katz and Nan Wang. Efficiency improvements for signature schemes with tight security reductions. In Sushil Jajodia, Vijayalakshmi Atluri, and Trent

- Jaeger, editors, *ACM CCS 2003: 10th Conference on Computer and Communications Security*, pages 155–164, Washington, DC, USA, October 27–30, 2003. ACM Press.
- [LGX25] Heming Liao, Jiangxia Ge, and Rui Xue. Non-adaptive one-way to hiding not only implies adaptive quantum reprogramming, but also does better. *Cryptology ePrint Archive*, Report 2025/1998, 2025.
- [LMM24] Fukang Liu, Mohammad Mahzoun, and Willi Meier. Modelling ciphers with overdefined systems of quadratic equations: Application to friday, vision, RAIN and biscuit. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part VII*, volume 15490 of *Lecture Notes in Computer Science*, pages 424–456, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
- [LMØM23] Fukang Liu, Mohammad Mahzoun, Morten Øyegarden, and Willi Meier. Algebraic attacks on RAIN and AIM using equivalent representations. *IACR Trans. Symmetric Cryptol.*, 2023(4):166–186, 2023.
- [MBS04] Jean-Charles Faugère Magali Bardet and Bruno Salvy. On the complexity of gröbner basis computation of semi-regular overdetermined algebraic equations. In *International Conference on Polynomial System Solving*, pages 71–74, 2004.
- [MR14] Ben Morris and Phillip Rogaway. Sometimes-recurse shuffle - almost-random permutations in logarithmic expected time. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 311–326, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
- [MS04] Alfred Menezes and Nigel P. Smart. Security of signature schemes in a multi-user setting. *Designs, Codes and Cryptography*, 33(3):261–274, 2004.
- [Nat01] National Institute of Standards and Technology. Advanced Encryption Standard (AES). Federal Information Processing Standards Publication (FIPS) NIST FIPS 197-upd1, Department of Commerce, Washington, D.C., 2001. Updated May 9, 2023.
- [Nat15] National Institute of Standards and Technology. Secure Hash Standard (SHS). Federal Information Processing Standards Publication FIPS PUB 180-4, National Institute of Standards and Technology, Gaithersburg, MD, August 2015.
- [Nat24] National Institute of Standards and Technology. NIST proposes to standardize a wider variant of AES. Computer Security Resource Center, NIST, December 2024.
- [ND15] National Institute of Standards and Technology and Morris J. Dworkin. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Federal Information Processing Standards Publication 202, National Institute of Standards and Technology, Gaithersburg, MD, August 2015.
- [NIS22] NIST. Call for additional digital signature schemes for the post-quantum cryptography standardization process. Technical report, National Institute of Standards and Technology, 2022.
- [Nyb94] Kaisa Nyberg. Differentially uniform mappings for cryptography. In Tor Helleseeth, editor, *Advances in Cryptology – EUROCRYPT’93*, volume 765 of *Lecture Notes in Computer Science*, pages 55–64, Lofthus, Norway, May 23–27, 1994. Springer Berlin Heidelberg, Germany.

- [Pat09] Jacques Patarin. The “coefficients H” technique (invited talk). In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *SAC 2008: 15th Annual International Workshop on Selected Areas in Cryptography*, volume 5381 of *Lecture Notes in Computer Science*, pages 328–345, Sackville, New Brunswick, Canada, August 14–15, 2009. Springer Berlin Heidelberg, Germany.
- [Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part I*, volume 13507 of *Lecture Notes in Computer Science*, pages 657–687, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [SAB⁺20] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, and Damien Stehlé. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [Sam26] Samsung SDS Open Source. AIMer reference implementation, 2026. GitHub repository. Verified on 2026-04-08. Public repository containing `AIMer-package.zip`; GitHub reports 1 commit, latest visible update Jan. 30, 2026.
- [SCL⁺25] Yimeng Sun, Shiyao Chen, Guowei Liu, Meiqin Wang, and Chao Niu. High exponents may not suffice to patch AIM (on attacks, weak parameters, and patches for AIM2). *Cryptology ePrint Archive*, Paper 2025/2272, 2025.
- [Sho04] Victor Shoup. Sequences of games: a tool for taming complexity in security proofs. *Cryptology ePrint Archive*, Report 2004/332, 2004.
- [SPH20] SPHINCS+ Team. SPHINCS+ round 3 submission package, 2020. NIST PQC Round 3 submission archive used for the SLH-DSA/SPHINCS+ benchmark.
- [SS21] Jan Ferdinand Sauer and Alan Szepeieniec. Sok: Gröbner basis algorithms for arithmetization oriented ciphers. *IACR Cryptol. ePrint Arch.*, page 870, 2021.
- [Str69] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13:354–356, 1969.
- [YSWW21] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2986–3001, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.
- [YZY⁺24] Hong-Sen Yang, Qun-Xiong Zheng, Jing Yang, Quan feng Liu, and Deng Tang. A new security evaluation method based on resultant for arithmetic-oriented algorithms. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part VII*, volume 15490 of *Lecture Notes in Computer Science*, pages 457–489, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
- [YZY25] Hong-Sen Yang, Qun-Xiong Zheng, and Jing Yang. Algebraic cryptanalysis of AO primitives based on polynomial decomposition - applications to rain and full AIM-I/III/V. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology*

- *ASIACRYPT 2025, Part I*, volume 16245 of *Lecture Notes in Computer Science*, pages 383–410, Melbourne, VIC, Australia, December 8–12, 2025. Springer, Singapore, Singapore.
- [ZFY23] Jiang Zhang, Dengguo Feng, and Di Yan. NEV: Faster and smaller NTRU encryption using vector decoding. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part VII*, volume 14444 of *Lecture Notes in Computer Science*, pages 157–189, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
- [Zha12] Mark Zhandry. How to construct quantum random functions. In *53rd Annual Symposium on Foundations of Computer Science*, pages 679–687, New Brunswick, NJ, USA, October 20–23, 2012. IEEE Computer Society Press.
- [Zha19] Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 239–268, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
- [Zha21] Mark Zhandry. Redeeming reset indistinguishability and applications to post-quantum security. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part I*, volume 13090 of *Lecture Notes in Computer Science*, pages 518–548, Singapore, December 6–10, 2021. Springer, Cham, Switzerland.
- [Zha25] Mark Zhandry. A Note on Quantum-Secure PRPs. *Quantum*, 9:1696, 2025.
- [ZWY⁺23] Kaiyi Zhang, Qingju Wang, Yu Yu, Chun Guo, and Hongrui Cui. Algebraic attacks on round-reduced rain and full AIM-III. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part III*, volume 14440 of *Lecture Notes in Computer Science*, pages 285–310, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
- [ZYF⁺20] Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: KEMs and signatures of smaller sizes. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 12111 of *Lecture Notes in Computer Science*, pages 37–65, Edinburgh, UK, May 4–7, 2020. Springer, Cham, Switzerland.

A Proving Knowledge of Secret Keys in NEV

NEV is a lattice-based key encapsulation mechanism [ZFY23]. In this section, we describe a method to prove the knowledge of secret key that corresponds to a given public key in NEV. We first recall the syntax and definition of the public/secret key pair in NEV, then demonstrate how to prove it using VOLEitH. This section serves as an example of the extensibility of VOLEitH when proving relations expressible as systems of low-degree multivariate polynomials.

A.1 Definition of NEV

We recall only the part of NEV that is needed for proving knowledge of a secret key corresponding to a public key. The full NEV-KEM construction is obtained from the passively secure NEV-PKE by applying the Fujisaki-Okamoto transform with implicit rejection [FO99, ZFY23].

The same transform is also applied to the optimized variant **NEV-PKE'** to obtain **NEV-KEM'**. In particular, the KEM key generation first runs the corresponding PKE key generation algorithm to obtain a pair (pk', sk') , and then stores additional KEM material in the secret key. These additional values do not change the algebraic public/secret key relation. Thus, for the purpose of proving the knowledge of secret key, it suffices to prove the public/secret key relation of the underlying IND-CPA encryption scheme.

Let

$$R = \mathbb{Z}[x]/(x^n + 1), \quad R_q = \mathbb{Z}_q[x]/(x^n + 1),$$

where n is a power of two and q is an odd prime. The plaintext space is $\{0, 1\}^\ell$. Let k be the largest integer such that $k \mid n$ and $n/k \geq \ell$. NEV sets

$$v = 1 - x^{n/k} \in R_q^*, \quad v^{-1} = \frac{q+1}{2} (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q^*.$$

The distribution B_η is the centered binomial distribution with parameter η , sampled coefficient-wise for ring elements. The distribution T_σ is the ternary distribution over $\{-1, 0, 1\}$ with probabilities $\Pr[1] = \Pr[-1] = \sigma$ and $\Pr[0] = 1 - 2\sigma$. In particular, $B_1 = T_{1/4}$. We describe the key generation algorithms of **NEV-PKE** and **NEV-PKE'** in [Algorithms 1](#) and [2](#).

Algorithm 1 **NEV-PKE.KeyGen**

Input: Parameters $(n, q, k, \ell, \chi_f, \chi_g)$ and $v = 1 - x^{n/k} \in R_q^*$.

Output: A public key $pk = h$ and a secret key $sk = f$.

- 1: Sample $f' \leftarrow \chi_f$ and $g \leftarrow \chi_g$ coefficient-wise such that $f := v f' + 1$ is invertible in R_q .
 - 2: Compute $h := g/f = g \cdot f^{-1}$ in R_q .
 - 3: **return** $(pk, sk) := (h, f) \in R_q \times R_q$.
-

Algorithm 2 **NEV-PKE'.KeyGen**

Input: Parameters $(n, q, k, \ell, \chi_f, \chi_g)$ and $v = 1 - x^{n/k} \in R_q^*$.

Output: A public key $pk = h$ and a secret key $sk = f$.

- 1: Sample $f' \leftarrow \chi_f$ and $g \leftarrow \chi_g$ coefficient-wise such that $f := f' + v^{-1}$ is invertible in R_q .
 - 2: Compute $h := g/f = g \cdot f^{-1}$ in R_q .
 - 3: **return** $(pk, sk) := (h, f) \in R_q \times R_q$.
-

We recall the selected parameter sets of [\[ZFY23\]](#) in [Table 18](#).

Table 18: Recommended parameters for the NEV parameter sets used in the public/secret key relation.

Parameter set	n	q	ℓ	k	v	(χ_f, χ_g)	(χ_r, χ_e)	(pk , ct)	Dec. failure	LWE est. (sk, ct)
NEV-512	512	769	256	2	$1 - x^{256}$	(B_1, B_1)	$(B_1, T_{1/6})$	(615, 615)	2^{-138}	(145, 141)
NEV'-512	512	769	256	2	$1 - x^{256}$	(B_1, B_1)	(B_1, B_1)	(615, 615)	2^{-200}	(145, 145)
NEV-1024	1024	769	256	4	$1 - x^{256}$	(B_1, B_1)	$(B_1, T_{1/6})$	(1229, 1229)	2^{-152}	(292, 281)
NEV'-1024	1024	769	256	4	$1 - x^{256}$	(B_1, B_1)	(B_1, B_1)	(1229, 1229)	2^{-200}	(292, 292)

A.2 Expressing Public/Secret Key Relation as Polynomials

As VOLEitH utilizes QuickSilver as its underlying proof system and QuickSilver excels at proving systems of low-degree polynomials, we first express the public/secret key relation of **NEV-PKE** and **NEV-PKE'** in this form.

NEV-PKE. In [Algorithm 1](#), the secret-key multiplier is $f = vf' + 1$, and the public key satisfies $h = g/f$ in R_q . Equivalently, a prover who knows the secret key can prove that there exist small polynomials $f', g \in R$ such that

$$h(vf' + 1) = g \pmod{q}$$

in R_q . Since h and v are public, this is a system of linear equations over $\mathbb{Z}_q$ in the coefficient variables of f' and g .

NEV-PKE'. In [Algorithm 2](#), the optimized variant instead sets $f = f' + v^{-1}$. Thus the corresponding public/secret key relation is the existence of small polynomials $f', g \in R$ such that

$$h(f' + v^{-1}) = g \pmod{q}$$

in R_q . Again, because h and v^{-1} are public constants, this gives only linear constraints over $\mathbb{Z}_q$ on the coefficients of f' and g .

Therefore, both variants require committing to the coefficients of the same two witness polynomials (f', g) . For all parameter sets in [Table 18](#), the key distributions are $(\chi_f, \chi_g) = (B_1, B_1)$, so every committed coefficient must lie in $\{-1, 0, 1\}$. Over $\mathbb{Z}_q$, this support condition for each coefficient z is the degree-3 equation

$$z(z - 1)(z + 1) = 0.$$

Consequently, the public/secret key relation for both NEV-PKE and NEV-PKE' is expressed as a combination of linear equations and degree-3 coefficient-range equations, which is well suited to the VOLEitH proof system.

A.3 Estimating the Proof Size

Although in Lynxer, we only utilize VOLEitH to prove degree-2 relations over $\mathbb{F}_2$, VOLEitH can be generalized to proving arbitrary low-degree polynomials over prime fields. Only the QuickSilver component needs to be tweaked. Here we provide estimations of proving the knowledge of secret keys for NEV, using the formalization described above.

Since we only need to commit to the coefficients of f', g , we have $\ell_{\text{wit}} = 2n$. By working in a larger prime field $\mathbb{F}_q$, we can choose a smaller field extension degree d_{ext} such that $|\mathbb{F}_{q^{d_{\text{ext}}}}| \geq 2^\lambda$. Moreover, since the polynomial system has maximum degree of 3, we need two random $\mathbb{F}_{q^{d_{\text{ext}}}}$ IT-MACs as masks. Moreover, to minimize proof size, we can use a random vector over $\mathbb{F}_{q^{d_{\text{ext}}}}$ as the linear universal hash function instead of [VOLEHash](#). Therefore, we can tweak [Eq.\(4\)](#) to get the proof size estimate as shown in [Eq.\(119\)](#).

$$\begin{aligned} \text{Size} = & \underbrace{2n \cdot \log(q)}_{\text{Fixing witness}} + \underbrace{(\tau - 1)(2n + 3d_{\text{ext}}) \cdot \log(q)}_{c_1, \dots, c_{\tau-1}} + \underbrace{(T_{\text{open}} + 2\tau)\lambda}_{\text{BAVC Opening}} \\ & + \underbrace{2d_{\text{ext}} \cdot \log(q)}_{\text{QuickSilver}} + \underbrace{d_{\text{ext}} \cdot \log(q)}_{\text{SoftSpoken}} + \underbrace{\lambda}_{\text{chall}_3} + \underbrace{120}_{\text{iv}_{\text{sig}}} + \underbrace{32}_{\text{Counter}} \end{aligned} \quad (119)$$

Using [Eq.\(119\)](#), we estimate the concrete proof sizes for the NEV parameter sets in [Table 19](#). The NEV-512 parameter sets correspond to 128-bit security, so we use the FAEST-128s VOLEitH parameters $(\lambda, \tau, w_{\text{grind}}, T_{\text{open}}) = (128, 11, 7, 102)$. The NEV-1024 parameter sets correspond to 256-bit security, so we use our Lynxer-256s VOLEitH parameters from [Table 5](#), namely $(\lambda, \tau, w_{\text{grind}}, T_{\text{open}}) = (256, 22, 12, 224)$. In the estimate, $\log(q)$ is instantiated as $\lceil \log_2(q) \rceil$ bits per field element.

Table 19: Estimated VOLEitH proof sizes for proving knowledge of an NEV secret key.

Parameter Set	λ	τ	w_{grind}	T_{open}	d_{ext}	Size (bits)	Size (bytes)
NEV-512	128	11	7	102	14	133,412	16,677
NEV'-512	128	11	7	102	14	133,412	16,677
NEV-1024	256	22	12	224	27	537,396	67,175
NEV'-1024	256	22	12	224	27	537,396	67,175

B Illustrations of the Nonlinear Component

B.1 Algebraic Metrics

The “degree” of the power function $f(x) = x^d$ over $\mathbb{F}_{2^n}$ has the following two definitions:

1. the degree over $\mathbb{F}_{2^n}$ is $d \bmod (2^n - 1)$;
2. the degree over $\mathbb{F}_2$ is $\text{wt}_2(d)$, i.e., the number of ‘1’s in the binary expansion of d .

To ensure that $f(x) = x^d$ is a permutation over $\mathbb{F}_{2^n}$, it is necessary that $\gcd(d, 2^n - 1) = 1$. In particular, to ensure that $f(x) = x^{2^i - 1}$ is a permutation, the condition reduces to $\gcd(i, n) = 1$.

B.2 Statistic Metrics

For $f(x) = x^{-1}$ over $\mathbb{F}_{2^n}$, $\text{diff}_f = 2^{2-n}$ and $\text{cor}_f = 2^{1-n/2}$ [Ny94].

For $f(x) = x^d$ with $\gcd(d, 2^n - 1) = 1$, the differential equation $(x + \Delta_x)^d + x^d = \Delta_f$ is a polynomial equation of degree $d - 1$. Hence, there are at most $d - 1$ solutions for $\Delta_x \neq 0$ and $\text{diff}_f \leq (d - 1)/2^n$. Moreover, when d is not a power of 2, $\text{cor}_f \leq (d - 1)2^{-n/2}$ referring to [KSW19].

For $f(x, y) = x \cdot y$ over $\mathbb{F}_{2^n}$, $\text{diff} = 2^{-n}$, as the differential equation is a linear equation in (x, y) .

For $f(x, y, z) = x \cdot y \cdot z^{2^n - 2}$ over $\mathbb{F}_{2^n}$, $\text{diff} = 2/2^n - 1/2^{2n}$, and the conditions for achieving the maximum value are:

1. $\Delta_z = 0$, $(\Delta_x, \Delta_y) \neq (0, 0)$, and $\Delta_f = 0$: When $\Delta_z = 0$, the equation simplifies to $\Delta_f = (\Delta_x y + d_y x + d_x d_y) \cdot z^{2^n - 2}$. If $z = 0$, then $\Delta_f = 0$ holds for all x, y ; thus, there are 2^{2n} solutions. If $z \neq 0$, then the equation becomes the affine linear relation $\Delta_f z = \Delta_x y + \Delta_y x + \Delta_x \Delta_y$, and each z corresponds to 2^n solutions of (x, y) ; thus, there are $2^n(2^n - 1)$ solutions. Combining both cases, the total number of solutions is $2^{2n+1} - 2^n$.
2. $\Delta_z \neq 0$, $\Delta_x \Delta_y = 0$ and $\Delta_f = 0$: When $\Delta_z \neq 0$: For $z = 0$, the equation simplifies to $(x + \Delta_x)(y + \Delta_y) = \Delta_f \Delta_z$, and the number of solutions is $2^{n+1} - 1$ when $\Delta_f = 0$; For $z = \Delta_z$, the equation simplifies to $xy = \Delta_f \Delta_z$, and the number of solutions is $2^{n+1} - 1$ when $\Delta_f = 0$; For $z \notin \{0, \Delta_z\}$, the equation simplifies to $(x + \frac{\Delta_y}{\Delta_z} z)(y + \frac{\Delta_x}{\Delta_z} z) = \frac{z(z + \Delta_z)(\Delta_x \Delta_y + \Delta_f \Delta_z)}{\Delta_z^2}$, and when $\Delta_f = 0$ and $\Delta_x \Delta_y = 0$, each z corresponds to $2^{n+1} - 1$ solutions, which sums up to $(2^n - 2)(2^{n+1} - 1)$. Combining these three cases, the total number of solutions is $2^{2n+1} - 2^n$.

For $f(x, y) = x \cdot y$, $\text{cor}_F = 1/2^n$. For $\Gamma_f \neq 0$, calculate the Walsh spectrum $W_f(\Gamma_x, \Gamma_y, \Gamma_f) = \sum_{x, y \in \mathbb{F}_{2^n}} (-1)^{\text{Tr}(\Gamma_f xy + \Gamma_x x + \Gamma_y y)}$. Substitute $x \leftarrow \Gamma_f x$, $a \leftarrow \Gamma_x \Gamma_f^{-1}$, $b \leftarrow \Gamma_y$, the equation simplifies to $W(a, b) = \sum_{x, y} (-1)^{\text{Tr}(xy + ax + by)}$. Fix x and sum over y , since

$$\sum_{y \in \mathbb{F}_{2^n}} (-1)^{\text{Tr}((x+b)y)} = \begin{cases} 2^n, & x = b \\ 0, & x \neq b \end{cases}$$

for the orthogonality of the trace function, $W(a, b)$ is non-zero only when $x = b$, and equals $(-1)^{\text{Tr}(ab)} \cdot 2^n$.

For $f(x, y) = x \cdot y \cdot z^{2^n-2}$, $\text{cor}_F = 1/2^n - 1/2^{2n}$. For $\Gamma_f \neq 0$, divide the Walsh sum into two parts of $z = 0$ and $z \neq 0$. For $z = 0$, $f(x, y, 0) = 0$, and the sum is $\sum_{x,y} (-1)^{\text{Tr}(\Gamma_x x + \Gamma_y y)}$, taking the maximum value of 2^{2n} when $\Gamma_x = \Gamma_y = 0$. For fixed $z \neq 0$, $f(x, y, z) = xyz^{-1}$, using the Walsh result of the binary multiplication function, $\sum_{x,y} (-1)^{\text{Tr}(\Gamma_f xyz^{-1})} = 2^n$ when $\Gamma_z = 0$. Finally, the maximum Walsh spectrum is obtained: $W_f = 2^{2n} + 2^n(2^n - 1) = 2^{2n+1} - 2^n$.